VNUHCM - UNIVERSITY OF SCIENCE
FACULTY OF INFORMATION TECHNOLOGY

PA4 Report

Group 6: PDF

CSC13002 - Introduction to Software Engineering

Author:
Nguyễn Tuấn An - 24127011
Nguyễn Hải Nam - 24127088
Nguyễn Hải Trọng - 24127311
Phạm Đinh Gia Khánh - 24127420
Võ An Phú - 24127494

Lecturer:
Ms. Nguyễn Thị Minh Tuyền
Mr. Hồ Tuấn Thanh
Mr. Mai Anh Tuấn

July 08th, 2026

Table of Contents

- A - Revised Use-Case Specification
  - Revision History
  - Use-Case Specification: Sign Up
    - 1. Sign Up
      - 1.1 Brief Description
    - 2. Preconditions
    - 3. Flow of Events
      - 3.1 Basic Flow
      - 3.2 Alternative Flows
        - 3.2.1 Email Already Exists
        - 3.2.2 Weak Password
        - 3.2.3 Password Too Long
        - 3.2.4 Rate Limit Exceeded
        - 3.2.5 Confirm Password Does Not Match
        - 3.2.6 Username Already Taken, Reserved, or Invalid
        - 3.2.7 Sign Up with Google Account
        - 3.2.8 Google Sign Up Username Collision
    - 4. Special Requirements
    - 5. Postconditions
    - 6. Extension Points
    - 7. Prototype Requirement
  - Use-Case Specification: Log In
    - 1. Log In
      - 1.1 Brief Description
    - 2. Preconditions
    - 3. Flow of Events
      - 3.1 Basic Flow
      - 3.2 Alternative Flows
        - 3.2.1 Invalid Credentials
        - 3.2.2 Account Suspended or Banned
        - 3.2.3 Rate Limit Exceeded
        - 3.2.4 Sign In with Google Account
        - 3.2.5 Log In with Non-Existent Google Account
    - 4. Special Requirements
    - 5. Postconditions
    - 6. Extension Points
    - 7. Prototype Requirement

- Use-Case Specification: Log Out
  - 1. Log Out
    - 1.1 Brief Description
  - 2. Preconditions
  - 3. Flow of Events
    - 3.1 Basic Flow
    - 3.2 Alternative Flows
      - 3.2.1 User Cancels Log Out
      - 3.2.2 Session Already Expired or Invalid
  - 4. Special Requirements
  - 5. Postconditions
  - 6. Extension Points
  - 7. Prototype Requirement
- Use-Case Specification: Manage Profile
  - 1. Manage Profile
    - 1.1 Brief Description
  - 2. Preconditions
  - 3. Flow of Events
    - 3.1 Basic Flow
    - 3.2 Alternative Flows
      - 3.2.1 Invalid Input Format
      - 3.2.2 Input Exceeds Maximum Limits
      - 3.2.3 Username Handle Collision on Update
      - 3.2.4 User Cancels Profile Edit
      - 3.2.5 Update Avatar Image
  - 4. Special Requirements
  - 5. Postconditions
  - 6. Extension Points
  - 7. Prototype Requirement
- Use-Case Specification: Change Password
  - 1. Change Password
    - 1.1 Brief Description
  - 2. Preconditions
  - 3. Flow of Events
    - 3.1 Basic Flow
    - 3.2 Alternative Flows
      - 3.2.1 Incorrect Current Password
      - 3.2.2 New Password Matches Old Password
      - 3.2.3 Confirm Password Does Not Match

        - 3.2.4 Weak or Too Long New Password
        - 3.2.5 User Cancels
        - 3.2.6 Rate Limit Exceeded
    - 4. Special Requirements
    - 5. Postconditions
    - 6. Extension Points
    - 7. Prototype Requirement
- Use-Case Specification: Change Avatar
    - 1. Change Avatar
        - 1.1 Brief Description
    - 2. Preconditions
    - 3. Flow of Events
        - 3.1 Basic Flow
        - 3.2 Alternative Flows
            - 3.2.1 File Too Large
            - 3.2.2 User Cancels
            - 3.2.3 Wrong File Type
    - 4. Special Requirements
    - 5. Postconditions
    - 6. Extension Points
    - 7. Prototype Requirement
- Use-Case Specification: Browse Feed
    - 1. Browse Feed
        - 1.1 Brief Description
    - 2. Preconditions
    - 3. Flow of Events
        - 3.1 Basic Flow
        - 3.2 Alternative Flows
            - 3.2.1 Switch feed mode
            - 3.2.2 No Posts Available
            - 3.2.3 Scroll to Load More Posts
            - 3.2.4 View a long content post
    - 4. Special Requirements
    - 5. Postconditions
    - 6. Extension Points
    - 7. Prototype Requirement
- Use-Case Specification: Search Content
    - 1. Search Content
        - 1.1 Brief Description

- 2. Preconditions
- 3. Flow of Events
  - 3.1 Basic Flow
  - 3.2 Alternative Flows
    - 3.2.1 Search feature unavailable
    - 3.2.2 Search Query Exceeds Character Limit
    - 3.2.3 Switch Category (Posts and Profile)
    - 3.2.4 No matching results
    - 3.2.5 Click a hashtag in "TRENDING TOPICS"
    - 3.2.6 Click a profile in "Suggested Creators"
    - 3.2.7 Click a post in "Trending Discussions"
- 4. Special Requirements
- 5. Postconditions
- 6. Extension Points
- 7. Prototype Requirement

- Use-Case Specification: View Post Details
  - 1. View Post Details
    - 1.1 Brief Description
  - 2. Preconditions
  - 3. Flow of Events
    - 3.1 Basic Flow
    - 3.2 Alternative Flows
      - 3.2.1 No comments
      - 3.2.2 Post Not Found
  - 4. Special Requirements
  - 5. Postconditions
  - 6. Extension Points
  - 7. Prototype Requirement
- Use-Case Specification: Create Post
  - 1. Create Post
    - 1.1 Brief Description
  - 2. Preconditions
  - 3. Flow of Events
    - 3.1 Basic Flow
    - 3.2 Alternative Flows
      - 3.2.1. Post creation canceled
      - 3.2.2 Content Validation Error
      - 3.2.3 Media Attachment Error
      - 3.2.4 Media Attachment Cancelled

        - 3.2.5 Exceeds Limit of Characters
        - 3.2.6 Exceeds Limit of Number of Media Attachments
        - 3.2.7 Valid Image with Large Size
        - 3.2.8 Network Timeout after Attaching Any Image
    - 4. Special Requirements
    - 5. Postconditions
    - 6. Extension Points
    - 7. Prototype Requirement
- Use-Case Specification: Interact with Post
    - 1. Interact with Post
        - 1.1 Brief Description
    - 2. Preconditions
    - 3. Flow of Events
        - 3.1 Basic Flow
        - 3.2 Alternative Flows
            - 3.2.1. The user selects **Reply**.
            - 3.2.2. The user selects **Like**.
            - 3.2.3. The user selects **Report**.
    - 4. Special Requirements
    - 5. Postconditions
    - 6. Extension Points
    - 7. Prototype Requirement
- Use-Case Specification: Leave Comment
    - 1. Leave Comment
        - 1.1 Brief Description
    - 2. Preconditions
    - 3. Flow of Events
        - 3.1 Basic Flow
        - 3.2 Alternative Flows
            - 3.2.1 Empty Comment
            - 3.2.2 Comment Exceeds Maximum Length
            - 3.2.3 Long Comment Display
            - 3.2.4 Newline Input with Shift+Enter
            - 3.2.5 Duplicate Submission Prevention
            - 3.2.6 Malicious Script Injection (XSS) Attempt
            - 3.2.7 Emoji and Unicode Input
            - 3.2.8 Nested Reply
            - 3.2.9 Offline Submission
            - 3.2.10 Multi-Level Nested Replies (Depth 1–2)

- 3.2.11 Deep Nested Replies (Depth > 2)
- 3.2.12 URL in Comment
- 3.2.13 Comment Rate Limit Exceeded
- 3.2.14 Authentication Expired During Commenting
- 3.2.15 Post Deleted While Commenting
- 4. Special Requirements
- 5. Postconditions
- 6. Extension Points
- 7. Prototype Requirement

- Use-Case Specification: Like/Unlike
  - 1. Like/Unlike
    - 1.1 Brief Description
  - 2. Preconditions
  - 3. Flow of Events
    - 3.1 Basic Flow
    - 3.2 Alternative Flows
      - 3.2.1 Target Post or Comment Deleted / Not Found
  - 4. Special Requirements
  - 5. Postconditions
  - 6. Extension Points
  - 7. Prototype Requirement
- Use-Case Specification: Request AI Summary
  - 1. Request AI Summary
    - 1.1 Brief Description
  - 2. Preconditions
  - 3. Flow of Events
    - 3.1 Basic Flow
    - 3.2 Alternative Flows
      - 3.2.1 Post Not Found or No Longer Accessible
      - 3.2.2 AI Provider Error
      - 3.2.3 Invalid AI Response
      - 3.2.4 Rate Limit Exceeded
      - 3.2.5 Network Offline
      - 3.2.6 Unauthorized or Suspended Access
  - 4. Special Requirements
  - 5. Postconditions
  - 6. Extension Points
  - 7. Prototype Requirement
- Use-Case Specification: Delete Own Post

- 1. Delete Own Post
  - 1.1 Brief Description
- 2. Preconditions
- 3. Flow of Events
  - 3.1 Basic Flow
  - 3.2 Alternative Flows
    - 3.2.1 User Cancels Deletion
    - 3.2.2 Post Not Found or Forbidden
    - 3.2.3 Deletion Failure
- 4. Special Requirements
- 5. Postconditions
- 6. Extension Points
- 7. Prototype Requirement

- Use-Case Specification: Report Inappropriate Content
  - 1. Report Inappropriate Content
    - 1.1 Brief Description
  - 2. Preconditions
  - 3. Flow of Events
    - 3.1 Basic Flow
    - 3.2 Alternative Flows
      - 3.2.1 User Cancels Report
      - 3.2.2 Validation Error
      - 3.2.3 Content Already Reported by User
      - 3.2.4 Content Not Found or No Longer Reportable
      - 3.2.5 Reporting Rate Limit Exceeded
      - 3.2.6 Report Submission Failure
  - 4. Special Requirements
  - 5. Postconditions
  - 6. Extension Points
  - 7. Prototype Requirement
- Use-Case Specification: Review Reports
  - 1. Review Reports
    - 1.1 Brief Description
  - 2. Preconditions
  - 3. Flow of Events
    - 3.1 Basic Flow
    - 3.2 Alternative Flows
      - 3.2.1 No Reports Available
      - 3.2.2 Actor Leaves Without a Decision

        - 3.2.3 Report Not Found or Already Processed
        - 3.2.4 Reported Content No Longer Exists
    - 4. Special Requirements
    - 5. Postconditions
    - 6. Extension Points
    - 7. Prototype Requirement
- Use-Case Specification: Forward Report to Admin
    - 1. Forward Report to Admin
        - 1.1 Brief Description
    - 2. Preconditions
    - 3. Flow of Events
        - 3.1 Basic Flow
        - 3.2 Alternative Flows
            - 3.2.1 Missing or Insufficient Escalation Rationale
            - 3.2.2 Report Not Found
            - 3.2.3 Report Already Processed
            - 3.2.4 Escalation Update Failure
        - 3.2.5 Concurrent Escalation
    - 4. Special Requirements
    - 5. Postconditions
    - 6. Extension Points
    - 7. Prototype Requirement
- Use-Case Specification: Delete Any Content
    - 1. Delete Any Content
        - 1.1 Brief Description
    - 2. Preconditions
    - 3. Flow of Events
        - 3.1 Basic Flow
        - 3.2 Alternative Flows
            - 3.2.1 Administrator Cancels Deletion
            - 3.2.2 Missing or Insufficient Rationale
            - 3.2.3 Content Not Found or Already Deleted
            - 3.2.4 Deletion Transaction Fails
    - 4. Special Requirements
    - 5. Postconditions
    - 6. Extension Points
    - 7. Prototype Requirement
- Use-Case Specification: Permanently Remove User
    - 1. Permanently Remove User

- 1.1 Brief Description
- 2. Preconditions
- 3. Flow of Events
  - 3.1 Basic Flow
  - 3.2 Alternative Flows
    - 3.2.1 Administrator Cancels Deletion
    - 3.2.2 Missing or Insufficient Rationale
    - 3.2.3 Data-Wipe Transaction Fails
- 4. Special Requirements
- 5. Postconditions
- 6. Extension Points
- 7. Prototype Requirement

- Use-Case Specification: Assign Moderator Role
  - 1. Assign Moderator Role
    - 1.1 Brief Description
  - 2. Preconditions
  - 3. Flow of Events
    - 3.1 Basic Flow
    - 3.2 Alternative Flows
      - 3.2.1 Administrator Cancels Role Assignment
      - 3.2.2 Missing or Insufficient Rationale
      - 3.2.3 Target Not Found or Not Eligible
      - 3.2.4 Concurrent Role Update
      - 3.2.5 Role Update or Audit Failure
  - 4. Special Requirements
  - 5. Postconditions
  - 6. Extension Points
  - 7. Prototype Requirement
- B - Software Architecture: System Context Diagram
  - 1. Technology Stack
    - Frontend
    - Backend
    - Database
    - External Services
    - Shared Module
  - 2. C4 Level 1 - System Context Diagram
    - Diagram Explanation
- C - Software Architecture: Container Diagram and Component Diagram
  - 3. C4 Level 2 - Container Diagram

        - 3.1 Diagram
        - 3.2 Container Descriptions
        - 3.3 Relationships Summary
    - 4. C4 Level 3 - Component Diagrams
        - 4.1 Frontend - Web Application
            - 4.1.1 Diagram
            - 4.1.2 Component Descriptions
        - 4.2 Backend - API Server
            - 4.2.1 Diagram A - Content & Discovery (Feed · Search · Profile · Hashtags · Media · Reports · AI TL;DR)
            - 4.2.2 Diagram B - Identity, Governance & AI (Auth · OAuth · Reactions · Moderation · Admin)
            - 4.2.3 Component Descriptions
                - Middleware Pipeline (API Gateway)
                - Route Layer
                - Service Layer
                - Data Access (`src/backend/db/`)
                - Infrastructure
- D - Deployment Diagram
    - 1. Production Deployment Diagram
        - 1.1 Diagram
            - How the request flows
        - 1.2 Node Descriptions
        - 1.3 Communication Protocols Summary
    - 2. Development Deployment Diagram
        - 2.1 Diagram
            - Key development details
        - 2.2 Node Descriptions
        - 2.3 Communication Protocols Summary
    - 3. Environment Comparison
- E - Implement 2 Functional Groups using Spec Kit
- F - AI Usage Report and Weekly Report
    - AI Usage Report
        - Declaration of No AI Use
        - AI Usage Notes
    - Weekly Report
        - Sprint Planning
        - Scrum Meetings
        - Sprint Review

- Jira Board

A - Revised Use-Case Specification

Performed by: Nguyễn Hải Trọng | Reviewed by: Phạm Đinh Gia Khánh

Revision History

Date	Version	Description	Author
19/Jul/2026	1.0	Initial draft: Documented all 21 Use-Case Specifications covering Auth, Core Interactions, and Moderation.	Nguyễn Hải Trọng
06/Aug/2026	1.1	Update the Alternative Flow and Reordered sections for clarity	Nguyễn Hải Trọng
22/Aug/2026	1.2	Update UCS 01–05 to reflect current app implementation (onboarding redirection, role-based login routing, username collision handling, avatar management, and NIST password constraints)	Nguyễn Tuấn An

Use-Case Specification: Sign Up

Performed by: Nguyễn Hải Trọng | Reviewed by: Nguyễn Tuấn An | Edited by: Nguyễn Tuấn An
Project Name: FOZ
Version: 1.2
Document identifier: FOZ-UCS-01

1. Sign Up

- **Use-Case ID:** UC-01
- **Actor(s):** Guest

1.1 Brief Description

This use case allows an unauthenticated guest to create a new user account and initial profile on the FOZ platform using an email, unique username, and secure password, or via Google OAuth authentication.

2. Preconditions

- The user must be a Guest (unauthenticated).

3. Flow of Events

3.1 Basic Flow

1. The use case begins when the Guest navigates to the Registration page (`/register`).
2. The user enters a unique username, a valid email address, a strong password, and confirms the password.
3. The user clicks "Sign up" (or "Create account").
4. The system validates the inputs (email format, username length 3–30 characters and allowed characters `[a-z0-9._]` , password length between 8 and 128 characters, and password confirmation match).
5. The system checks if the email or username is already registered in the database.
6. The system securely hashes the password using bcrypt (cost factor 12).
7. The system creates a new record in the `user_accounts` table (role: `standard` , status: `active`) and a default profile record in the `profiles` table (with display name initialized to username).
8. The system issues a JWT access token and creates an active session in `refresh_sessions` , setting the `refresh_token` http-only cookie.
9. The system redirects the user to the Onboarding Profile Setup page (`/onboarding/avatar`) to configure their display name and avatar (or proceed to the Home Feed).

3.2 Alternative Flows

3.2.1 Email Already Exists

1. If at step 5, the entered email address already exists in the database.
2. The system aborts the registration and returns HTTP 409 (`EMAIL_TAKEN`).
3. The system displays an error message: "An account with this email already exists".
4. The flow resumes at step 2.

3.2.2 Weak Password

1. If at step 4, the password is less than 8 characters.
2. The system highlights the password field and displays: "Password must be at least 8 characters".

3. The flow resumes at step 2.

3.2.3 Password Too Long

1. If at step 4, the password exceeds 128 characters.
2. The system highlights the password field and displays: "Password is too long".
3. The flow resumes at step 2.

3.2.4 Rate Limit Exceeded

1. If the user makes more than 15 registration attempts within 1 minutes.
2. The system rejects the request with HTTP 429 (`RATE_LIMITED`) and displays: "Too many requests, slow down."
3. The use case ends.

3.2.5 Confirm Password Does Not Match

1. If at step 4, the confirmation password does not match the entered password.
2. The system highlights the confirmation field and displays: "Passwords do not match".
3. The flow resumes at step 2.

3.2.6 Username Already Taken, Reserved, or Invalid

1. If at step 4 or 5, the chosen username already exists in the database, is a reserved keyword (e.g., `admin` , `moderator` , `system`), or contains characters other than lowercase letters, numbers, underscores, and periods.
2. The system aborts registration and displays the specific validation error (e.g., "This username is already taken", "This username is reserved", or "Username can only contain lowercase letters, numbers, underscores, and periods").
3. The flow resumes at step 2.

3.2.7 Sign Up with Google Account

1. At step 2, the user chooses to "Continue with Google".
2. The system initiates Google OAuth (popup or redirect to Google OAuth consent screen).
3. The user successfully authenticates with Google.
4. The system automatically provisions a `user_accounts` record (passwordless) and `profiles` record using Google profile data, deriving a unique base username from the email prefix.
5. The system issues access and refresh tokens, displays a success notification ("Account created! Set up your profile."), and redirects the user to the onboarding page (`/onboarding/avatar`).

3.2.8 Google Sign Up Username Collision

1. If at step 4 of the Google Sign Up flow, the derived base username is already taken by another user or is a reserved keyword.
2. The system automatically generates a unique username by appending an incremental numeric counter (e.g., base name + "1") to the base name.
3. The system proceeds to provision the account with the newly generated unique username.

4. Special Requirements

- **Security:** Passwords must be hashed via bcrypt with a minimum cost factor of 12. Plaintext passwords must never be logged. JWT access tokens are stored in client memory; refresh tokens are stored in http-only `SameSite=Lax` cookies.
- **Rate Limiting:** Maximum 100 registration attempts per 15 minutes per IP address.

5. Postconditions

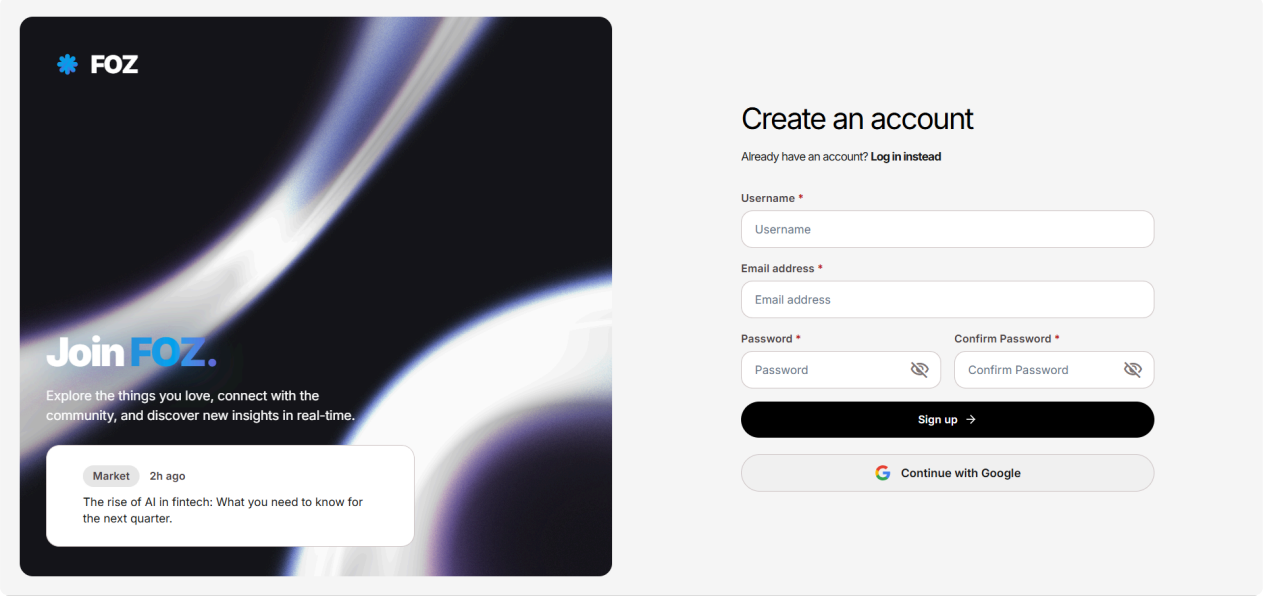
- A new active user account and profile are saved in the database. An active session is stored in `refresh_sessions`. The user is authenticated.

6. Extension Points

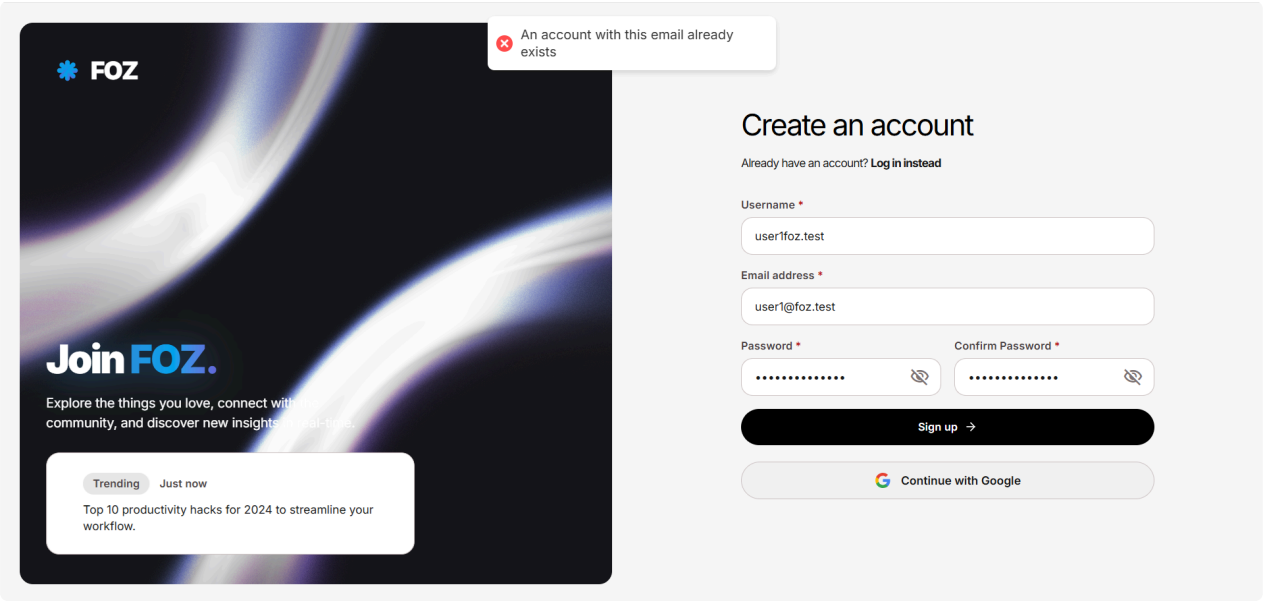
- None.

7. Prototype Requirement

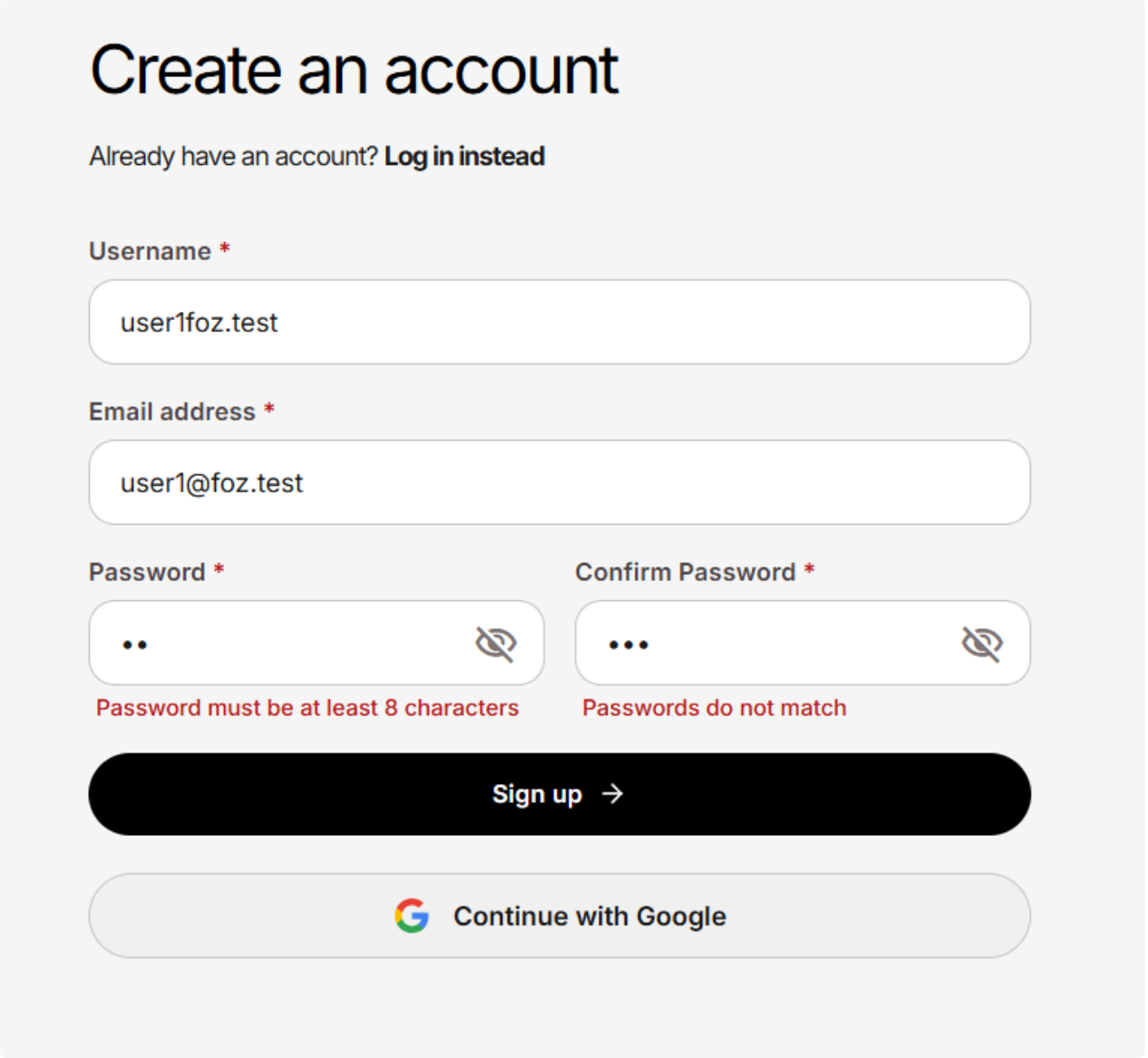
- **Screen 1 (Basic Flow):**



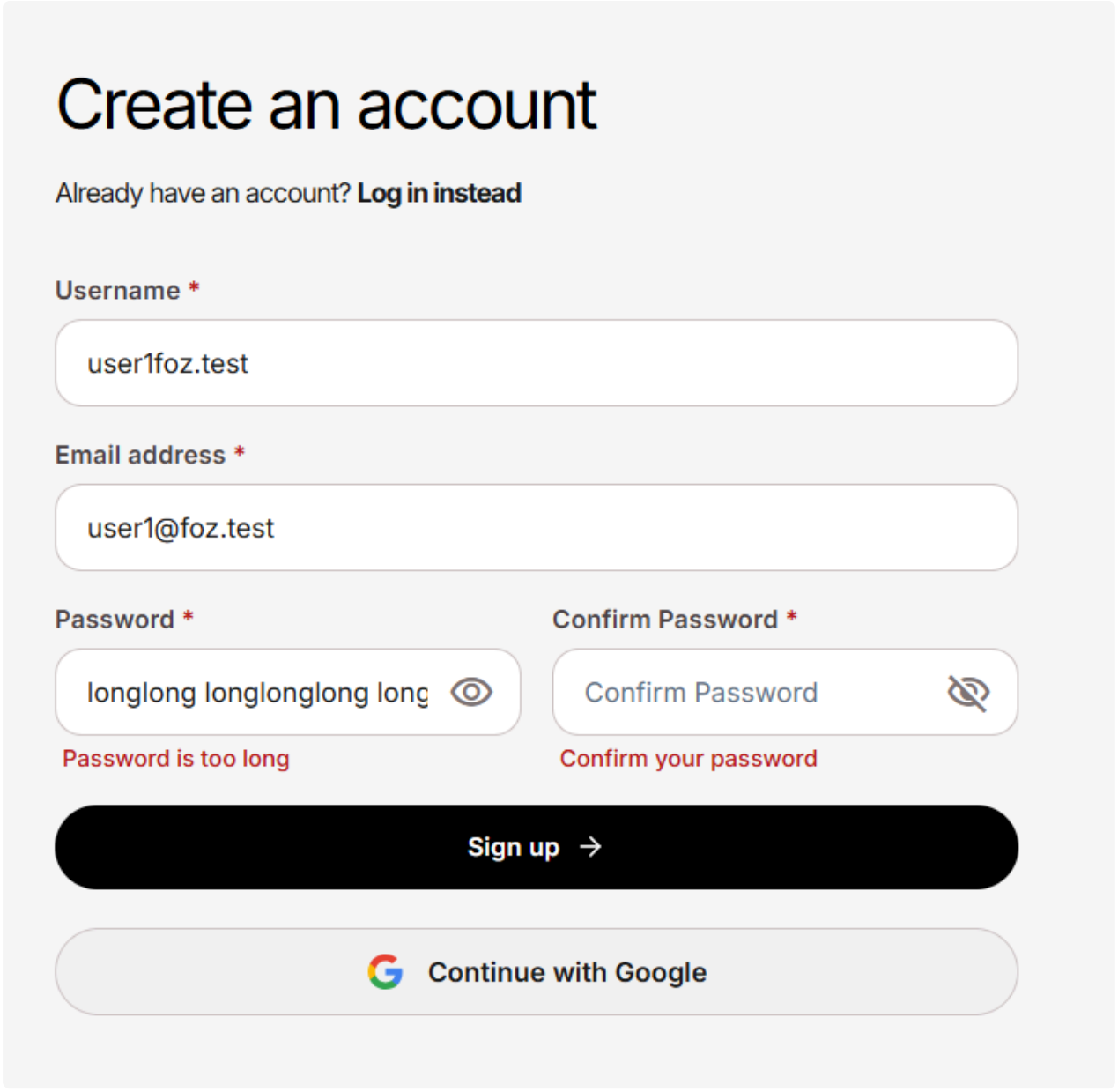
- Screen 2 (Alt Flow - Email Already Exists):



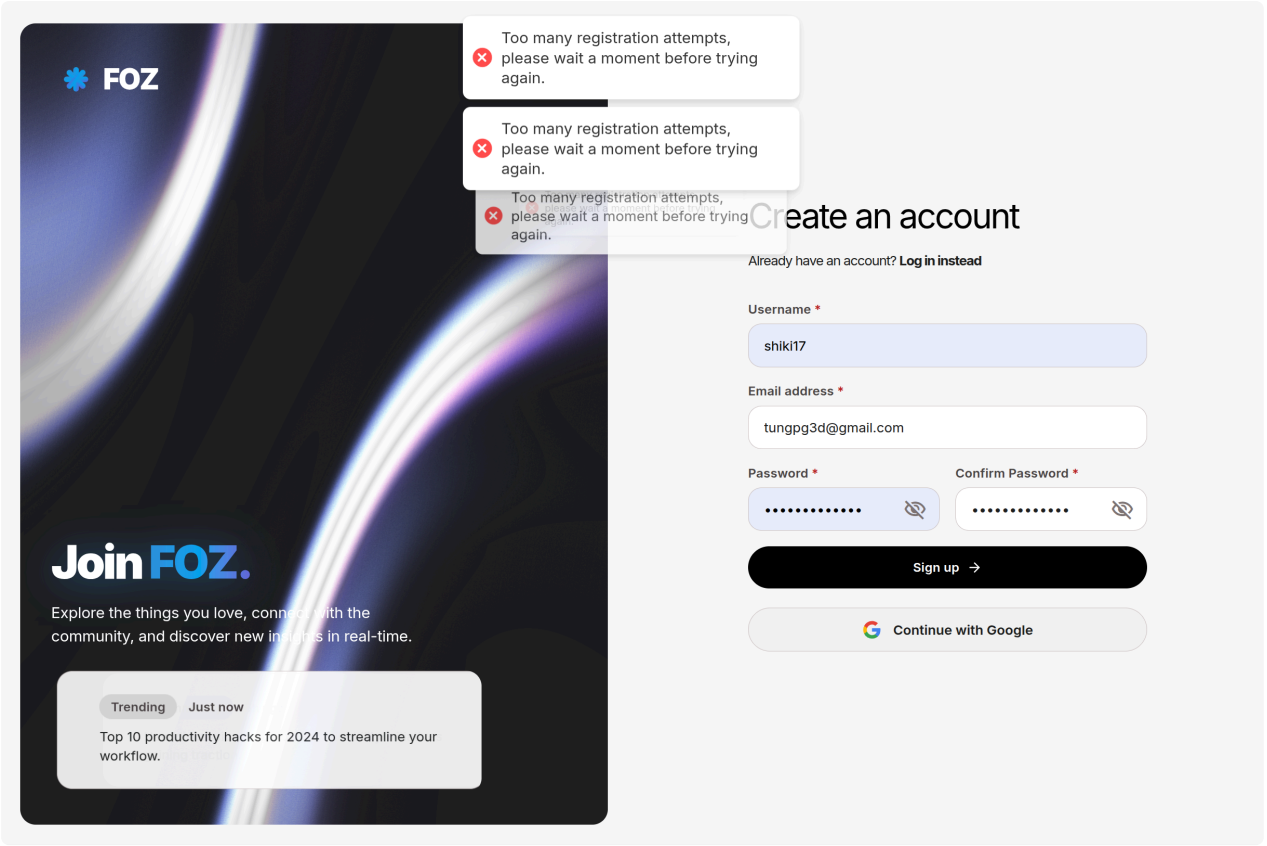
- Screen 3 (Alt Flow - Weak Password):



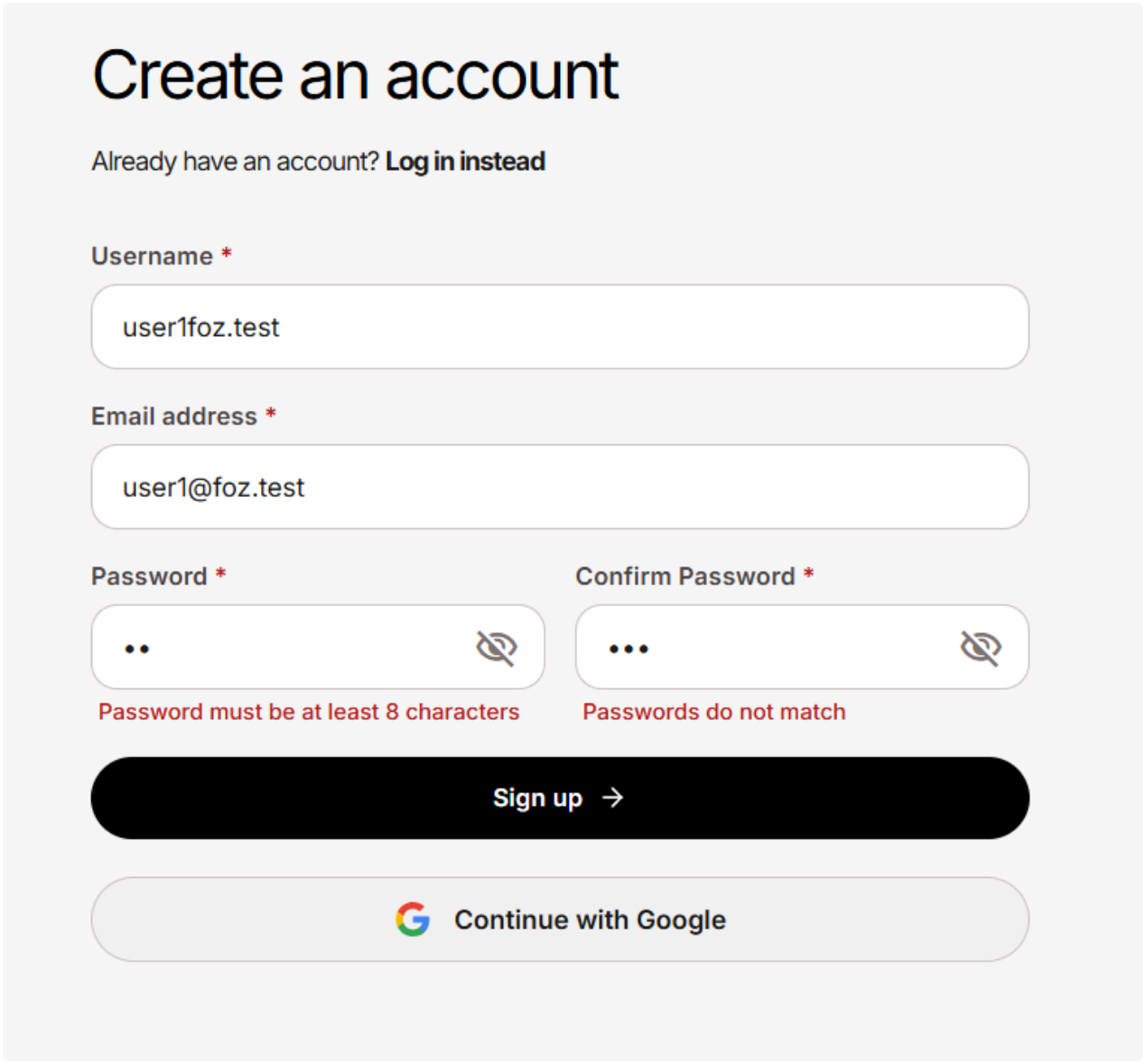
- Screen 4 (Alt Flow - Password Too Long):



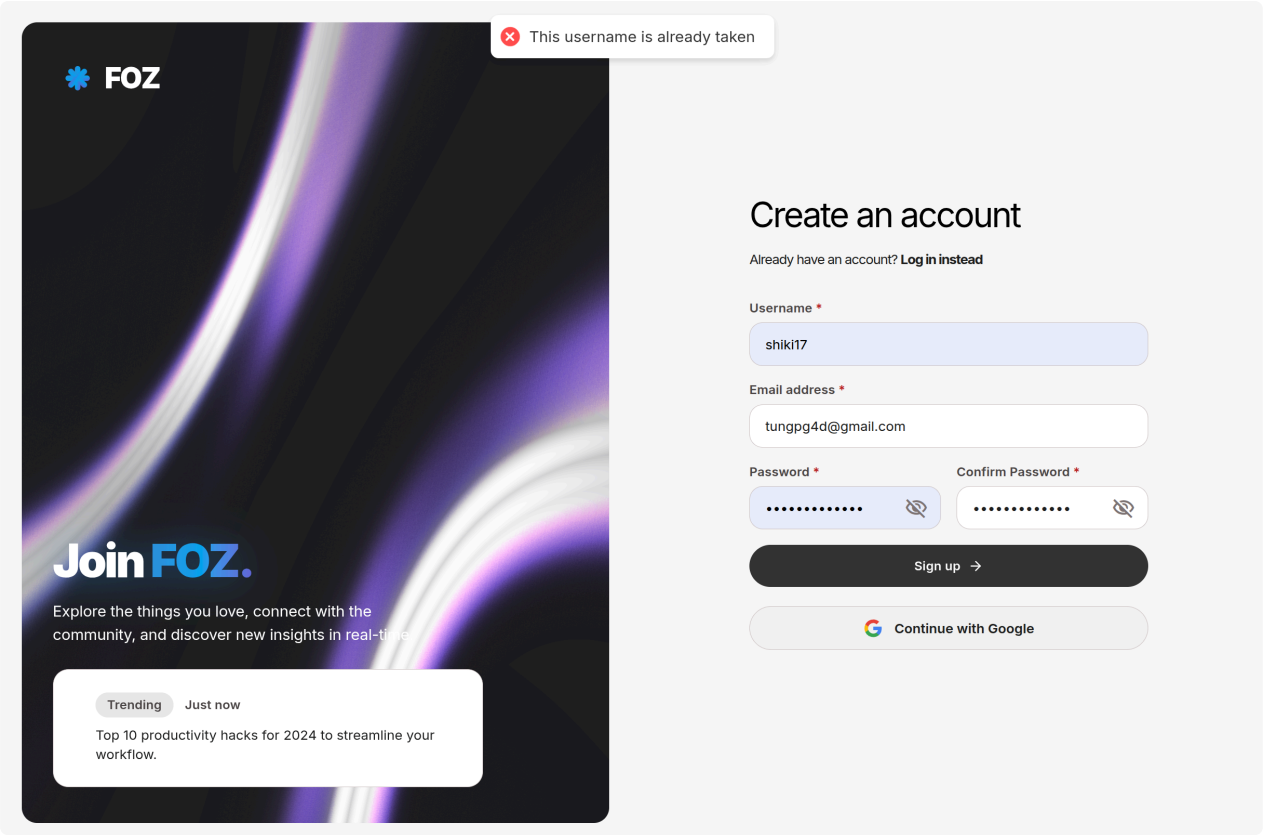
- **Screen 5 (Alt Flow - Rate Limit Exceeded):**



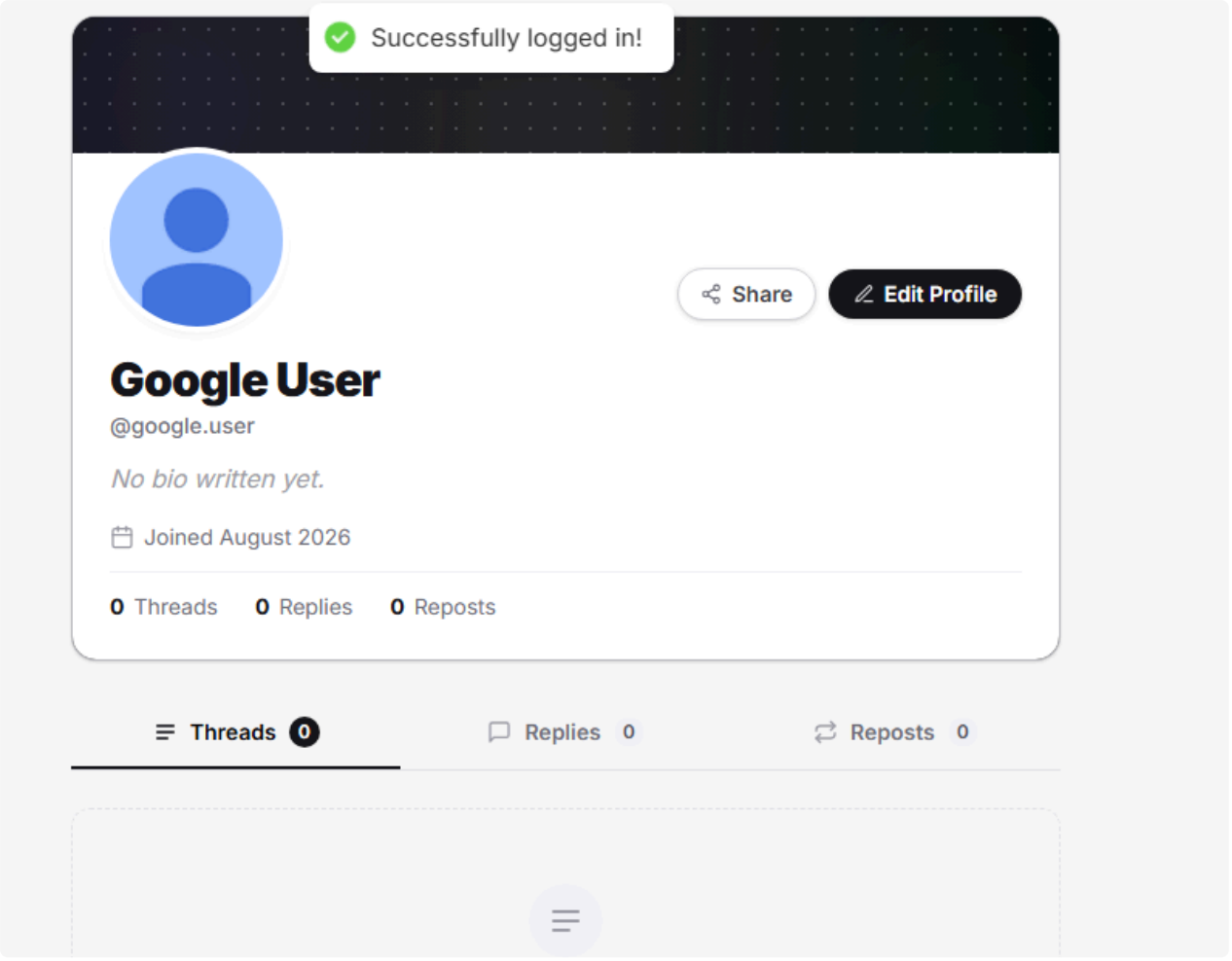
- **Screen 6 (Alt Flow - Confirm Password Does Not Match):**



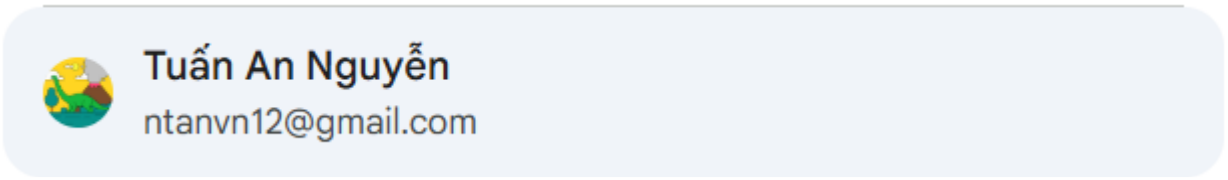
- Screen 7 (Alt Flow - Username Already Taken):



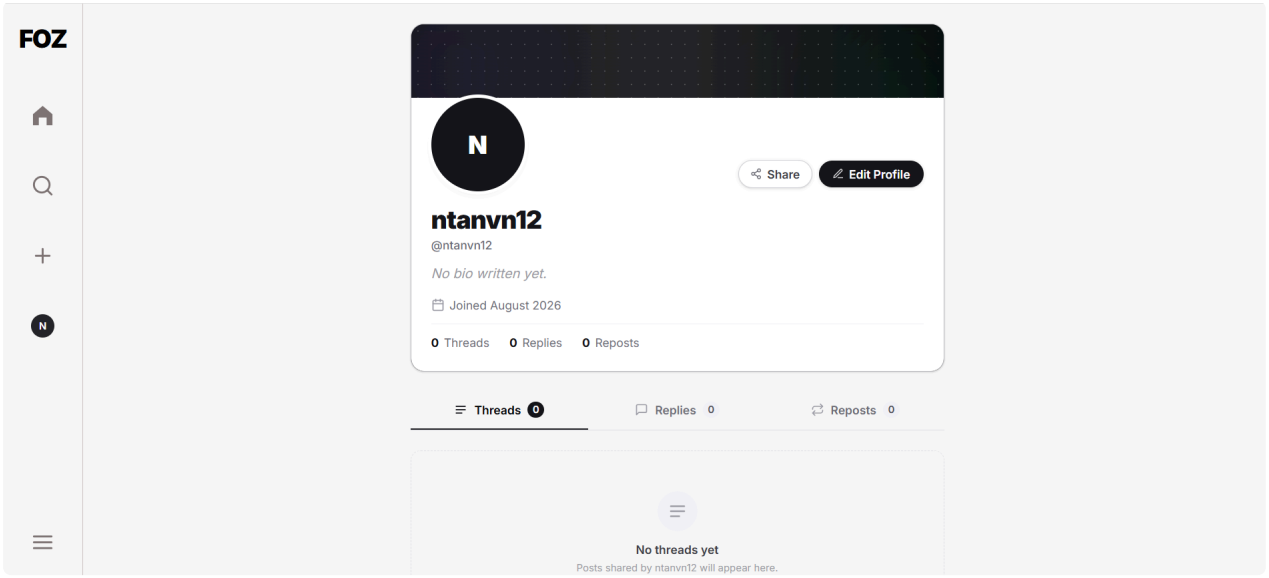
- Screen 8 (Alt Flow - Sign Up with Google Account):



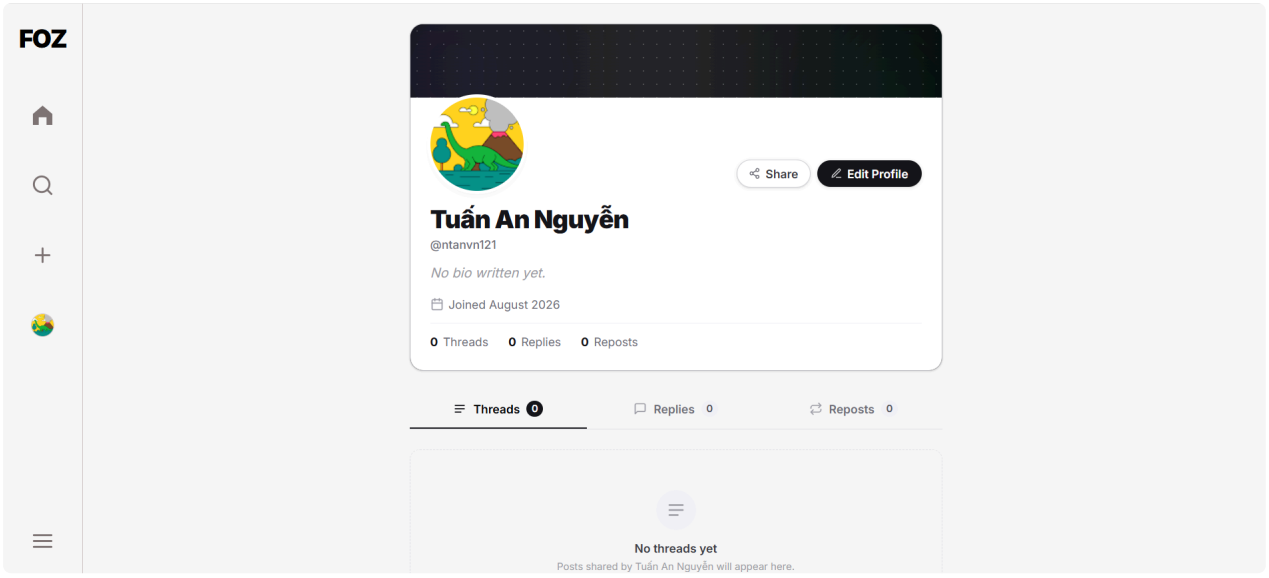
- **Screen 9 (Alt Flow - Google Sign Up Username Collision):**



This is the email to test the alternative flow.



The website already has a username that matches the Google account email prefix.



The system automatically generates a unique username by appending number "1" to the base name.

Use-Case Specification: Log In

Performed by: Nguyễn Hải Trọng | Reviewed by: Nguyễn Tuấn An | Edited by: Nguyễn Hải Trọng

Project Name: FOZ

Version: 1.2

Document identifier: FOZ-UCS-02

1. Log In

- **Use-Case ID:** UC-02
- **Actor(s):** Guest

1.1 Brief Description

This use case describes how a registered user accesses their account by providing valid credentials (email and password) or authenticating via Google OAuth.

2. Preconditions

- The user is unauthenticated (Guest).

3. Flow of Events

3.1 Basic Flow

1. The use case begins when the Guest navigates to the Login page (`/login`).
2. The user enters their registered email address and password, then clicks "Log in".
3. The system finds the user in `user_accounts` and verifies the password hash using bcrypt.
4. The system checks if the user account status is `active` .
5. The system issues a short-lived JWT Access Token and sets an http-only Refresh Token cookie (recording the session in `refresh_sessions`).
6. The system redirects the user based on their role: Administrators to `/admin` , Moderators to `/moderator` , and Standard Users to the Home Feed (`/`).

3.2 Alternative Flows

3.2.1 Invalid Credentials

1. If at step 3, the email is not found in the database or the password does not match.
2. The system rejects the request with HTTP 401 (`INVALID_CREDENTIALS`) and displays: "Invalid email or password".
3. The flow resumes at step 2.

3.2.2 Account Suspended or Banned

1. If at step 4, the account status is `suspended` or `banned` (not `active`).
2. The system rejects authentication with HTTP 403 (`ACCOUNT_NOT_ACTIVE`) and displays: "Your account is not currently available."
3. The use case ends.

3.2.3 Rate Limit Exceeded

1. If the user makes more than 15 login attempts within 1 minutes.

2. The system rejects the request with HTTP 429 (`RATE_LIMITED`) and displays: "Too many requests, slow down."
3. The use case ends.

3.2.4 Sign In with Google Account

1. If at step 2, the user chooses to "Sign in with Google".
2. The system initiates Google OAuth (popup or redirect to Google OAuth consent screen).
3. The user successfully authenticates with Google.
4. The system checks the database for an existing Google social link or matching email in `user_accounts` .
5. The system verifies the account is active, issues an Access Token, sets the Refresh Token cookie, and redirects the user based on role (or to onboarding if new).
6. The use case ends.

3.2.5 Log In with Non-Existent Google Account

1. If at step 4 of the Google Sign In flow, the system finds no existing user account associated with the Google profile or email.
2. The system automatically provisions a new user account on the fly (including auto-generating a unique username and saving profile details) instead of rejecting the login.
3. The login attempt is seamlessly converted into a successful registration, and the user is redirected to the onboarding profile setup page (`/onboarding/avatar`).
4. The use case ends.

4. Special Requirements

- **Rate Limiting:** Maximum 100 attempts per 15 minutes per IP to prevent brute-force attacks.
- **Security:** Passwords verified via timing-safe comparison; access tokens kept in memory; refresh tokens in http-only cookies.

5. Postconditions

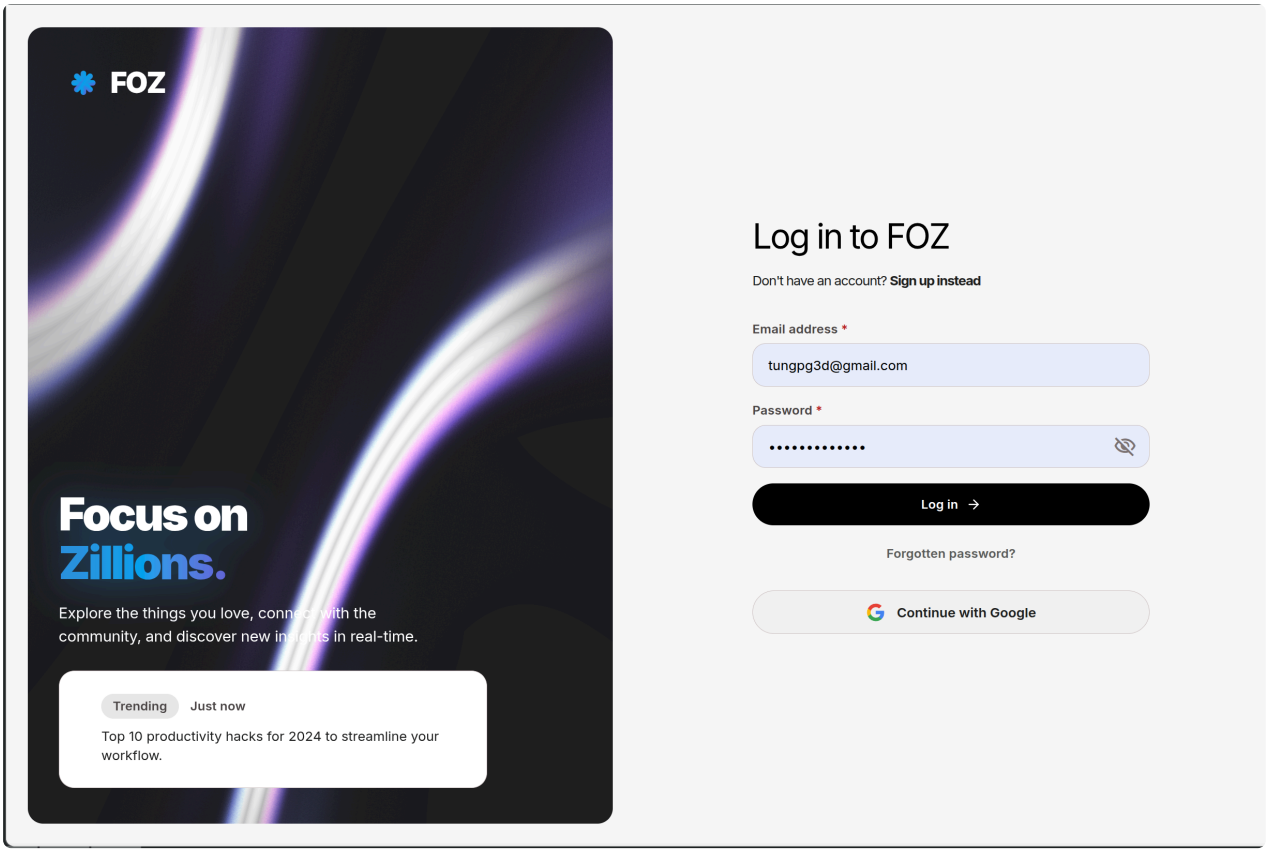
- The user is authenticated, an active session is recorded in `refresh_sessions` , and the user is navigated to the appropriate landing view.

6. Extension Points

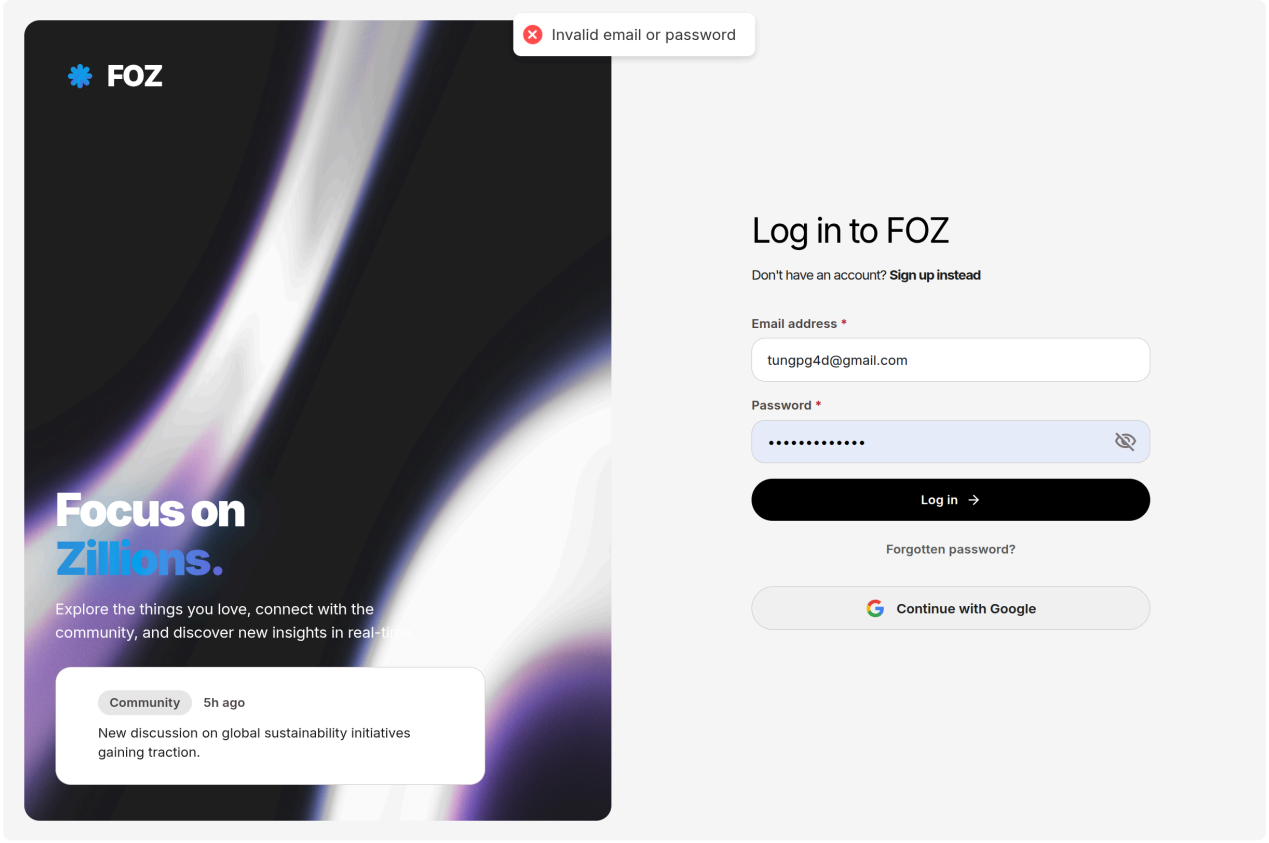
- None.

7. Prototype Requirement

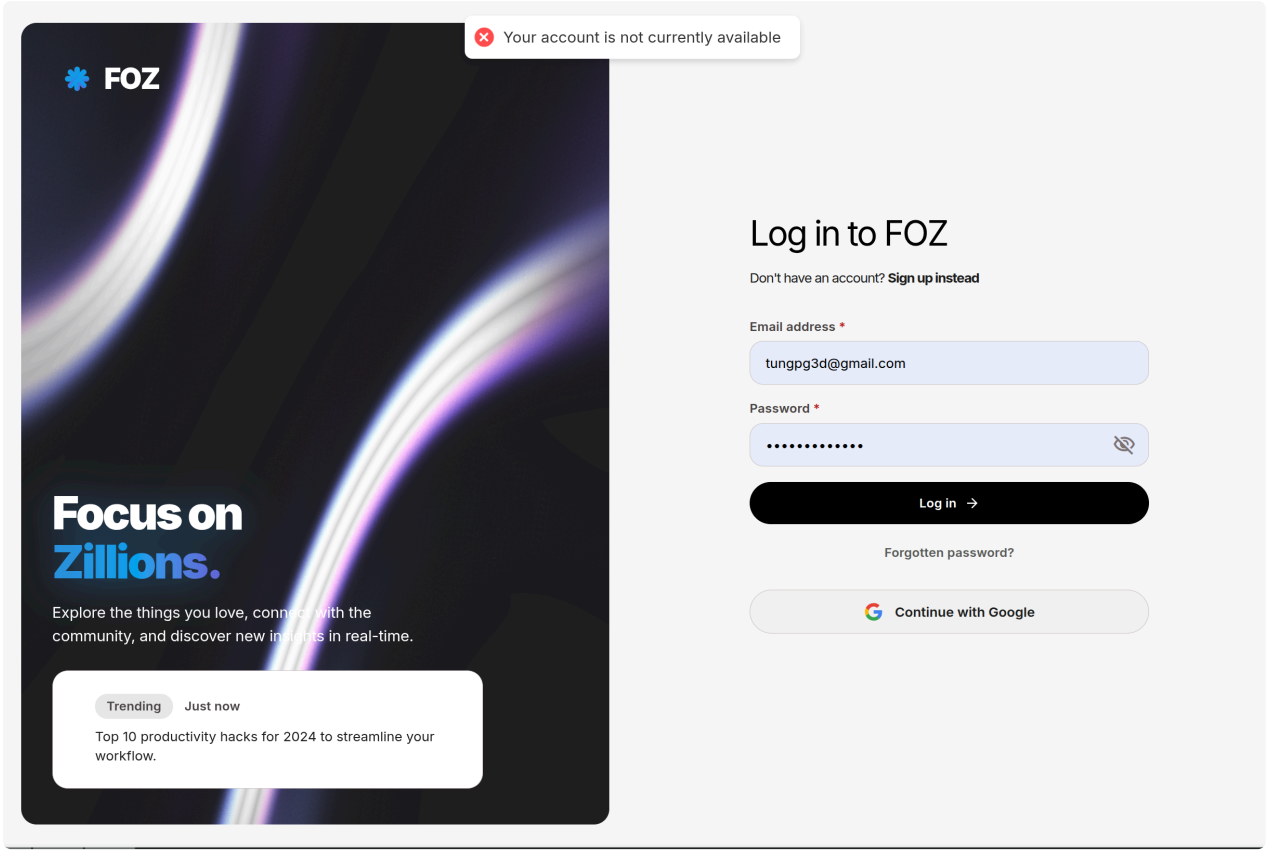
- Screen 1 (Basic Flow):



- Screen 2 (Alt Flow - Invalid Credentials):

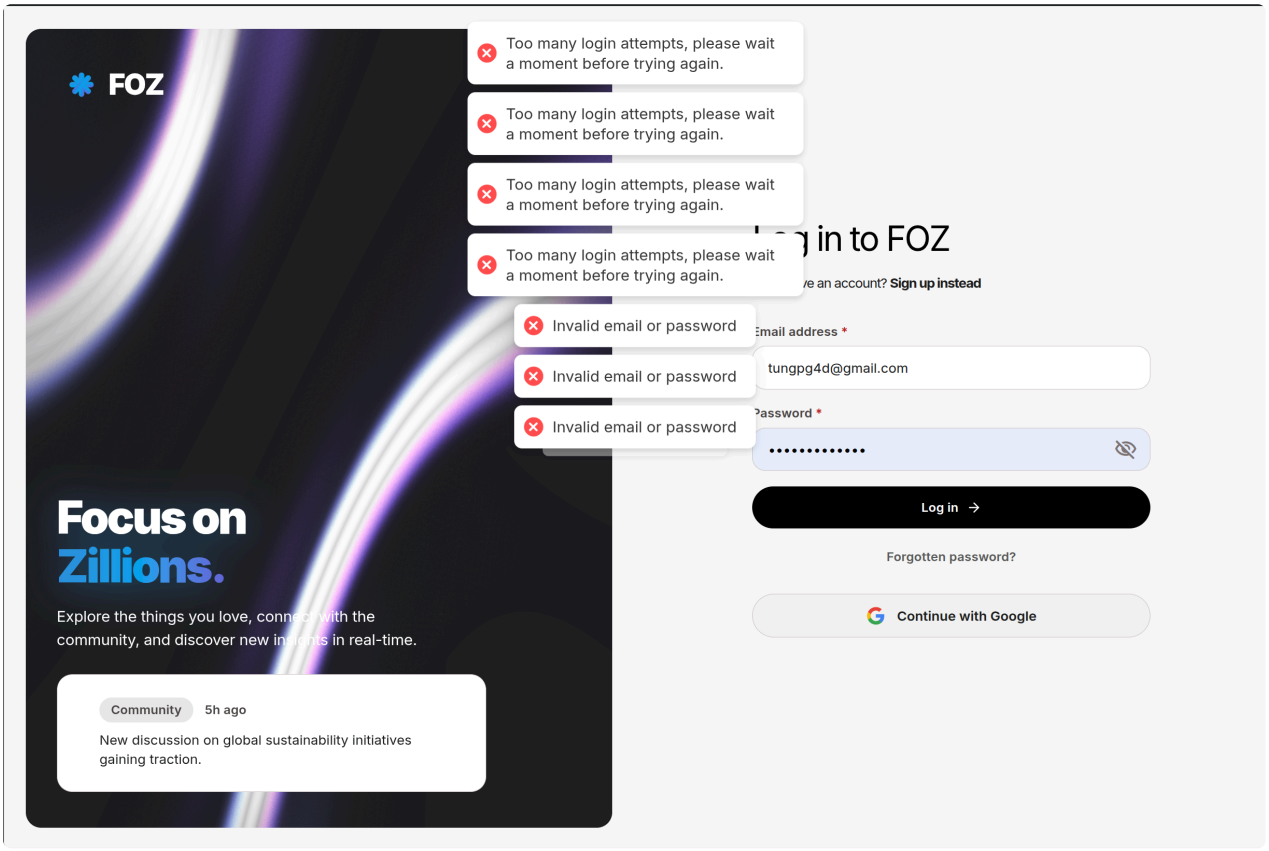


- Screen 3 (Alt Flow - Account Suspended):

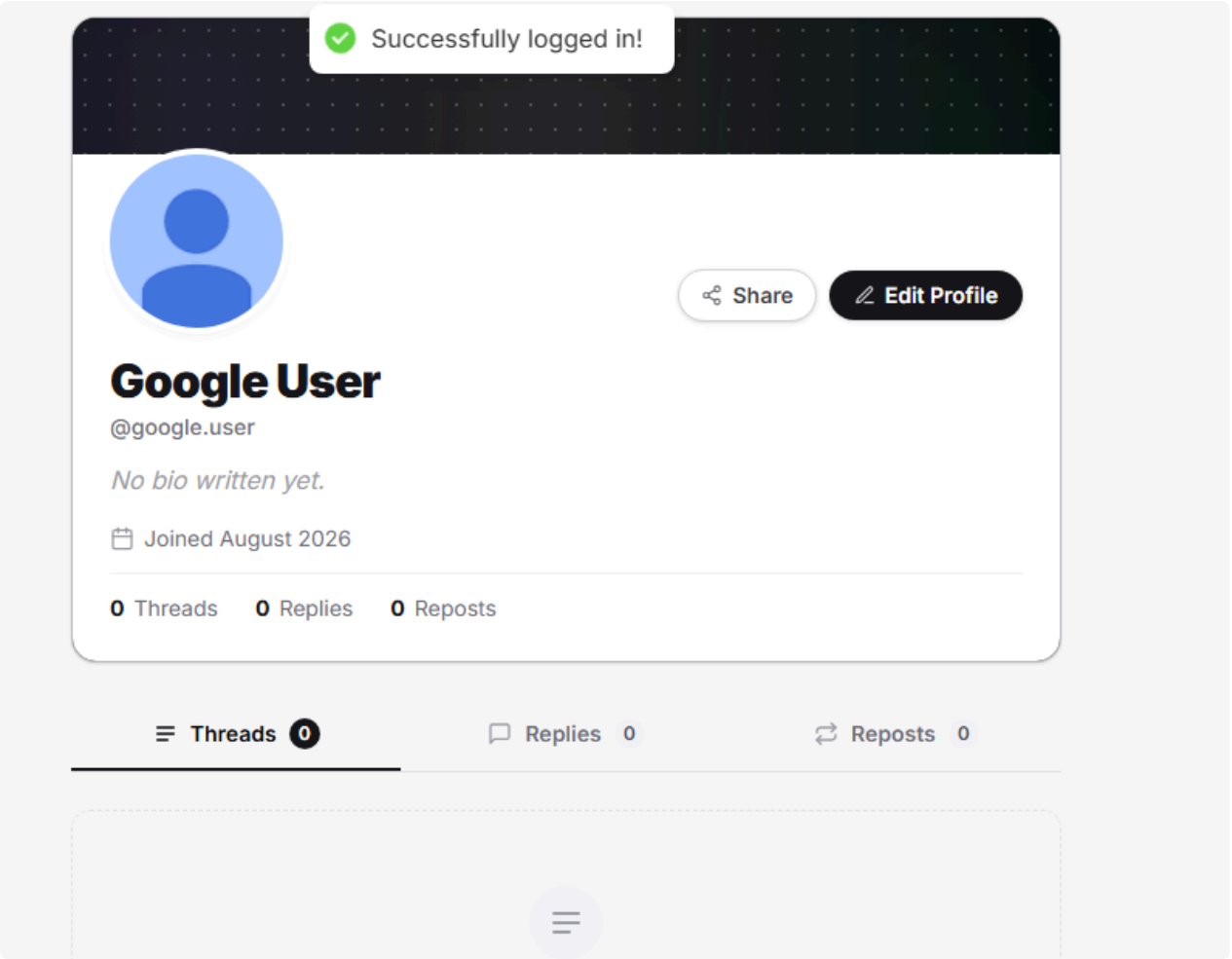


FLAG

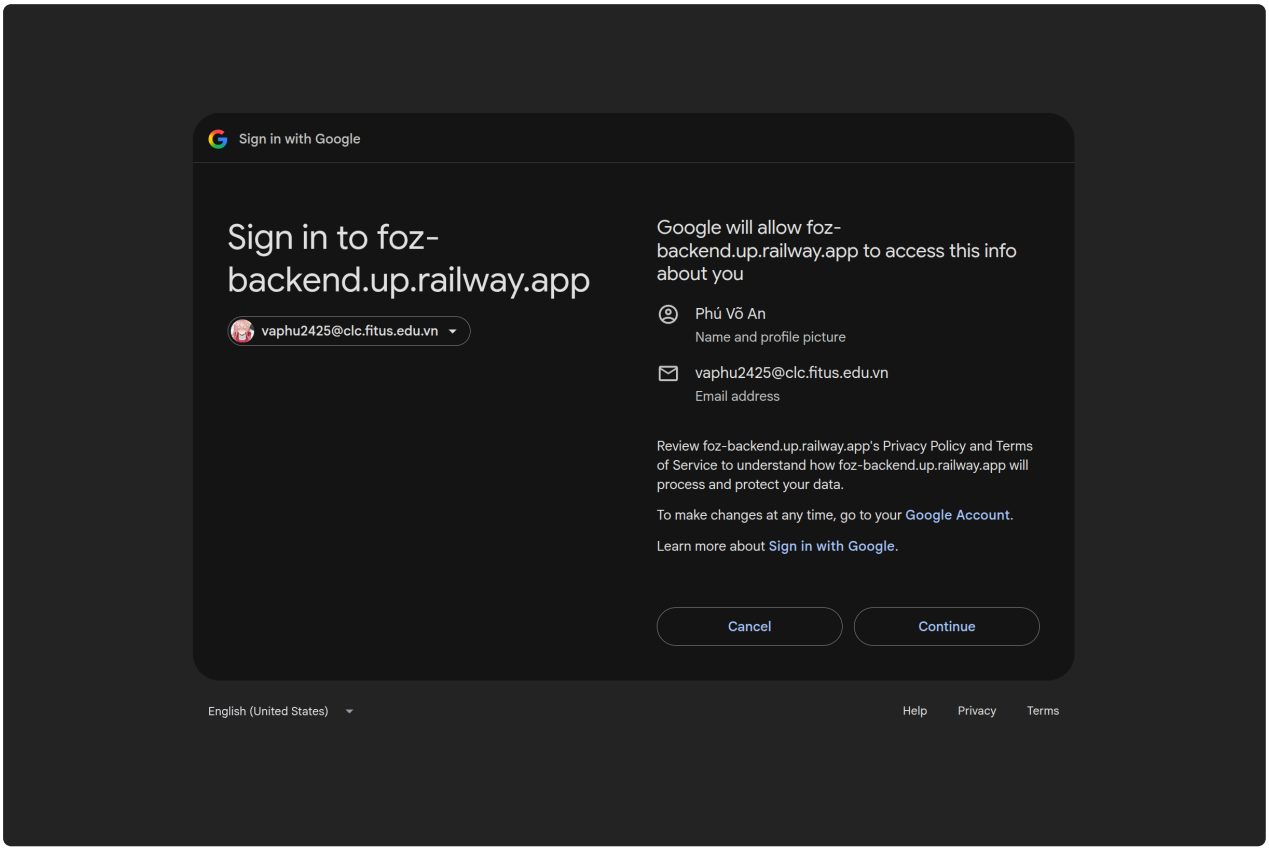
- Screen 4 (Alt Flow - Rate Limit Exceeded):



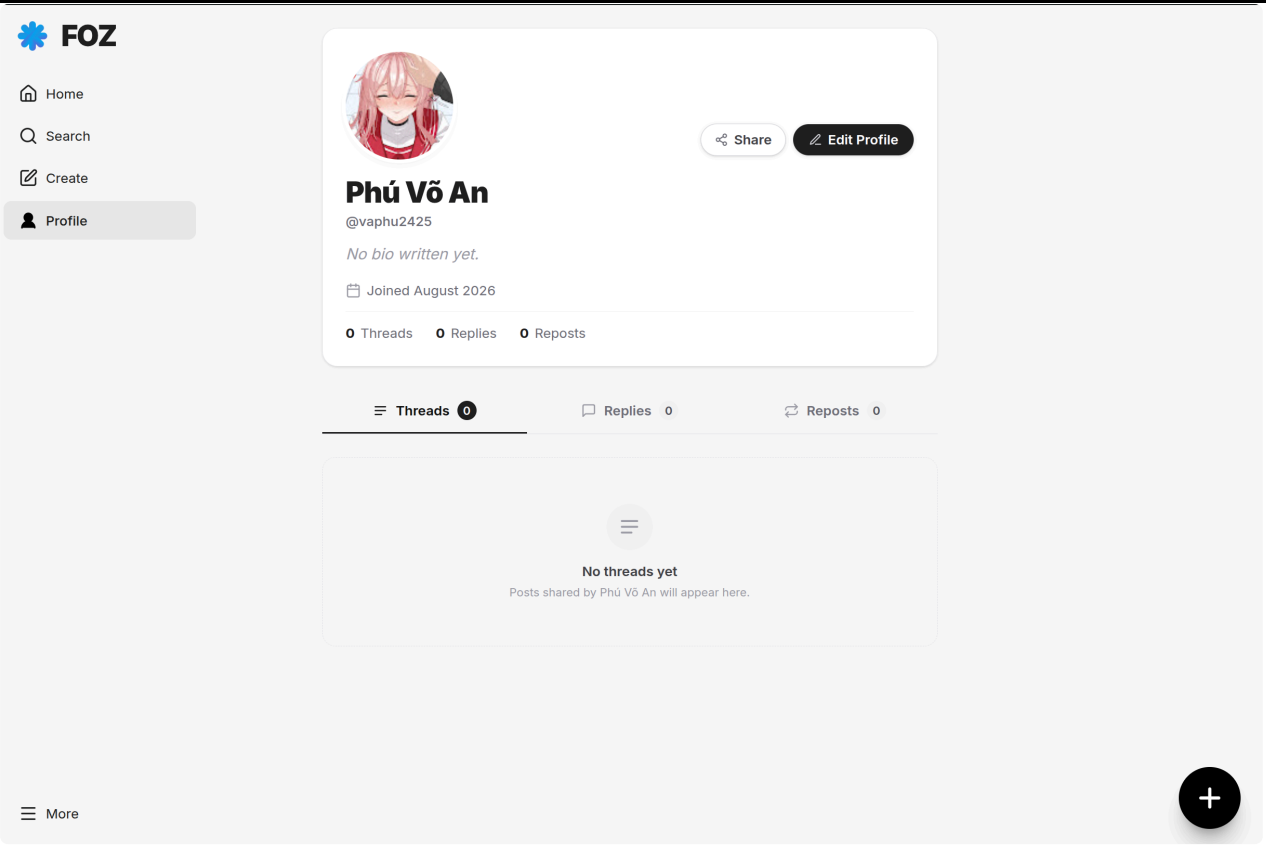
- **Screen 5 (Alt Flow - Sign In with Google Account):**



- **Screen 6 (Alt Flow - Log In with Non-Existent Google Account):**



This email hasn't been logged in to the website.



After login, the system automatically creates a new user account.

Use-Case Specification: Log Out

Performed by: Nguyễn Hải Trọng | Reviewed by: Nguyễn Tuấn An | Edited by: Nguyễn Hải Trọng
Project Name: FOZ
Version: 1.1
Document identifier: FOZ-UCS-03

1. Log Out

- **Use-Case ID:** UC-03
- **Actor(s):** Authenticated User (Standard User, Moderator, Administrator)

1.1 Brief Description

Allows an authenticated user to securely end their active session on the current device.

2. Preconditions

- The user must be authenticated.

3. Flow of Events

3.1 Basic Flow

1. The use case begins when the user clicks the "Log out" / "Sign out" button in the navigation or sidebar.
2. The system opens the Logout confirmation modal (`LogoutDialog`), displaying the user's avatar, account handle/email, and confirmation prompt ("Log out of your account?").
3. The user clicks "Log out" in the dialog.
4. The client sends a `POST /api/v1/auth/logout` request.
5. The backend validates the refresh cookie and updates the `revoked_at` timestamp in the `refresh_sessions` table to invalidate the session.
6. The backend clears the `refresh_token` http-only cookie from the response.
7. The client clears the JWT access token and user state from the local authentication store.
8. The modal closes and the user state updates to unauthenticated (Guest).

3.2 Alternative Flows

3.2.1 User Cancels Log Out

1. At step 2, the user clicks "Cancel", presses the Escape key, or clicks outside the modal.
2. The system closes the dialog and no action is taken.
3. The user remains authenticated and the use case ends.

3.2.2 Session Already Expired or Invalid

1. If at step 4, the refresh session token is already expired, revoked, or missing.
2. The backend clears the refresh cookie and responds successfully.
3. The client clears local authentication state and the use case ends.

4. Special Requirements

- **Security:** Immediate server-side session revocation in `refresh_sessions` and complete purging of client authentication credentials.

5. Postconditions

- The user's session is marked revoked in `refresh_sessions` , authentication cookies/tokens are cleared, and the client returns to an unauthenticated state.

6. Extension Points

- None.

7. Prototype Requirement

- **Screen 1 (Basic Flow):**

FOZ

Home

Search

Create

Profile

Phú Võ An

@vaphu2425

No bio written yet.

Joined August 2026

0 Threads **0** Replies

Threads 0

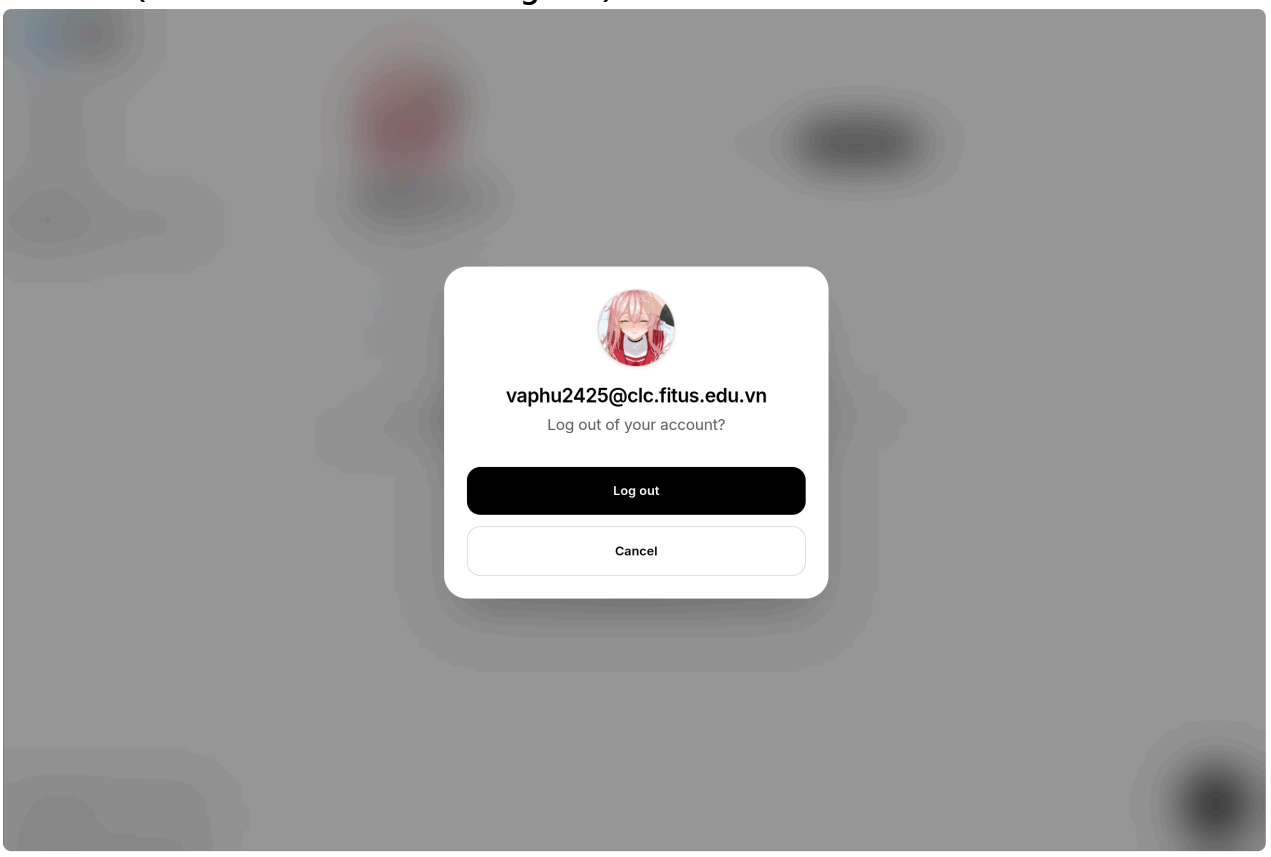
Settings

Feedback

Log out

More

- **Screen 2 (Alt Flow - User Cancels Log Out):**



Use-Case Specification: Manage Profile

Performed by: Nguyễn Hải Trọng | Reviewed by: Nguyễn Tuấn An | Edited by: Nguyễn Hải Trọng
Project Name: FOZ
Version: 1.2
Document identifier: FOZ-UCS-04

1. Manage Profile

- **Use-Case ID:** UC-04
- **Actor(s):** Standard User

1.1 Brief Description

Allows an authenticated user to view and update their public profile details, including display name, unique username handle, bio, location, website URL, and avatar image.

2. Preconditions

- The user must be authenticated and viewing their own profile page (`/profiles/@username` or `/profile`).

3. Flow of Events

3.1 Basic Flow

1. The use case begins when the user navigates to their profile page and clicks the "Edit profile" button.
2. The system opens the `ProfileEditModal`, pre-populating fields (Avatar, Username handle, Display Name, Bio, Location, Website) with current data from the `profiles` and `user_accounts` tables.
3. The user modifies text fields (Display Name, Bio, Location, Website, or Username) and clicks "Save".
4. The system validates the inputs (Display Name 1–50 characters and valid characters, Bio at most 160 characters, Location at most 100 characters, Website at most 200 characters with valid URL format, Username 3–30 characters and valid format).
5. The client submits a `PATCH /api/v1/profile/me` request.
6. The backend updates the `profiles` table (and `user_accounts` if username was changed) and updates the `updated_at` timestamp.
7. The system displays a success toast ("Profile updated successfully"), closes the modal, and refreshes the profile view with updated details.

3.2 Alternative Flows

3.2.1 Invalid Input Format

1. If at step 4, the website URL is invalid (e.g. missing domain dot or bad protocol) or Display Name contains disallowed special characters.
2. The system displays field validation error messages below the respective input fields.
3. The flow resumes at step 3.

3.2.2 Input Exceeds Maximum Limits

1. If at step 4, any text input exceeds its maximum character limit (Display Name: 50, Bio: 160, Location: 100, Website: 200).
2. The system displays specific field validation errors indicating character limits.
3. The flow resumes at step 3.

3.2.3 Username Handle Collision on Update

1. If at step 3, the user modifies their username handle to one that already exists for another account.
2. The system displays a collision confirmation dialog: "The username has already existed. The system will generate a unique name by adding a number. Do you want to proceed? [Yes] [No]".
3. If the user clicks "Yes", the system re-submits the request with auto-generation enabled, appends an incremental numeric suffix (e.g., `username1`), updates the username, and saves the profile.

4. If the user clicks "No", the confirmation dialog closes and the user can manually change the username handle.

3.2.4 User Cancels Profile Edit

1. If at step 3, the user clicks "Cancel", clicks the close icon, or clicks outside the modal.
2. The system discards all pending changes and closes the modal without modifying database records.
3. The use case ends.

3.2.5 Update Avatar Image

1. At step 2, the user clicks on the avatar preview to select an image file.
2. The client validates that the file is a supported image format (PNG, JPEG, GIF, WebP) and does not exceed 2MB.
3. The client compresses the image to WebP and uploads it to Cloudinary (or encodes as base64 fallback), updating the avatar via `POST /api/v1/profile/me/avatar`.
4. The system updates the avatar preview and persists the new `avatar_url`.
5. The flow resumes at step 3.

4. Special Requirements

- **Performance:** Avatar images are compressed on the client before upload to minimize bandwidth and storage.
- **Cleanup:** When an avatar hosted on Cloudinary is replaced, the previous asset is asynchronously destroyed.

5. Postconditions

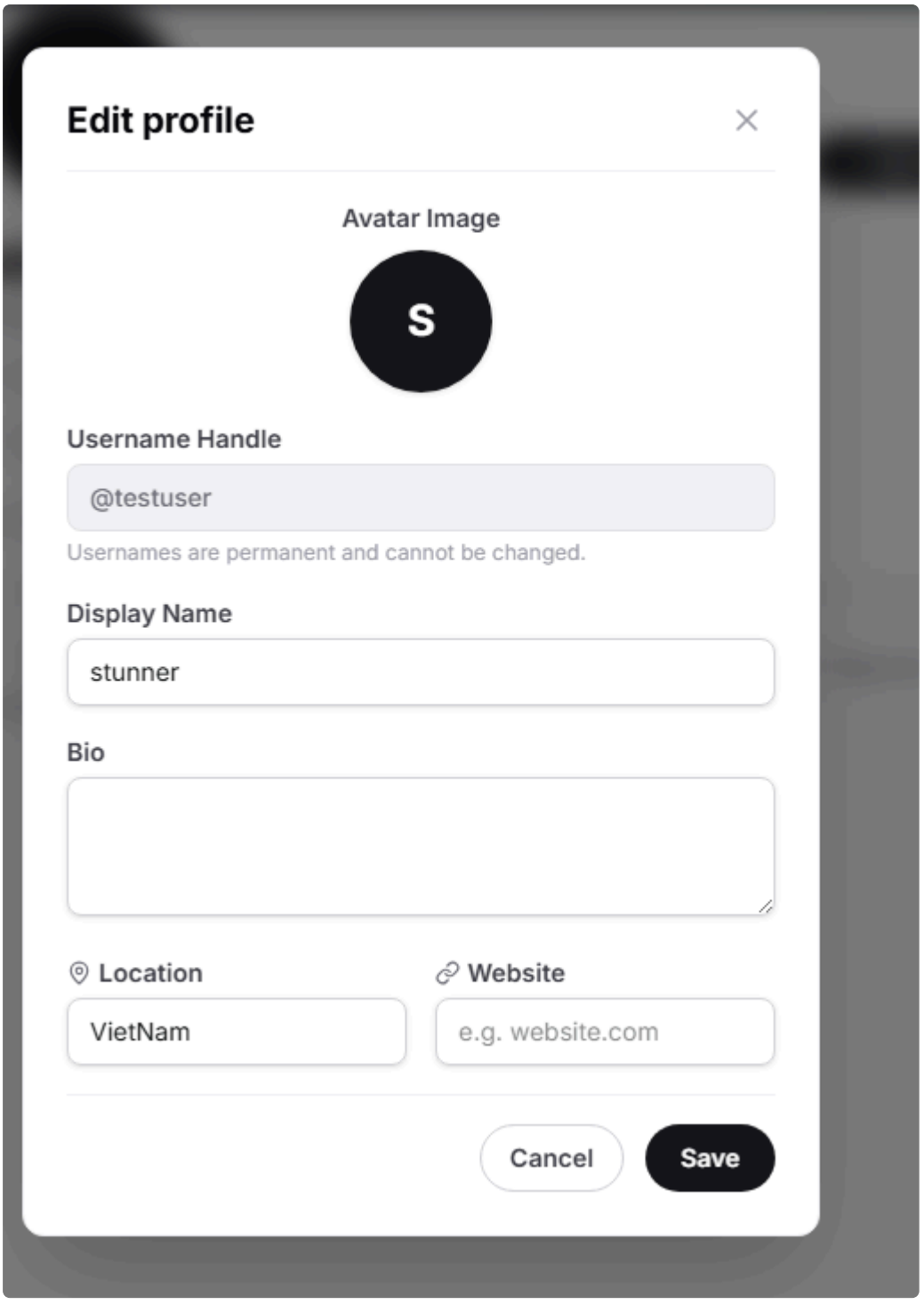
- The user's profile information (and username if changed) is updated in the database and visible publicly.

6. Extension Points

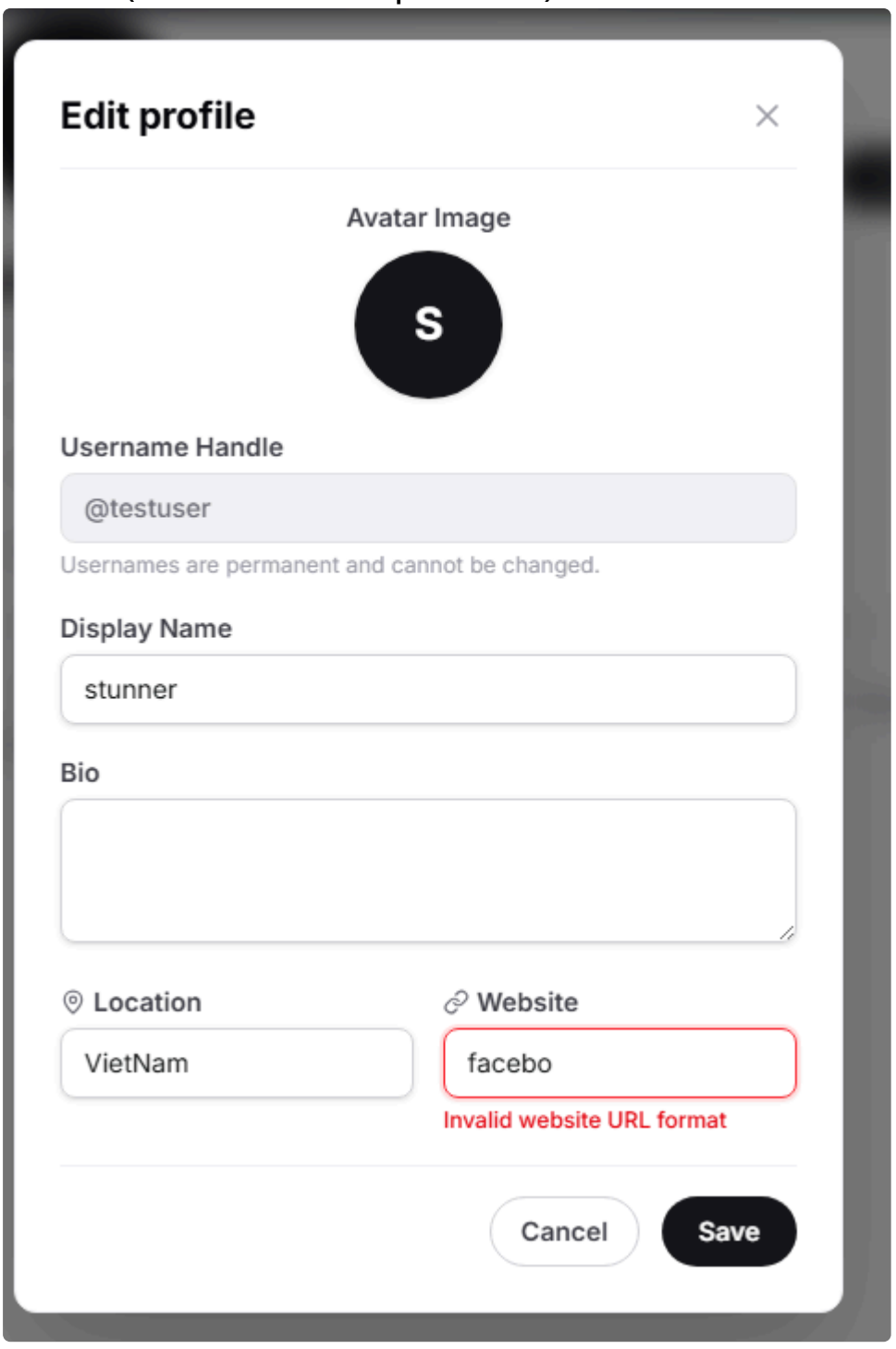
- **Change Password:** Extended via UC-05.
- **Change Avatar:** Extended via UC-06.

7. Prototype Requirement

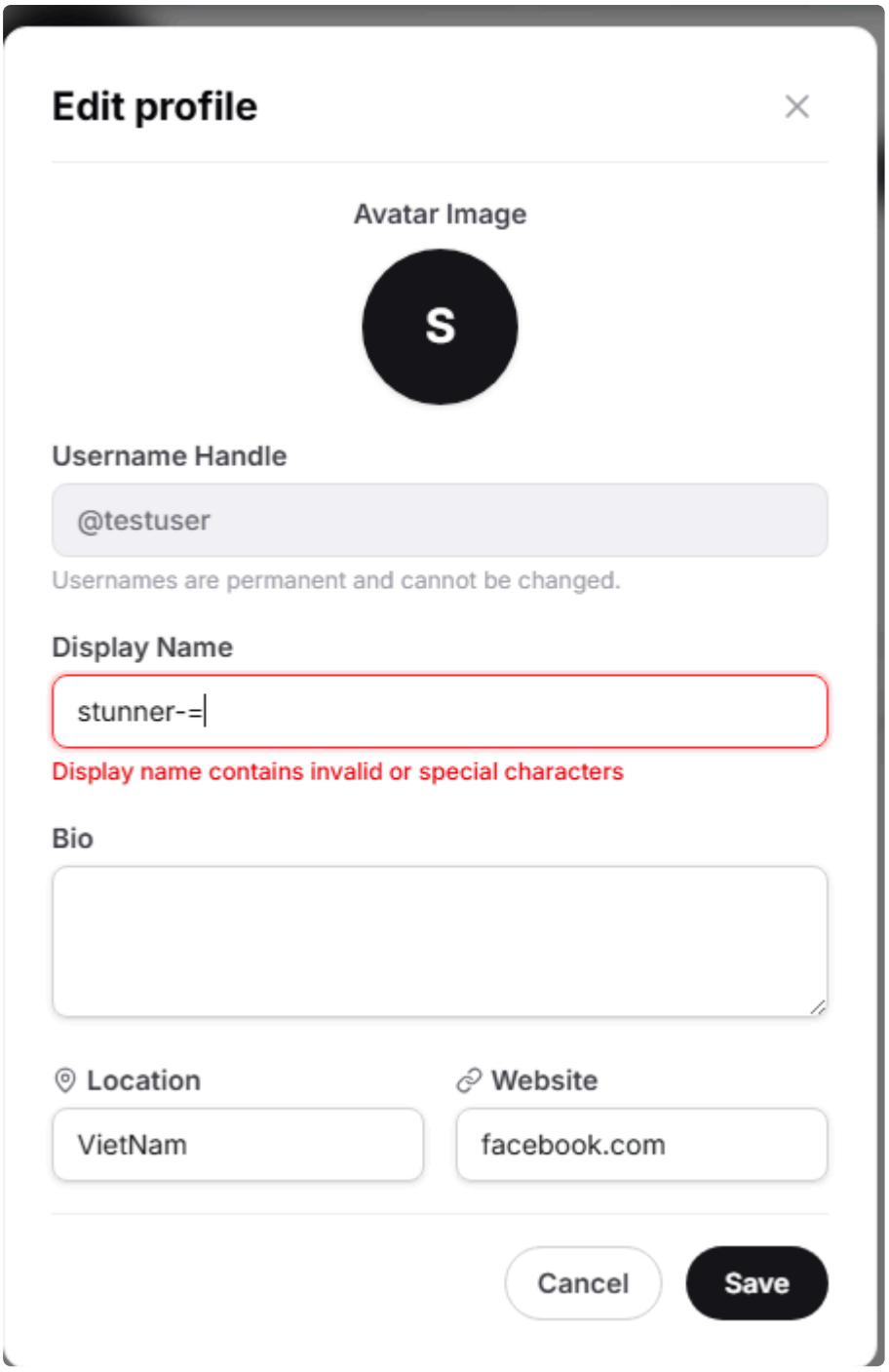
- **Screen 1 (Basic Flow):**



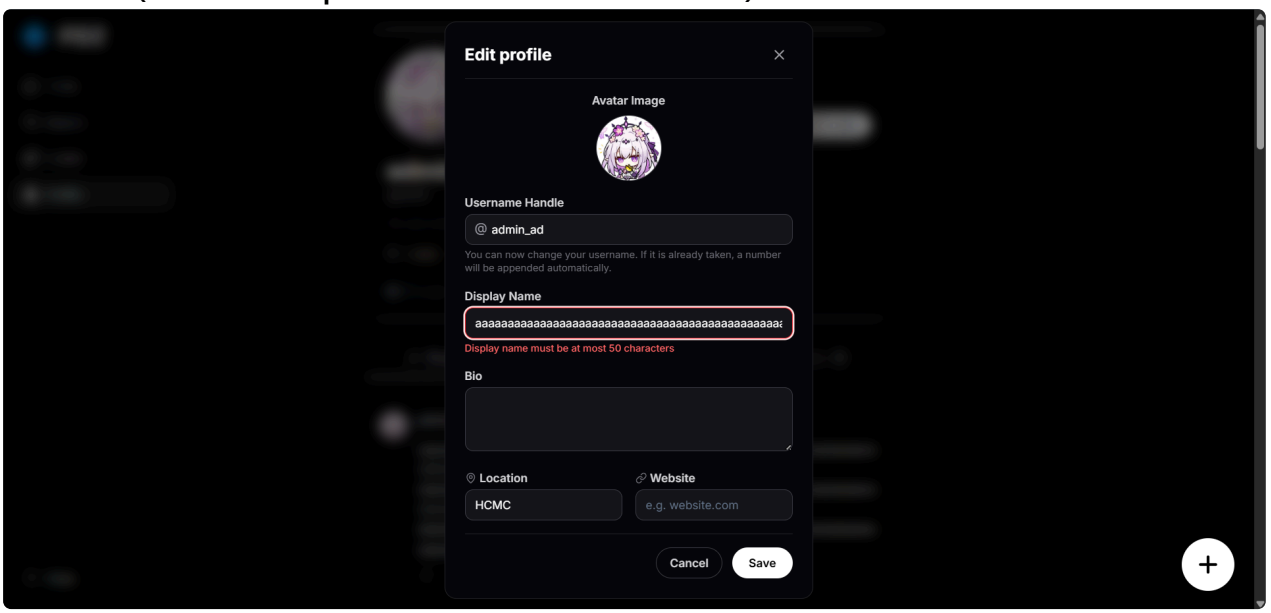
- **Screen 2 (Alt Flow - Invalid Input Format):**



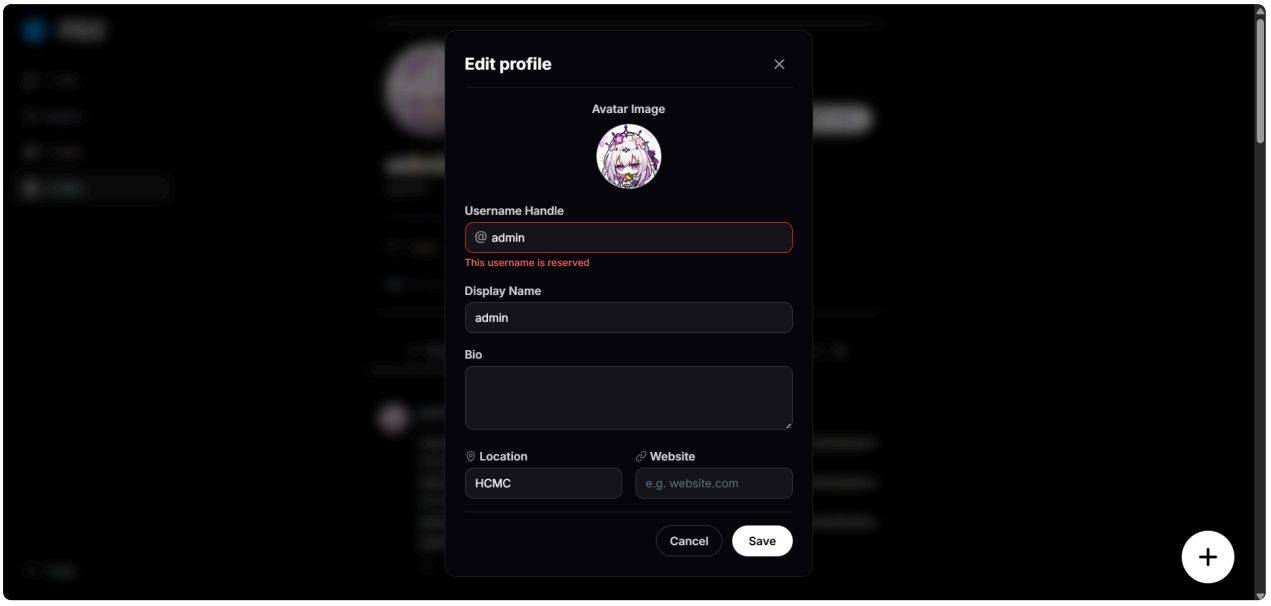
- **Screen 3 (Alt Flow - Validation Errors):**



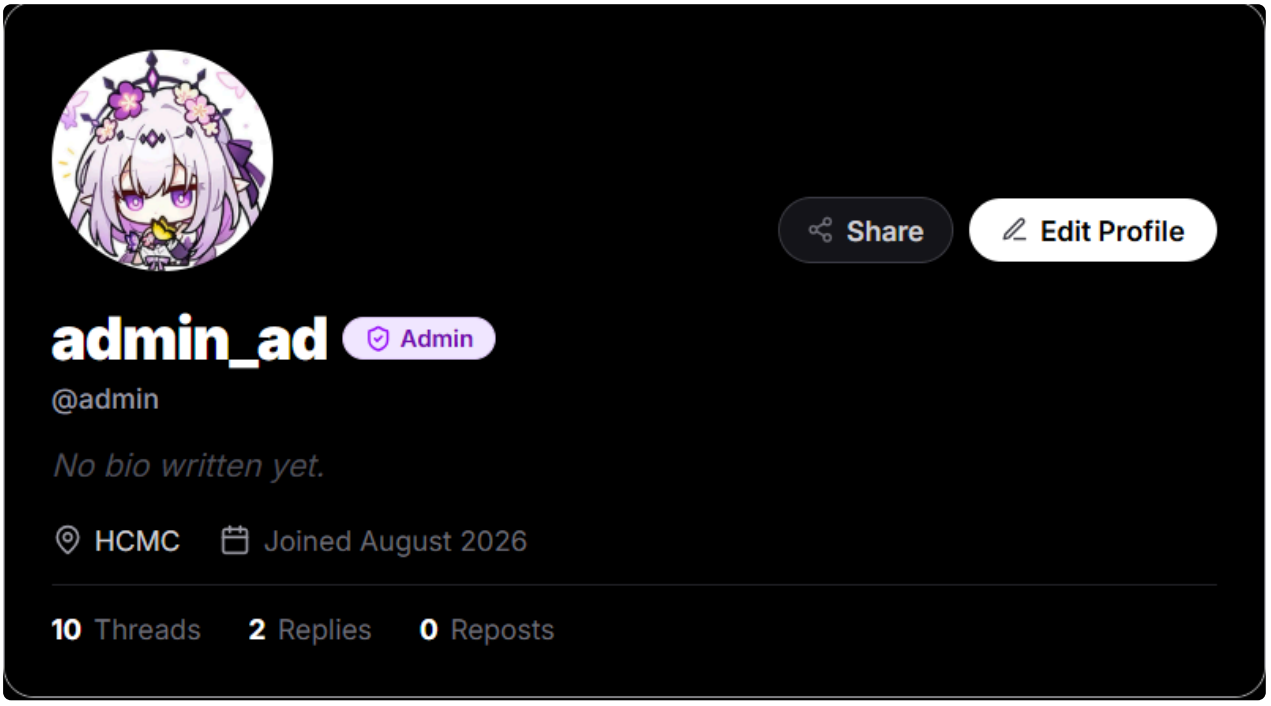
- Screen 4 (Alt Flow - Input Exceeds Maximum Limits):



- Screen 5 (Alt Flow - Username Handle Collision on Update):



- Screen 6 (Alt Flow - User Cancels Profile Edit):



Use-Case Specification: Change Password

Performed by: Nguyễn Hải Trọng | Reviewed by: Nguyễn Tuấn An | Edited by: Nguyễn Hải Trọng
Project Name: FOZ
Version: 1.2
Document identifier: FOZ-UCS-05

1. Change Password

- **Use-Case ID:** UC-05
- **Actor(s):** Standard User

1.1 Brief Description

Allows an authenticated user to change their password securely by verifying their current password and providing a new password meeting NIST security standards.

2. Preconditions

- The user must be authenticated.

3. Flow of Events

3.1 Basic Flow

1. The use case begins when the user navigates to the Change Password interface (accessible via `/settings` under Account Settings or `/change-password`).
2. The user enters their current password, a new password, and confirms the new password.
3. The user clicks "Update password".
4. The system verifies the current password against the stored `password_hash` in `user_accounts` using bcrypt.
5. The system validates the new password (length 8–128 characters, differs from current password, and matches confirmation password).
6. The system hashes the new password with bcrypt (cost factor 12) and updates `user_accounts.password_hash` and `updated_at`.
7. The system displays a success toast message ("Password updated"), resets the input fields, and redirects to the Home page (if on the standalone page).

3.2 Alternative Flows

3.2.1 Incorrect Current Password

1. If at step 4, the current password entered does not match the stored hash.
2. The system rejects the request with HTTP 401 (`INCORRECT_CURRENT_PASSWORD`).

3. The system highlights the current password field with "Incorrect current password" and displays an error toast.
4. The flow resumes at step 2.

3.2.2 New Password Matches Old Password

1. If at step 5, the new password is identical to the current password.
2. The system rejects the request and displays: "New password must differ from the current one".
3. The flow resumes at step 2.

3.2.3 Confirm Password Does Not Match

1. If at step 5, the confirmation password does not match the new password.
2. The system highlights the confirmation field and displays: "Passwords do not match".
3. The flow resumes at step 2.

3.2.4 Weak or Too Long New Password

1. If at step 5, the new password is less than 8 characters or exceeds 128 characters.
2. The system rejects the request and displays the appropriate error (e.g., "Password must be at least 8 characters" or "Password is too long").
3. The flow resumes at step 2.

3.2.5 User Cancels

1. If at step 2, the user clicks "Back" or "Back to home" instead of "Update password".
2. The system discards all inputs and returns to the previous screen.
3. The use case ends.

3.2.6 Rate Limit Exceeded

1. If the user makes more than 100 password change requests within 15 minutes.
2. The system rejects the request with HTTP 429 (`RATE_LIMITED`) and displays: "Too many requests, slow down."
3. The use case ends.

4. Special Requirements

- **Security:** Passwords must be hashed using bcrypt (cost 12); current password must be verified before changes are applied; new password must not be identical to the old password.

5. Postconditions

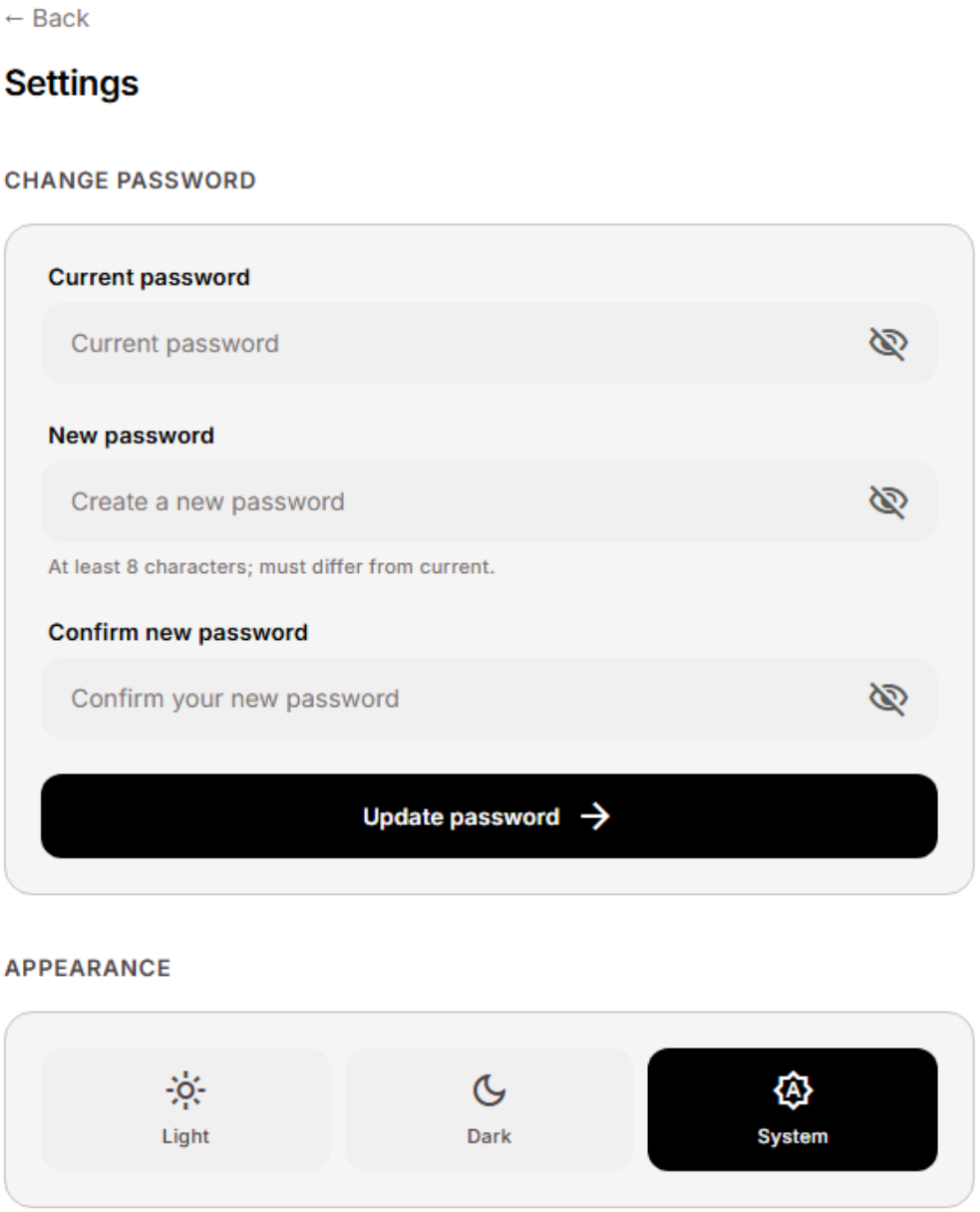
- The user's `password_hash` in `user_accounts` is updated with the new hash.

6. Extension Points

- None.

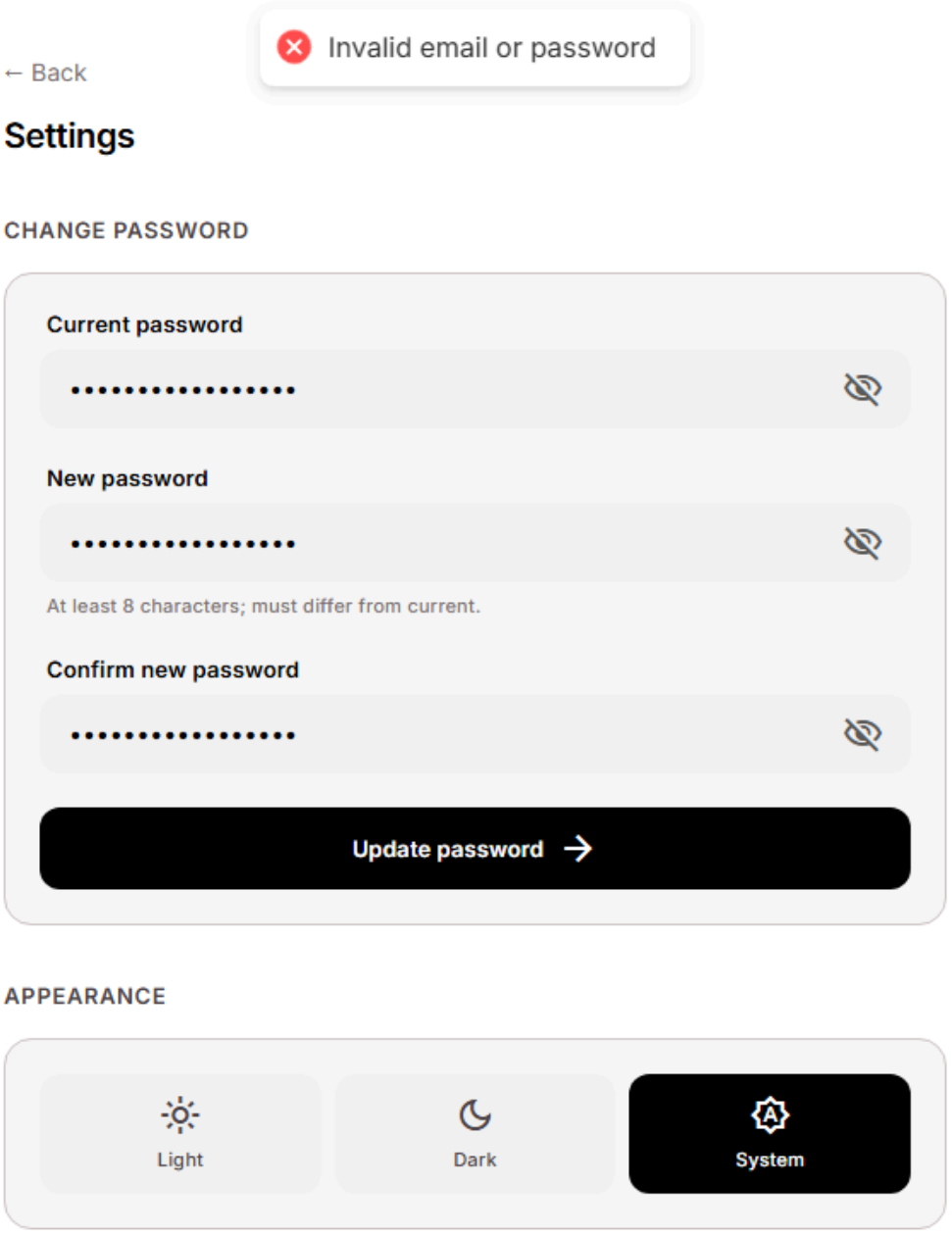
7. Prototype Requirement

- **Screen 1 (Basic Flow):**

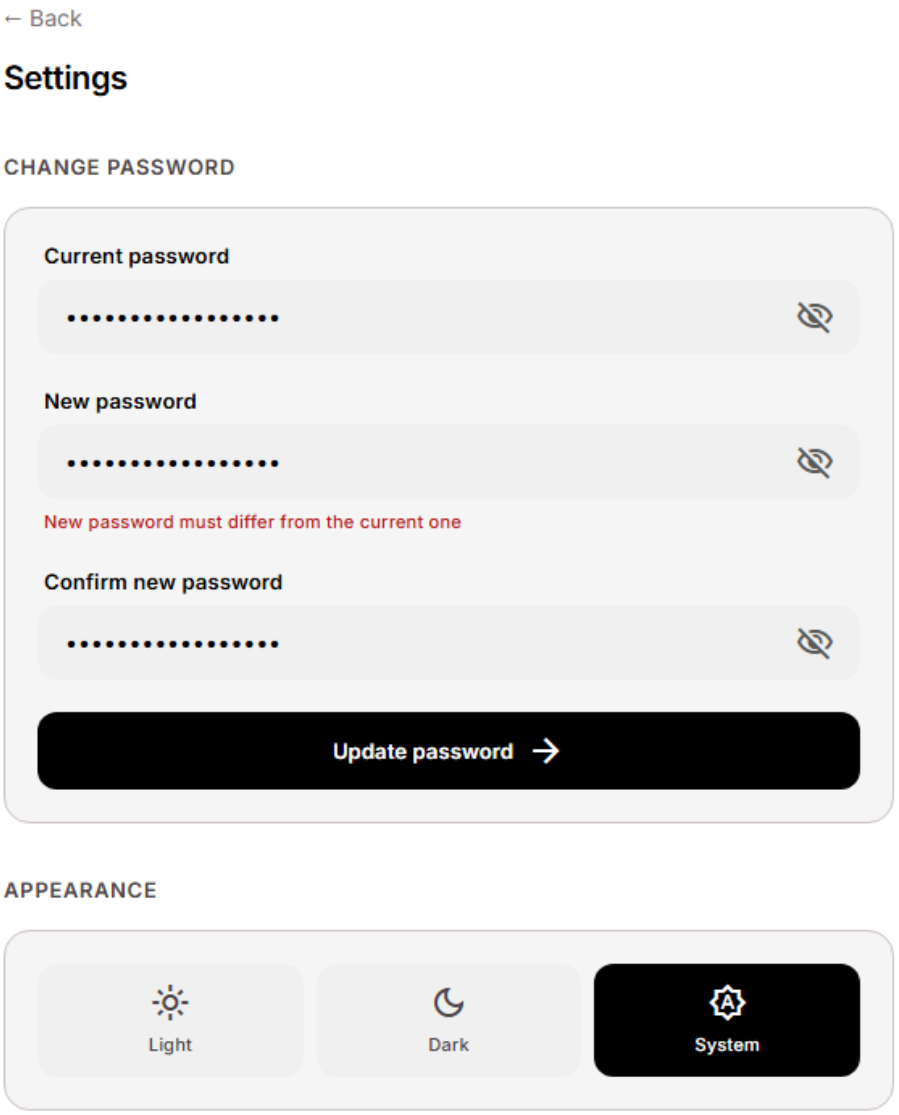


Change Password will be in Settings

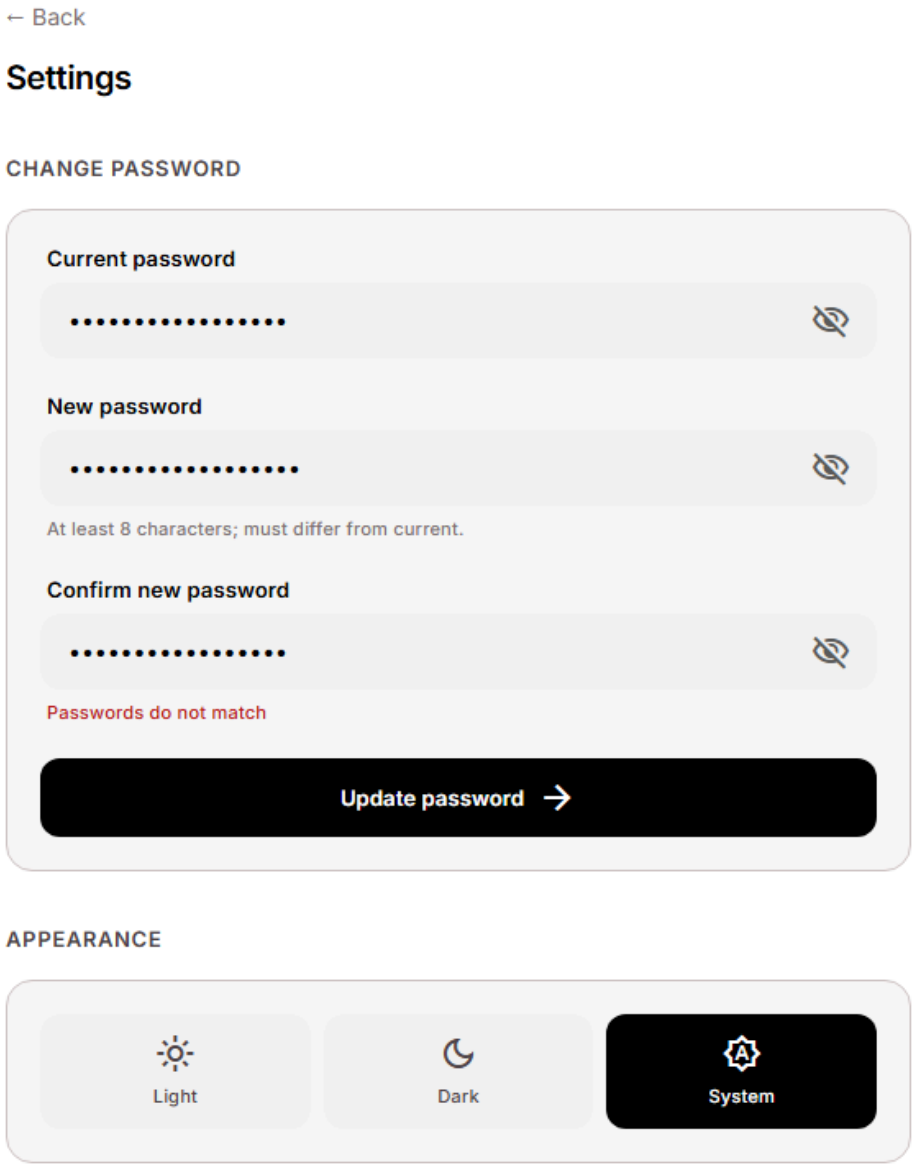
- Screen 2 (Alt Flow - Incorrect Current Password):



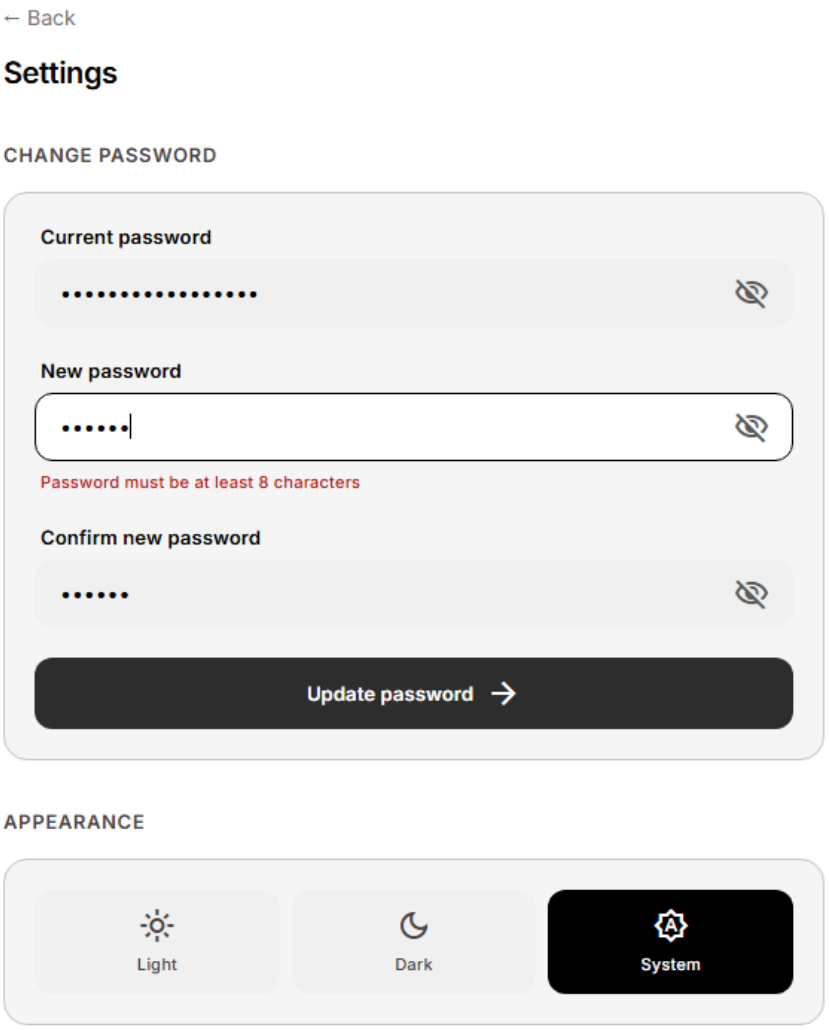
- Screen 3 (Alt Flow - New Password Matches Old Password):



- **Screen 4 (Alt Flow - Confirm Password Does Not Match):**



- **Screen 5 (Alt Flow - Weak New Password):**



Use-Case Specification: Change Avatar

Performed by: Nguyễn Hải Trọng | Reviewed by: Nguyễn Tuấn An | Edited by: Nguyễn Hải Trọng
Project Name: FOZ
Version: 1.1
Document identifier: FOZ-UCS-06

1. Change Avatar

- **Use-Case ID:** UC-06
- **Actor(s):** Standard User

1.1 Brief Description

Allows an authenticated user to upload a new profile picture.

2. Preconditions

- The user is authenticated.

3. Flow of Events

3.1 Basic Flow

1. The use case extends UC-04 when the user clicks the "Upload picture" icon.
2. The user selects an image file from their device.
3. The system validates the file type and size.
4. The system uploads the file to storage and retrieves the image URL.
5. The system updates the `avatar_url` in the `profiles` table.
6. The system refreshes the avatar display on the UI.

3.2 Alternative Flows

3.2.1 File Too Large

1. If at step 3, the file exceeds the 25MB raw upload limit.
2. The system aborts the upload and displays "Image file exceeds maximum raw size limit of 25MB."
3. The flow resumes at step 1.

3.2.2 User Cancels

1. If at step 2, the user closes the file picker without selecting an image.
2. The system aborts the upload process.
3. The use case ends.

3.2.3 Wrong File Type

1. If at step 3, the file uploaded is not a supported image format (e.g., PDF, DOCX).
2. The system aborts the upload and displays "Invalid file format. Please upload an image."
3. The flow resumes at step 1.

4. Special Requirements

- **Space:** Raw images are compressed heavily on the client and server to conserve server storage.

5. Postconditions

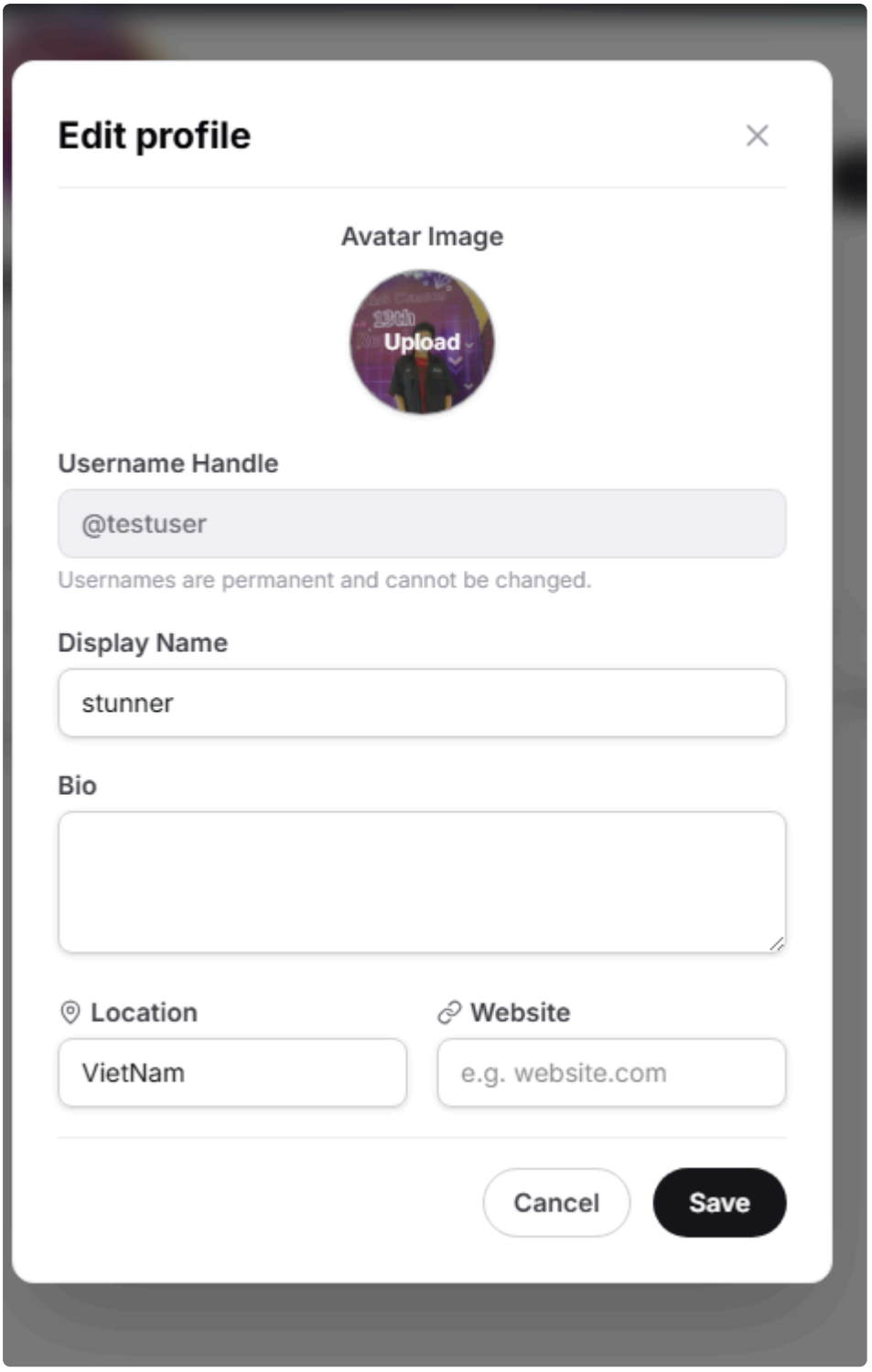
- The new avatar is visible publicly on the user's profile.

6. Extension Points

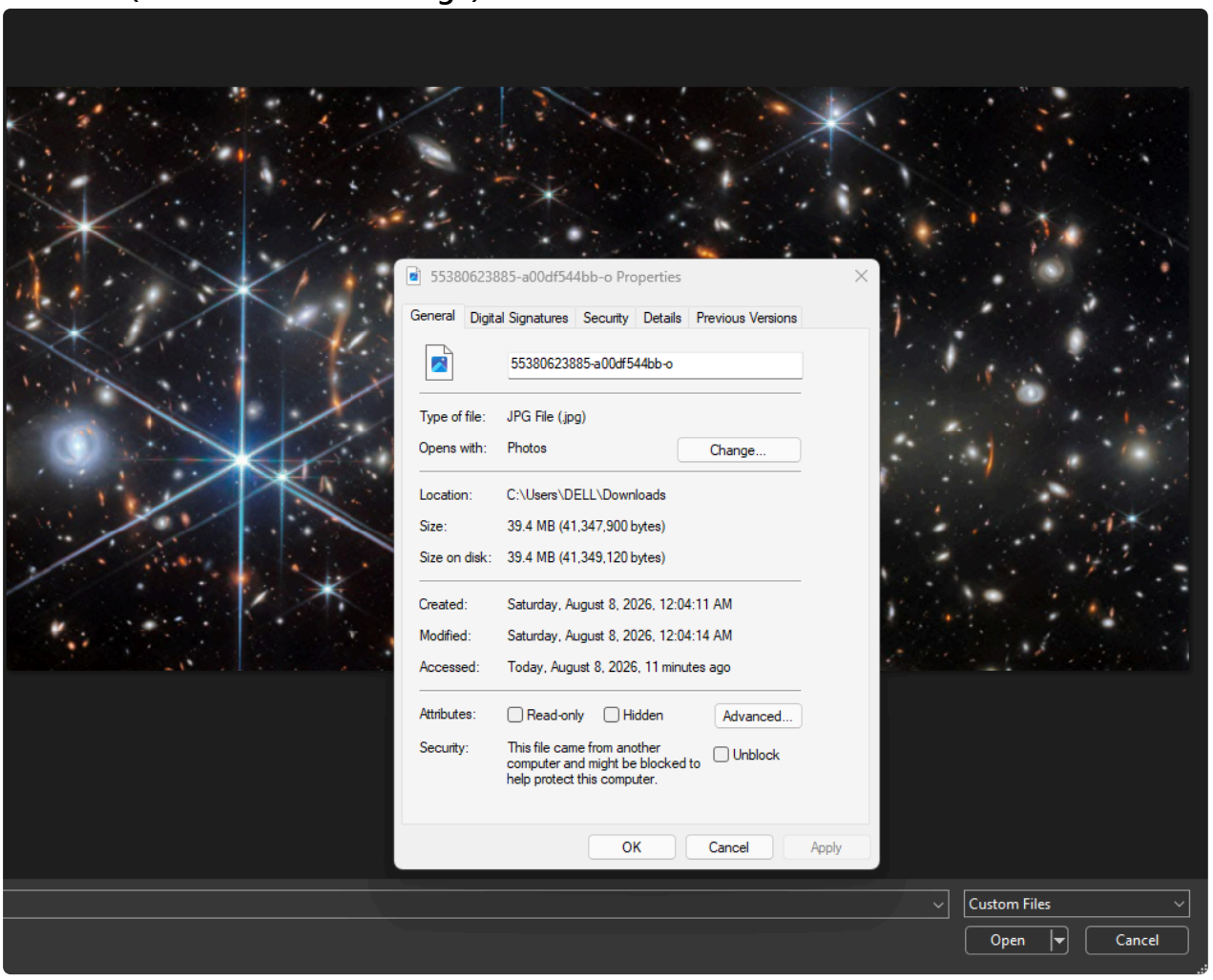
- None.

7. Prototype Requirement

- **Screen 1 (Basic Flow):**



- **Screen 2 (Alt Flow - File Too Large):**



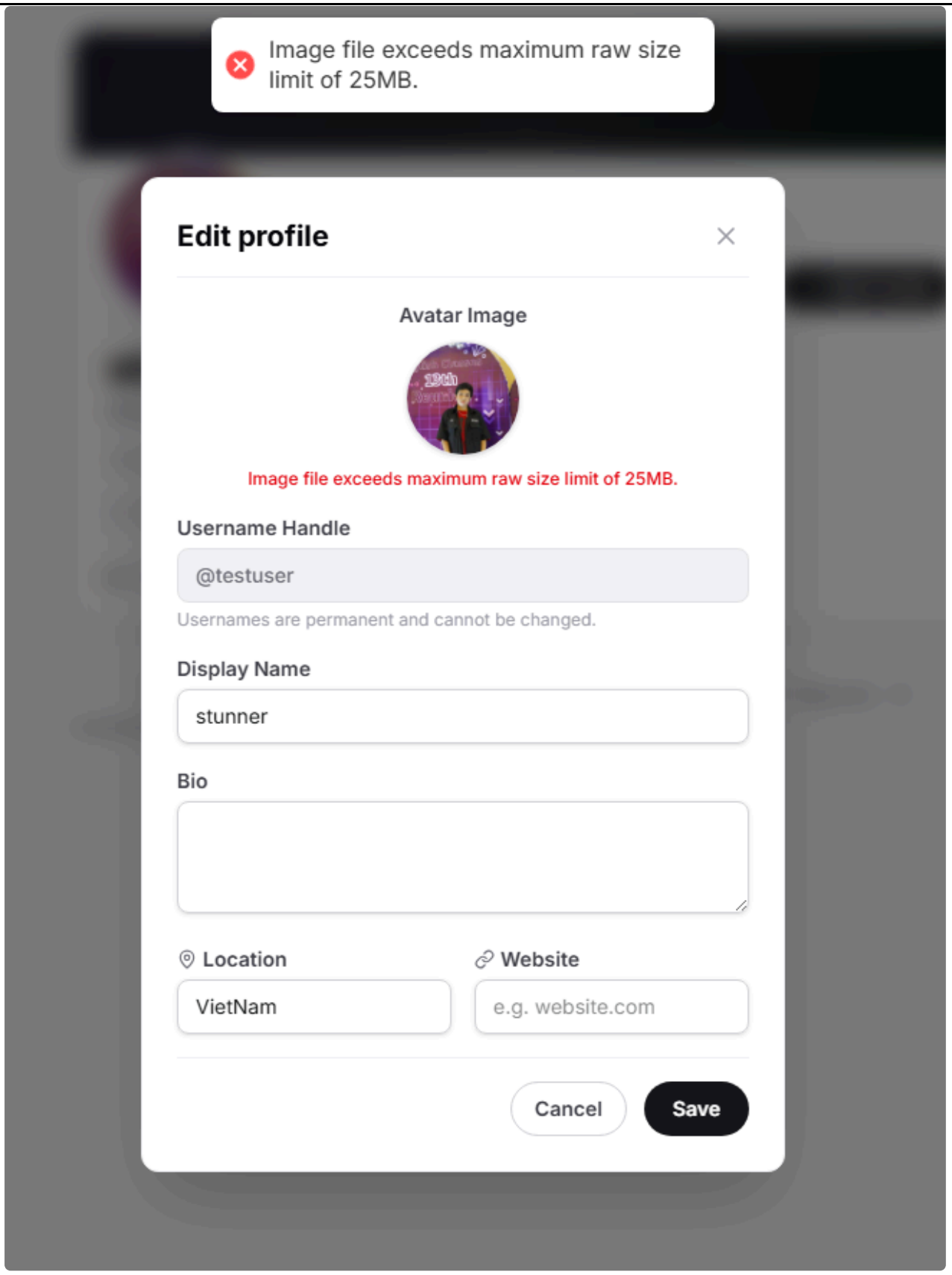
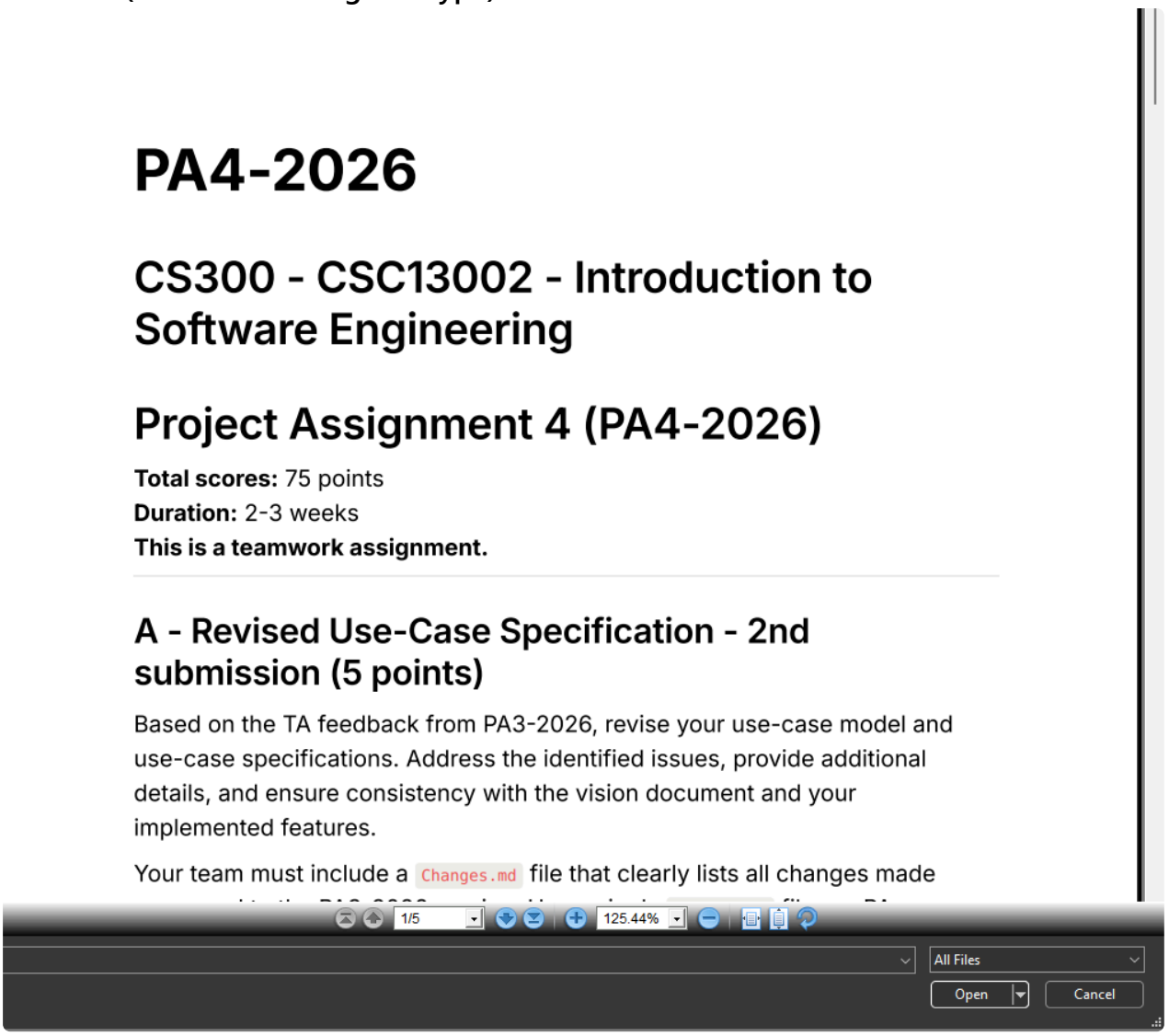
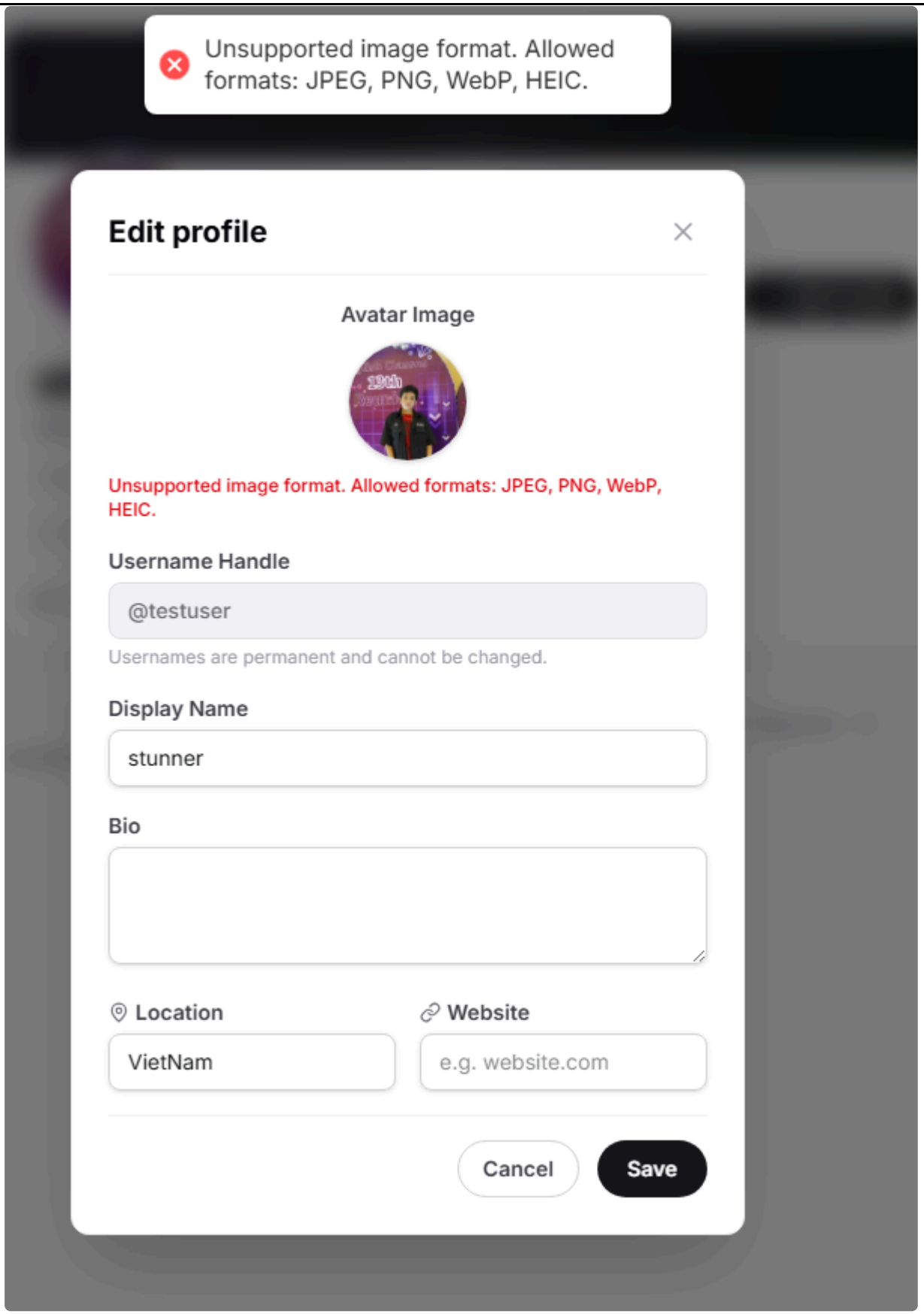


Image file exceeds maximum raw size limit of 25MB.
Edit profile
Avatar Image
Image file exceeds maximum raw size limit of 25MB.
Username Handle
@testuser
Usernames are permanent and cannot be changed.
Display Name
stunner
Bio
Location
VietNam
Website
e.g. website.com
Cancel
Save

- **Screen 3 (Alt Flow - Wrong File Type):**





Use-Case Specification: Browse Feed

Project Name: FOZ
Version: 1.0
Document identifier: FOZ-UCS-07

1. Browse Feed

- **Use-Case ID:** UC-07
- **Actor(s):** Guest, Standard User, Moderator, Admin

1.1 Brief Description

Allows users to view a dynamic timeline of forum posts sorted by recent engagement metrics or chronological order.

2. Preconditions

- None.

3. Flow of Events

3.1 Basic Flow

1. The use case begins when the user navigates to the application's homepage, which presents the feed tabs "For You" (default) and "Latest".
2. The system queries the `posts` table for content with `visibility_state = 'public'` and `deleted_at IS NULL`, unwrapping reposts so the original post content is shown.
3. The system orders the posts according to the selected feed mode:
   - **For You:** ranking based on engagement, and recency.
   - **Latest:** sorted by posting time, with the newest first.
4. The system displays the feed with post content, author avatars, and interaction counts (likes, comments, reposts).
5. The user scrolls to browse feed.

3.2 Alternative Flows

3.2.1 Switch feed mode

1. The user switches between the "For You" and "Latest" feed tabs.
2. The system reloads and displays the posts according to the newly selected mode.

3.2.2 No Posts Available

1. If at step 2, the database returns no posts.
2. The system displays a "No discussions found. Be the first to post!" placeholder.
3. The use case ends.

3.2.3 Scroll to Load More Posts

1. If at step 5, the user scrolls to the bottom of the visible posts.
2. The system triggers pagination displaying "Load more" button.

3. The user clicks "Load more" button.
4. The system fetches and displays the next set of posts.
5. The use case ends.

3.2.4 View a long content post

1. If at step 4, the post content is too long, the system displays a few first lines of post along with a button "Read more".
2. The user clicks "Read more" button.
3. The system displays the full content of the post along with a button "Show less".
4. The user clicks "Show less" button.
5. The system displays the first few lines of the post.
6. The use case ends.

4. Special Requirements

- **Performance:** The initial feed must load in `< 2 seconds` under good network conditions.

5. Postconditions

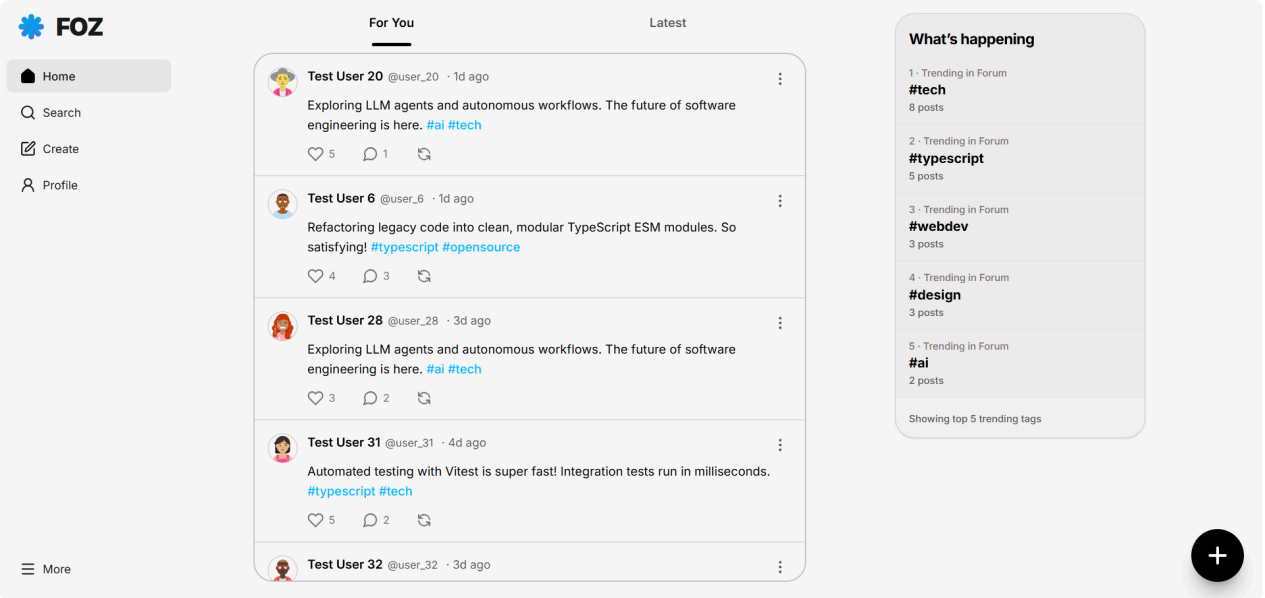
- None.

6. Extension Points

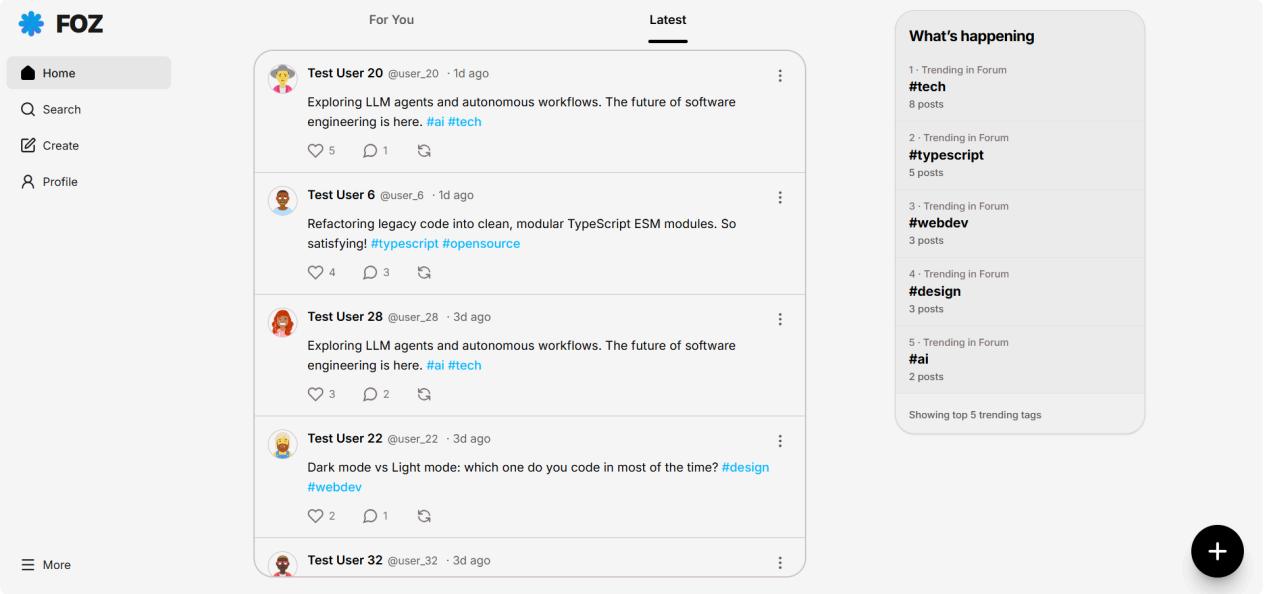
- None.

7. Prototype Requirement

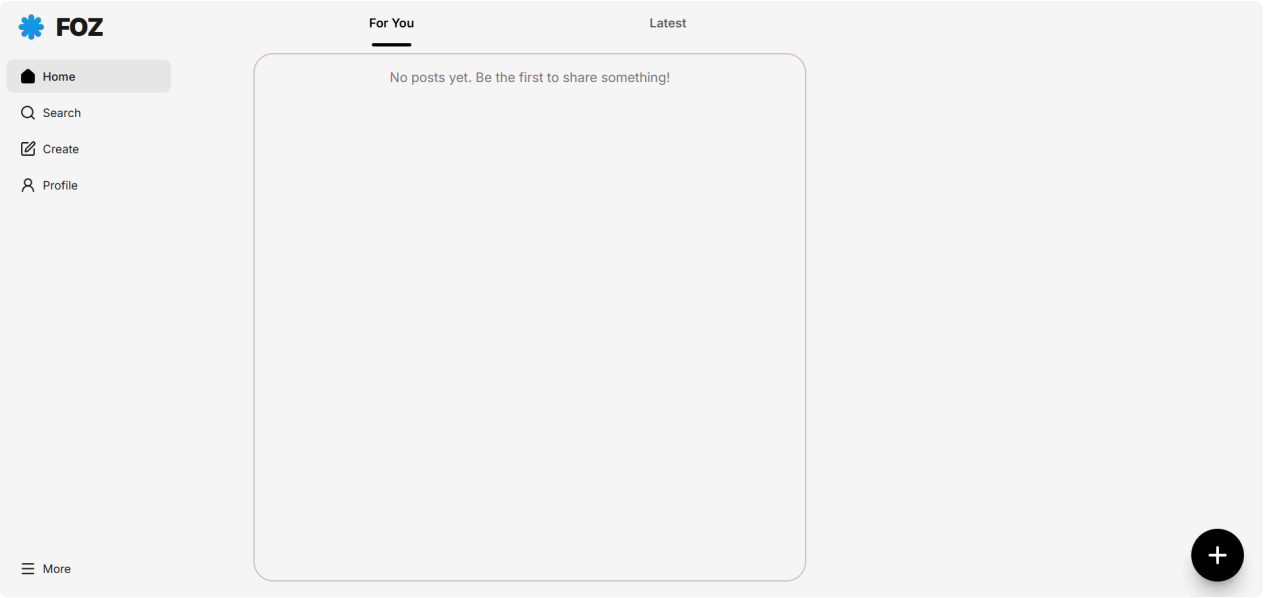
- Screen 1 (Basic Flow):



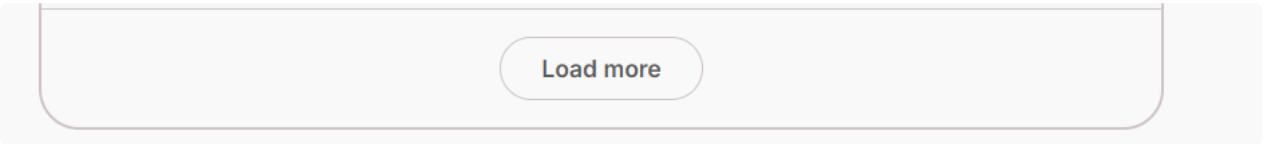
Screen 2 (Alternative Flow 1): Latest tab



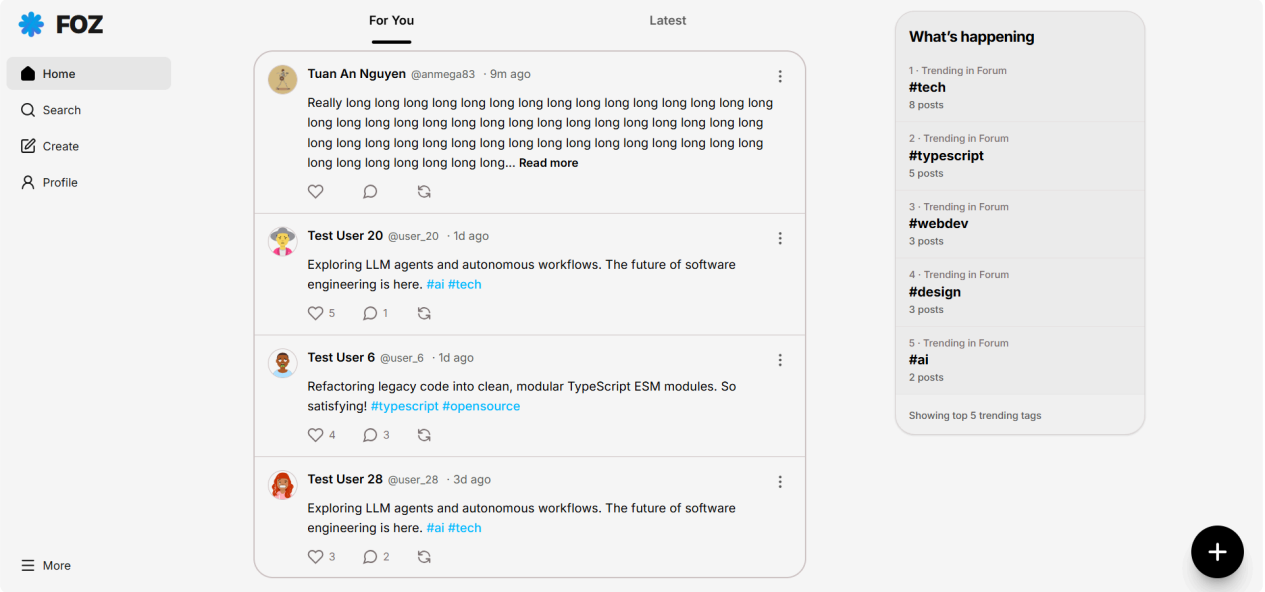
Screen 3 (Alternative Flow 2): No post available



Screen 4 (Alternative Flow 3): Scroll to load more posts



Screen 5 (Alternative Flow 4): View a long content post


FOZ
Home
Search
Create
Profile
More
For You
Latest
Tuan An Nguyen @anmega83 · 9m ago
Really long long long

3. Flow of Events

3.1 Basic Flow

1. The use case begins when the user navigates to the application's search page, then enters a keyword into the search bar and presses Enter (or clicks the search icon).
2. The system sends the query to the backend.
3. The backend searches the `posts` (body/title) and `profiles` (display_name/username) tables for matches.
4. The system filters out content that is deleted, hidden, or authored by banned users.
5. The system displays a categorized list of results (Posts, Profile) with the **Posts** tab selected by default.

3.2 Alternative Flows

3.2.1 Search feature unavailable

1. If at step 1, the search query is empty or contains only whitespace.
2. The system does not display the search button until a valid search term is entered.
3. The use case ends.

3.2.2 Search Query Exceeds Character Limit

1. If at step 1 of the basic flow, the user enters a search query exceeding the maximum length (500 characters) and attempts to submit.
2. The system blocks query submission and displays an error message notifying the user (e.g., "Search query is too long. Please limit to 500 characters.").
3. The user remains on the search interface to edit the query.
4. The use case ends.

3.2.3 Switch Category (Posts and Profile)

1. At step 5 of the basic flow, the user switches between the "Posts" and "Profile" category tabs on the search results page.
2. The system updates the view to display the search results corresponding to the selected category (matching posts or matching user profiles).
3. The use case ends.

3.2.4 No matching results

1. The user enters a search query.
2. The system finds no matching content.
3. The system displays **"No results for search query."**
4. The use case ends.

3.2.5 Click a hashtag in "TRENDING TOPICS"

1. The user clicks a hashtag under the **"TRENDING TOPICS"** section on the search page.
2. The system automatically populates the search bar with the selected hashtag and executes the search query.
3. The system displays posts matching the selected topic/hashtag.
4. The use case ends.

3.2.6 Click a profile in "Suggested Creators"

1. The user clicks a creator profile under the **"Suggested Creators"** section on the search page.
2. The system navigates the user to the selected creator's profile page.
3. The use case ends.

3.2.7 Click a post in "Trending Discussions"

1. The user clicks a discussion post under the **"Trending Discussions"** section on the search page.
2. The system navigates the user to the selected post's detail page.
3. The use case ends.

4. Special Requirements

- **Performance:** Search results should be displayed within 2–3 seconds under good network conditions.

5. Postconditions

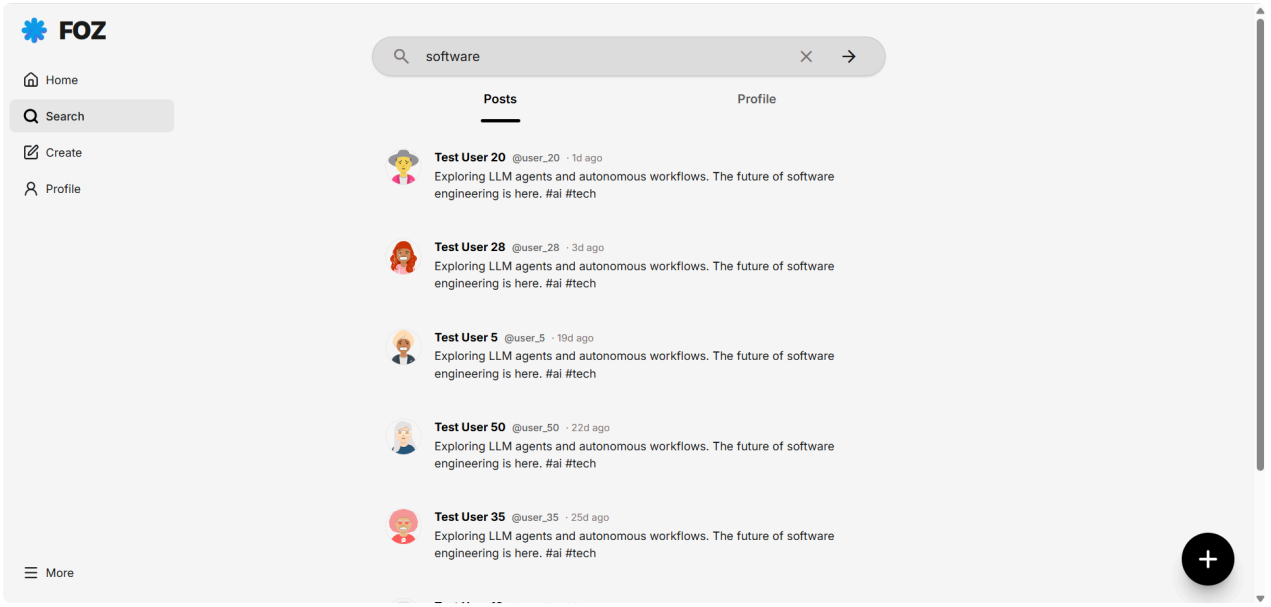
- None.

6. Extension Points

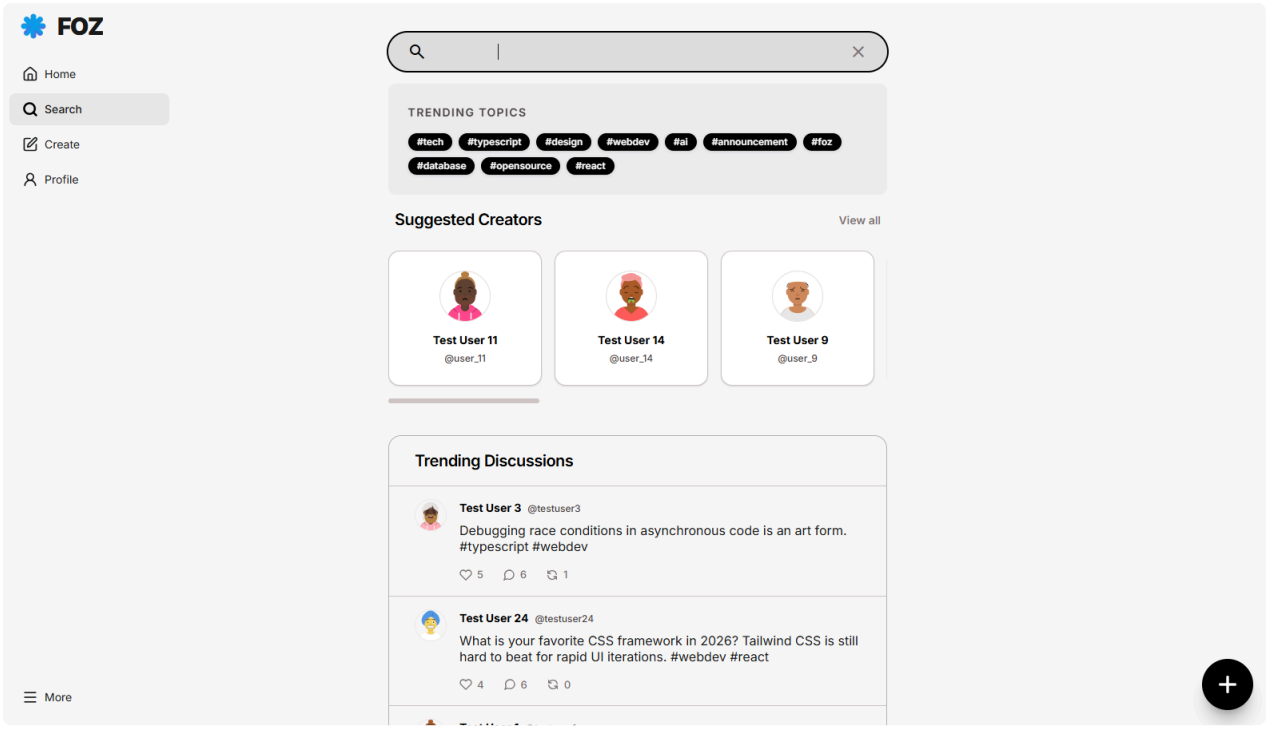
- None.

7. Prototype Requirement

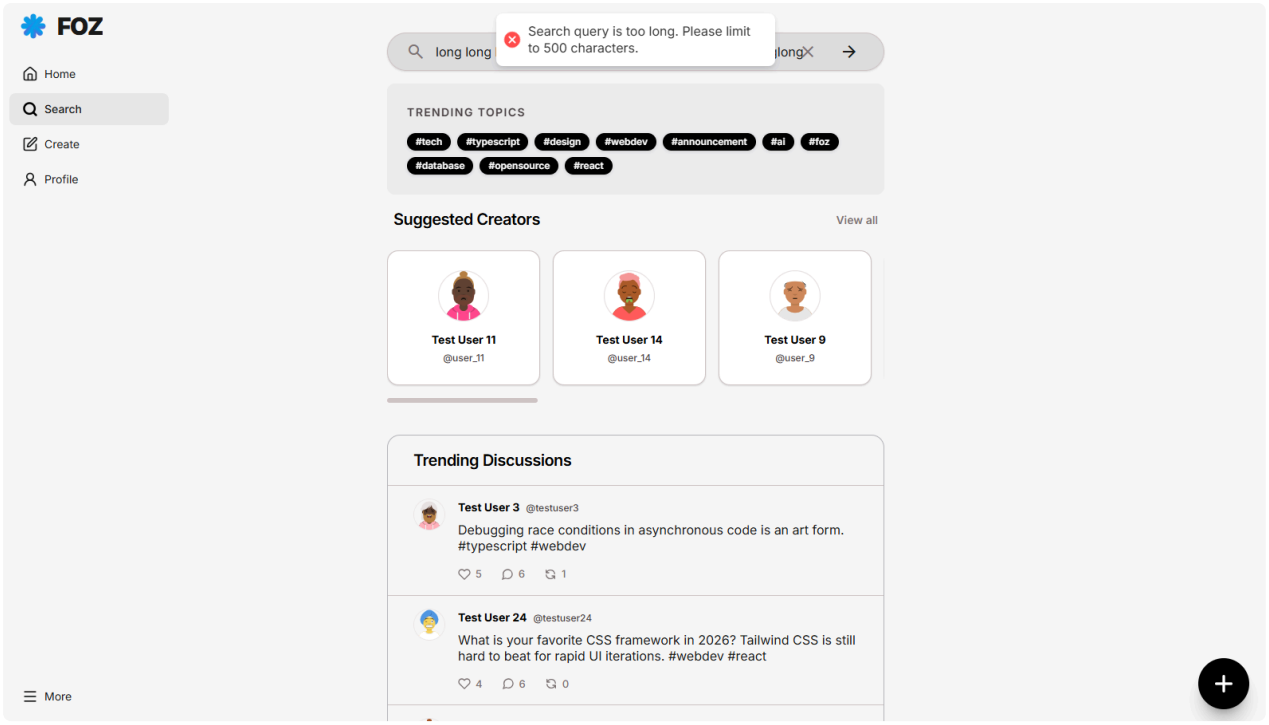
- Screen 1 (Basic Flow):



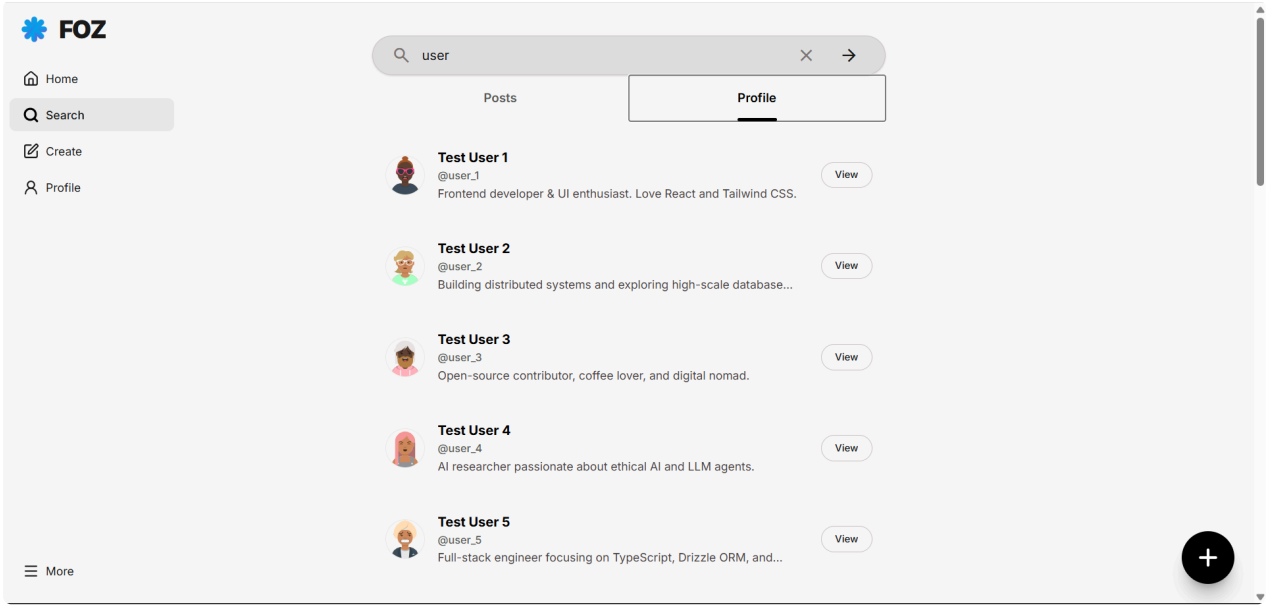
Screen 2 (Alternative Flow 1 - Empty):



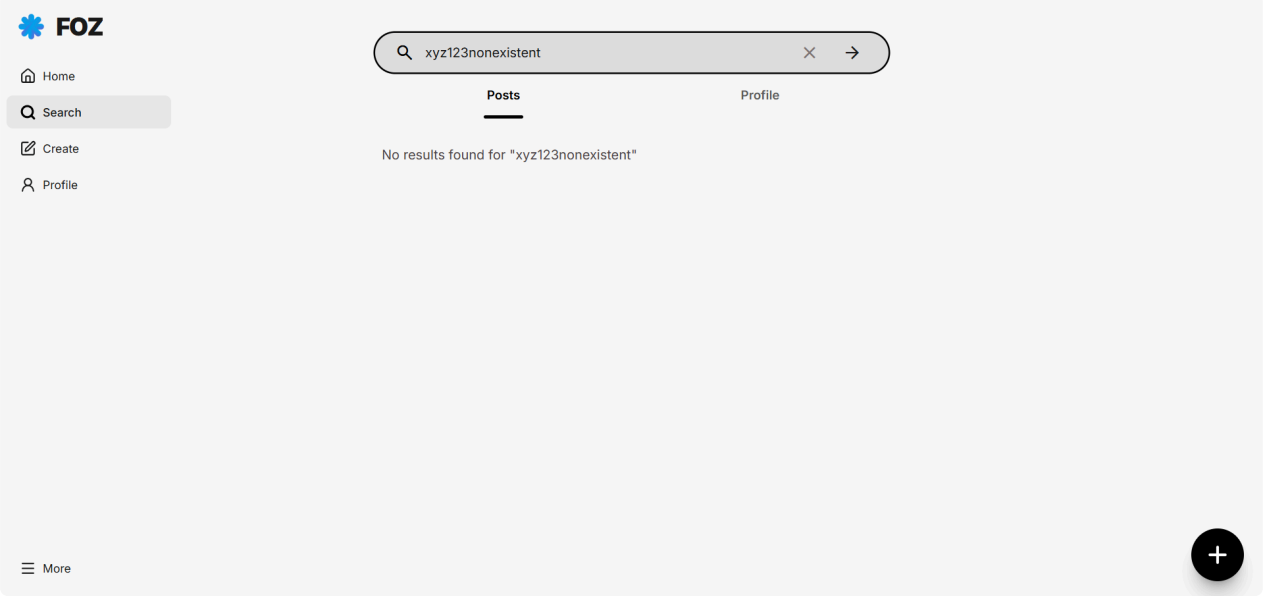
Screen 3 (Alternative Flow 2 - Query limit):



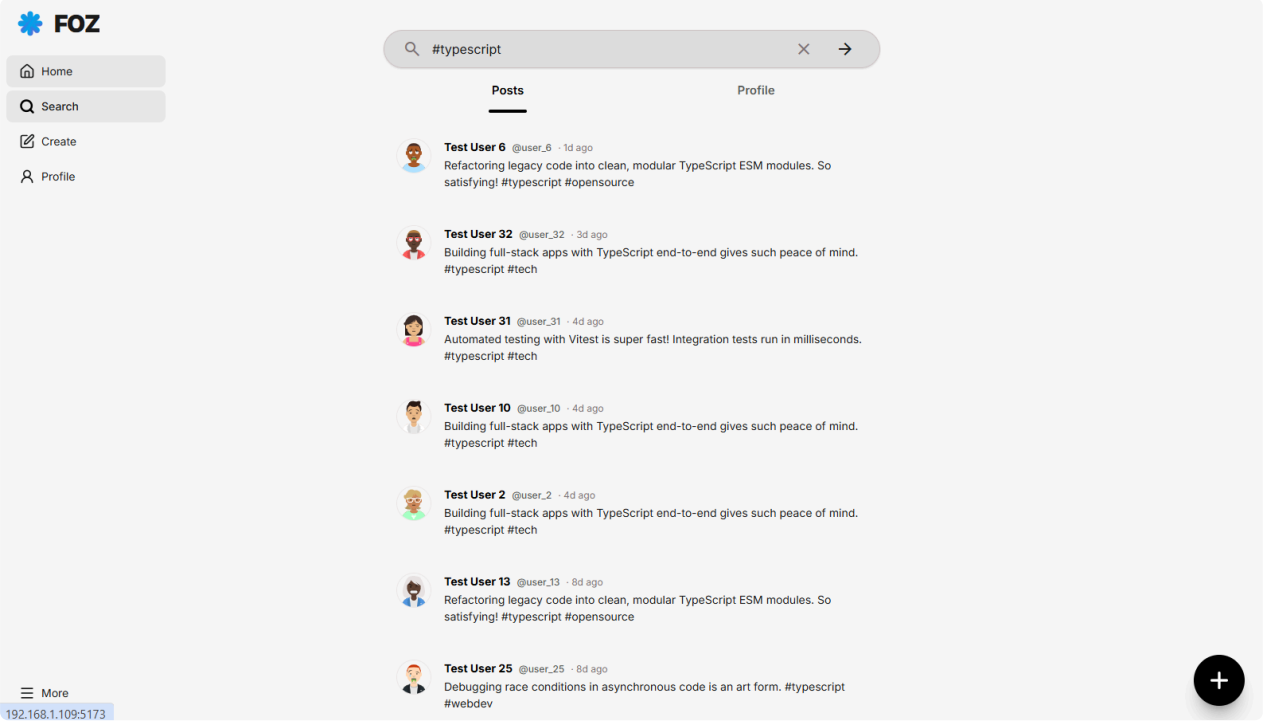
Screen 4 (Alternative Flow 3 - Switch Category):



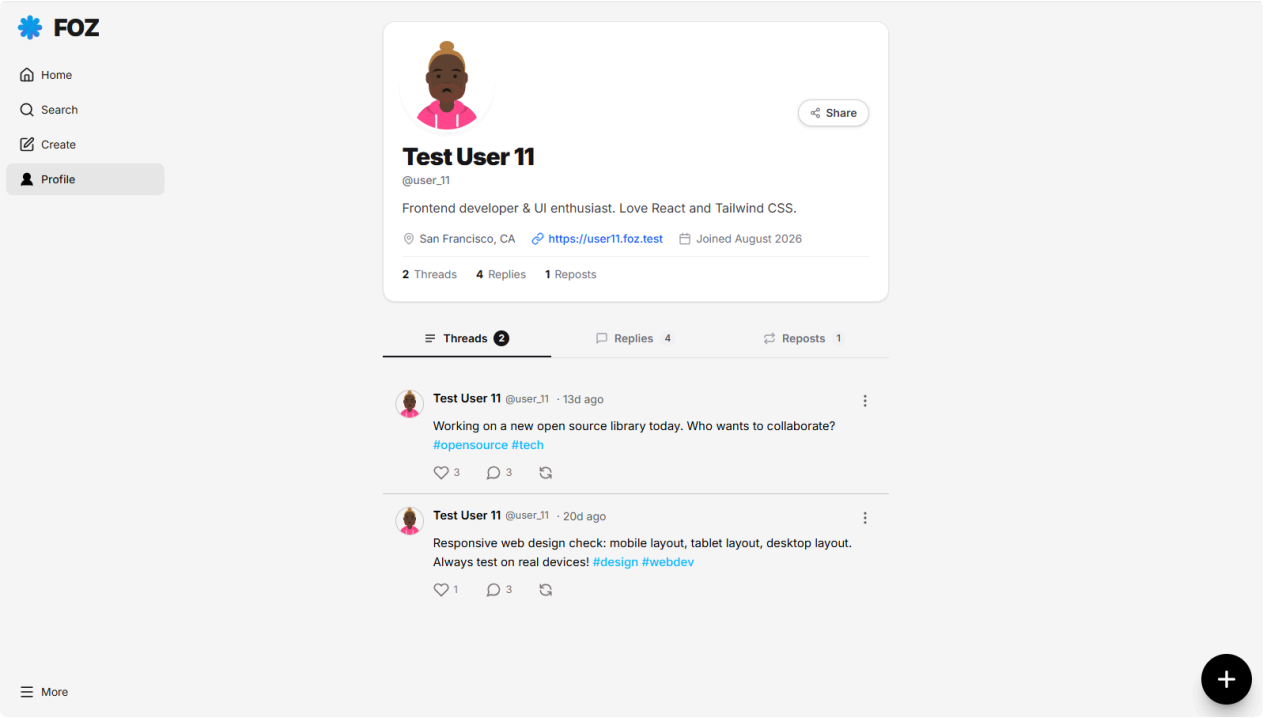
Screen 5 (Alternative Flow 4 - No results):



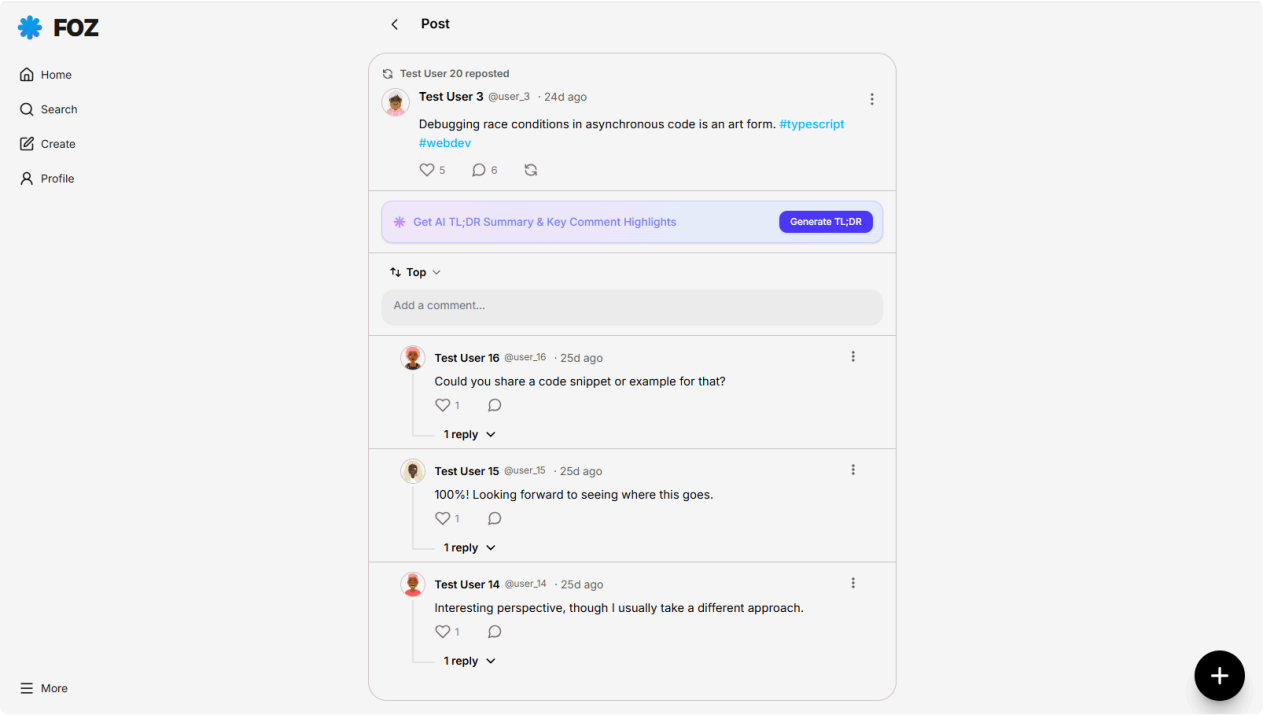
Screen 6 (Alternative Flow 5 - "TRENDING TOPICS"):



Screen 7 (Alternative Flow 6 - "Suggested Creators"):



Screen 8 (Alternative Flow 7 - "Trending Discussions"):



Use-Case Specification: View Post Details

Project Name: FOZ
Version: 1.1
Document identifier: FOZ-UCS-09

1. View Post Details

- **Use-Case ID:** UC-09
- **Actor(s):** Guest, Standard User

1.1 Brief Description

Allows a user to read the full body of a post along with its entire nested comment thread.

2. Preconditions

- The post ID must be valid.

3. Flow of Events

3.1 Basic Flow

1. The use case begins when the user clicks on a post from the feed or search results.
2. The system queries the `posts` and `comments` tables matching the Post ID.
3. The system formats the comment tree hierarchically.
4. The system displays the full post content, author details, and comments.

3.2 Alternative Flows

3.2.1 No comments

1. The post has no comments.
2. The system displays **"No comments yet. Be the first to reply!"**

3.2.2 Post Not Found

1. If at step 2, the requested Post ID does not exist in the database, or the post's lifecycle_state is removed_by_moderation.
2. The system displays a 404 Not Found error page.
3. The use case ends.

4. Special Requirements

- None.

5. Postconditions

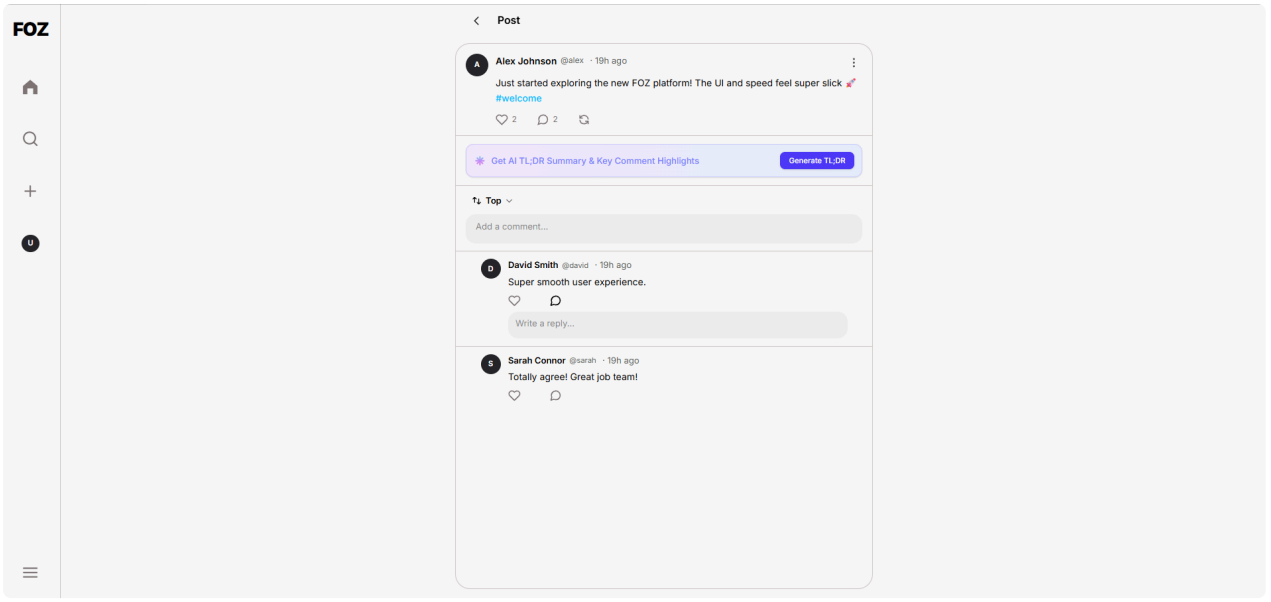
- None.

6. Extension Points

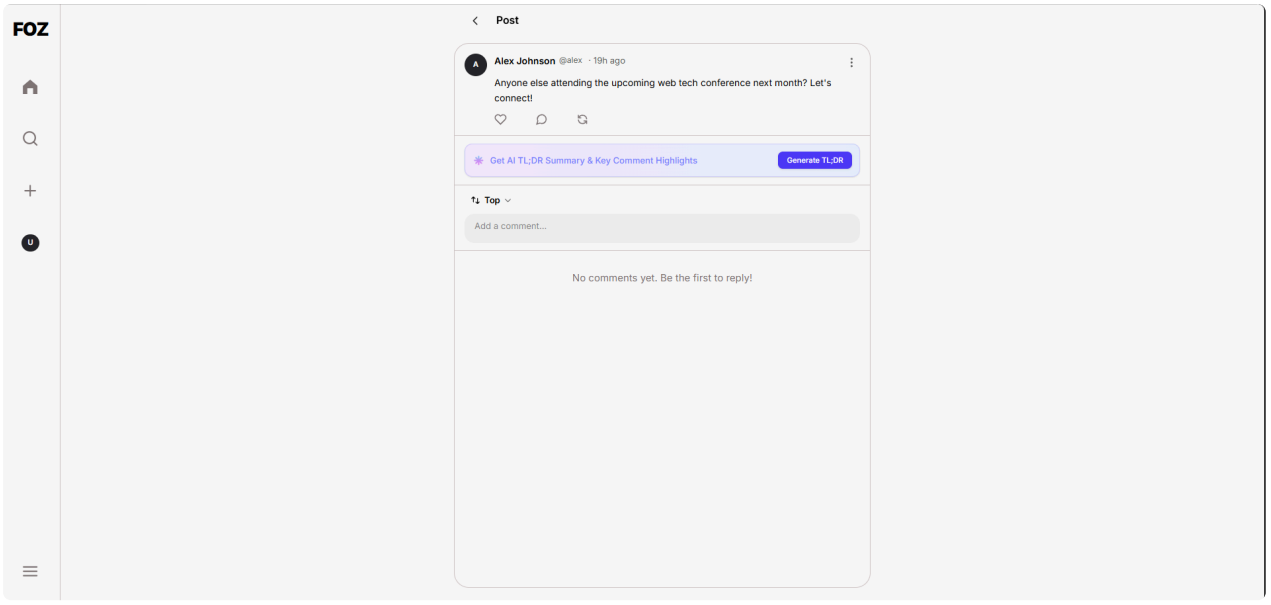
- **Request AI Summary:** Extended via UC-14.

7. Prototype Requirement

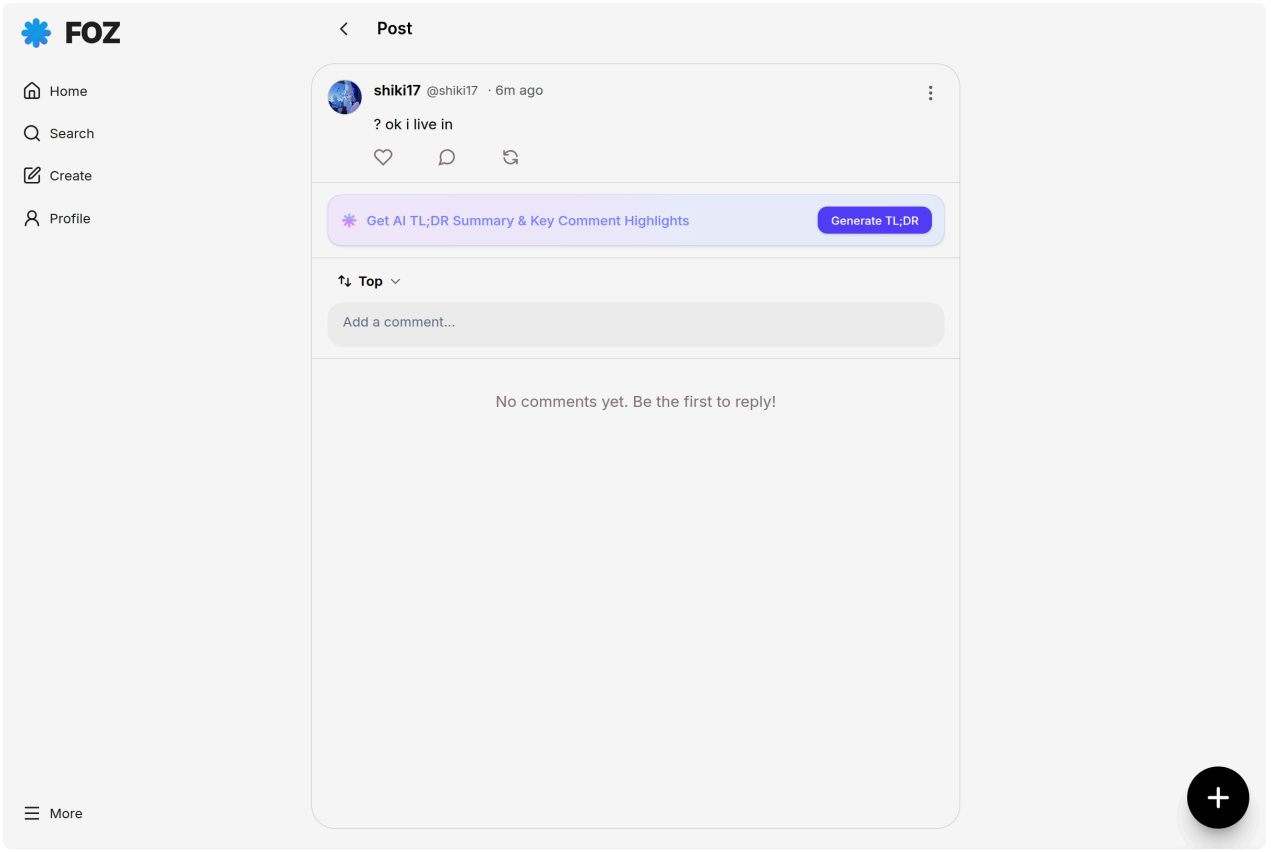
- Screen 1 (Basic Flow):

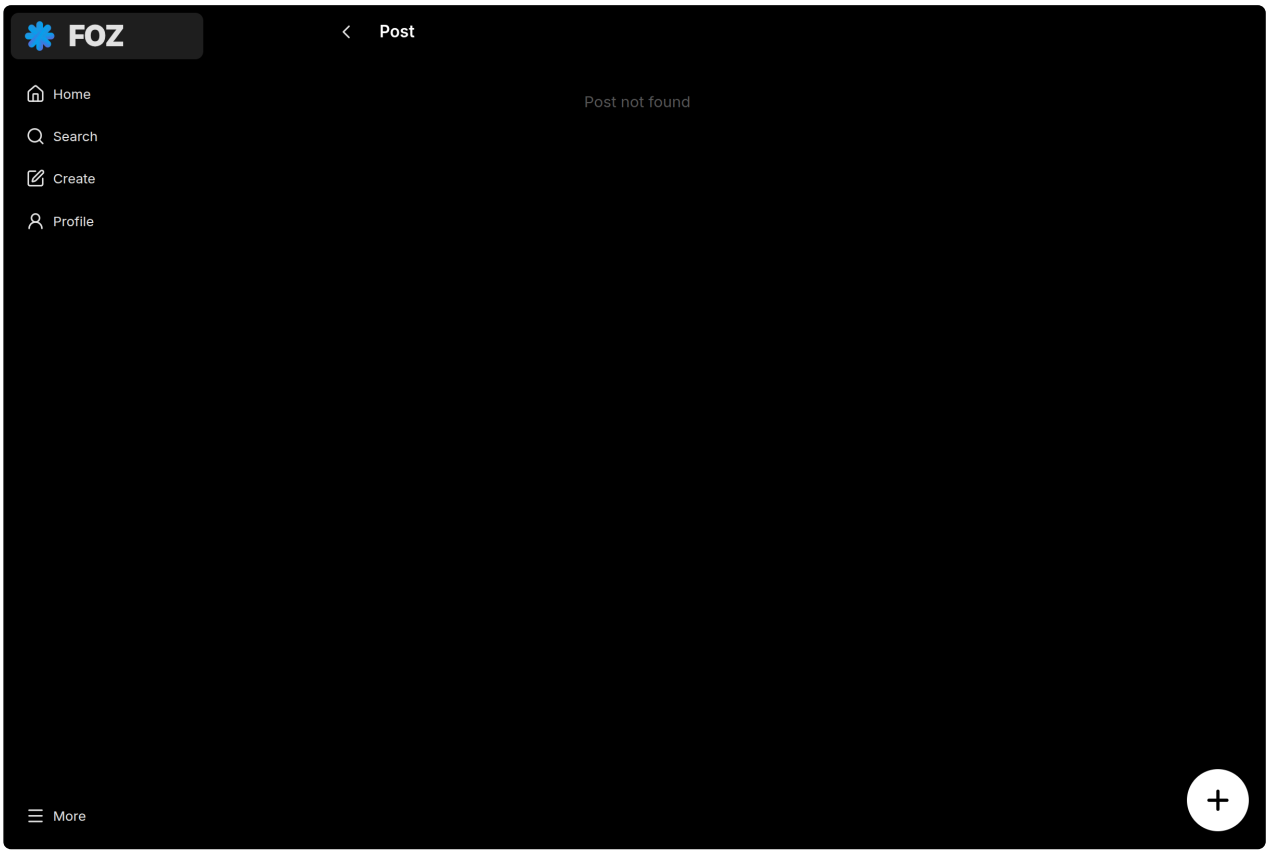


- Screen 2 (Alternative Flow 1 - No comments):



- Screen 2 (Alternative Flow 2 - Post Not Found):





Use-Case Specification: Create Post

Project Name: FOZ
Version: 1.1
Document identifier: FOZ-UCS-10

1. Create Post

- **Use-Case ID:** UC-10
- **Actor(s):** Standard User

1.1 Brief Description

Allows an authenticated user to publish a new text-based post to initiate a discussion.

2. Preconditions

- The user is authenticated and their account status is `active`.

3. Flow of Events

3.1 Basic Flow

1. The use case begins when the user clicks "Create Post".
2. The system displays an editor.
3. The user adds the content and clicks "Post".
4. The system validates the input lengths.
5. The system saves the record in the `posts` table (lifecycle_state: `active`).
6. The system redirects to the new post's detail page.

3.2 Alternative Flows

3.2.1. Post creation canceled

1. The user cancels the post creation.
2. The system remains the draft and returns to the previous screen.

3.2.2 Content Validation Error

1. If at step 4, the post body is empty.
2. The system disallows user to click submit button.
3. The flow resumes at step 3.

3.2.3 Media Attachment Error

1. If at step 3, the users attachs wrong file type of the image to the post.
2. The system aborts and displays a validation error message.
3. The flow resumes at step 3.

3.2.4 Media Attachment Cancelled

1. If at step 3, the user cancels the media attachment.

2. The post body is left empty.
3. The system aborts and displays a validation error message.
4. The flow resumes at step 3.

3.2.5 Exceeds Limit of Characters

1. If at step 4, the post body exceeds 5,000 characters.
2. The system aborts and displays a validation error message.
3. The flow resumes at step 3.

3.2.6 Exceeds Limit of Number of Media Attachments

1. If at step 4, there are more than 5 media attachments.
2. The system aborts and displays a validation error message.
3. The flow resumes at step 3.

3.2.7 Valid Image with Large Size

1. If at step 3, a valid type image is attached to a post with the size more than 5MB.
2. The system automatically compresses the image before uploading.
3. The flow resumes at step 5.

3.2.8 Network Timeout after Attaching Any Image

1. If at step 3, the user attachs an image to a post.
2. The network is timeout.
3. The system displays pending "Posting" button.

4. Special Requirements

- **Security:** Inputs must be sanitized against XSS.

5. Postconditions

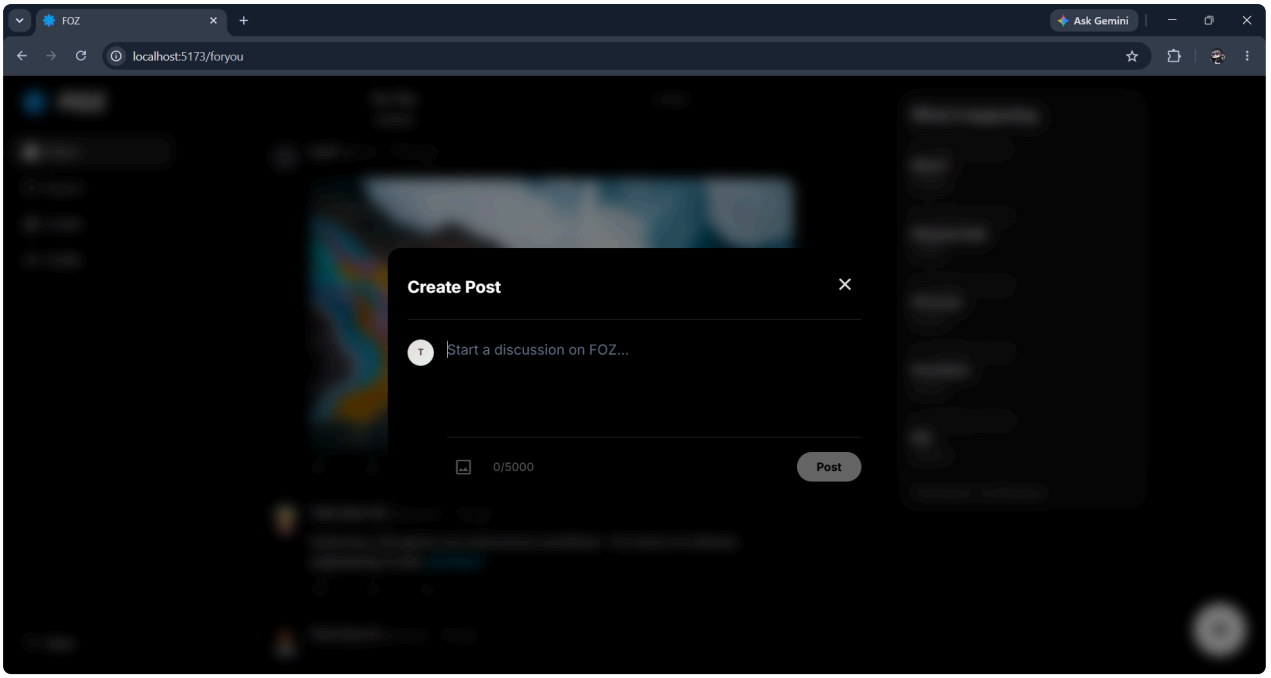
- A new record exists in the `posts` table.

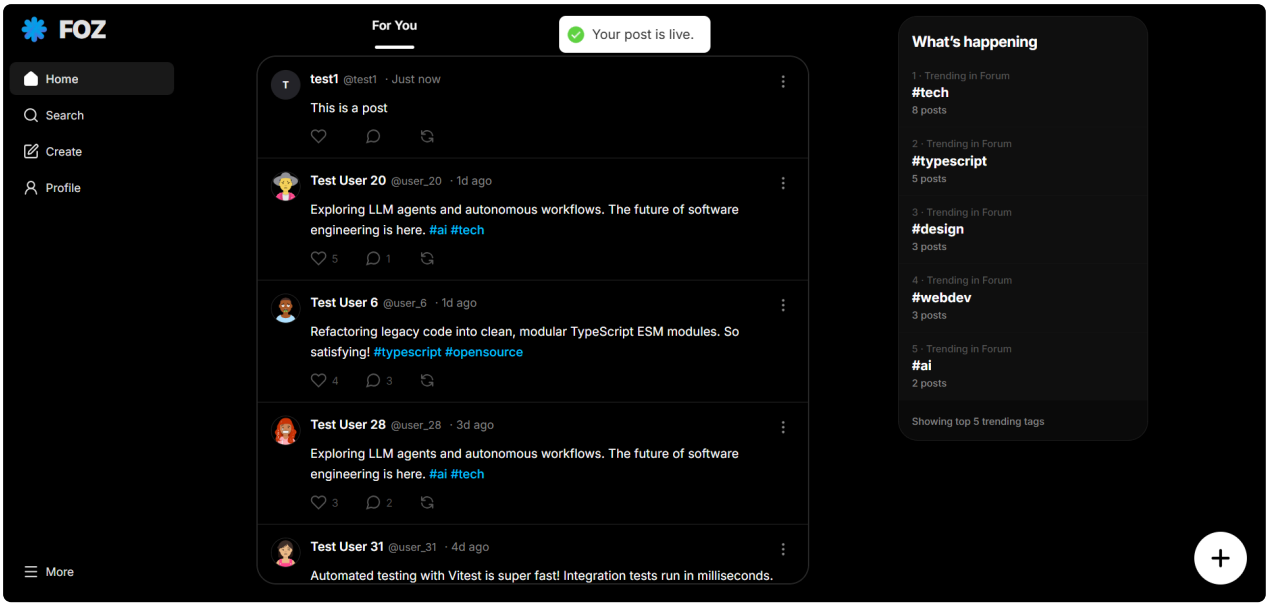
6. Extension Points

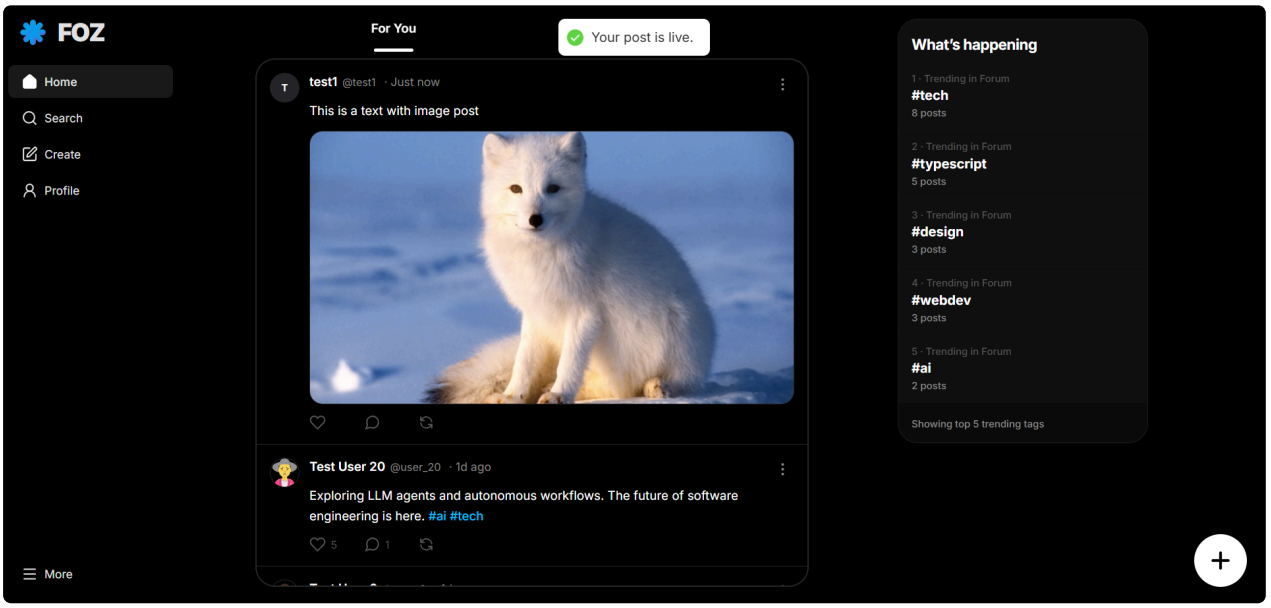
- None.

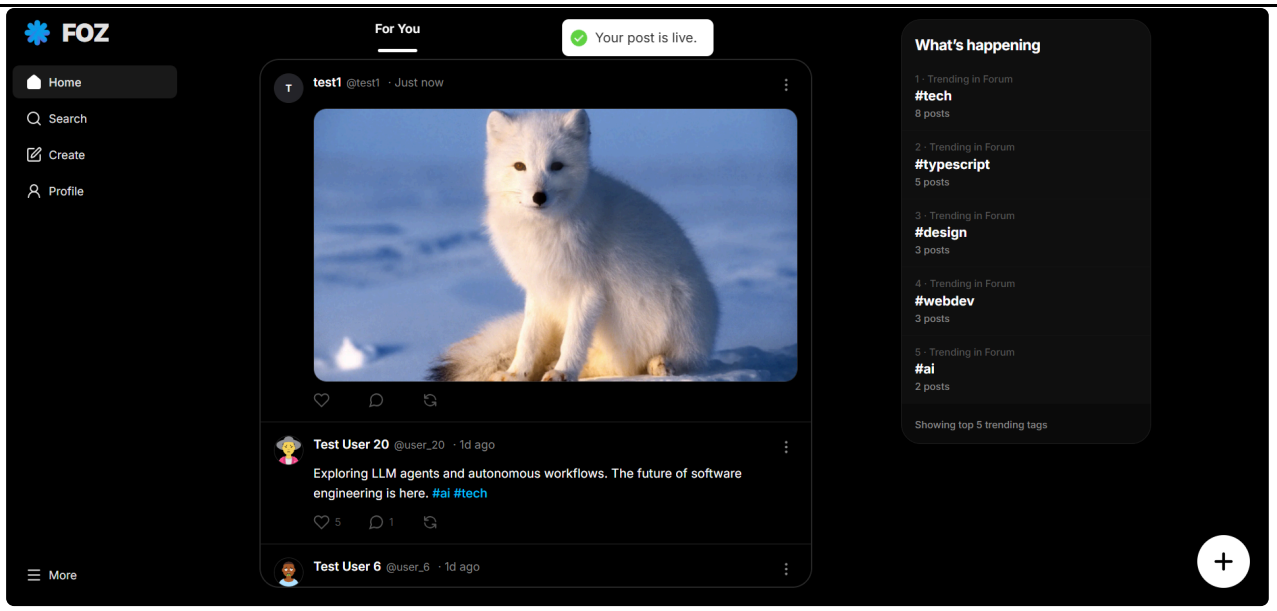
7. Prototype Requirement

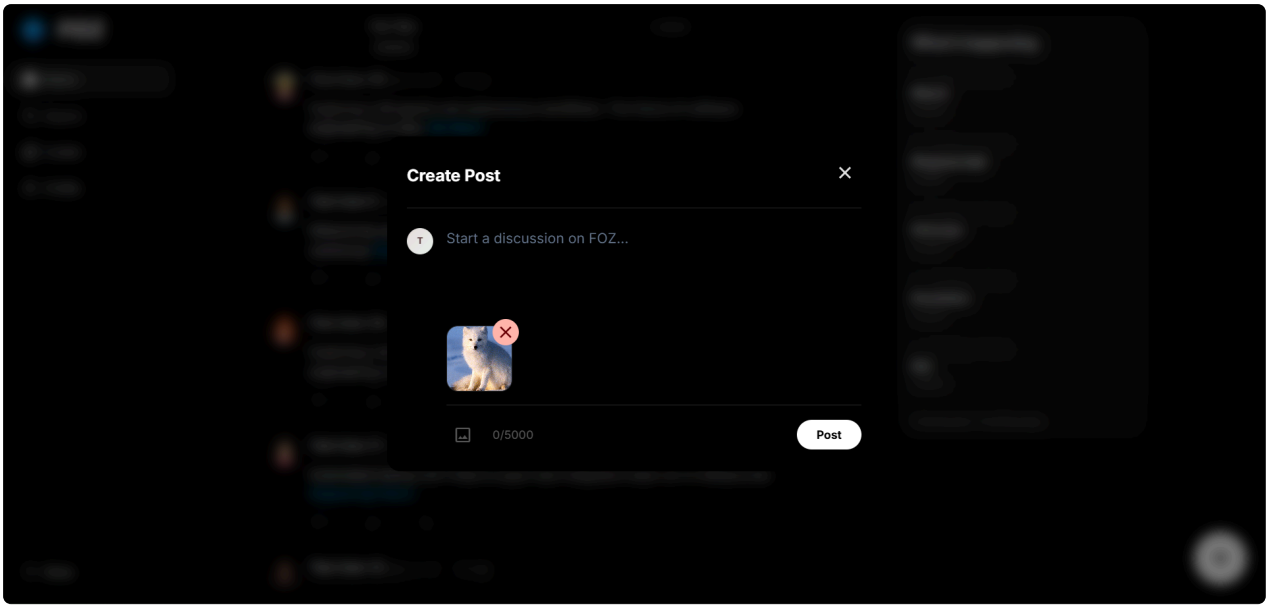
- Screen 1 (Basic Flow):

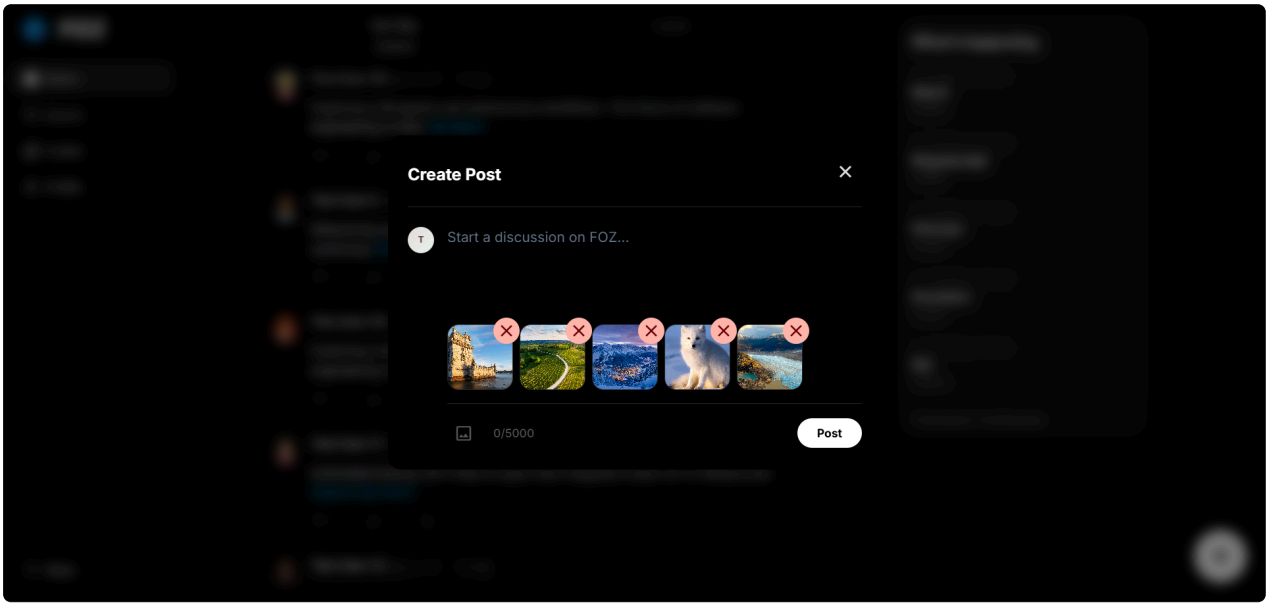




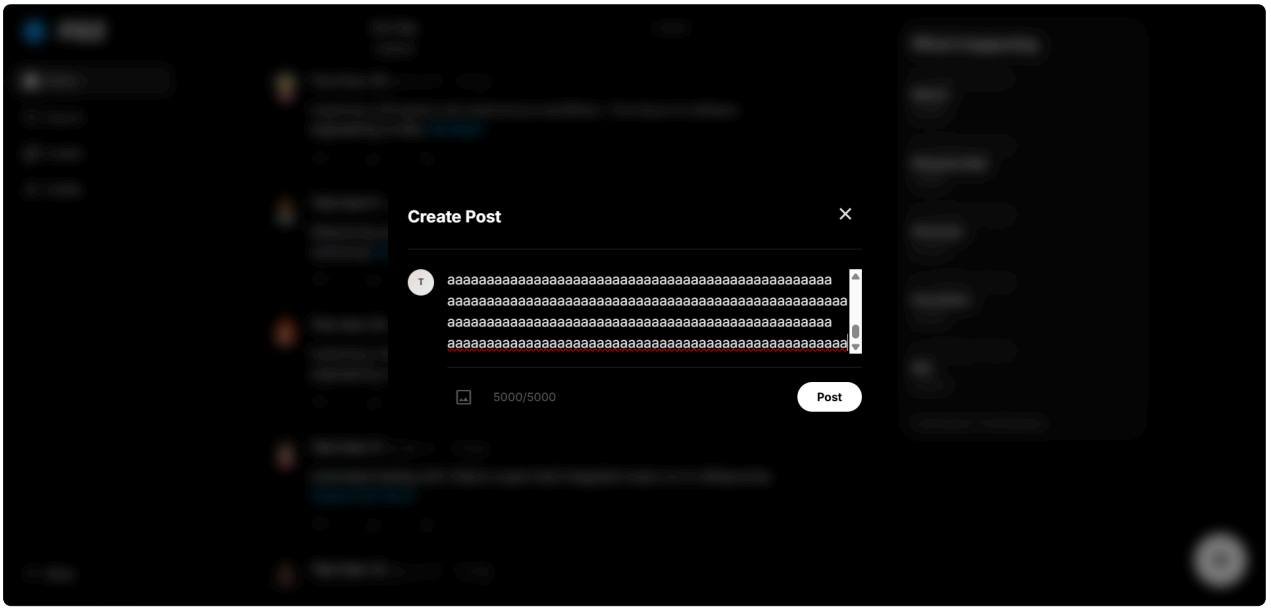


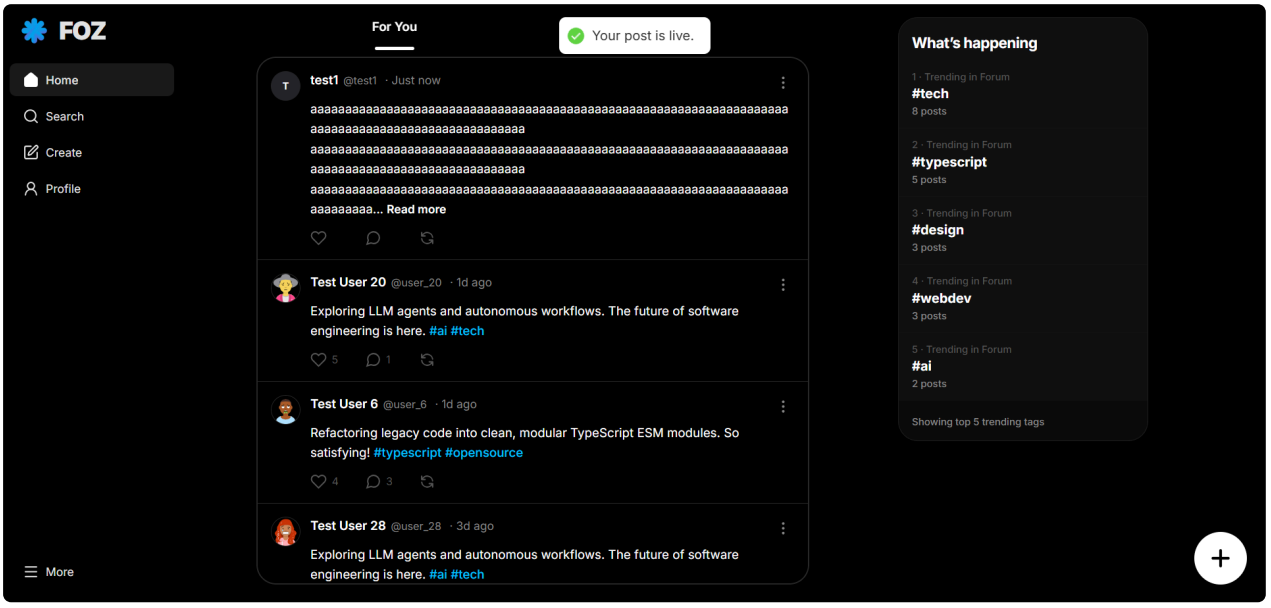

FOZ
Home
Search
Create
Profile
More
For You
Your post is live.
test1 @test1 · Just now
Test User 20 @user_20 · 1d ago
Exploring LLM agents and autonomous workflows. The future of software engineering is here. #ai #tech
Test User 6 @user_6 · 1d ago
What's happening
1 · Trending in Forum
#tech
8 posts
2 · Trending in Forum
#typescript
5 posts
3 · Trending in Forum
#design
3 posts
4 · Trending in Forum
#webdev
3 posts
5 · Trending in Forum
#ai
2 posts
Showing top 5 trending tags


Create Post
Start a discussion on FOZ...
0/5000
Post

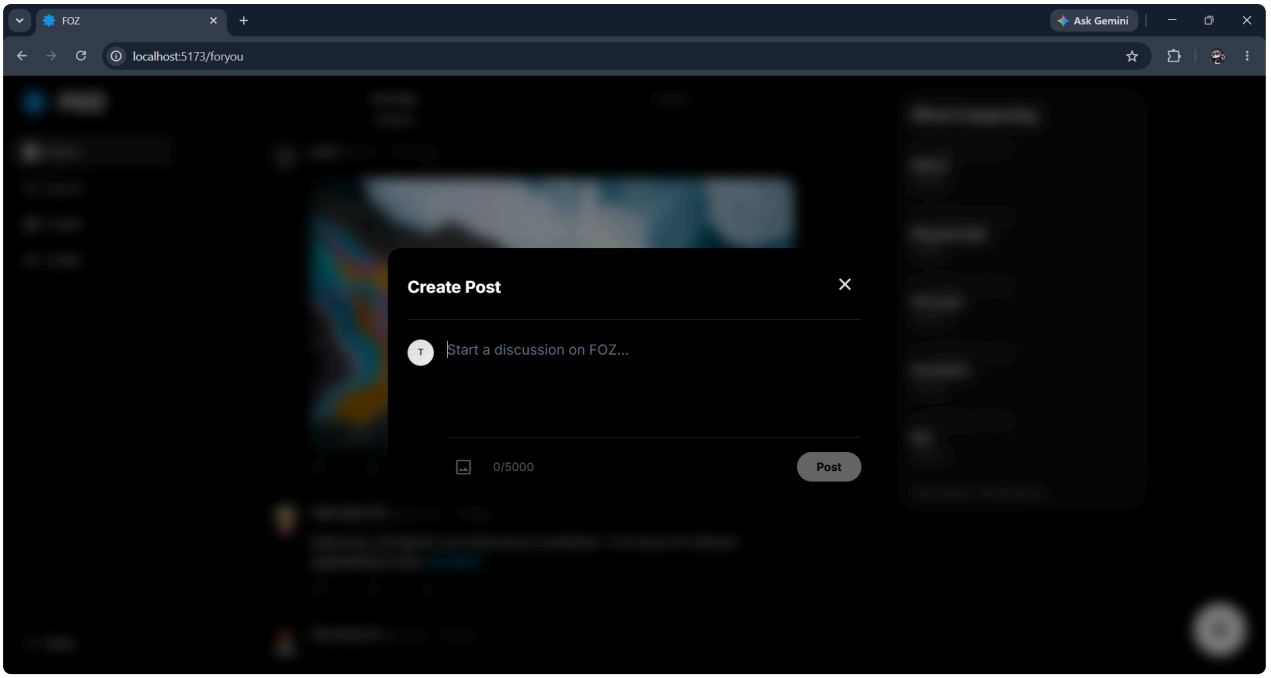

Create Post
Start a discussion on FOZ...
0/5000
Post

FOZ
Home
Search
Create
Profile
More
For You
Latest
test1 @test1 · Just now
Test User 20 @user_20 · 1d ago
Exploring LLM agents and autonomous workflows. The future of software engineering is here. #ai #tech
What's happening
1 · Trending in Forum
#tech
8 posts
2 · Trending in Forum
#typescript
5 posts
3 · Trending in Forum
#design
3 posts
4 · Trending in Forum
#webdev
3 posts
5 · Trending in Forum
#ai
2 posts
Showing top 5 trending tags

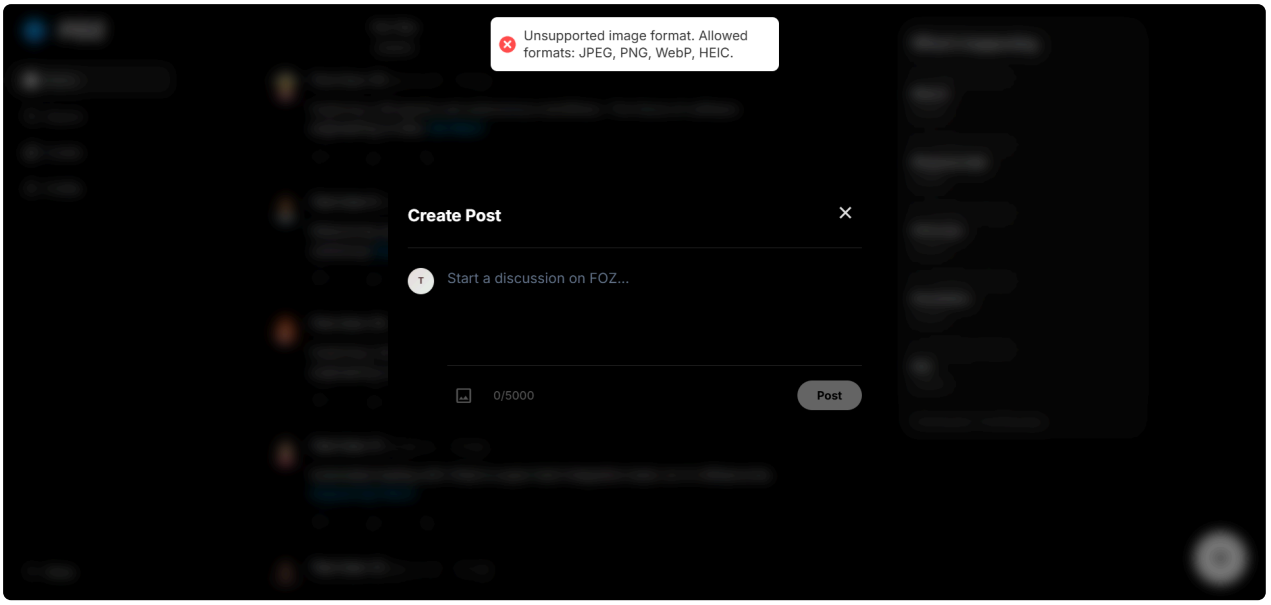

Create Post
5000/5000
Post


FOZ
Home
Search
Create
Profile
More
For You
Your post is live.
test1 @test1 · Just now
... Read more
Test User 20 @user_20 · 1d ago
Exploring LLM agents and autonomous workflows. The future of software engineering is here. #ai #tech
Test User 6 @user_6 · 1d ago
Refactoring legacy code into clean, modular TypeScript ESM modules. So satisfying! #typescript #opensource
Test User 28 @user_28 · 3d ago
Exploring LLM agents and autonomous workflows. The future of software engineering is here. #ai #tech
What's happening
1 · Trending in Forum
#tech
8 posts
2 · Trending in Forum
#typescript
5 posts
3 · Trending in Forum
#design
3 posts
4 · Trending in Forum
#webdev
3 posts
5 · Trending in Forum
#ai
2 posts
Showing top 5 trending tags

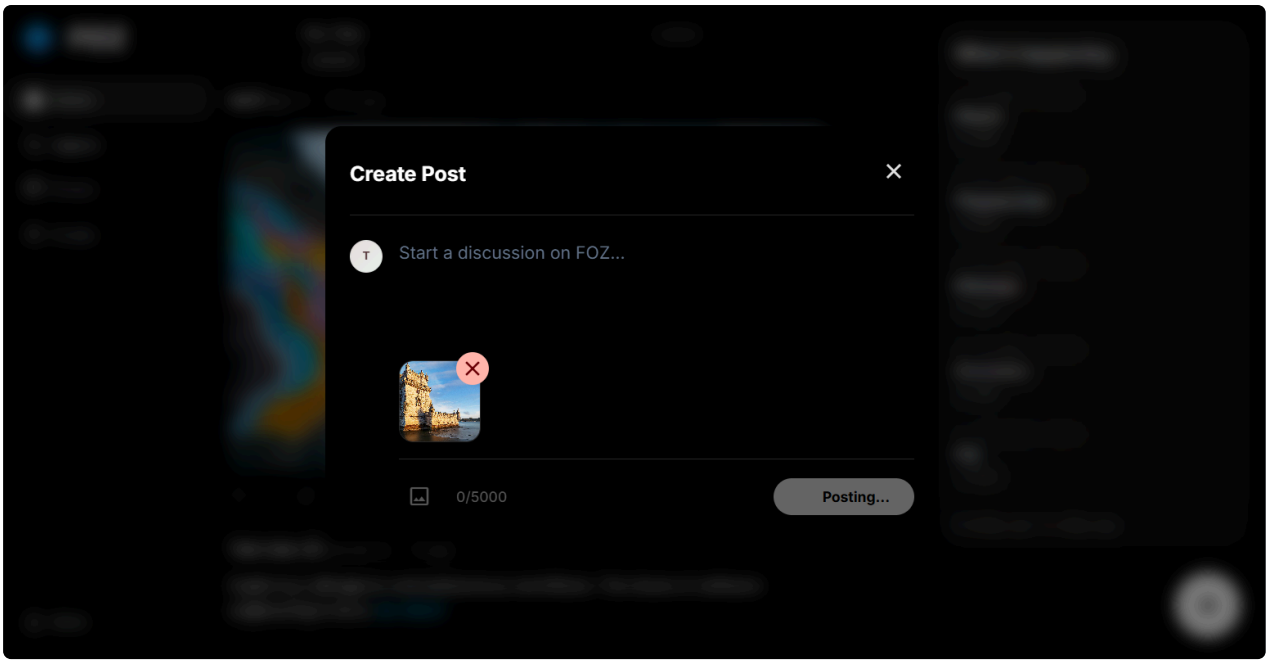
- Screen 2 (Alternative Flow 2 - Content Validation Error):

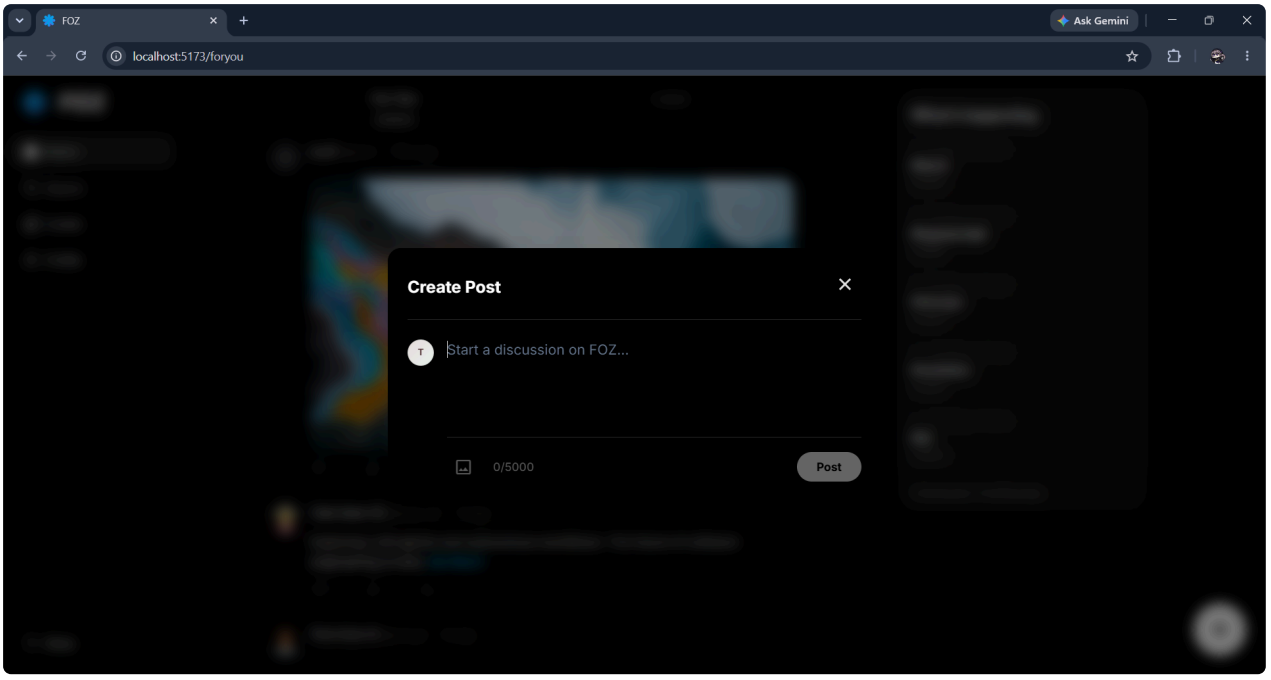


- Screen 3 (Alternative Flow 3 - Media Attachment Error):

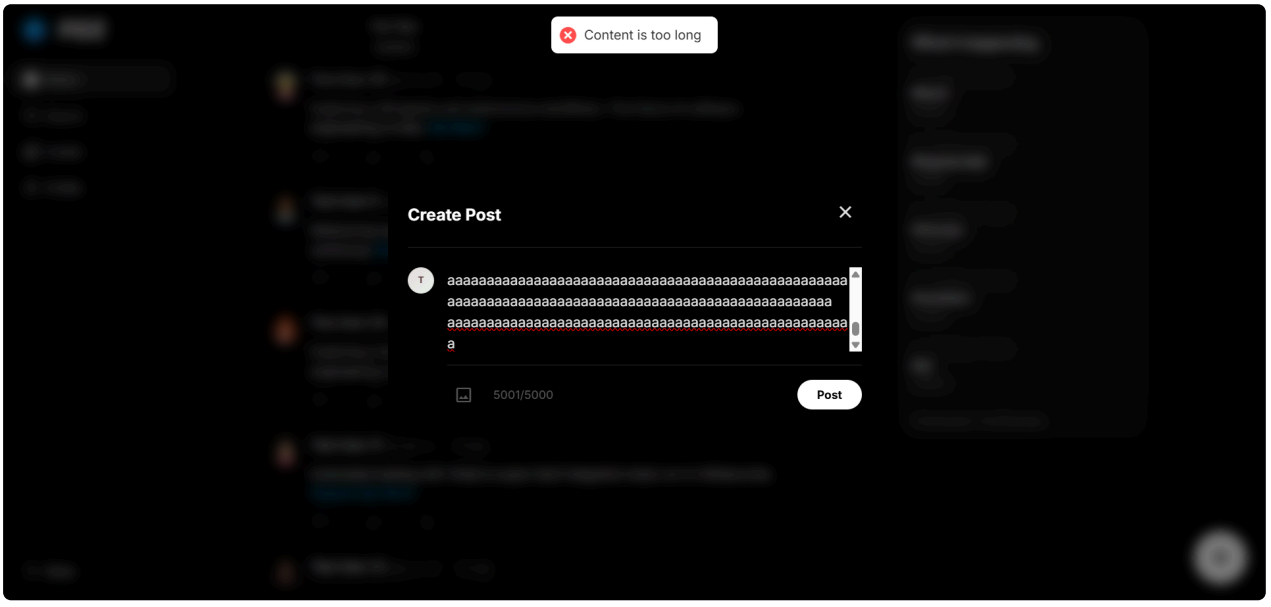


- Screen 4 (Alternative Flow 4 - Media Attachment Cancelled):

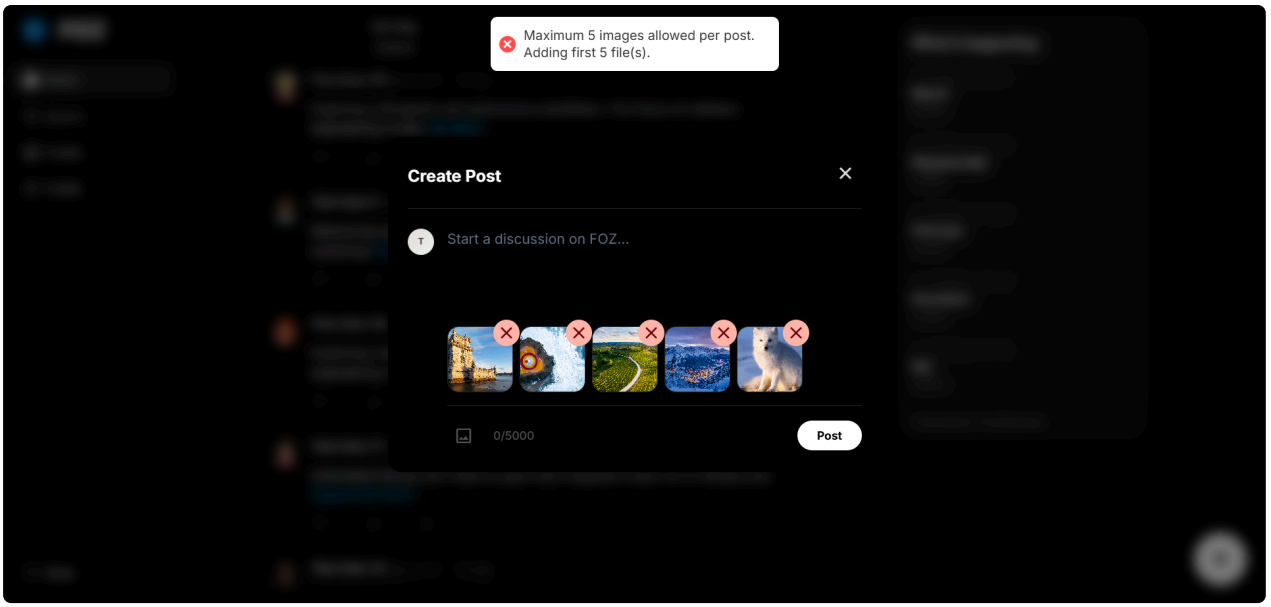




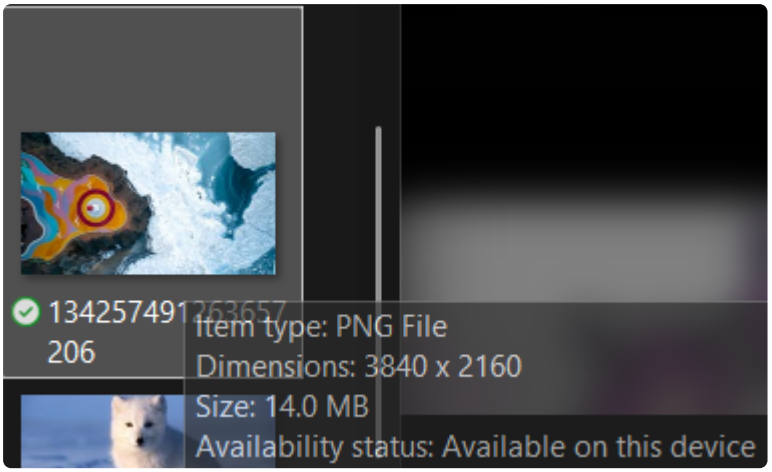
- Screen 5 (Alternative Flow 5 - Exceeds Characters Limit):

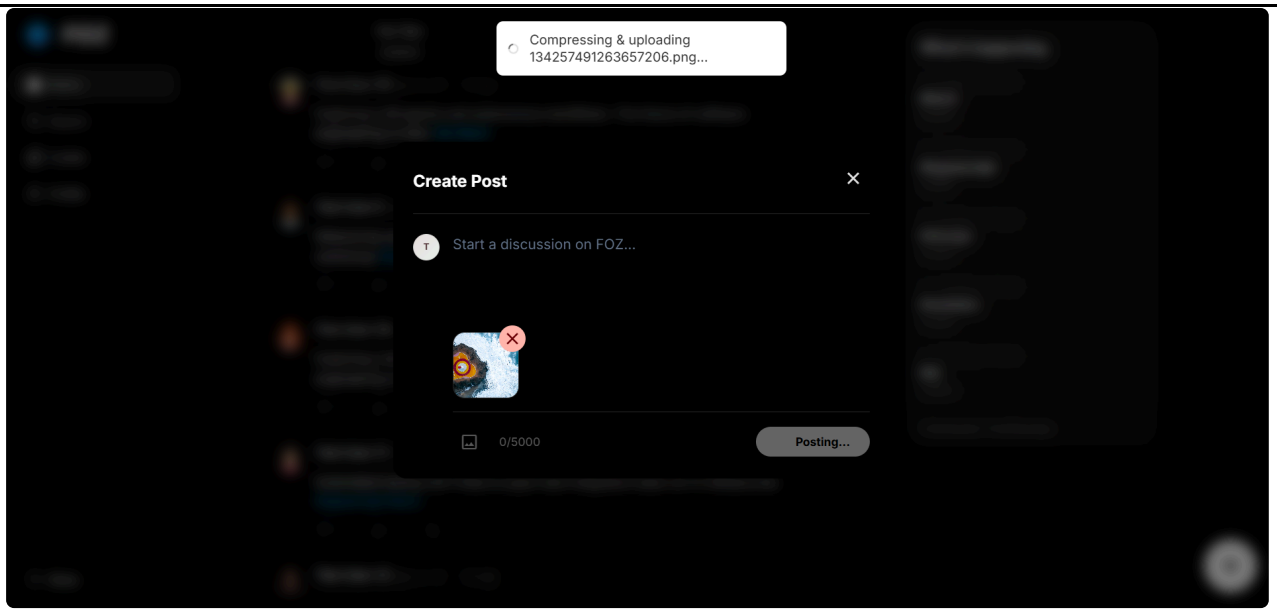

Content is too long
Create Post
aaaa

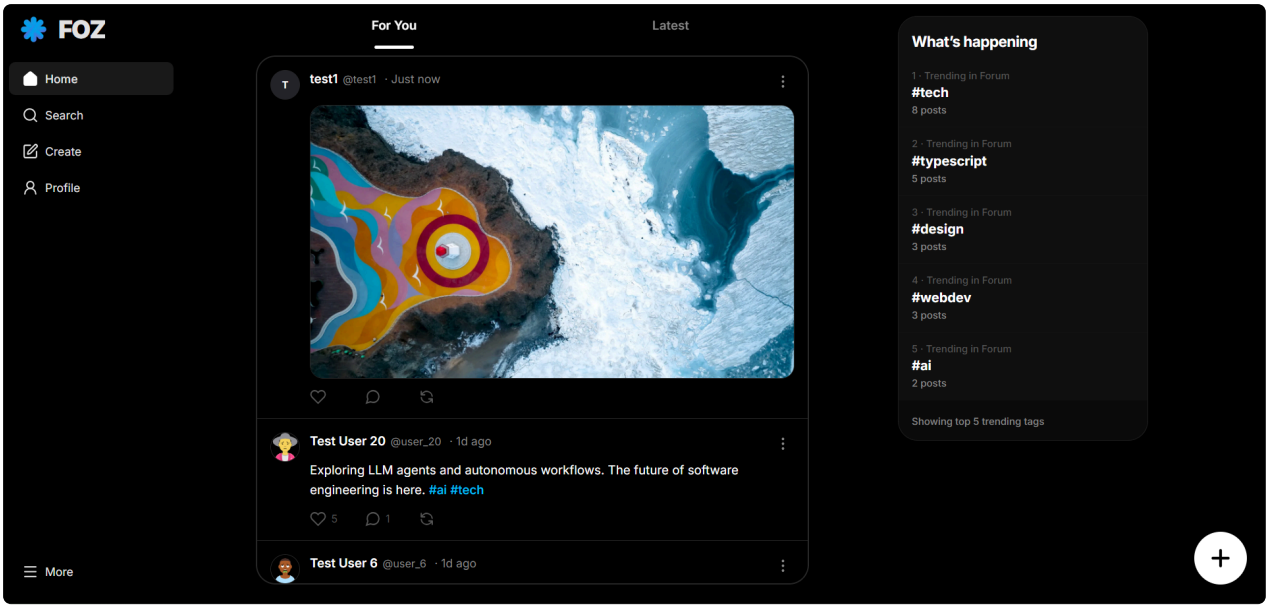
- **Screen 6 (Alternative Flow 6 - Exceeds Media Attachments Limit):**



- **Screen 7 (Alternative Flow 7 - Valid Image with Large Size):**







- Screen 8 (Alternative Flow 8 - Network Timeout after Attaching Any Image):

Use-Case Specification: Interact with Post

Project Name: FOZ
Version: 1.0
Document identifier: FOZ-UCS-11

1. Interact with Post

- **Use-Case ID:** UC-11
- **Actor(s):** Standard User

1.1 Brief Description

Acts as the generalized use case for users engaging with a post via various actions.

2. Preconditions

- The user is authenticated. Post is active.

3. Flow of Events

3.1 Basic Flow

1. The use case begins when an authenticated user opens a post.
2. The system presents interaction buttons (Like, Reply, Report, Repost).
3. The user selects an interaction option.
4. The system triggers the specific extended workflow.

3.2 Alternative Flows

3.2.1. The user selects **Reply**.

- The use case is extended by **UC-12: Leave Comment**.

3.2.2. The user selects **Like**.

- The use case is extended by **UC-13: Like/Unlike**.

3.2.3. The user selects **Report**.

- The use case is extended by **UC-16: Report Inappropriate Content**.

4. Special Requirements

- None.

5. Postconditions

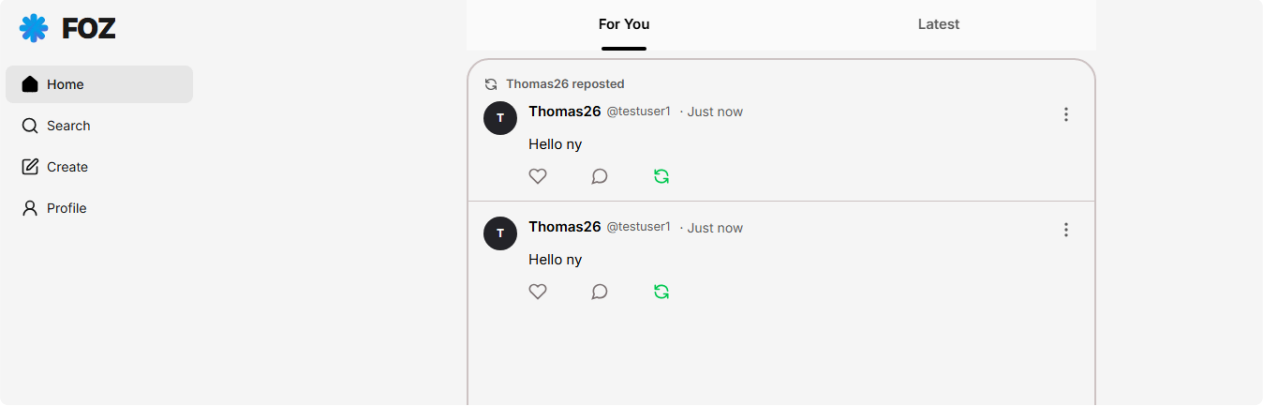
- None.

6. Extension Points

- **Leave Comment:** Extended via UC-12.
- **Like/Unlike:** Extended via UC-13.
- **Report Inappropriate Content:** Extended via UC-16.

7. Prototype Requirement

- Screen 1 (Basic Flow):



Use-Case Specification: Leave Comment

Project Name: FOZ
Version: 1.2
Document identifier: FOZ-UCS-12

1. Leave Comment

- **Use-Case ID:** UC-12
- **Actor(s):** Standard User

1.1 Brief Description

Allows an authenticated user to reply to a post or to another comment.

2. Preconditions

- User is authenticated.

3. Flow of Events

3.1 Basic Flow

1. Extends UC-11 when the user clicks "Reply" or the post.
2. The system opens a small text input field.
3. The user types their comment and clicks "Post Reply".
4. The system validates the text.
5. The system inserts a record into the `comments` table, mapping `post_id` and optionally `parent_comment_id`.
6. The system updates the UI to show the new comment inline.

3.2 Alternative Flows

3.2.1 Empty Comment

1. If at step 4, the comment is empty or contains only whitespace.
2. The system disables the "Post Reply" button
3. The flow resumes at step 3.

3.2.2 Comment Exceeds Maximum Length

1. If at step 4, the submitted comment content exceeds the 5,000-character limit.
2. The system rejects the submission and displays a validation error message ("Content is too long").
3. The user modifies the comment body to meet the character requirement.
4. The flow resumes at step 3.

3.2.3 Long Comment Display

1. When displaying a comment that exceeds 240 characters or 4 lines of text.
2. The system automatically truncates the body text preview and appends a "Read more" toggle action.
3. The user clicks "Read more" to expand and view the full comment content.

3.2.4 Newline Input with Shift+Enter

1. If at step 3, the user presses **Shift+Enter** while typing their comment.
2. The system inserts a newline into the comment text instead of submitting the comment.
3. The flow resumes at step 3.

3.2.5 Duplicate Submission Prevention

1. If at step 3, the user submits the comment multiple times in rapid succession (e.g., pressing Enter or the send button repeatedly).
2. The system disables the send button or otherwise ignores duplicate submissions while the first request is still pending.
3. Only one comment record is created; the flow resumes at step 5.

3.2.6 Malicious Script Injection (XSS) Attempt

1. If at step 4, the submitted comment contains malicious markup such as `<script>alert('xss')</script>`.
2. The system sanitizes the input and renders it as plain text; no script executes.
3. The comment remains safely displayed after a page reload; the flow resumes at step 5.

3.2.7 Emoji and Unicode Input

1. If at step 4, the submitted comment contains emojis or text typed from the keyboard.

2. The system accepts and renders the emojis exactly as typed.
3. The flow resumes at step 5.

3.2.8 Nested Reply

1. If at step 1, the user clicks "Reply" on an existing comment instead of on the post.
2. The system opens the reply input for that specific comment.
3. The user types the reply (e.g., "Nested standard reply") and submits it.
4. The system inserts the record into the `comments` table with `parent_comment_id` pointing to the selected comment.
5. The system displays the reply under the selected parent with the correct indentation and reply count, not as a separate top-level comment.
6. The flow resumes at step 6.

3.2.9 Offline Submission

1. If at step 3, the user submits the comment while their device has no internet connection.
2. The request fails to reach the server.
3. The system displays a toast notification notifying the user they are offline.
4. The typed comment is preserved in the input box; the user can retry once the connection is restored.
5. The flow resumes at step 3.

3.2.10 Multi-Level Nested Replies (Depth 1–2)

1. If at step 1, the user replies to a reply (i.e., a comment already nested one level deep).
2. The system displays replies at depth 1 and 2 indented to the right.
3. The replies are clearly indicated with line markers, and a "Hide replies" button connects to the parent comment of the replies.
4. The flow resumes at step 6.

3.2.11 Deep Nested Replies (Depth > 2)

1. If at step 1, the user replies to a comment at depth greater than 2.
2. The system no longer indents the reply further to the right.
3. The reply stays below its parent, and below it a "Hide replies" button is shown.
4. The flow resumes at step 6.

3.2.12 URL in Comment

1. If at step 4, the submitted comment contains a valid URL (e.g., `https://example.com`).
2. The system accepts the comment and displays the URL as plain text (no auto-hyperlinking).
3. The flow resumes at step 5.

3.2.13 Comment Rate Limit Exceeded

1. If at step 3, the user submits 15–20 comments within a single minute.
2. The system displays a pop-up modal telling the user to slow down and prevents them from commenting for the next 10 seconds.
3. The use case ends.

3.2.14 Authentication Expired During Commenting

1. If at step 3, the user's authentication session expires while they are typing the comment.
2. The system displays a modal telling the user their session is expired and prompts them to log in again.
3. The use case ends; the typed draft is preserved so the user can continue after logging back in.

3.2.15 Post Deleted While Commenting

1. If at step 3, an administrator deletes the post while the user is typing a comment.
2. The system displays a modal telling the user that the post is no longer available.
3. The use case ends.

4. Special Requirements

- None.

5. Postconditions

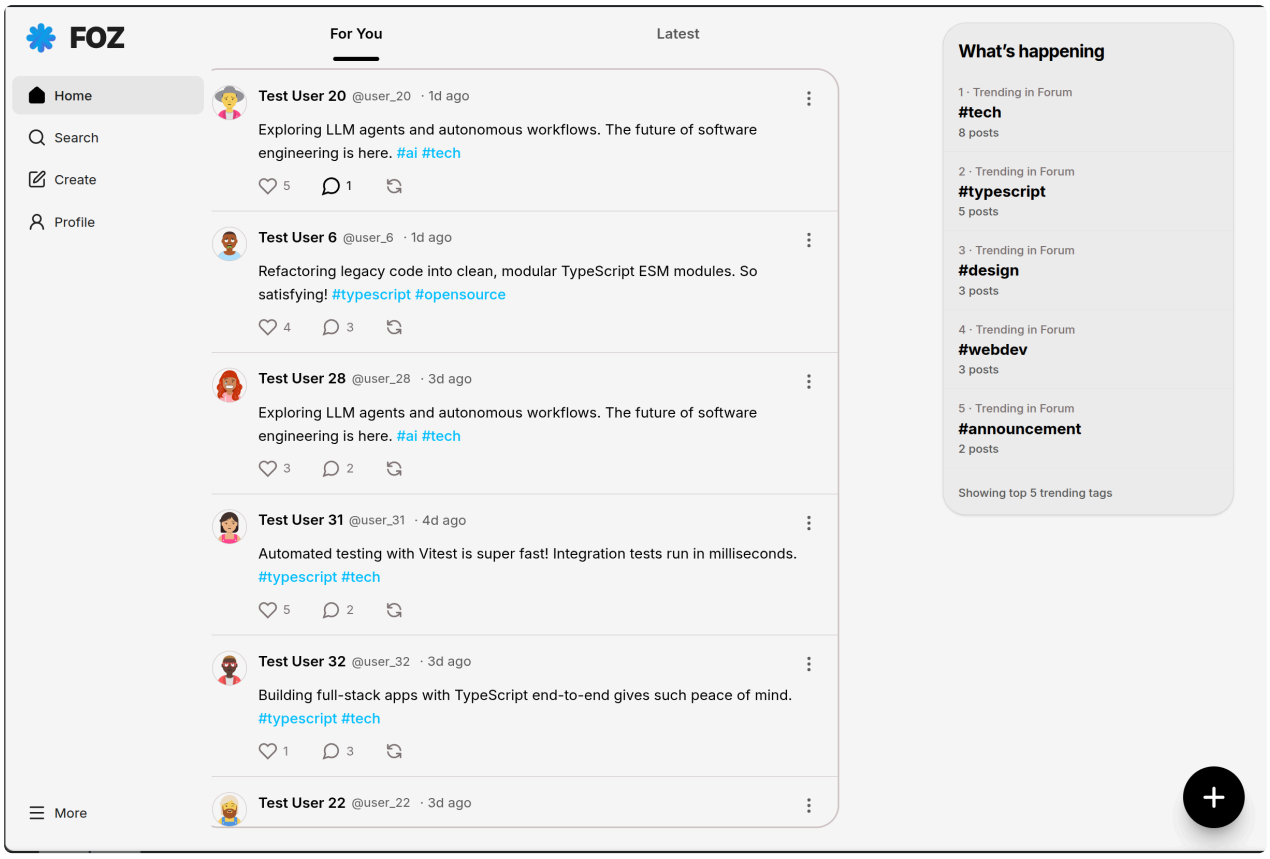
- A new comment is persisted to the database.

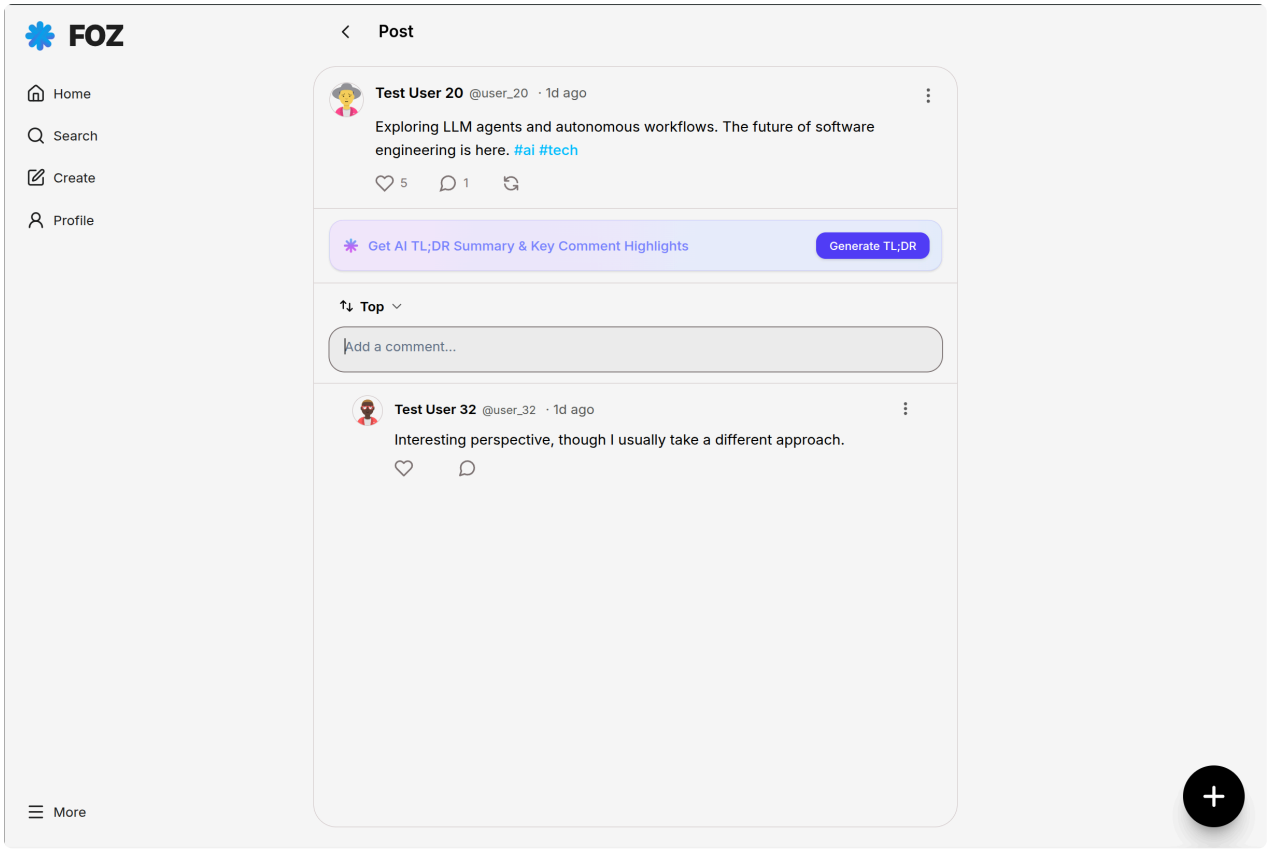
6. Extension Points

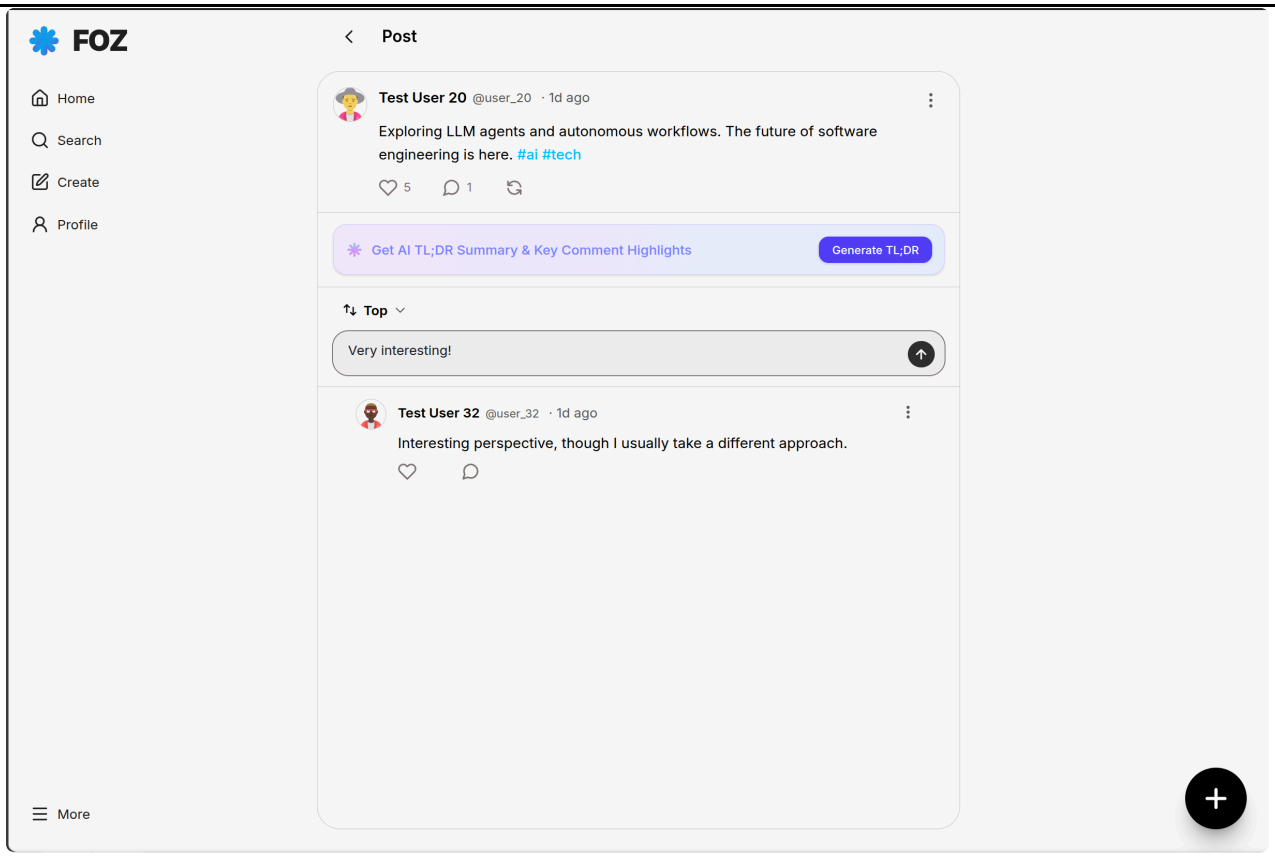
- None.

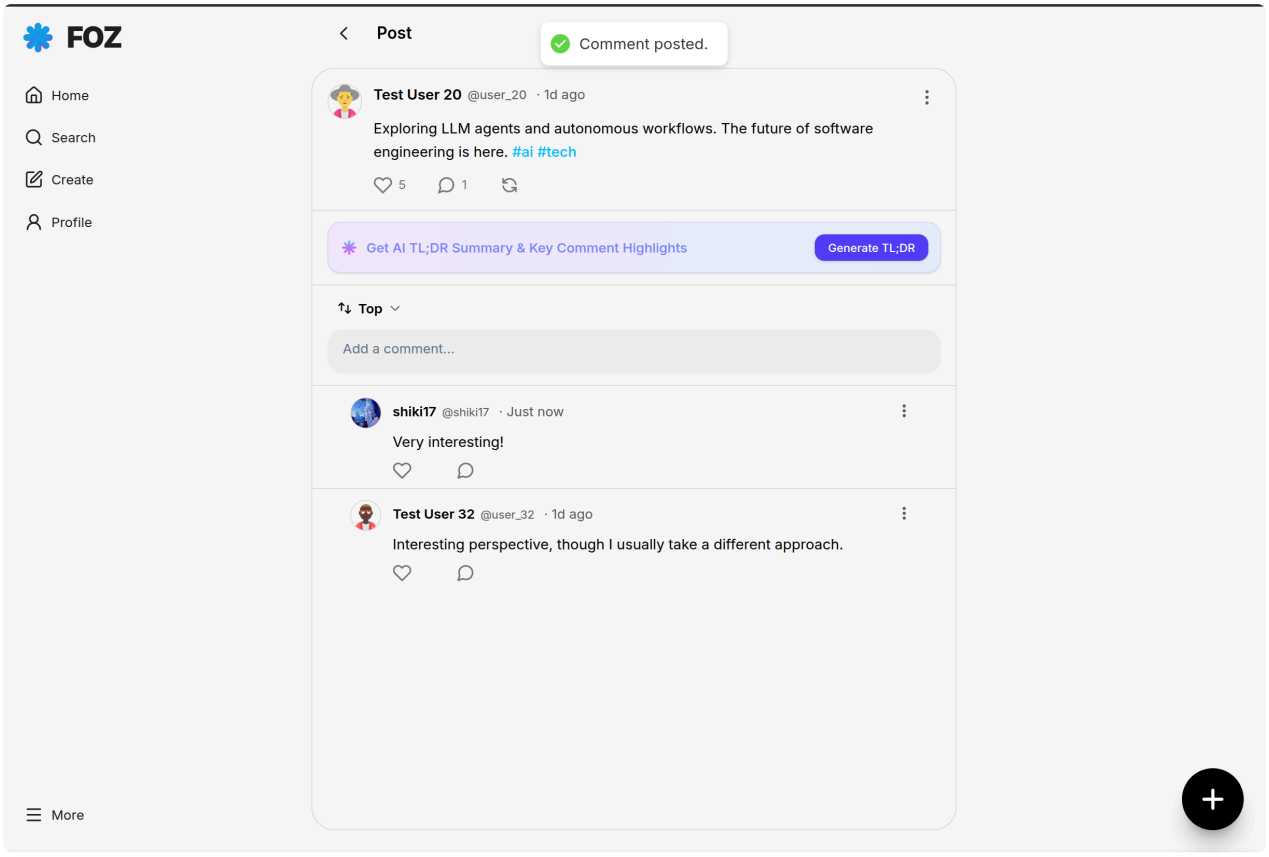
7. Prototype Requirement

- Screen 1 (Basic Flow):

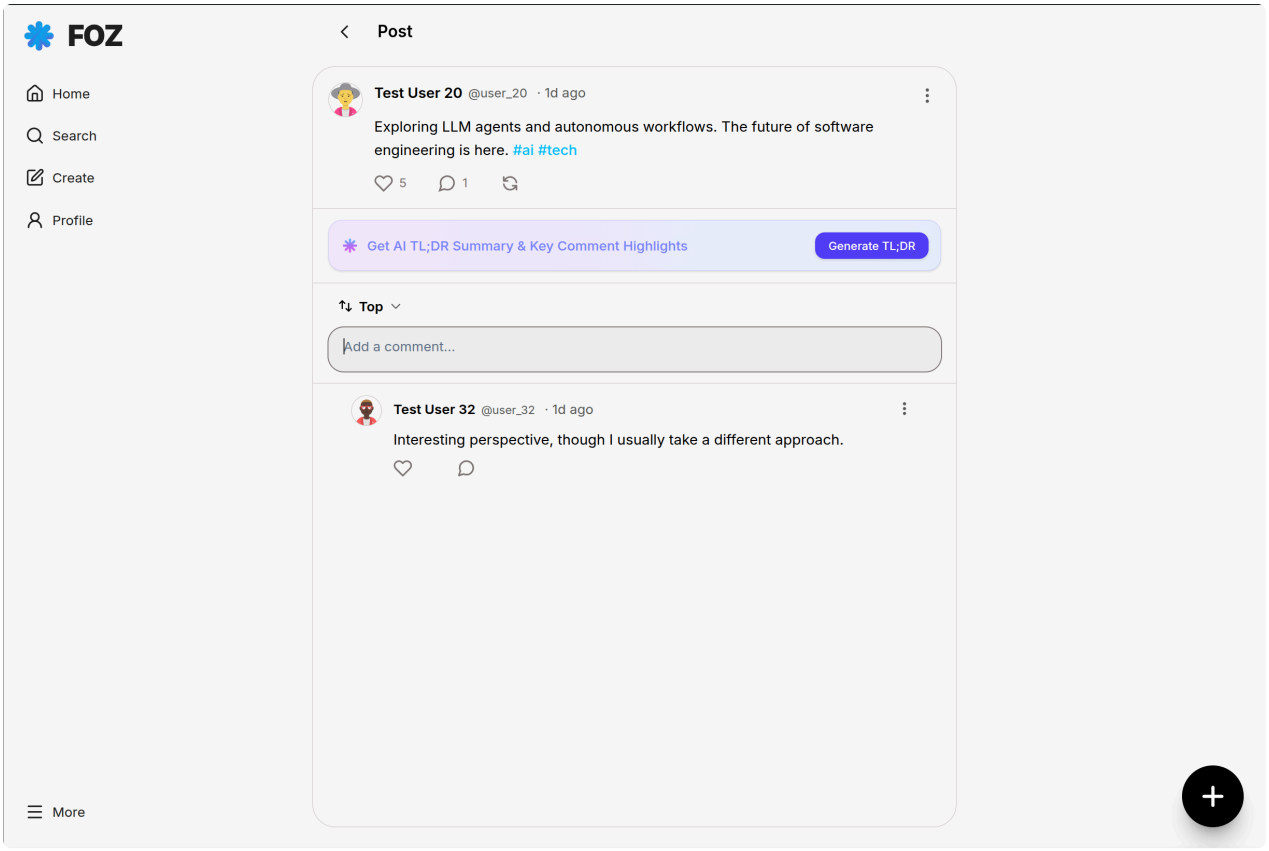




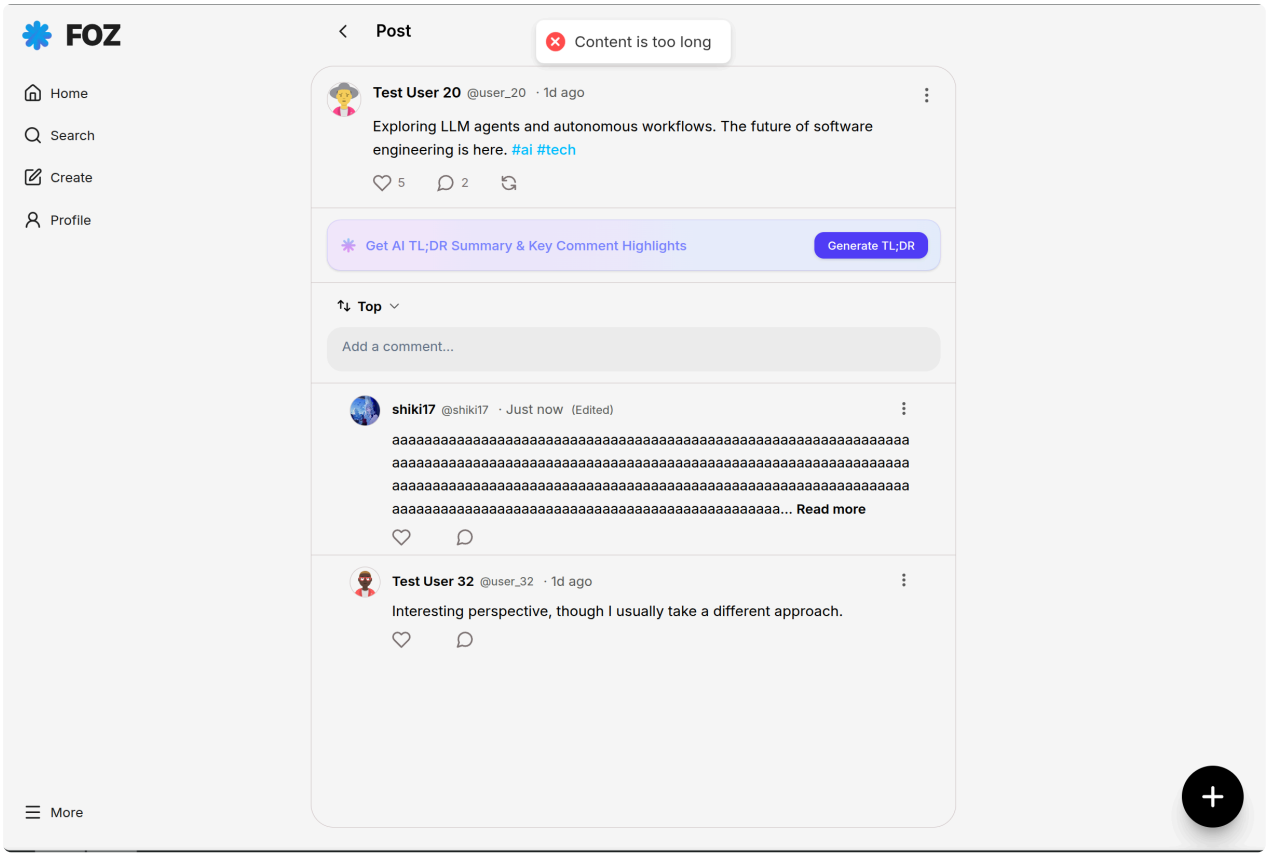

FOZ
Home
Search
Create
Profile
More
Post
Test User 20 @user_20 · 1d ago
Exploring LLM agents and autonomous workflows. The future of software engineering is here. #ai #tech
5
1
Get AI TL;DR Summary & Key Comment Highlights
Generate TL;DR
Top
Very interesting!
Test User 32 @user_32 · 1d ago
Interesting perspective, though I usually take a different approach.


FOZ
Home
Search
Create
Profile
More
Post
Comment posted.
Test User 20 @user_20 · 1d ago
Exploring LLM agents and autonomous workflows. The future of software engineering is here. #ai #tech
5
1
Get AI TL;DR Summary & Key Comment Highlights
Generate TL;DR
Top
Add a comment...
shiki17 @shiki17 · Just now
Very interesting!
Test User 32 @user_32 · 1d ago
Interesting perspective, though I usually take a different approach.

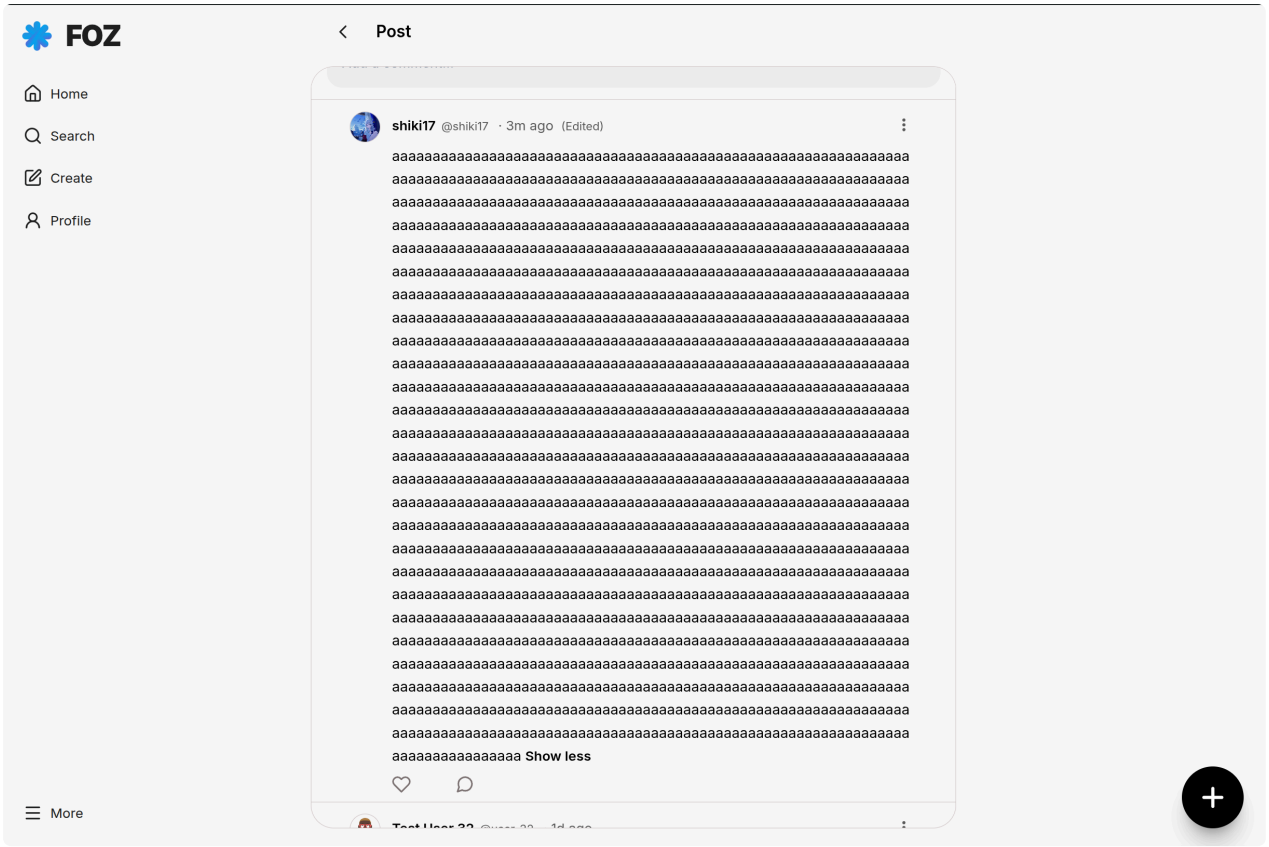
- Screen 2 (Alternative Flow 1 - Empty Comment):



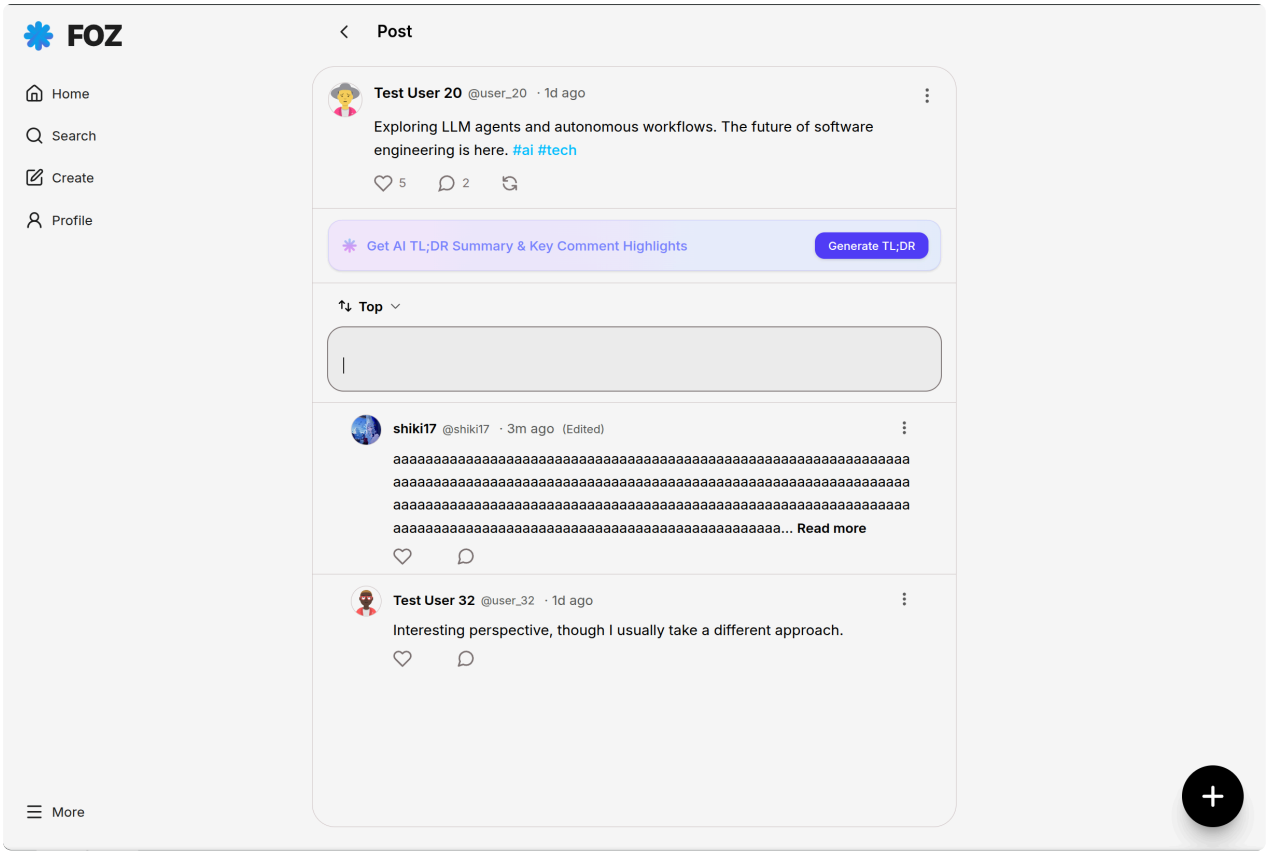
- Screen 3 (Alternative Flow 2 - Comment Exceeds Maximum Length):



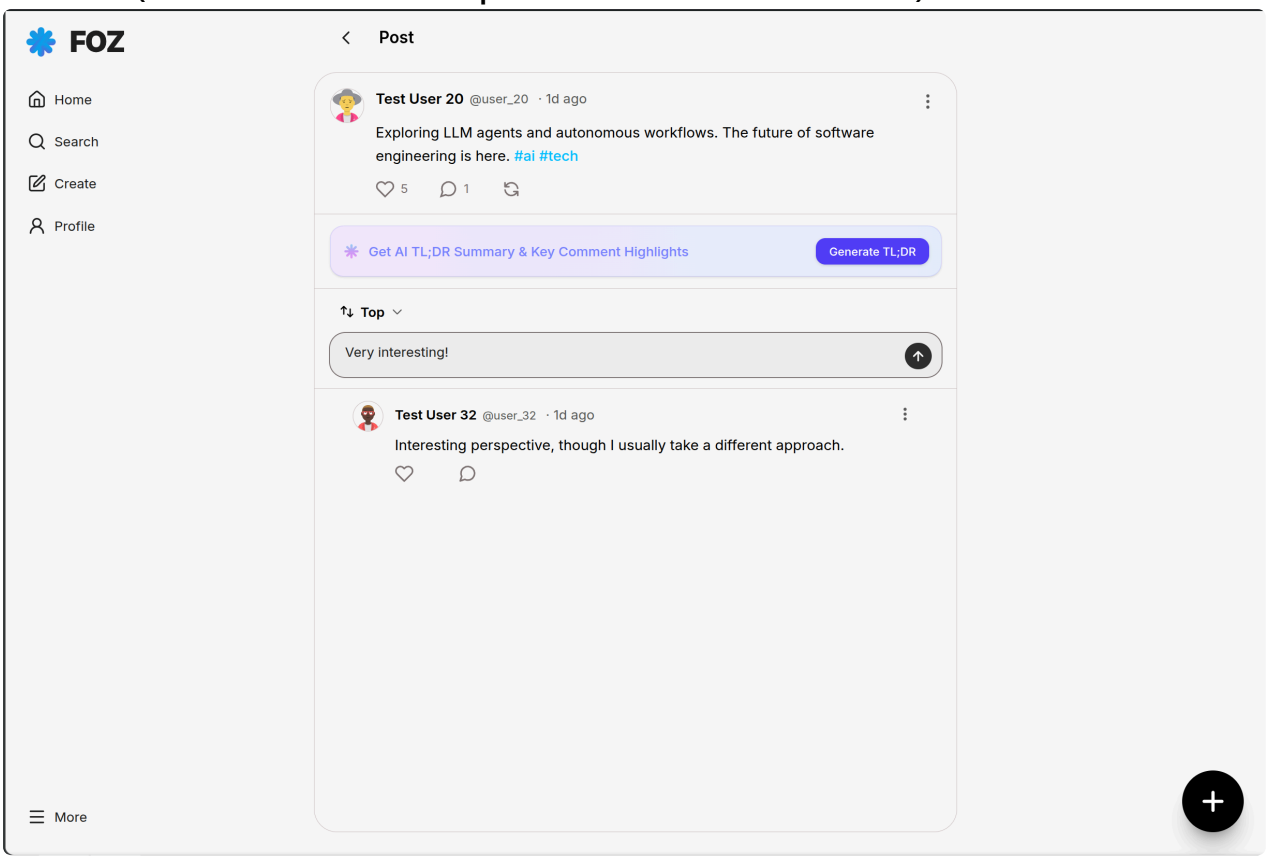
- Screen 4 (Alternative Flow 3 - Long Comment Display):

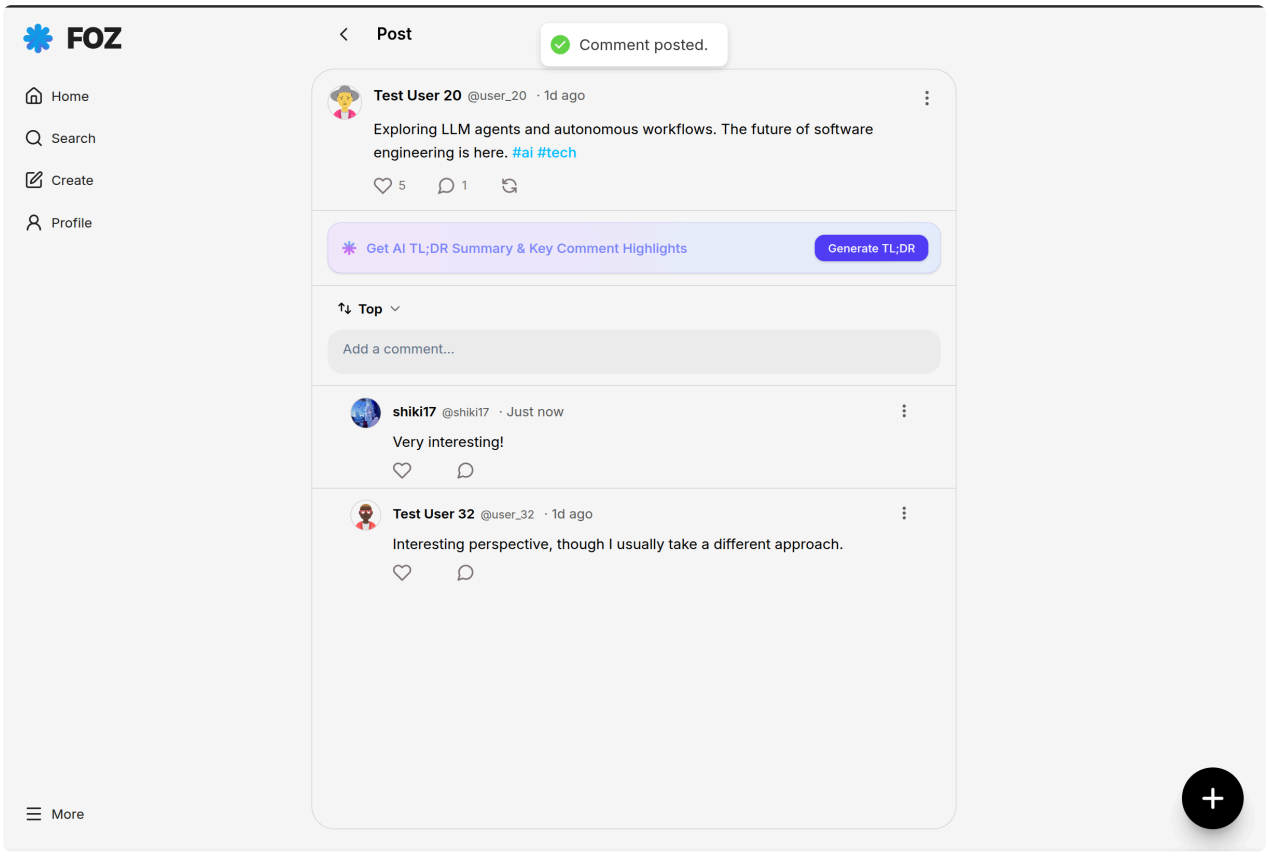


- Screen 5 (Alternative Flow 4 - Newline Input with Shift+Enter):

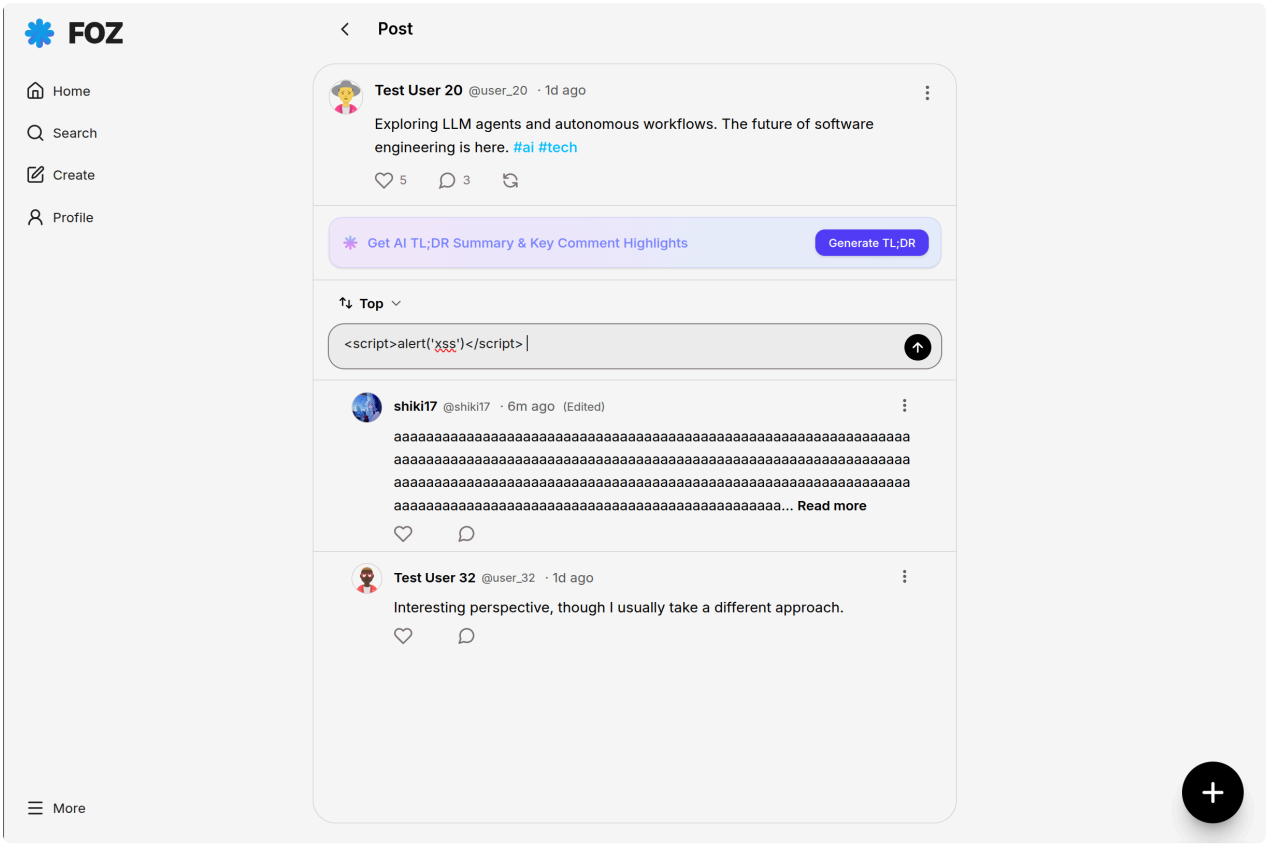


- ## Screen 6 (Alternative Flow 5 - Duplicate Submission Prevention):

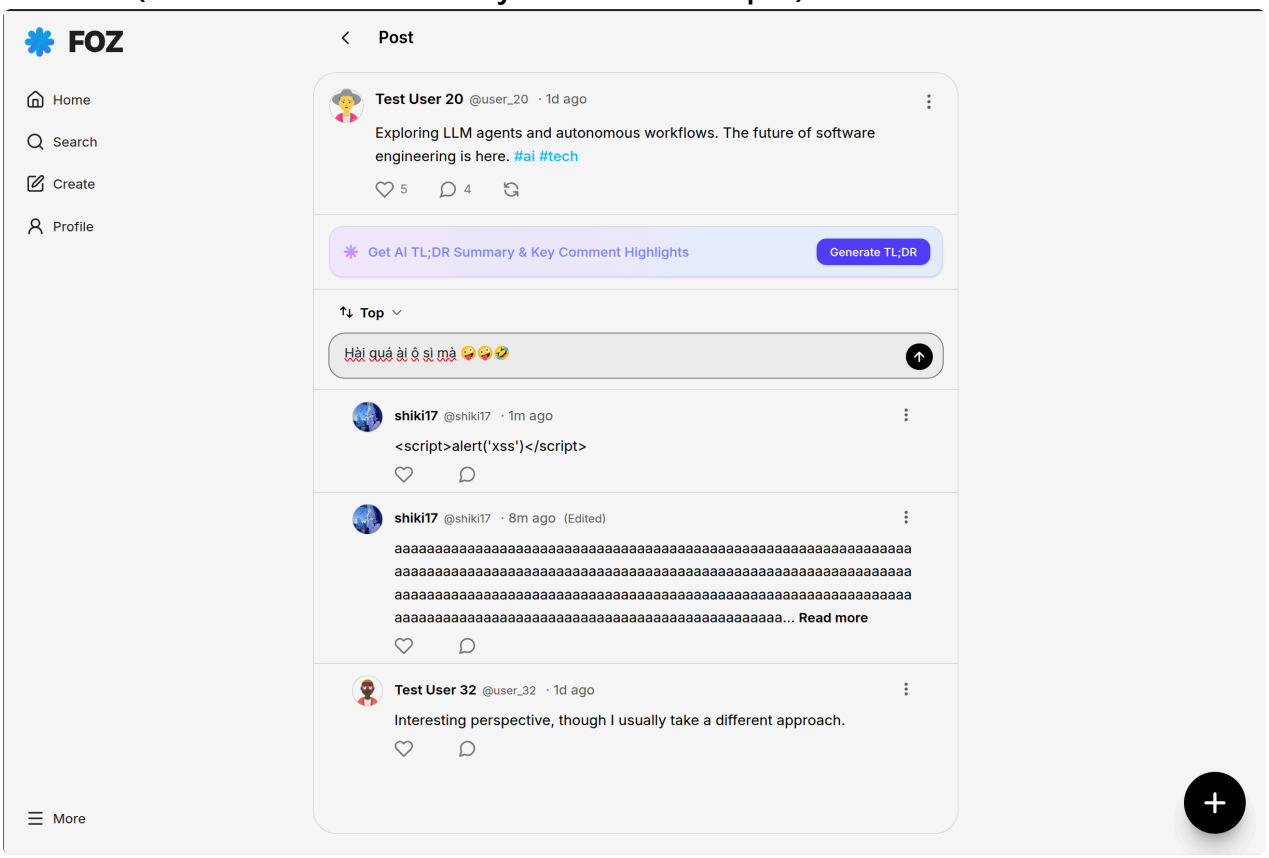




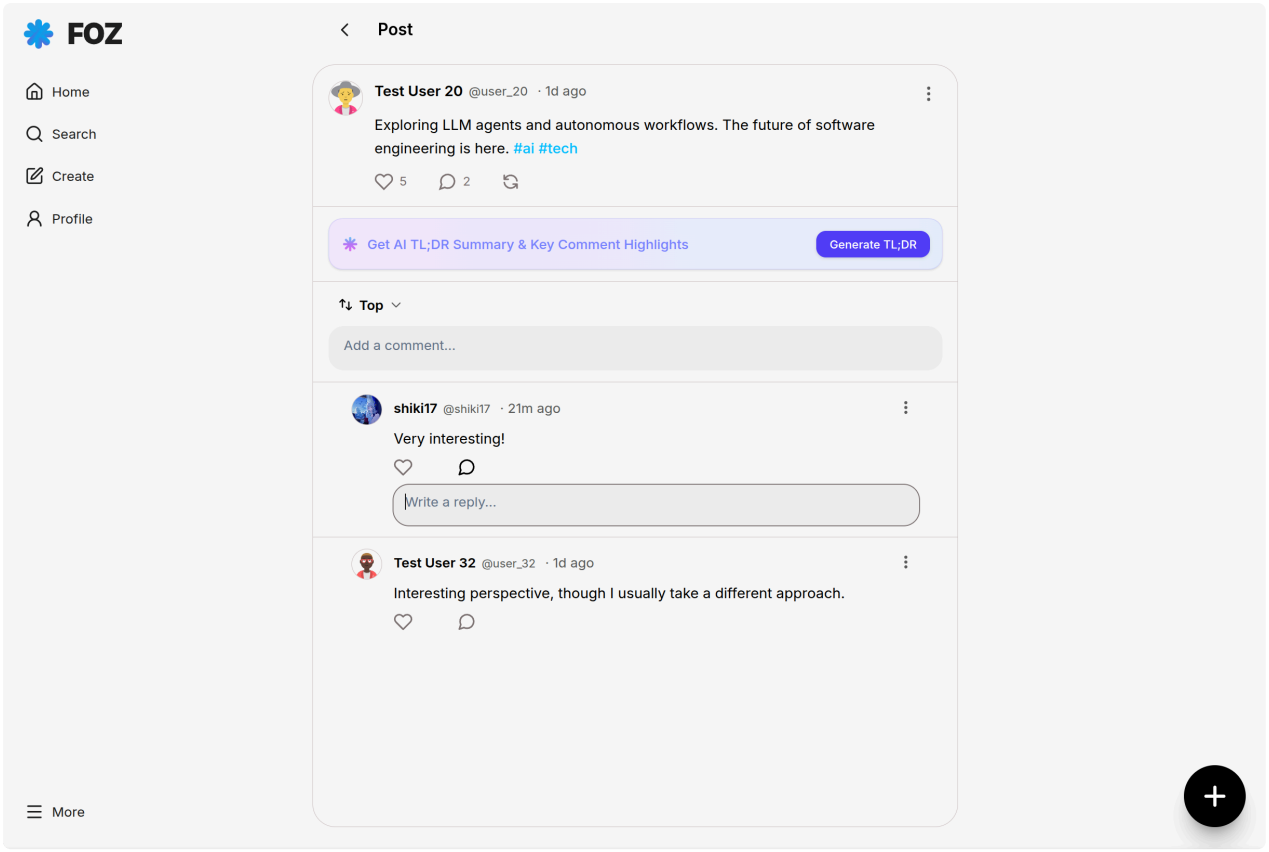
- ## Screen 7 (Alternative Flow 6 - Malicious Script Injection (XSS) Attempt):

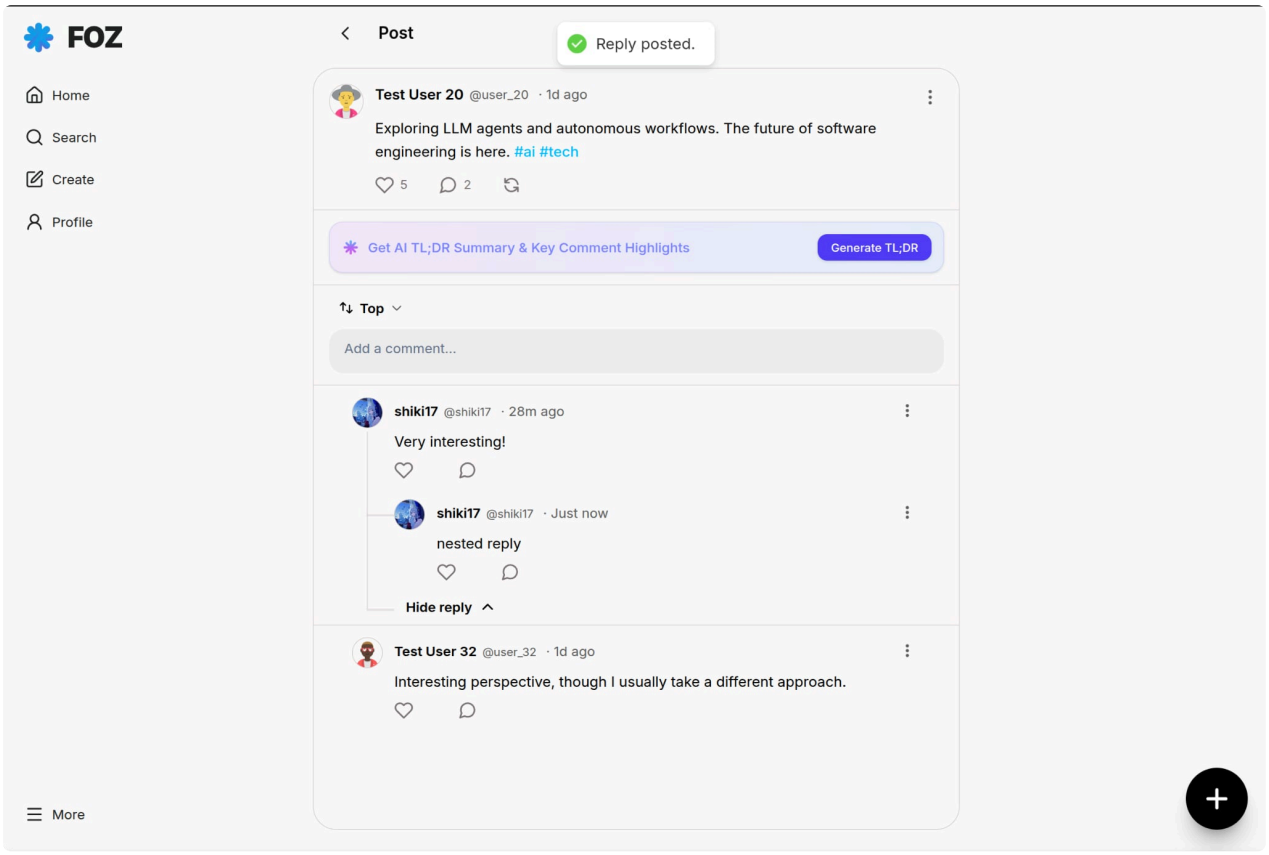

FOZ
Home
Search
Create
Profile
More
Post
Test User 20 @user_20 · 1d ago
Exploring LLM agents and autonomous workflows. The future of software engineering is here. #ai #tech
5
3
Get AI TL;DR Summary & Key Comment Highlights
Generate TL;DR
Top
<script>alert('xss')</script>
shiki17 @shiki17 · 6m ago (Edited)
aaaa

- ## Screen 8 (Alternative Flow 7 - Emoji and Unicode Input):

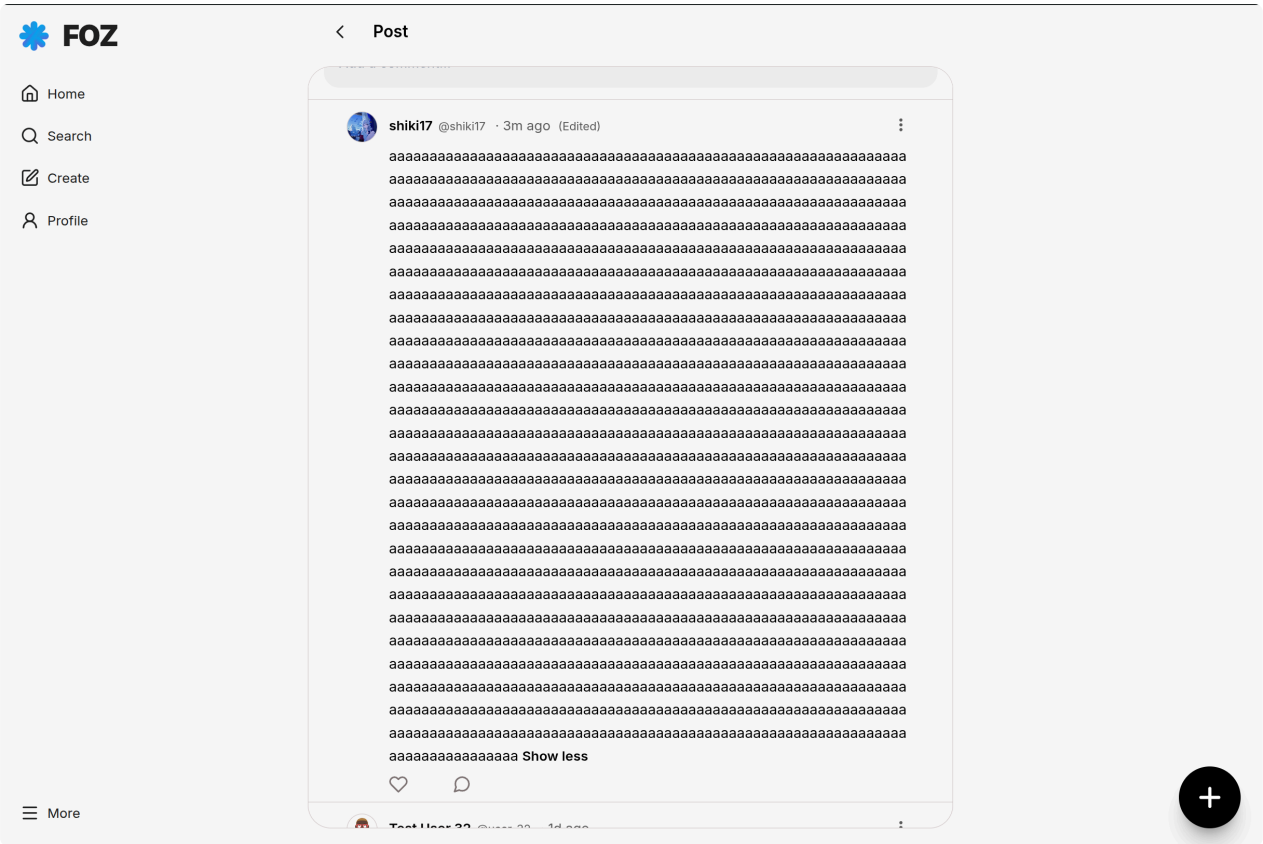

FOZ
Home
Search
Create
Profile
More
Post
Test User 20 @user_20 · 1d ago
Exploring LLM agents and autonomous workflows. The future of software engineering is here. #ai #tech
5
4
Get AI TL;DR Summary & Key Comment Highlights
Generate TL;DR
Top
Hài quá ài ố sì mà
shiki17 @shiki17 · 1m ago
<script>alert('xss')</script>
shiki17 @shiki17 · 8m ago (Edited)
aaaa

- ## Screen 9 (Alternative Flow 8 - Nested Reply):

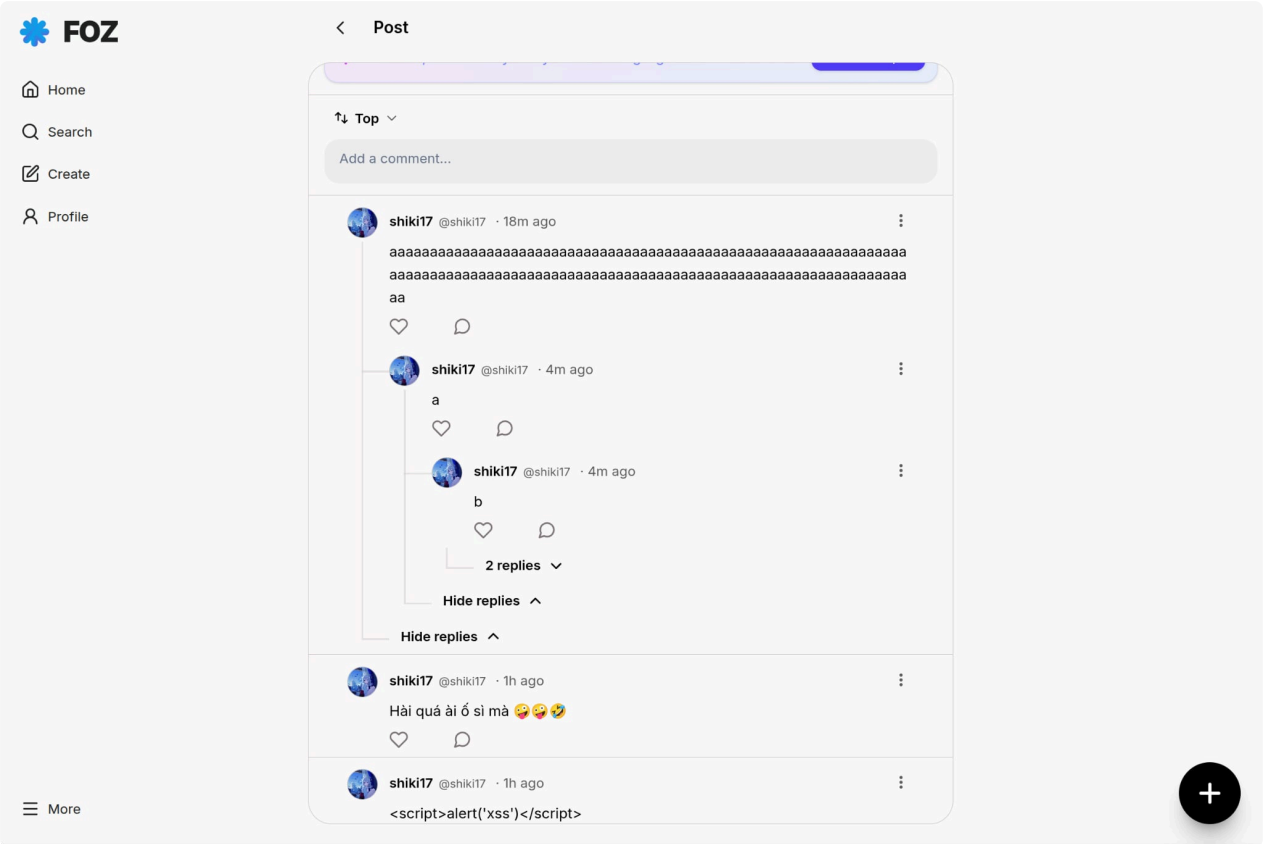




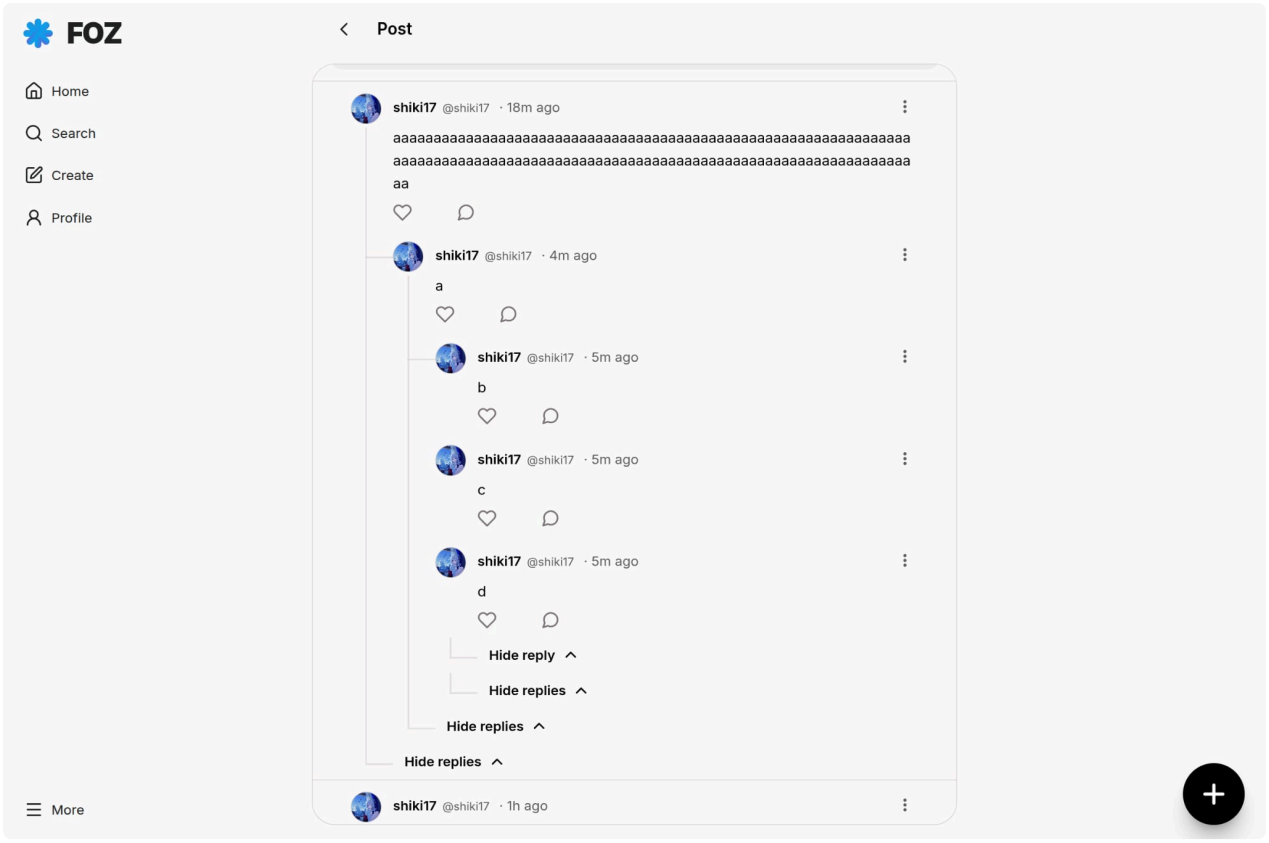
- Screen 10 (Alternative Flow 9 - Offline Submission):



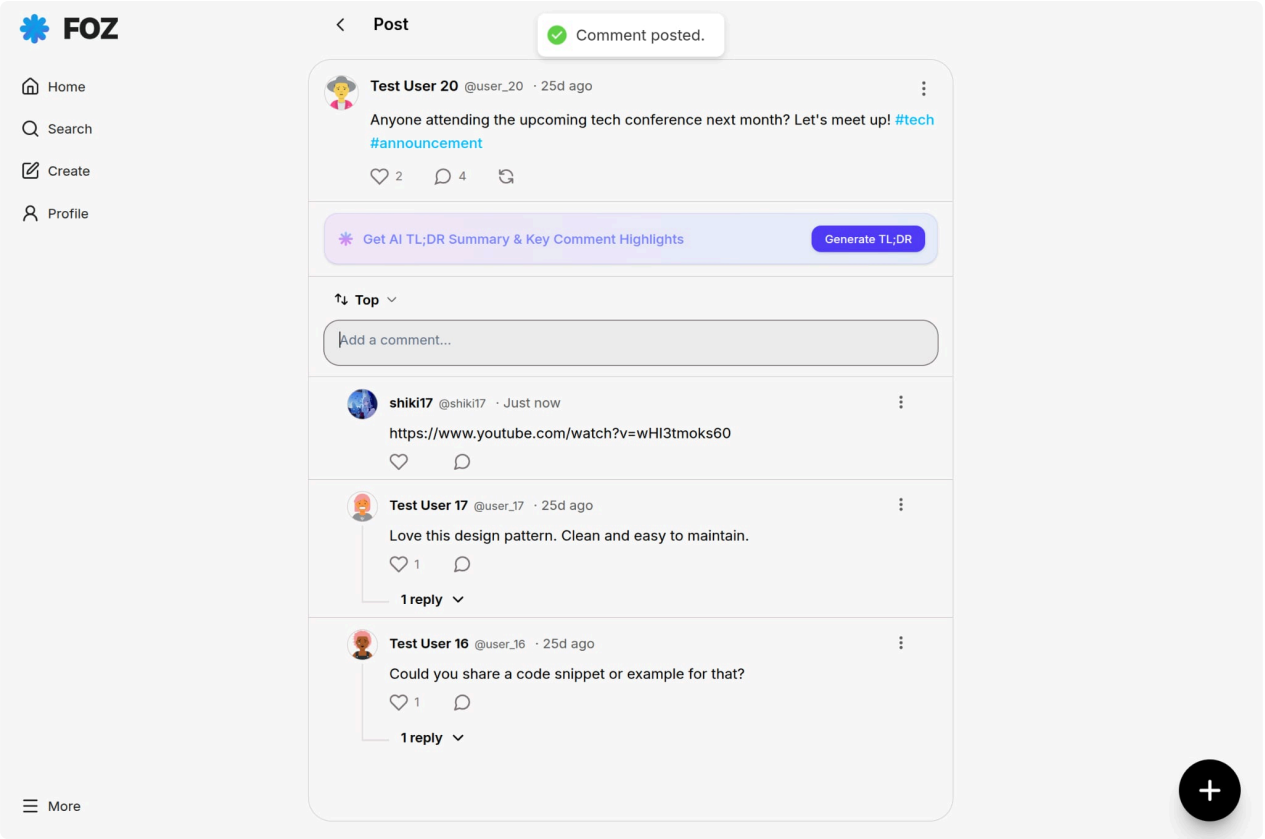
- Screen 11 (Alternative Flow 10 - Multi-Level Nested Replies (Depth 1–2)):



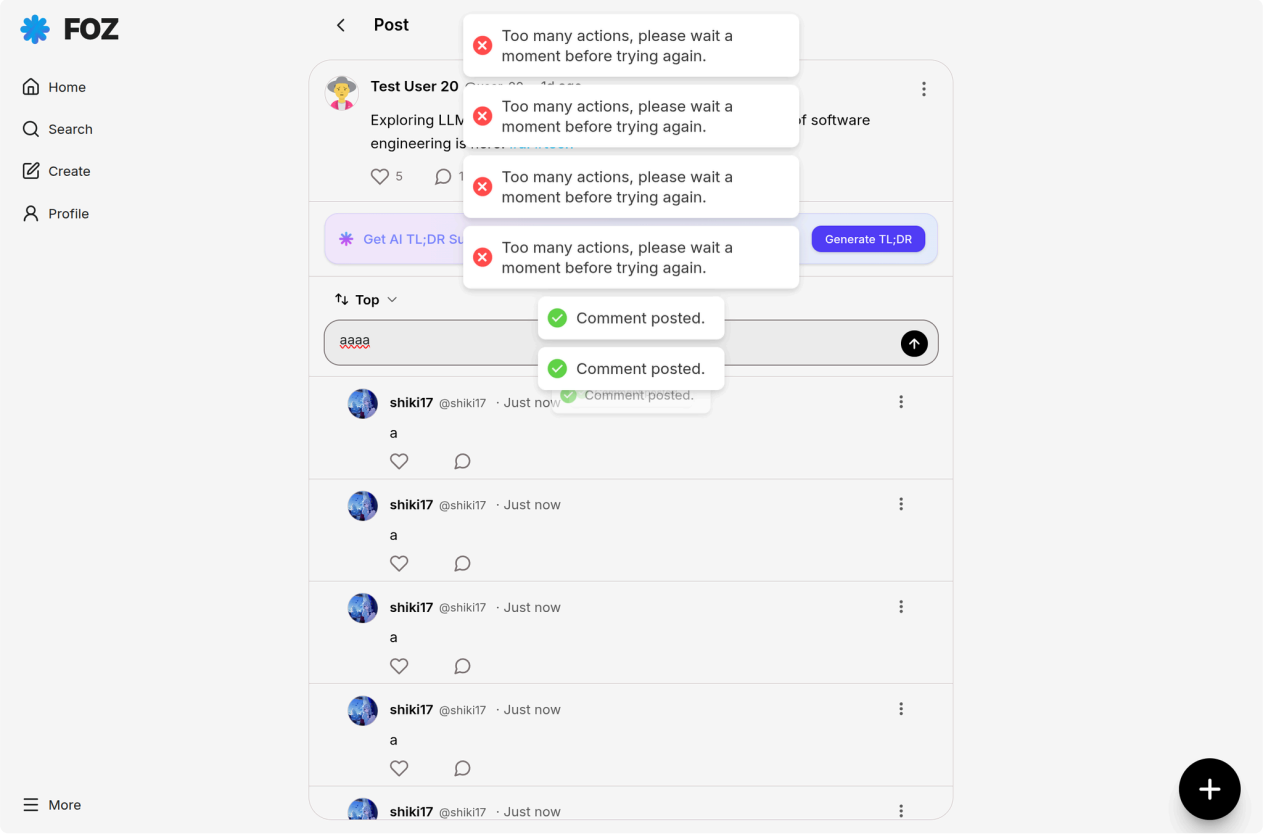
- Screen 12 (Alternative Flow 11 - Deep Nested Replies (Depth > 2)):


FOZ
Home
Search
Create
Profile
More
Post
shiki17 @shiki17 · 18m ago
aaaa

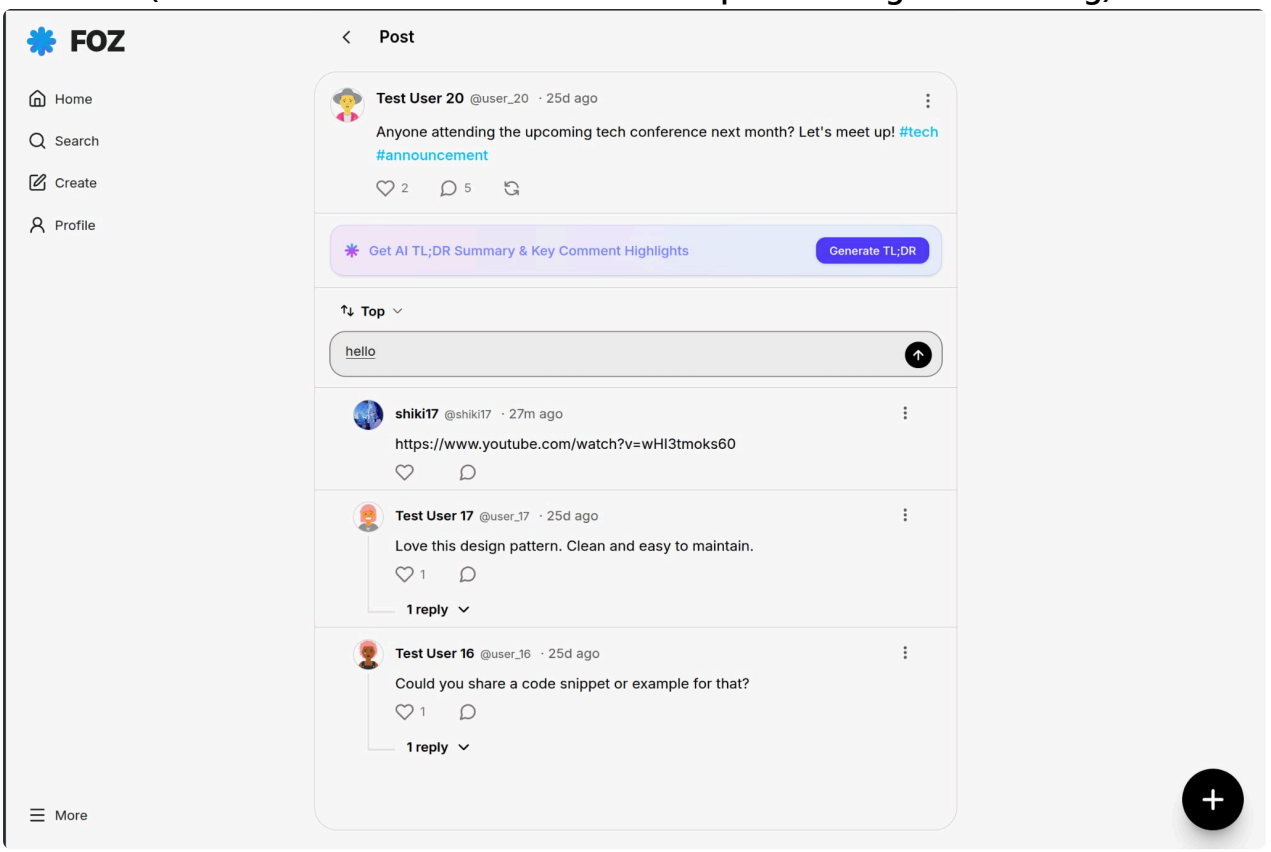
- Screen 13 (Alternative Flow 12 - URL in Comment):

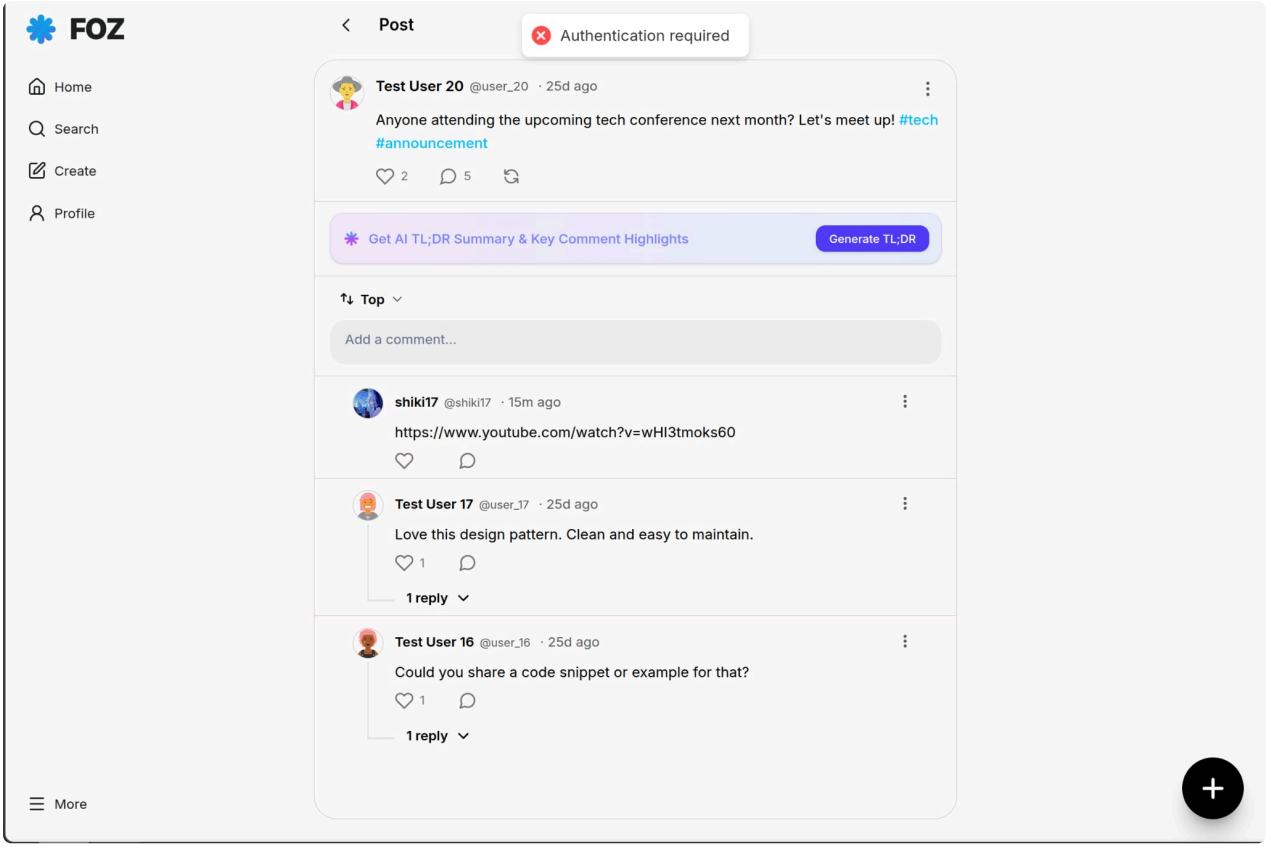


- Screen 14 (Alternative Flow 13 - Comment Rate Limit Exceeded):

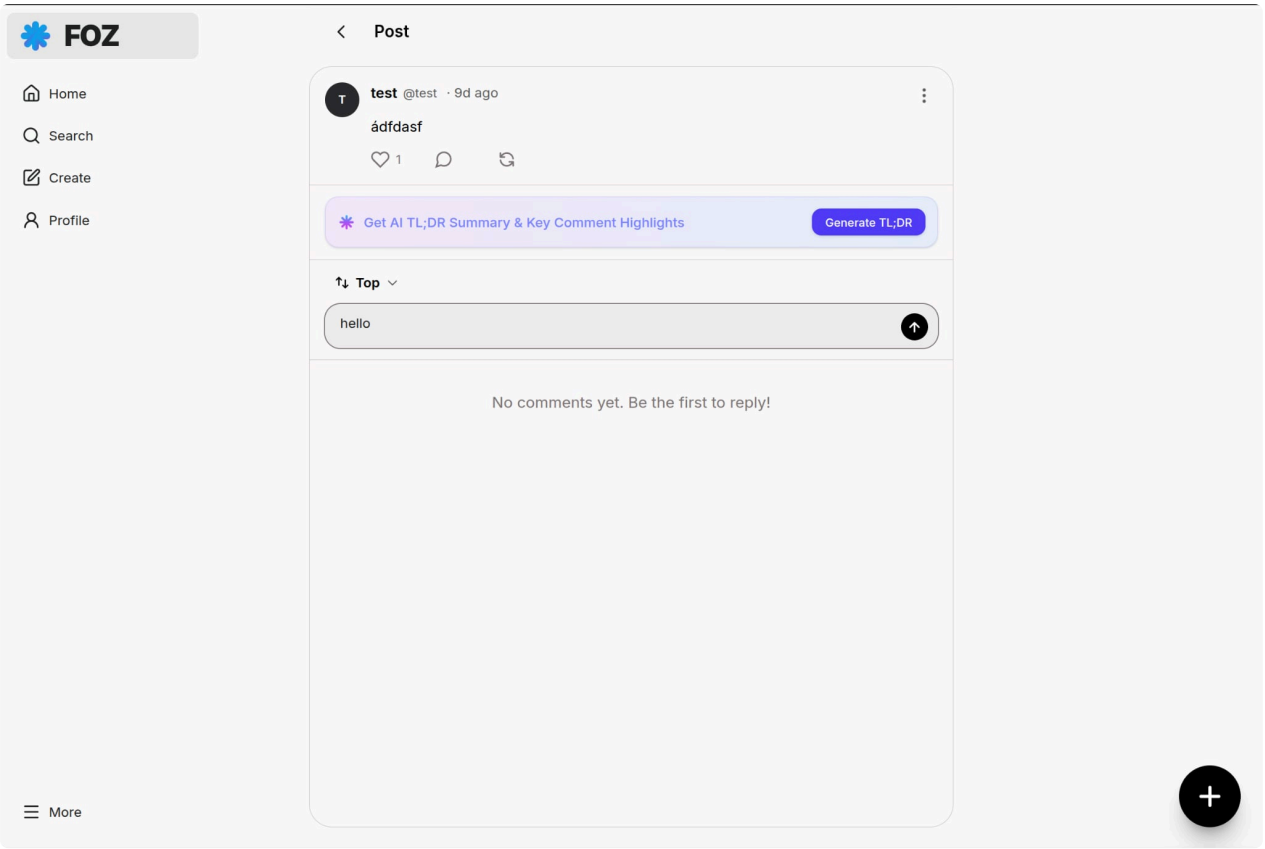


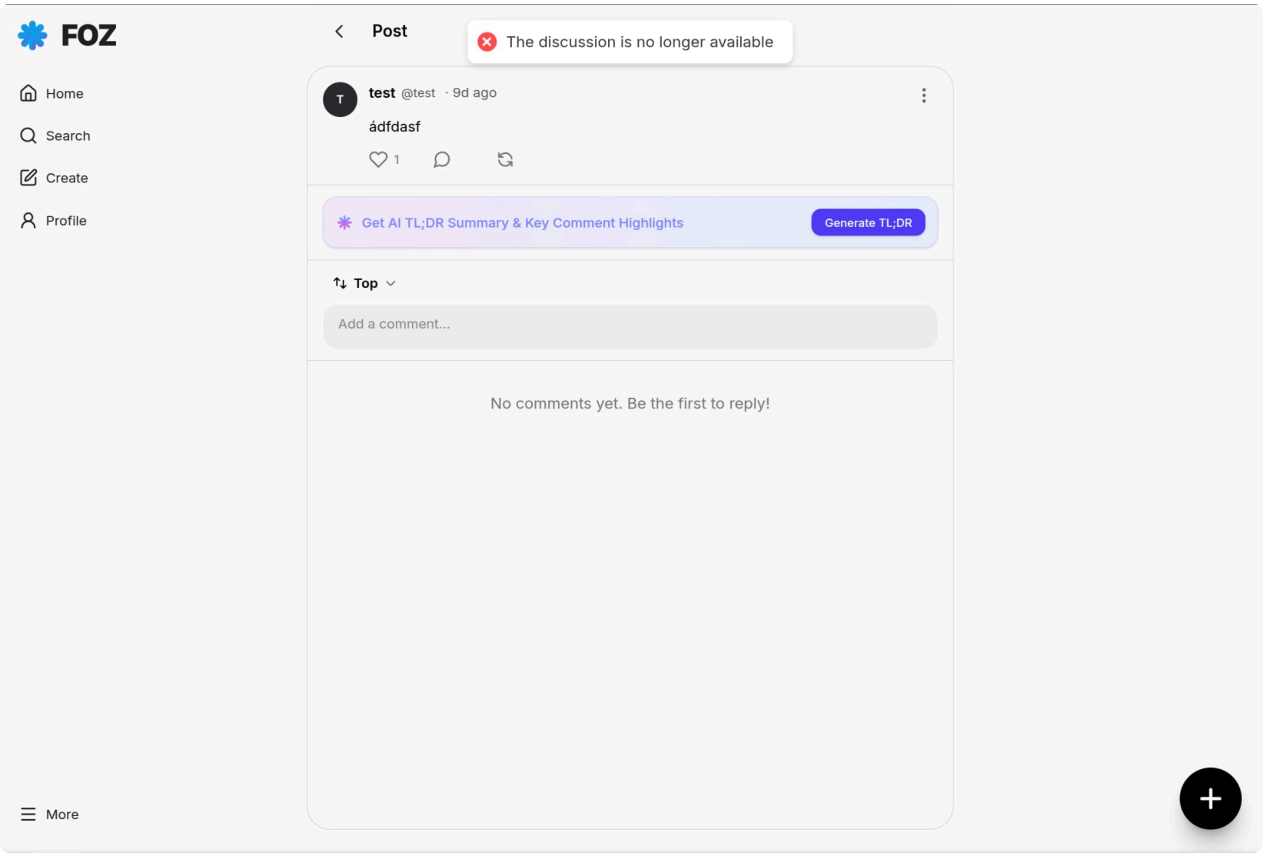
- ## Screen 15 (Alternative Flow 14 - Authentication Expired During Commenting):





- Screen 16 (Alternative Flow 15 - Post Deleted While Commenting):





Use-Case Specification: Like/Unlike

Project Name: FOZ
Version: 1.0
Document identifier: FOZ-UCS-13

1. Like/Unlike

- **Use-Case ID:** UC-13
- **Actor(s):** Standard User

1.1 Brief Description

Allows a user to toggle an appreciation metric on a post or comment.

2. Preconditions

- User is authenticated.

3. Flow of Events

3.1 Basic Flow

1. Extends UC-11 when the user clicks the "Heart/Like" icon.
2. The system checks the `reactions` table to see if a record already exists for this `user_account_id` and `target_id`.
3. If no record exists, the system inserts a new reaction (`reaction_kind: 'like'`) and increments the counter visually.
4. If a record exists, the system deletes it (Unlike) and decrements the counter.

3.2 Alternative Flows

3.2.1 Target Post or Comment Deleted / Not Found

1. If at step 2, the target post or comment has been deleted or is no longer available.
2. The system rejects the reaction request with a "Not Found" error.
3. The interface reverts the optimistic UI state and notifies the user that the content is no longer available.
4. The use case ends.

4. Special Requirements

- **Performance:**
  - **UI Response:** Reaction toggles update visually on the UI in < **50ms** (Optimistic UI).
  - **API Latency:** Database synchronization via API completes within <= **500ms** under normal network conditions.

5. Postconditions

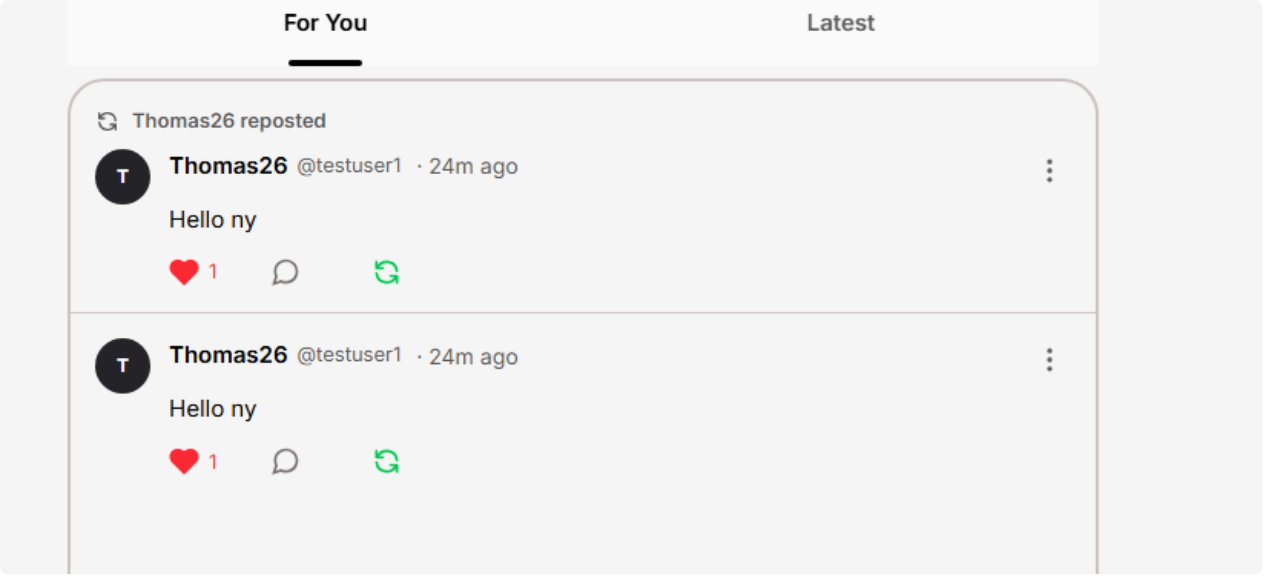
- The `reactions` table is updated.

6. Extension Points

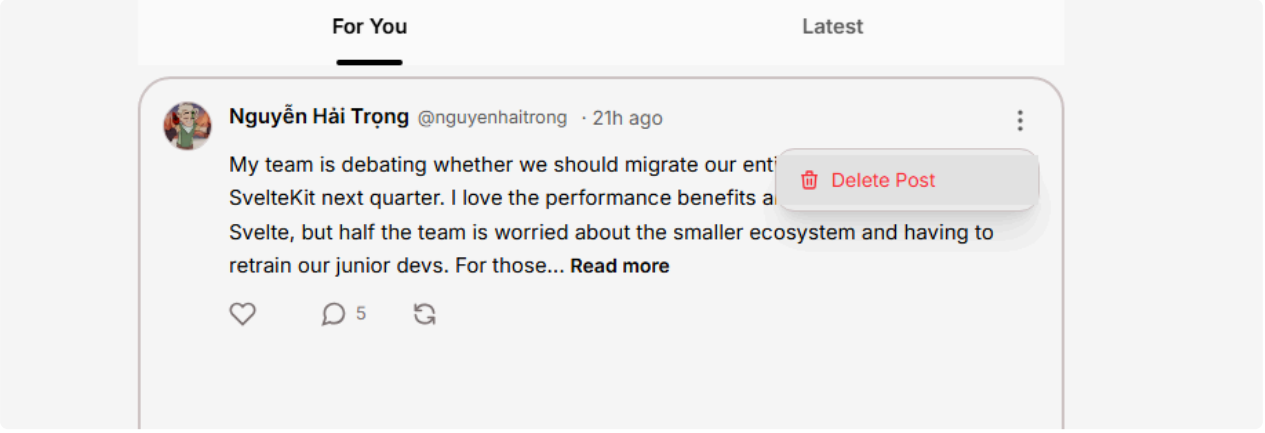
- None.

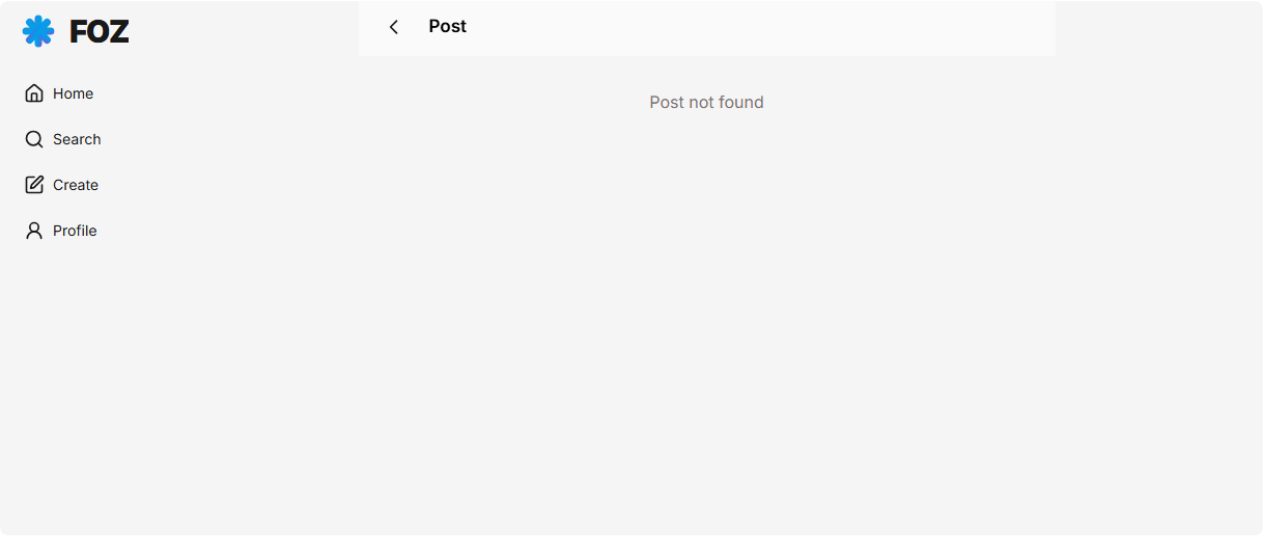
7. Prototype Requirement

- Screen 1 (Basic Flow):



- Screen 2 (Alternative Flow):





Use-Case Specification: Request AI Summary

Reviewed and Edited by: Nguyen Hai Nam | Reviewed by: Vo An Phu

Project Name: FOZ
Version: 1.1
Document identifier: FOZ-UCS-14

1. Request AI Summary

- **Use-Case ID:** UC-14
- **Actor(s):** Standard User

1.1 Brief Description

Allows an authenticated user to request an on-demand AI-generated TL;DR of a post and its comments.

2. Preconditions

- The user is authenticated and their account status is `active`.
- A valid post exists, is accessible to the user, and is not hidden or deleted.

3. Flow of Events

3.1 Basic Flow

1. Extends UC-09 when the authenticated user clicks "Generate TL;DR".
2. The system verifies that the post is still accessible and applies the per-user summary rate limit.
3. The system retrieves the post body and up to 10 active comments using a deduplicated 4:3:3 sample: four most liked, three most replied to by direct first-level replies, and three most recent. If fewer than 10 active comments exist, it retrieves all of them. Hidden, deleted, or moderated comments are excluded.
4. The system sends only the required post and comment text to the configured AI provider chain over HTTPS.
5. The AI provider returns a summary, and the system validates that the response contains usable summary text.
6. The system displays the generated summary in a modal.

3.2 Alternative Flows

3.2.1 Post Not Found or No Longer Accessible

1. If at step 2 or step 3 the post has been deleted, hidden, or cannot be found.
2. The system displays: "This post is no longer available."
3. The use case ends.

3.2.2 AI Provider Error

1. If at step 4 the primary AI provider times out, returns an error, or exceeds its quota.
2. The system attempts the configured fallback provider.
3. If every configured provider fails, the system safely catches the exception and displays: "Failed to load summary" along with a "Retry" button.
4. The use case ends. If a fallback provider succeeds, the flow resumes at step 5.

3.2.3 Invalid AI Response

1. If at step 5 the provider returns an empty, malformed, or unusable summary.
2. The system does not display the response and shows: "Failed to load summary" along with a "Retry" button.
3. The use case ends.

3.2.4 Rate Limit Exceeded

1. If at step 2 the user has already completed 10 summary requests in the rolling one-minute window.
2. The system rejects the request and displays: "Too many summary requests, please try again in a minute."
3. The use case ends.

3.2.5 Network Offline

1. If the user loses internet connection while the request is pending.
2. The system catches the network failure, stops the loading state, and displays an error message along with a "Retry" button.
3. The use case ends.

3.2.6 Unauthorized or Suspended Access

1. If the user is unauthenticated (guest) or their account is `suspended`.
2. The system immediately rejects the API request (HTTP 401 Unauthorized or HTTP 403 Forbidden).
3. The use case ends.

4. Special Requirements

- **Authentication:** Only authenticated Standard Users may trigger an AI request; the API must enforce this requirement.
- **Performance:** The complete provider chain must return a usable summary within `< 5 seconds`.
- **Rate Limit:** Maximum 10 requests per authenticated user in a rolling one-minute window.
- **Security:** User-generated text must be sanitized before it is sent to the external provider, and account identifiers or unrelated personal data must not be included in the payload.

- **Persistence:** The summary is displayed for the current request and is not persistently cached in v1.

5. Postconditions

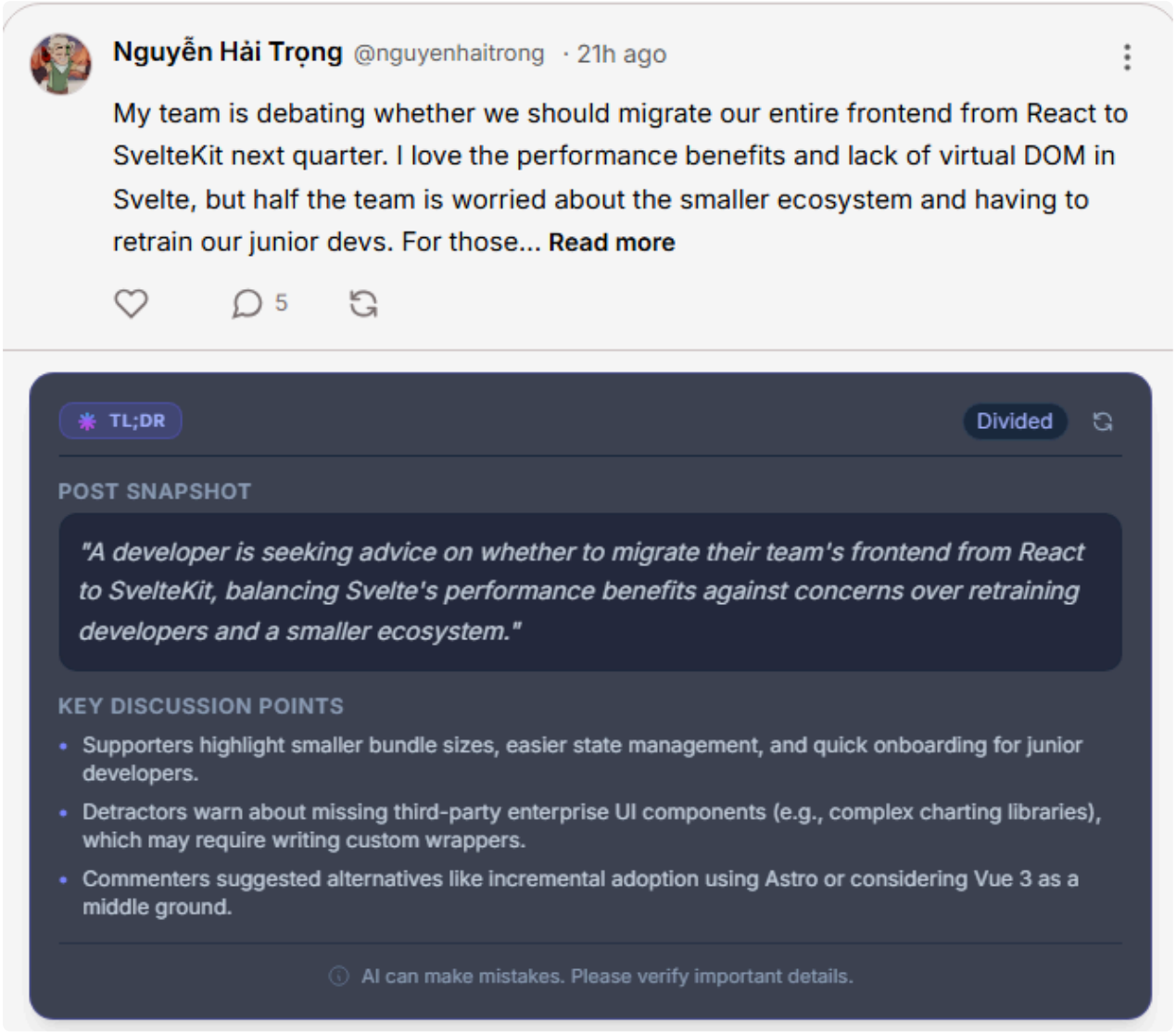
- On success, the generated summary is displayed to the user.
- On failure or rate limiting, no summary is displayed and the post remains unchanged.

6. Extension Points

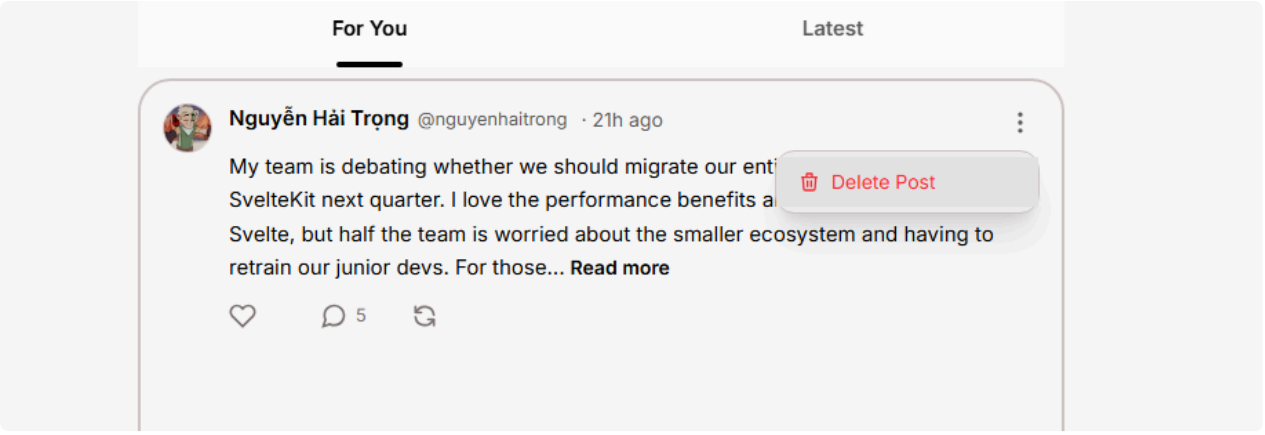
- None.

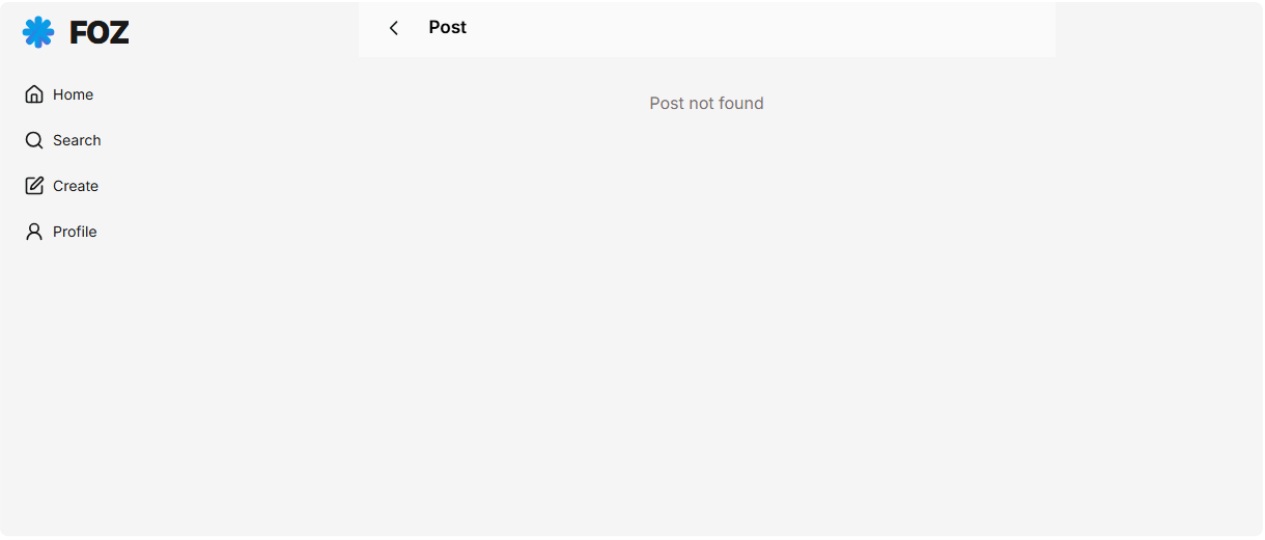
7. Prototype Requirement

- Screen 1 (Basic Flow):

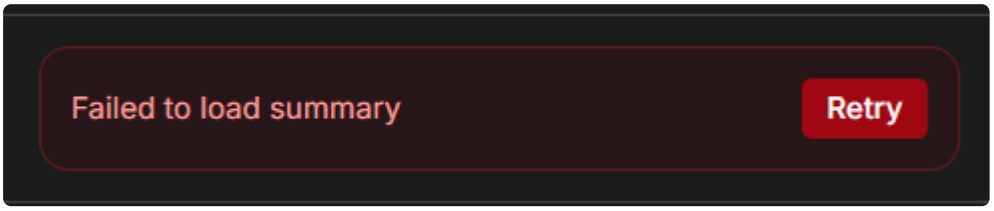


- Screen 2 (Alternative Flow 1 - Post Not Found):

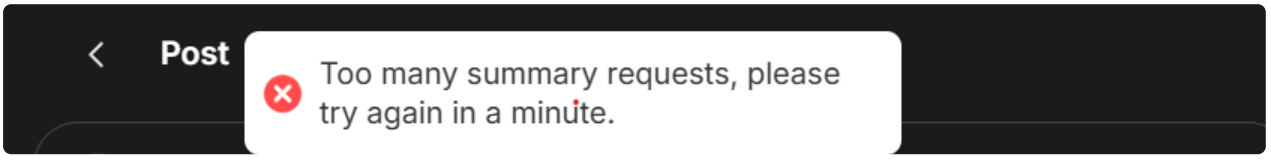




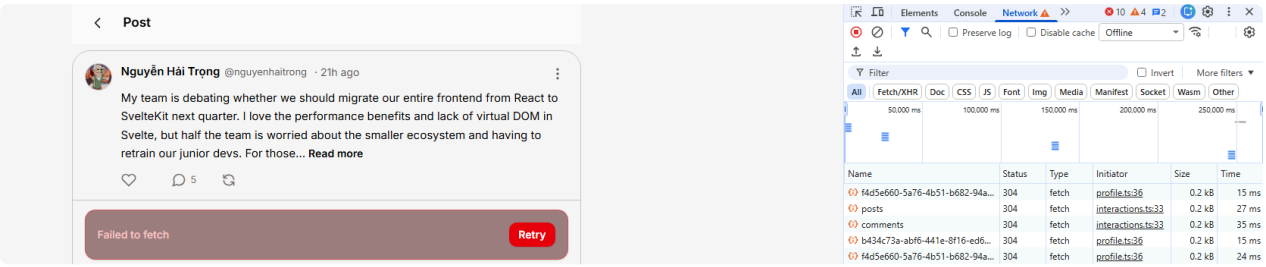
- Screen 3 (Alternative Flow 2 & 3 - AI Error / Invalid AI Response):



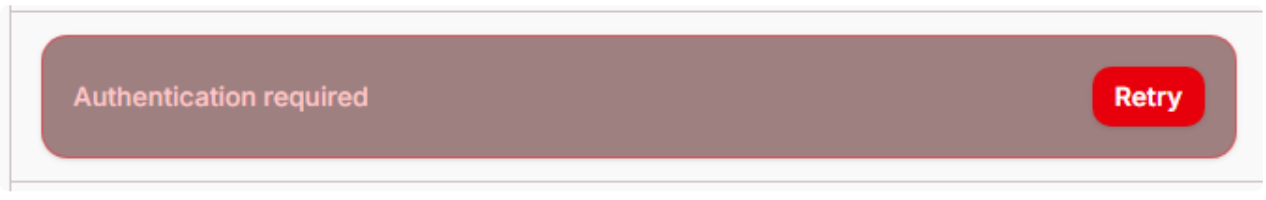
- Screen 4 (Alternative Flow 4 - Rate Limit Exceeded):



- Screen 5 (Alternative Flow 5 - Network Offline):



- Screen 6 (Alternative Flow 6 - Unauthorized Access):



Use-Case Specification: Delete Own Post

Reviewed and Edited by: Nguyen Hai Nam | Reviewed by: Vo An Phu

Project Name: FOZ
Version: 1.1
Document identifier: FOZ-UCS-15

1. Delete Own Post

- **Use-Case ID:** UC-15
- **Actor(s):** Standard User

1.1 Brief Description

Allows an authenticated user to permanently delete a post they authored. The post record is removed from the database and becomes inaccessible to everyone, including the author; it cannot be restored.

2. Preconditions

- The user is authenticated and their account status is `active`.
- The target post exists and the user is its `author_id`.

3. Flow of Events

3.1 Basic Flow

1. The use case begins when the user clicks the "Delete" option on their own post.
2. The system displays a confirmation dialog warning that the deletion is permanent and irreversible.
3. The user confirms the deletion.
4. The system verifies ownership at the API boundary, then the backend deletes the post row from the `posts` table; dependent comments and attachments are removed via cascade.
5. The system removes the post from all public feeds, search results, and post-detail views, then redirects the user to the home feed.

3.2 Alternative Flows

3.2.1 User Cancels Deletion

1. If at step 3 the user clicks "Cancel", closes the dialog, or presses Escape.
2. The dialog closes, no database changes are made, and the use case ends.

3.2.2 Post Not Found or Forbidden

1. If at step 4 the post no longer exists or the user is not its author.
2. The system makes no changes and displays: "This post cannot be deleted or you do not have permission to delete it."
3. The use case ends.

3.2.3 Deletion Failure

1. If at step 4 the database deletion fails.
2. The system displays: "The post could not be deleted. Please try again." and leaves the post unchanged.
3. The use case ends.

4. Special Requirements

- **Authorization:** Ownership must be checked at the API boundary as well as in the user interface.
- **Permanence:** The deletion is a hard delete — the record is removed from the `posts` table and is not recoverable by the author or any application user.
- **Data Integrity:** Dependent comments and attachments are removed via cascade; uploaded media assets are destroyed best-effort in the background.

5. Postconditions

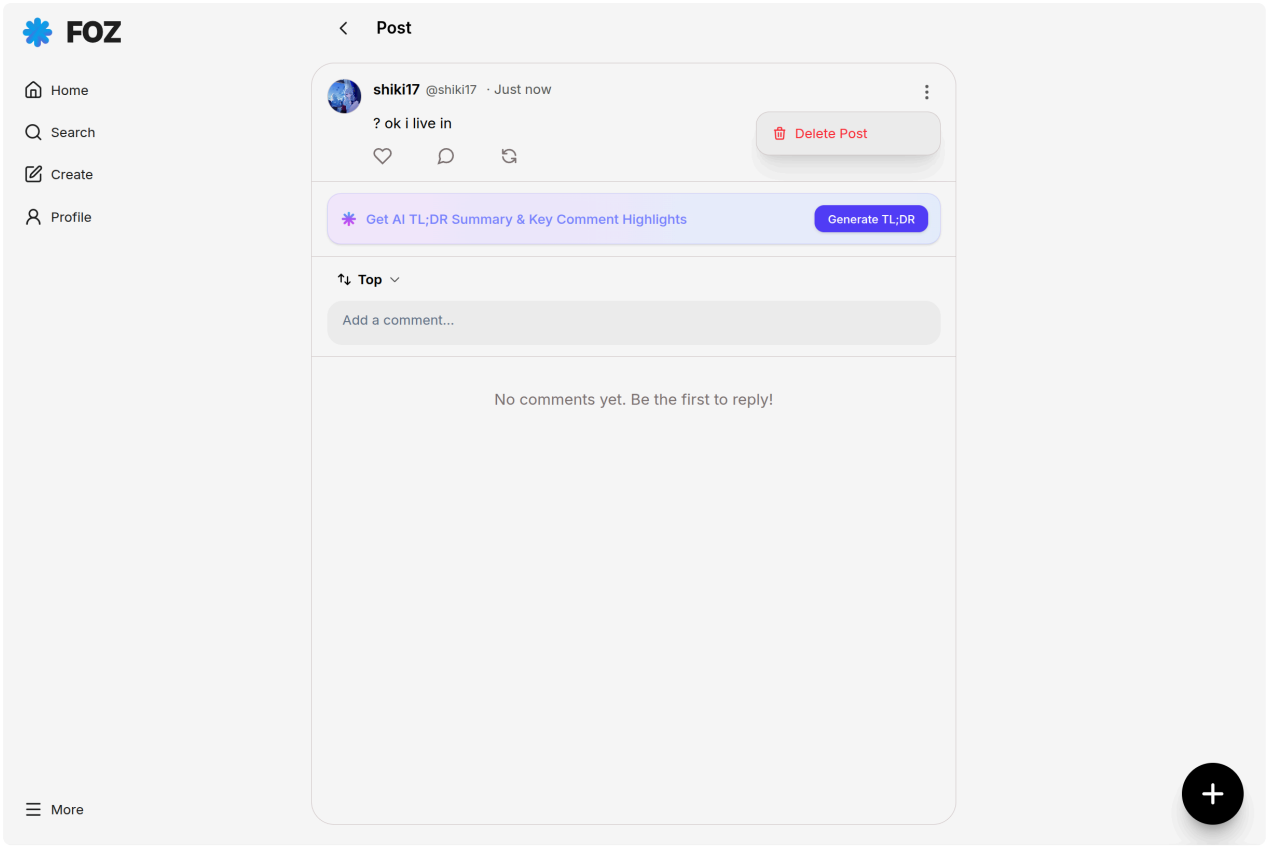
- The post record no longer exists in the `posts` table.
- The post is inaccessible to the author, the public, and all application queries.

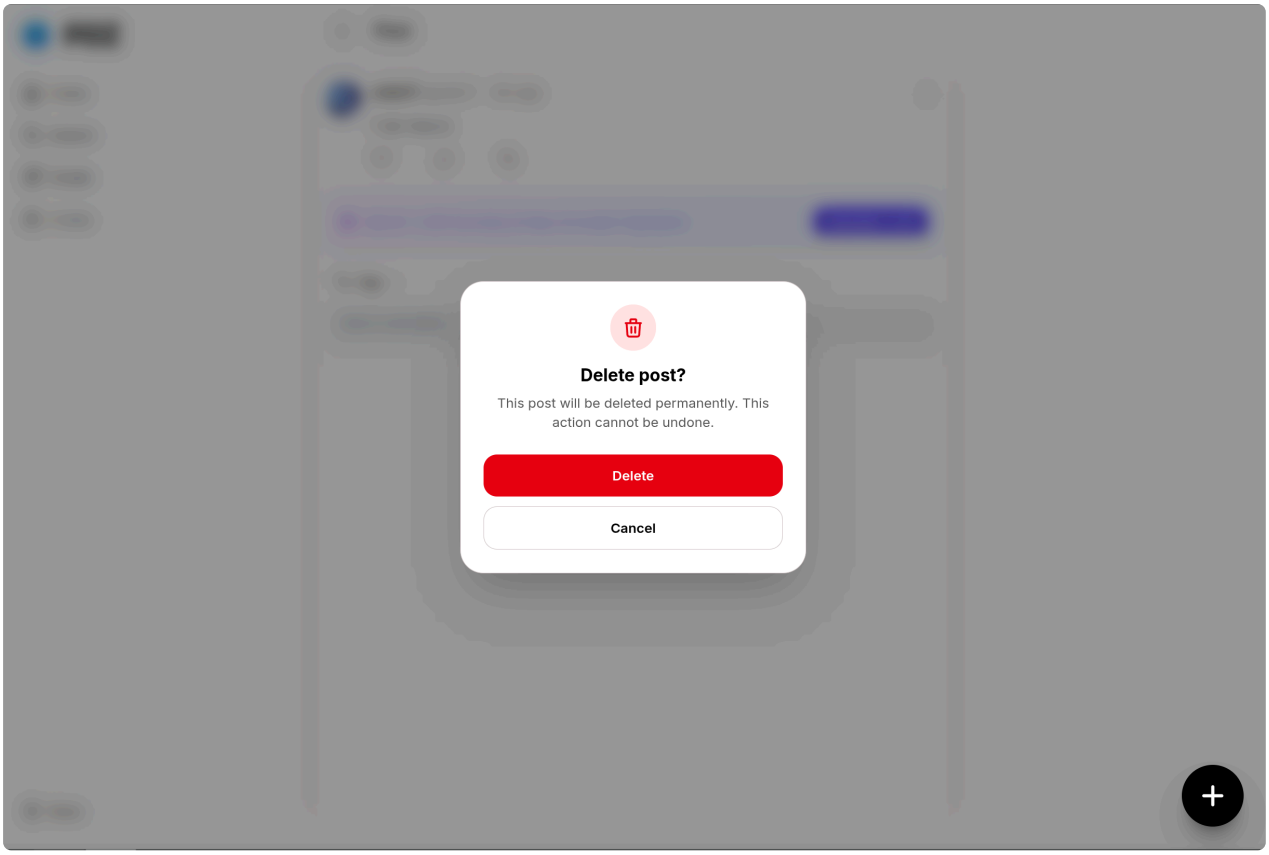
6. Extension Points

- None.

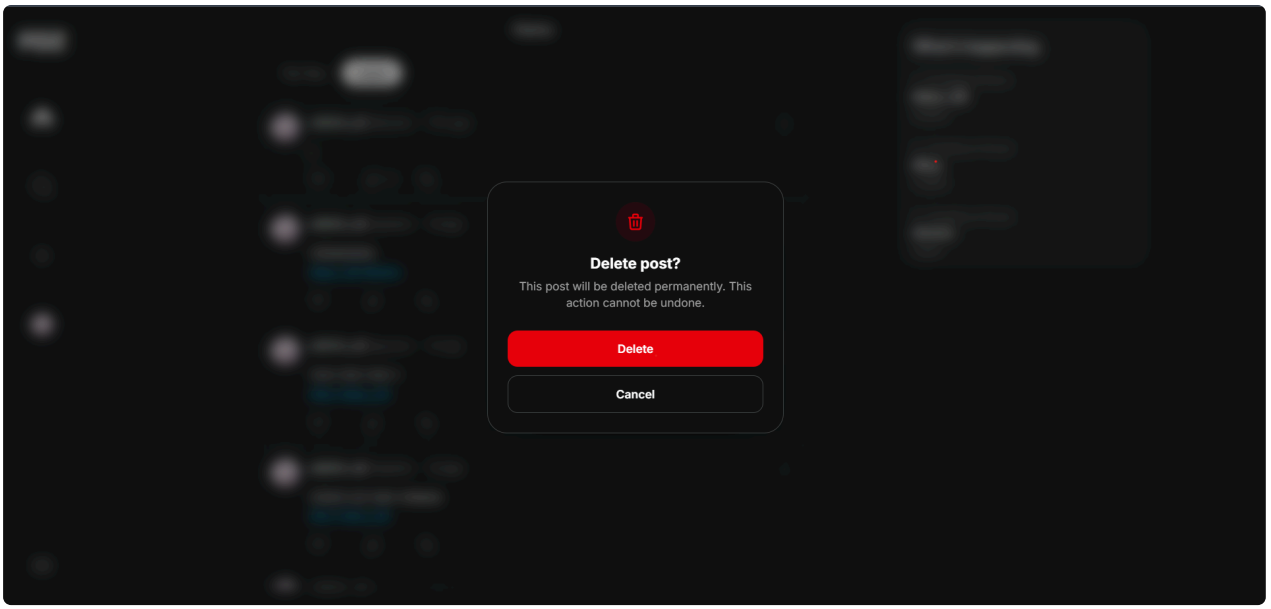
7. Prototype Requirement

- Screen 1 (Basic Flow):

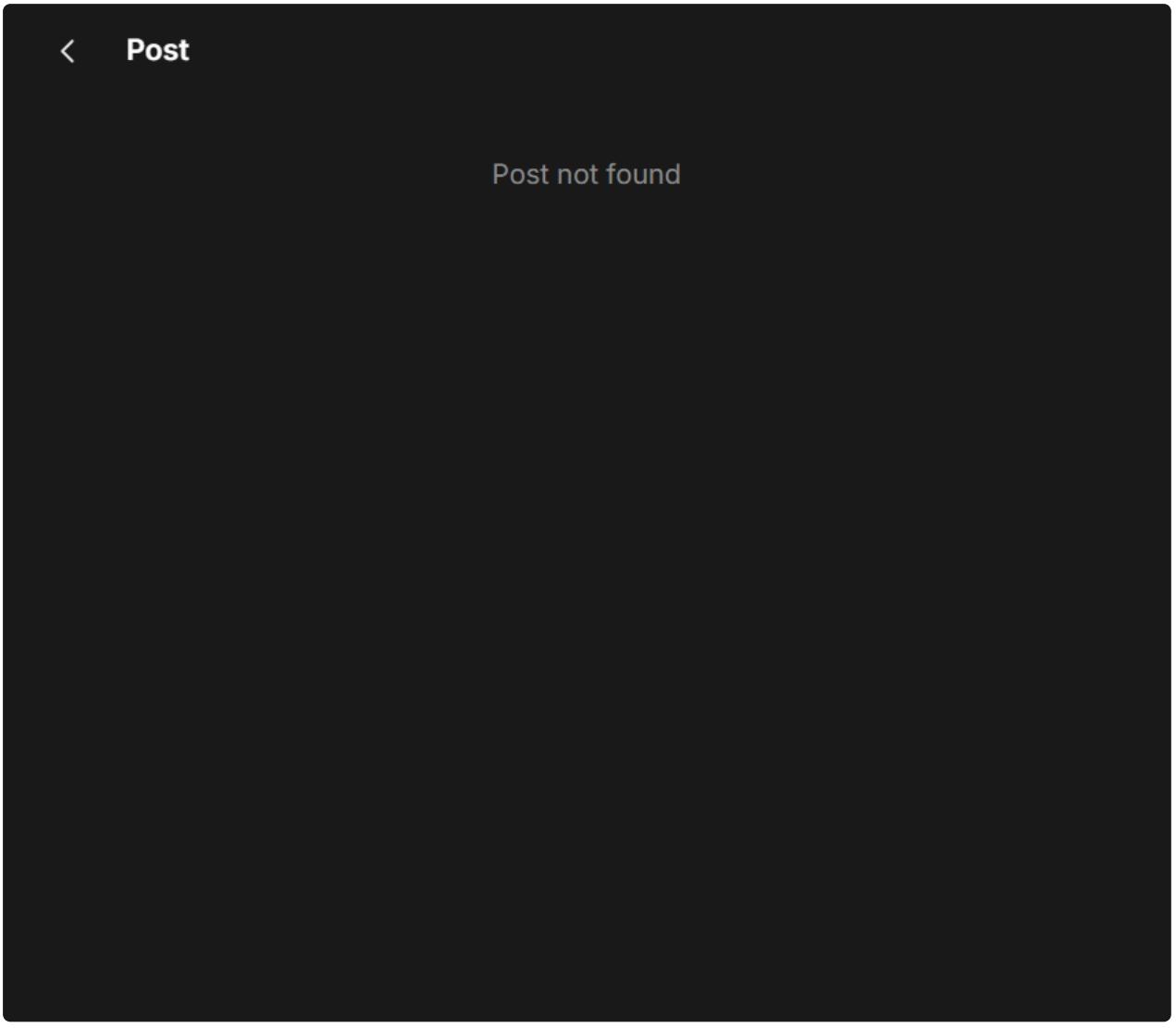




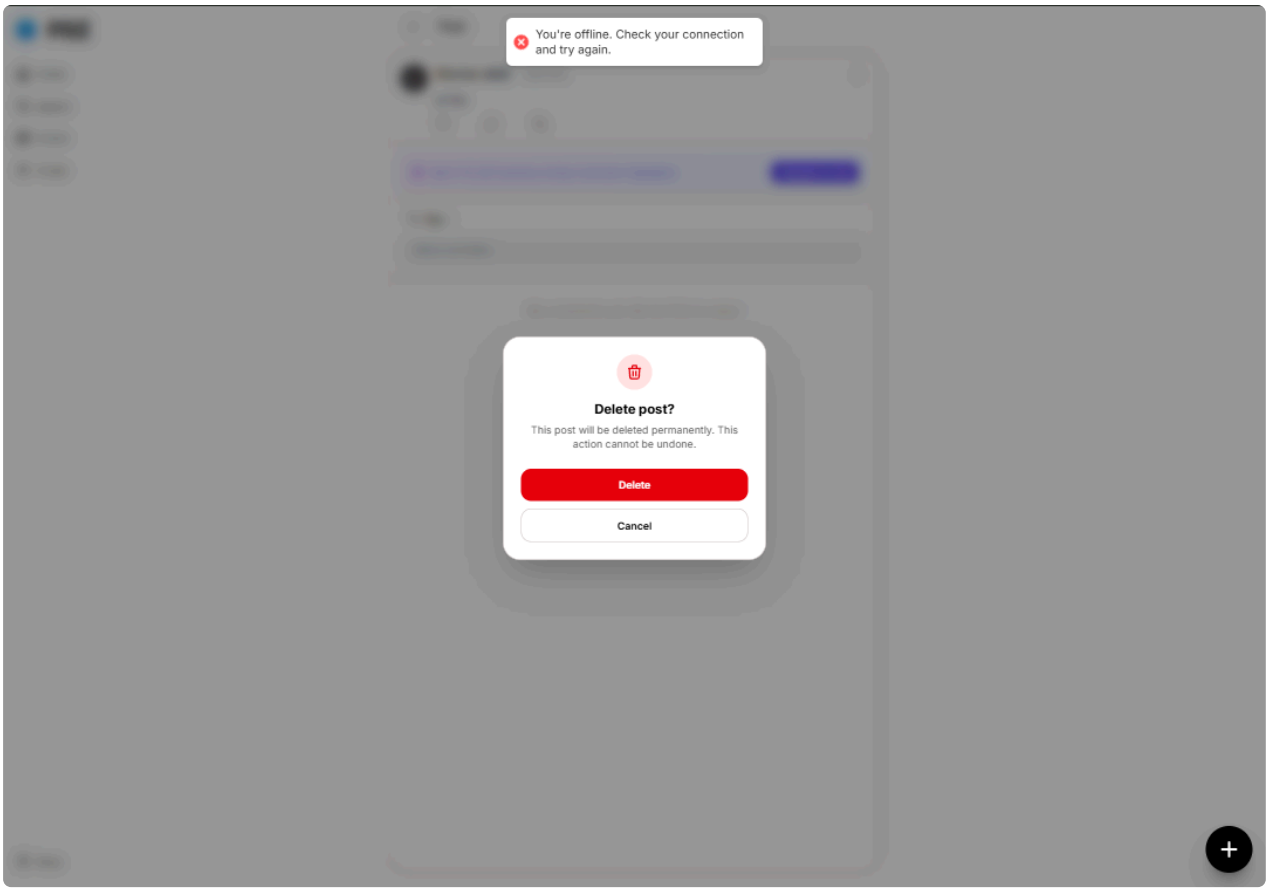
- Screen 2 (Alternative Flow 1):



- Screen 3 (Alternative Flow 2):



- **Screen 4 (Alternative Flow 3):**



Use-Case Specification: Report Inappropriate Content

Reviewed and Edited by: Nguyen Hai Nam | Reviewed by: Vo An Phu

Project Name: FOZ
Version: 1.1
Document identifier: FOZ-UCS-16

1. Report Inappropriate Content

- **Use-Case ID:** UC-16
- **Actor(s):** Standard User

1.1 Brief Description

Allows an authenticated user to report an active post or comment that may violate the community guidelines.

2. Preconditions

- The user is authenticated and their account status is `active`.
- The target is an existing, active post or comment that can be reported.

3. Flow of Events

3.1 Basic Flow

1. The use case begins when the user clicks "Report" on an active post or comment.
2. The system displays a report modal with the target content, the required reasons `Harassment`, `Spam`, `Inappropriate`, and `Other`, and an optional details field.
3. The user selects one reason, enters optional details of no more than 1,000 characters, and clicks "Submit Report". If the reason is `Other`, the user must provide details explaining the report.
4. The system validates the target, reason, details, duplicate-report rule, and reporting rate limit.
5. The system inserts a row into `reports` containing `reporter_id`, `subject_type`, `subject_id`, `reason`, `details`, and `status = 'open'`.
6. The system displays a "Thank you for your report" toast.

3.2 Alternative Flows

3.2.1 User Cancels Report

1. If at step 2 or step 3 the user closes the modal, clicks "Cancel", or presses Escape.
2. The system discards the entered data, makes no database changes, and the use case ends.

3.2.2 Validation Error

1. If at step 4 no reason is selected, the `Other` reason has no details, or the details exceed 1,000 characters.
2. The system highlights the invalid field and displays a specific validation error.
3. The flow resumes at step 3.

3.2.3 Content Already Reported by User

1. If at step 4 an open report already exists for the same `reporter_id`, `subject_type`, and `subject_id`.
2. The system does not insert another row and displays: "You have already reported this content."
3. The use case ends.

3.2.4 Content Not Found or No Longer Reportable

1. If at step 4 the target has been deleted, hidden, or cannot be found.
2. The system does not create a report and displays: "This content is no longer available for reporting."
3. The use case ends.

3.2.5 Reporting Rate Limit Exceeded

1. If at step 4 the user has already submitted 10 reports in the rolling 15-minute window.
2. The system rejects the request and displays: "Too many reports. Please try again later."
3. The use case ends.

3.2.6 Report Submission Failure

1. If at step 5 the database insert fails.
2. The system rolls back the operation and displays: "The report could not be submitted. Please try again."
3. The use case ends.

4. Special Requirements

- **Security:** Details must be sanitized and validated at the API boundary.
- **Rate Limit:** Maximum 10 report submissions per authenticated user in a rolling 15-minute window.
- **Data Integrity:** A uniqueness constraint must prevent duplicate open reports for the same reporter and target.

5. Postconditions

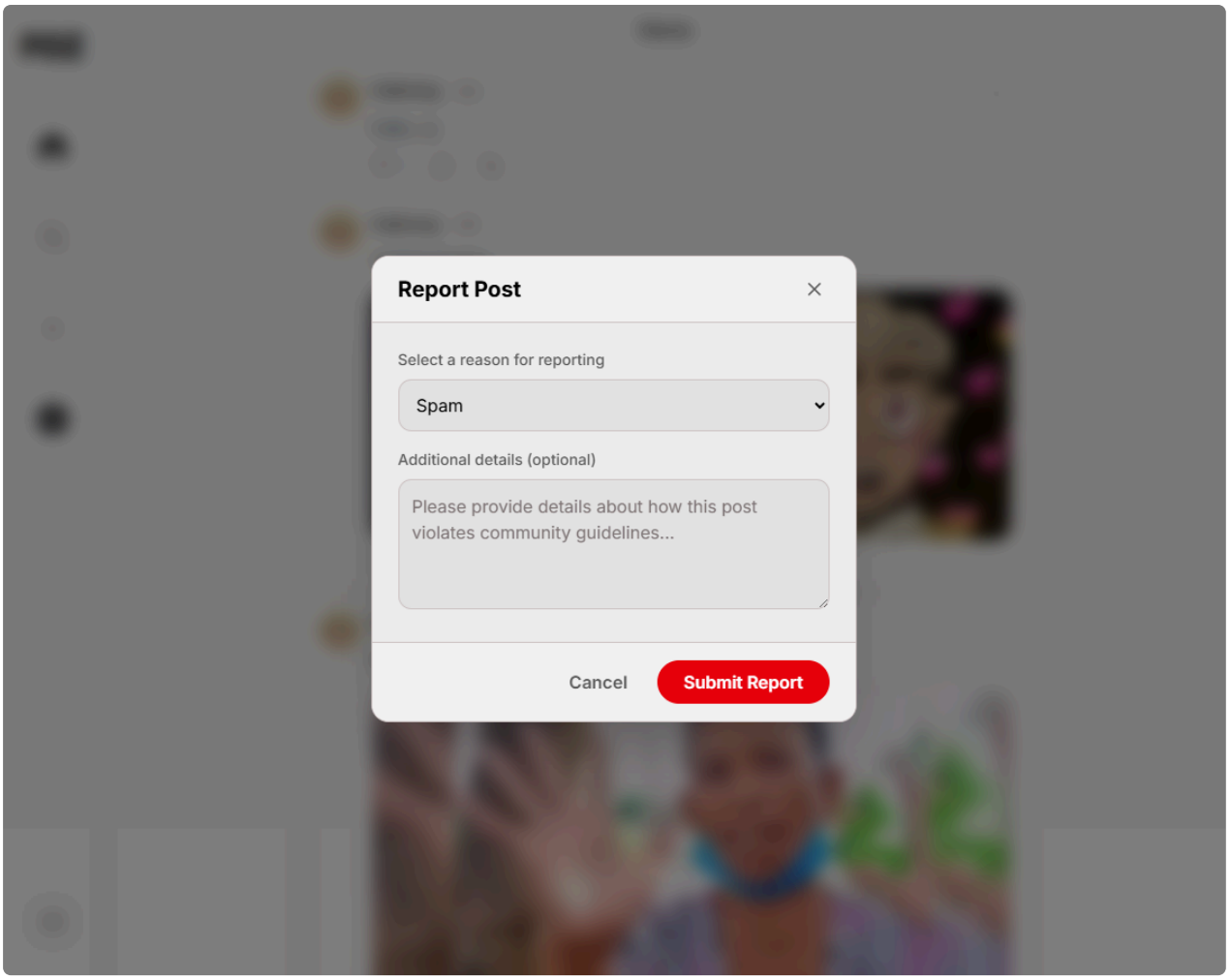
- On success, one new row exists in `reports` with status `open`.
- On cancellation, validation failure, duplicate submission, or an unavailable target, no new report row is created.

6. Extension Points

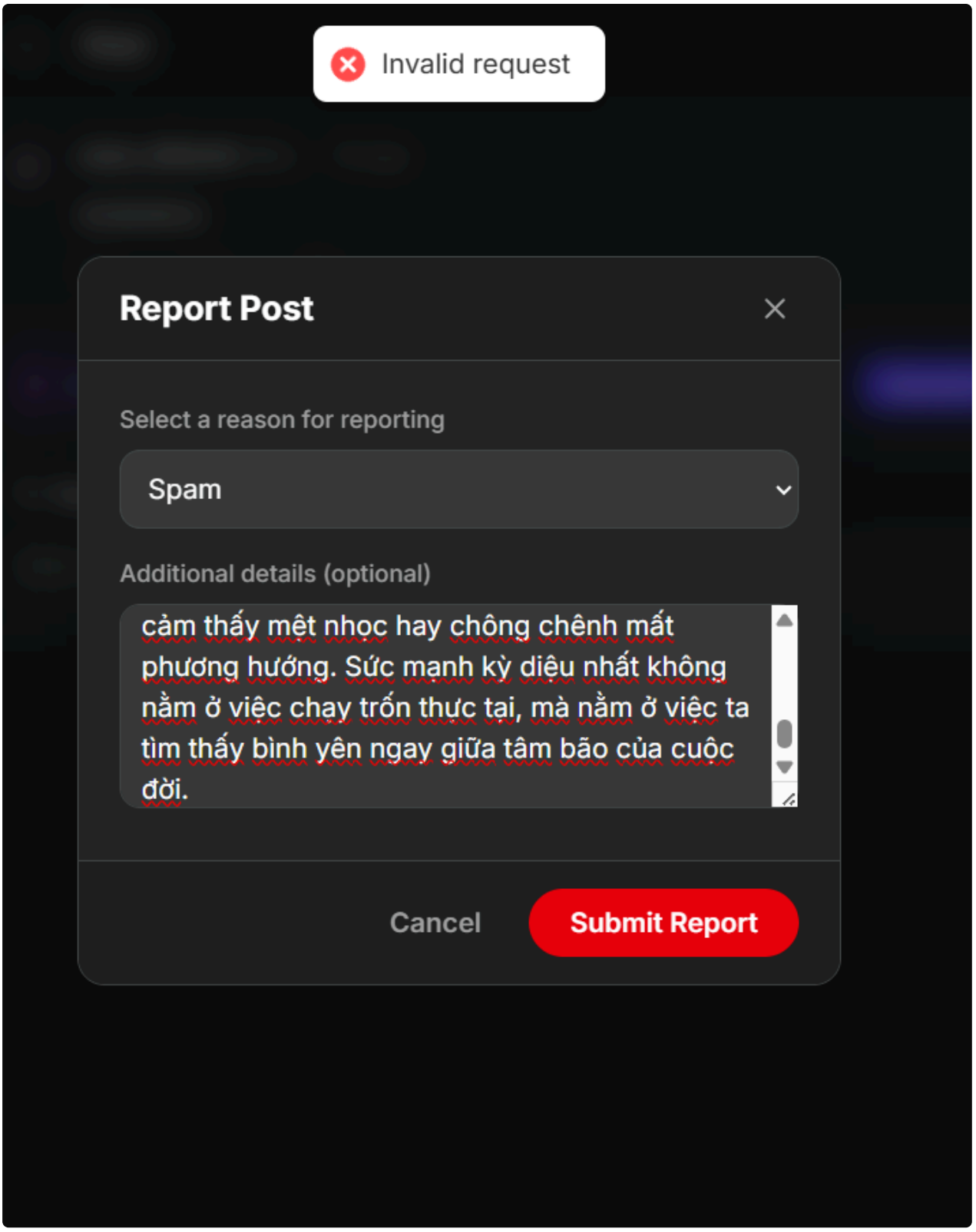
- None.

7. Prototype Requirement

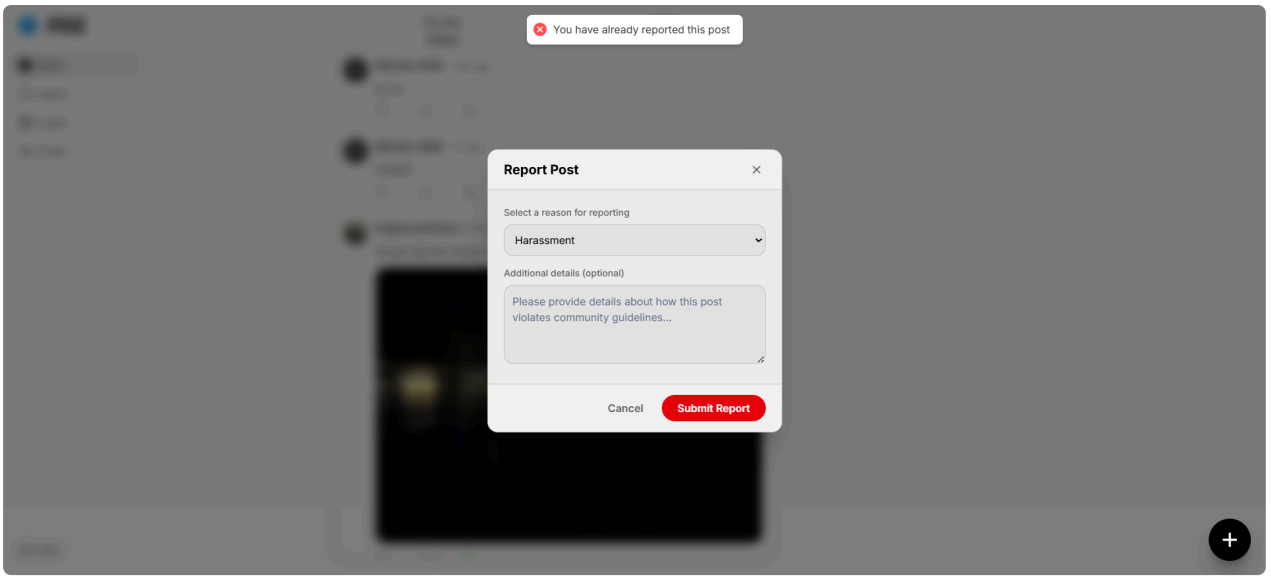
- **Screen 1 (Basic Flow):**



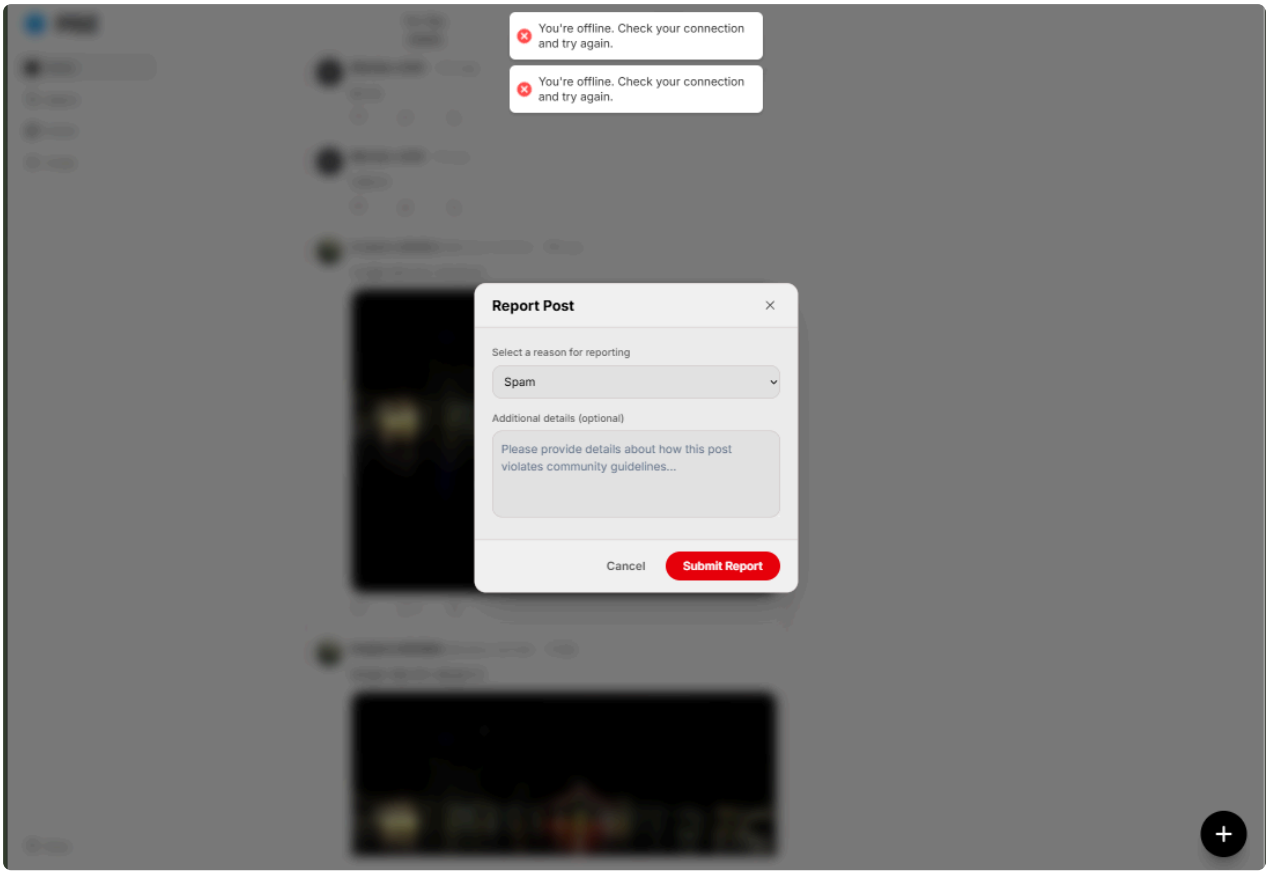
- **Screen 2 (Alternative Flow 2):**



- Screen 3 (Alternative Flow 3):



- Screen 4 (Alternative Flow 6):



Use-Case Specification: Review Reports

Reviewed and Edited by: Nguyen Hai Nam | Reviewed by: Vo An Phu

Project Name: FOZ
Version: 1.1

Document identifier: FOZ-UCS-17

1. Review Reports

- **Use-Case ID:** UC-17
- **Actor(s):** Moderator, Administrator

1.1 Brief Description

Allows Moderators to review community reports and dismiss or escalate them, and allows Administrators to review escalated reports and dismiss or apply an authorized administrative action.

2. Preconditions

- The user is authenticated and has role `moderator` or `administrator`.
- The selected report exists and has not already been resolved.

3. Flow of Events

3.1 Basic Flow

1. A Moderator opens the Moderation Dashboard, or an Administrator opens the Admin Dashboard.
2. The system displays reports assigned to the actor or available in that actor's queue: Moderators see unassigned reports with status `open` and their own `under_review` reports, while Administrators see `escalated` reports and their own `under_review` reports.
3. The actor selects a report. If the report is unassigned and eligible for the actor's queue, the system atomically assigns it to the actor and changes its status to `under_review`; if it is already `under_review`, the system verifies that it is assigned to the actor. The system then displays the reported content, report details, and prior review history.
4. The actor enters a decision rationale of at least five words and selects an action:
   - A Moderator selects `Dismiss` or `Escalate to Admin`.
   - An Administrator selects `Dismiss` or invokes UC-19 to permanently delete the reported content.
5. If the actor selects `Dismiss`, the system changes the report status to `dismissed` and records the decision rationale, actor, timestamp, and optional review notes in `moderation_reviews` and `audit_logs`.
6. If the Moderator selects `Escalate to Admin`, the system passes the decision rationale to UC-18.
7. If the Administrator invokes UC-19 and the deletion succeeds, UC-19 records the administrative rationale, closes the related report, and the system displays the completed review.

3.2 Alternative Flows

3.2.1 No Reports Available

1. If at step 2 no eligible reports are found.
2. The system displays: "No reports require review."
3. The use case ends.

3.2.2 Actor Leaves Without a Decision

1. If at step 4 the actor closes the report without selecting an action.
2. If the actor claimed the report during this use case, the system restores it to the previous queue status (`open` for a Moderator or `escalated` for an Administrator) and clears the assignment. If it was already assigned to the actor, the system leaves it `under_review`.
3. The system records no decision and the use case ends.

3.2.3 Report Not Found or Already Processed

1. If at step 3 the report does not exist or another reviewer has already claimed or resolved it.
2. The system displays the current report state and returns a not-found or conflict error without overwriting the other review.
3. The use case ends.

3.2.4 Reported Content No Longer Exists

1. If at step 3 the reported post or comment has already been removed.
2. The system marks the report `closed` with closure code `content_unavailable`, records the closure rationale "Reported content is no longer available.", and informs the actor that no further content action is possible.
3. The use case ends.

4. Special Requirements

- **Authorization:** Moderator and Administrator permissions must be enforced at the API boundary, not only in the dashboard.
- **Separation of Powers:** Moderators may dismiss or escalate reports but must not permanently remove content or punish users.
- **Concurrency:** Only one reviewer may claim a report at a time, and state transitions must be conditional on the current report status.
- **Decision Rationale:** Every Moderator or Administrator decision must include a non-empty rationale of at least five words after trimming whitespace.
- **Audit:** Every dismiss, escalation, content removal, and closure must record the decision rationale without storing unnecessary sensitive data.

5. Postconditions

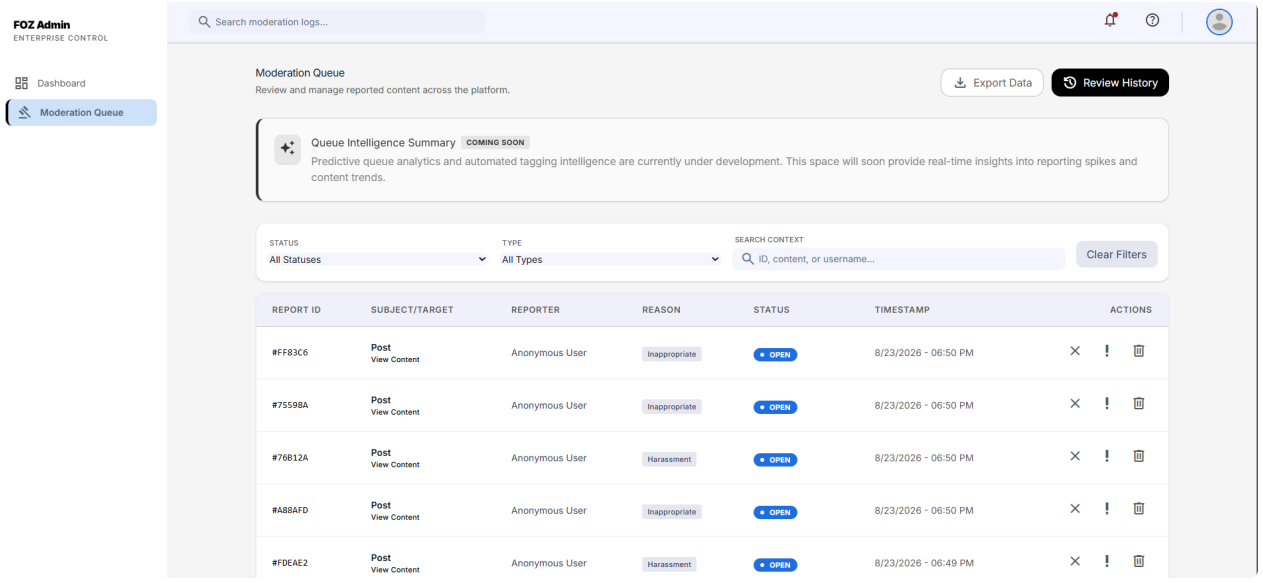
- A dismissed report has status `dismissed`, a decision rationale of at least five words, and a moderation review and audit entry.
- An escalated report has status `escalated` after UC-18 completes, with the Moderator's decision rationale recorded.
- A successfully force-deleted report has status `closed` after UC-19 completes, with the Administrator's rationale recorded.
- If the actor cancels before making a decision, no moderation decision is recorded.

6. Extension Points

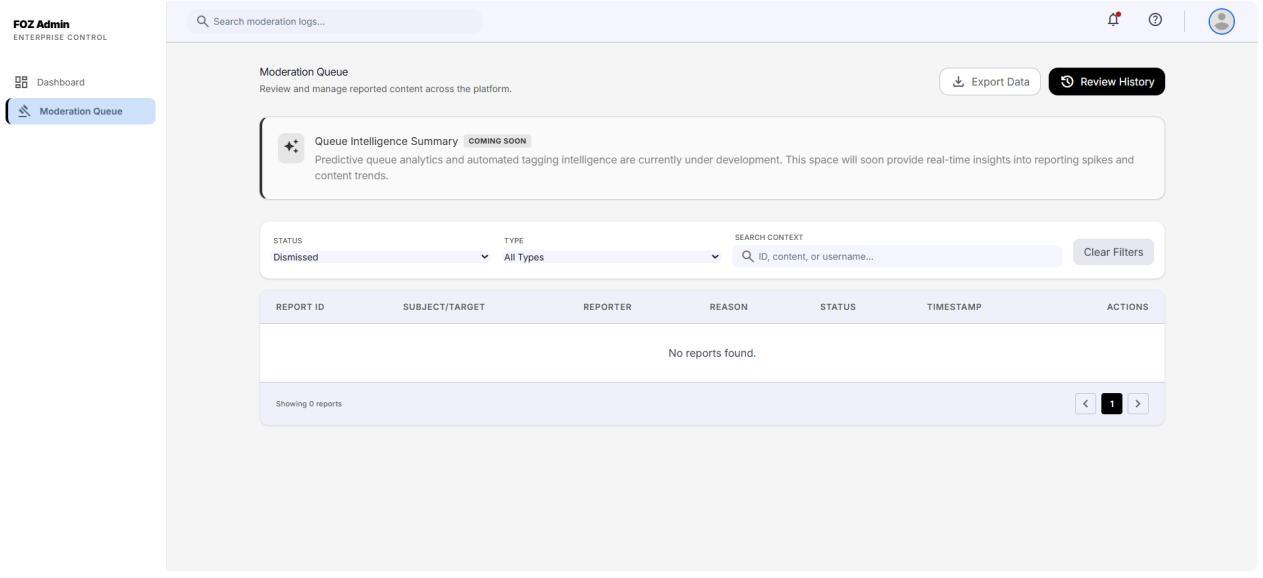
- **Forward Report to Admin:** Extended via UC-18.
- **Force Delete Content:** Invokes UC-19 for Administrator actions.

7. Prototype Requirement

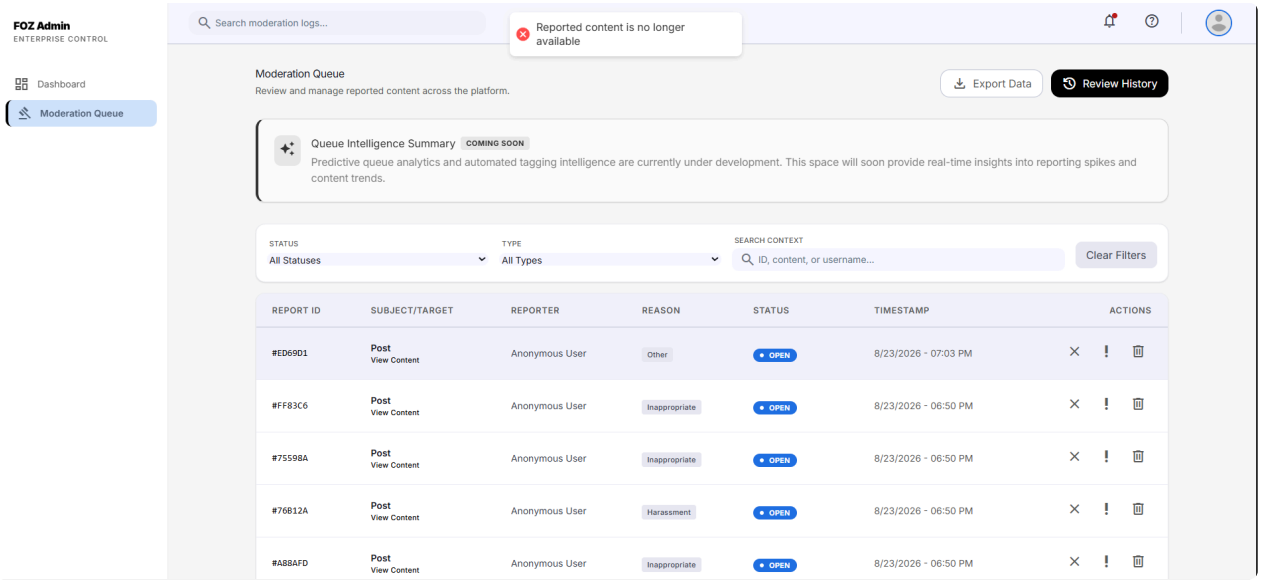
- Screen 1 (Basic Flow):



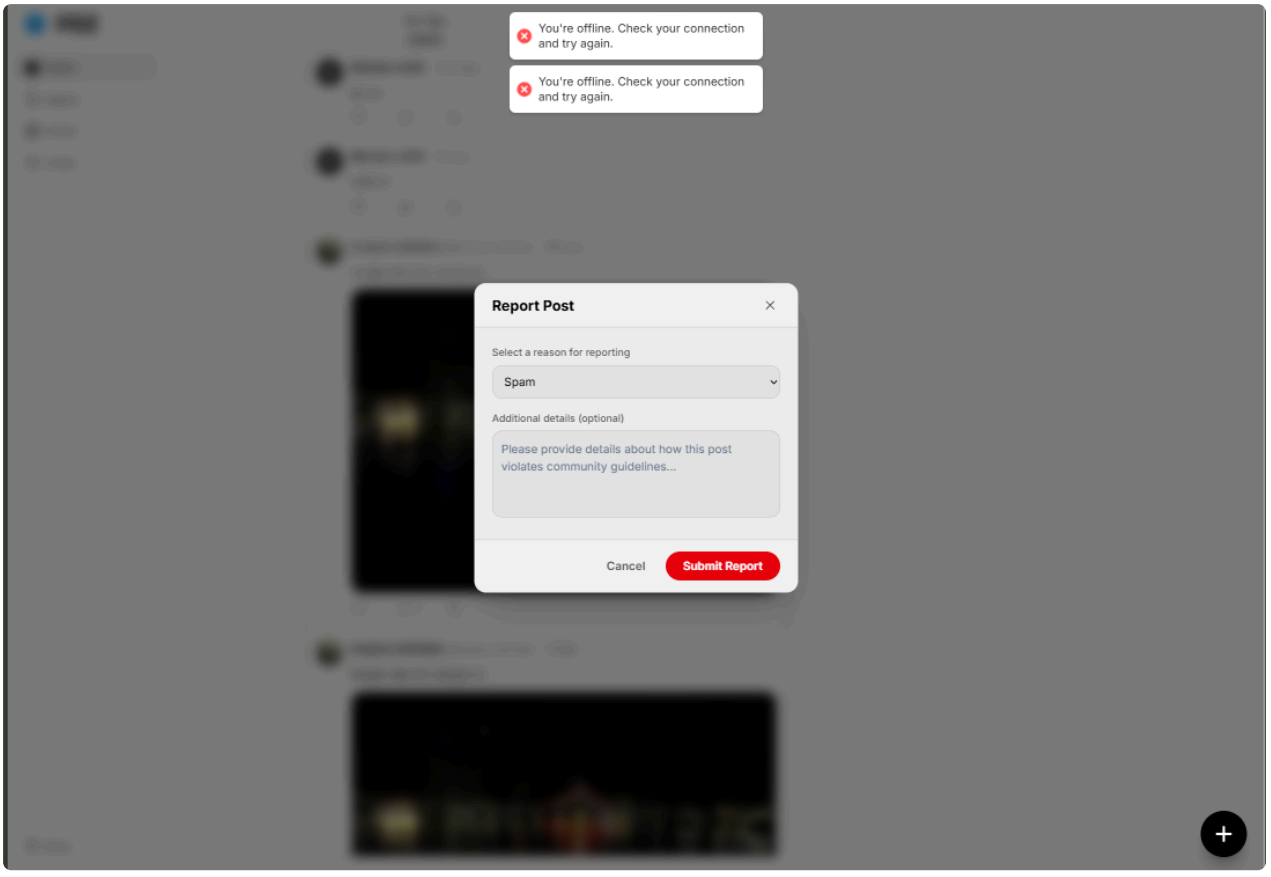
- Screen 2 (Alternative Flow 1):



- Screen 3 (Alternative Flow 3):



- Screen 4 (Alternative Flow 4):



Use-Case Specification: Forward Report to Admin

Reviewed and Edited by: Nguyen Hai Nam | *Reviewed by: Vo An Phu* | Refined by: Pham Dinh Gia Khanh

Project Name: FOZ
Version: 1.1

Document identifier: FOZ-UCS-18

1. Forward Report to Admin

- **Use-Case ID:** UC-18
- **Actor(s):** Moderator

1.1 Brief Description

Allows a Moderator to escalate a report that may require an administrative decision or punishment beyond the Moderator's authority.

2. Preconditions

- The user is authenticated and has role `moderator`.
- The report exists, is currently `under_review`, and has not been dismissed, escalated, or closed.
- The reported content is still available for administrative review.

3. Flow of Events

3.1 Basic Flow

1. Extends UC-17 when a Moderator selects `Escalate to Admin` for a report requiring administrative review.
2. The system opens an escalation modal showing the report and requiring an escalation rationale of at least five words.
3. The Moderator enters the escalation rationale and clicks "Escalate to Admin".
4. The system validates the report state and escalation rationale.
5. In one transaction, the system changes the report status to `escalated`, records the decision, Moderator, timestamp, and escalation rationale in `moderation_reviews`, and records the escalation in `audit_logs`.
6. The system places the report in the Admin-only queue.

3.2 Alternative Flows

3.2.1 Missing or Insufficient Escalation Rationale

1. If at step 4 the Moderator submits no rationale or a rationale shorter than five words after trimming whitespace.
2. The system displays: "A rationale of at least 5 words is required to escalate a report."
3. The modal remains open, escalation is prevented, and the flow resumes at step 3.

3.2.2 Report Not Found

1. If at step 4 or step 5 the report does not exist.
2. The system aborts the transaction and displays: "This report is no longer available."
3. The use case ends.

3.2.3 Report Already Processed

1. If at step 4 or step 5 the report is no longer `under_review` because another reviewer processed it.
2. The system aborts the transaction, returns a conflict response, and displays the report's current state.
3. The use case ends.

3.2.4 Escalation Update Failure

1. If at step 5 any status, review-log, or audit-log operation fails.
2. The system rolls back all changes and displays: "The report could not be escalated. Please try again."
3. The use case ends.

3.2.5 Concurrent Escalation

1. If at step 3, two moderators attempt to escalate the same report at nearly the same time.
2. The system accepts the earlier transaction, rolls back the second one, and notifies the second moderator: "This report has already been escalated."
3. The use case ends

4. Special Requirements

- **Authorization:** The Moderator role must be enforced at the API boundary.
- **Decision Rationale:** The escalation rationale must contain at least five words after trimming whitespace.
- **Audit:** The escalation decision and non-sensitive escalation rationale must be auditable.
- **Atomicity:** The report status, moderation review, and audit entry must be updated consistently; Admin queue visibility is derived from the resulting `escalated` status.

5. Postconditions

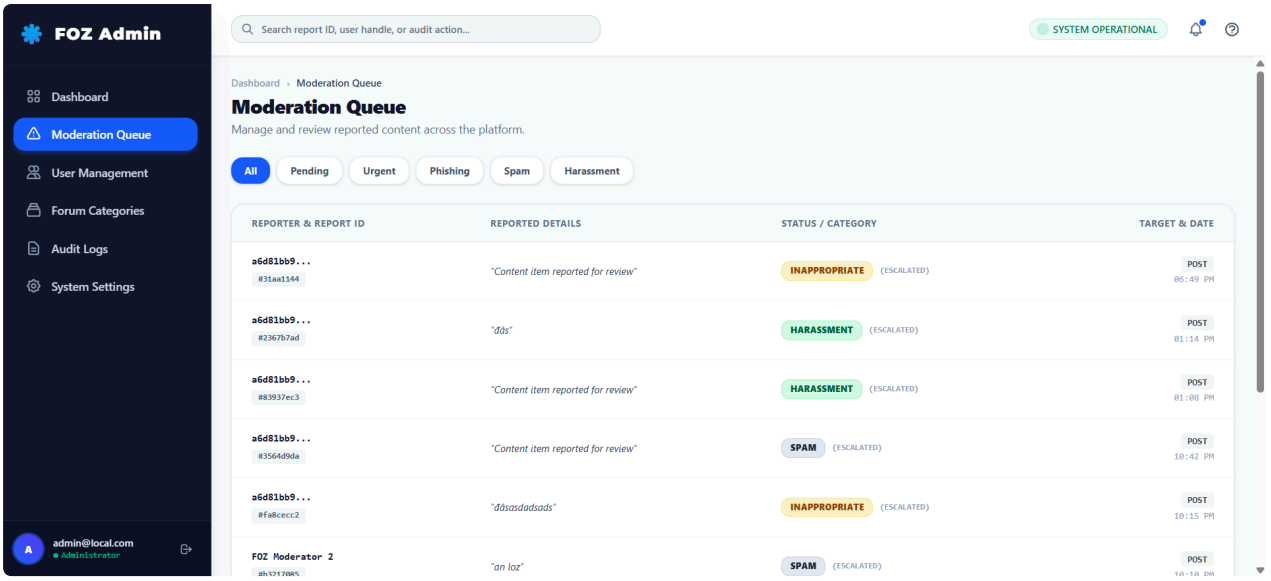
- The report status is `escalated`.
- A moderation review and audit entry record the escalation rationale.
- The report is visible in the Admin-only queue.

6. Extension Points

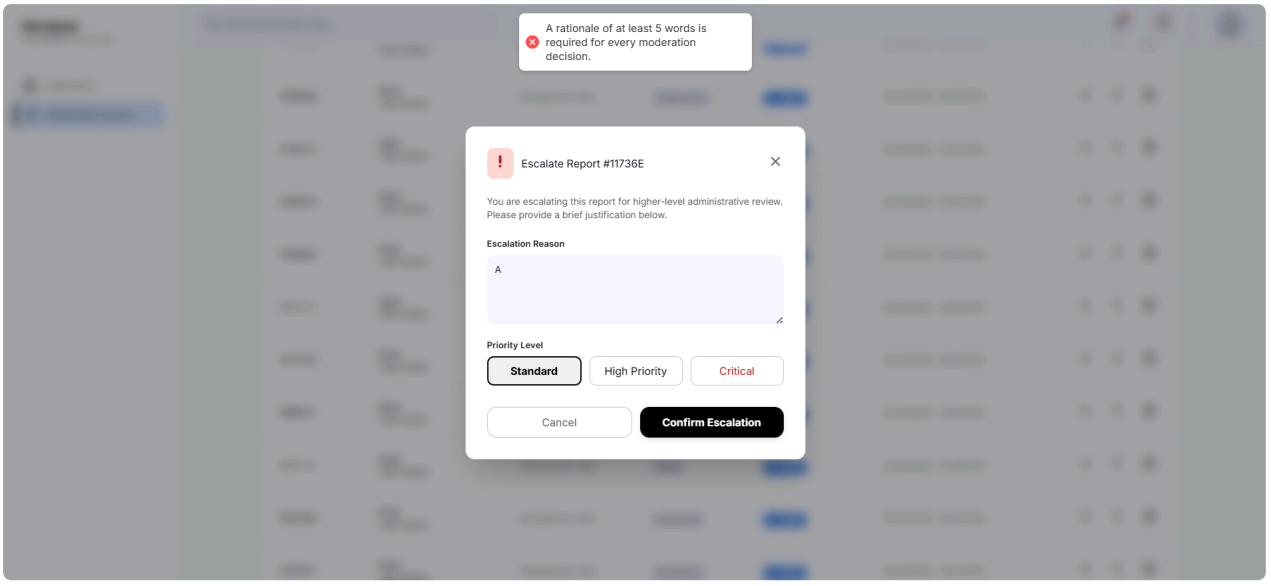
- None.

7. Prototype Requirement

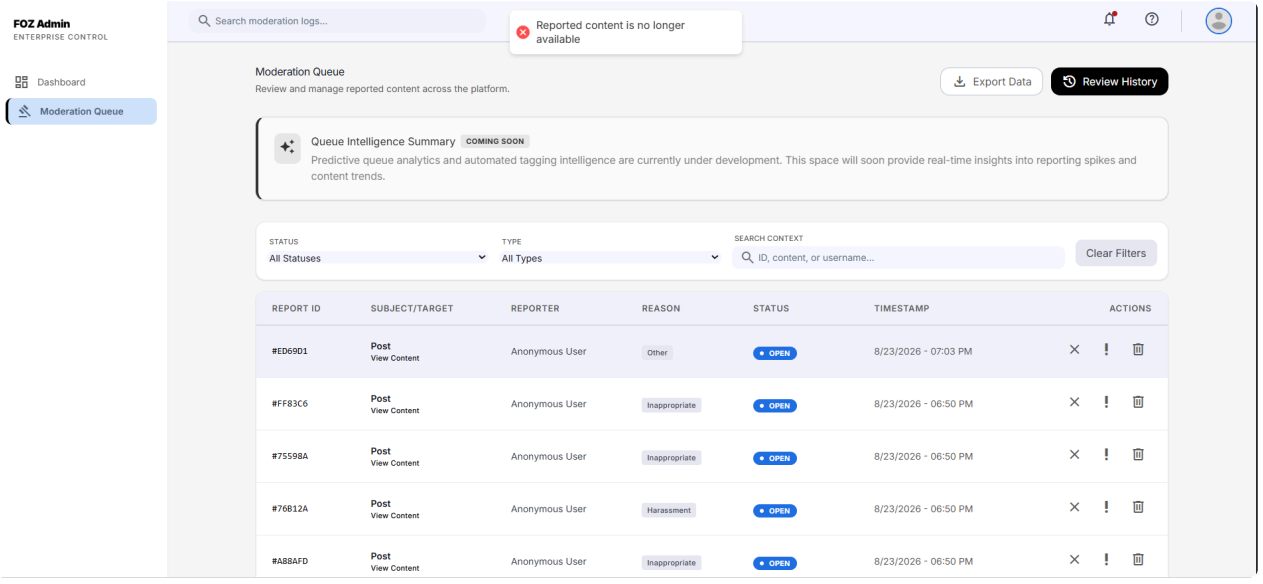
- Screen 1 (Basic Flow):



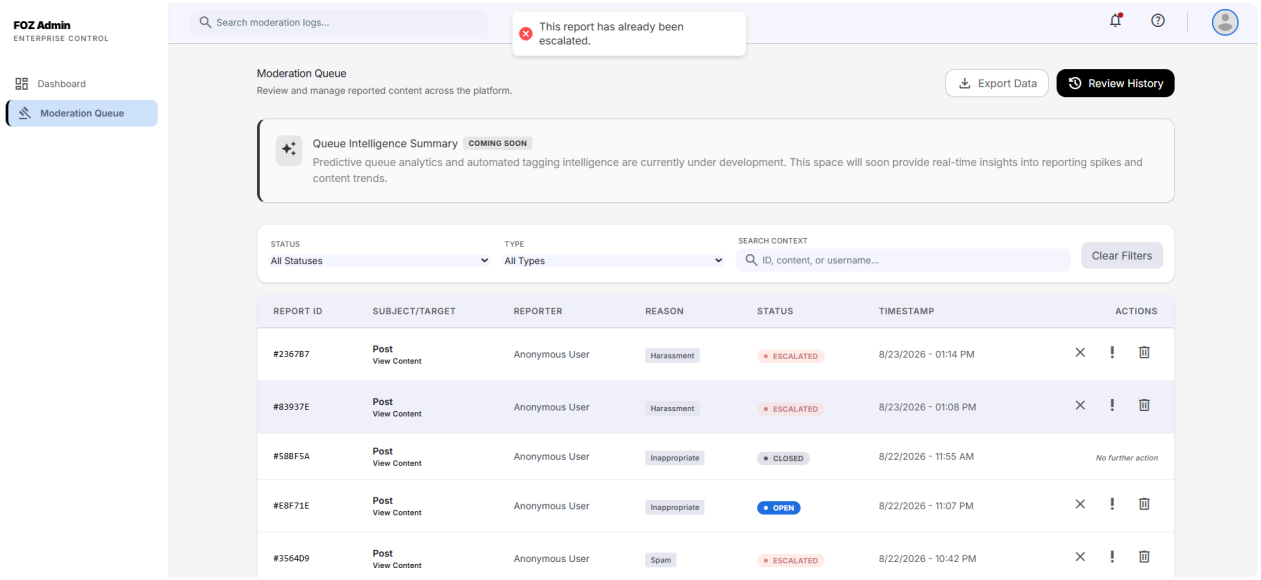
- Screen 2 (Alternative Flow 1):



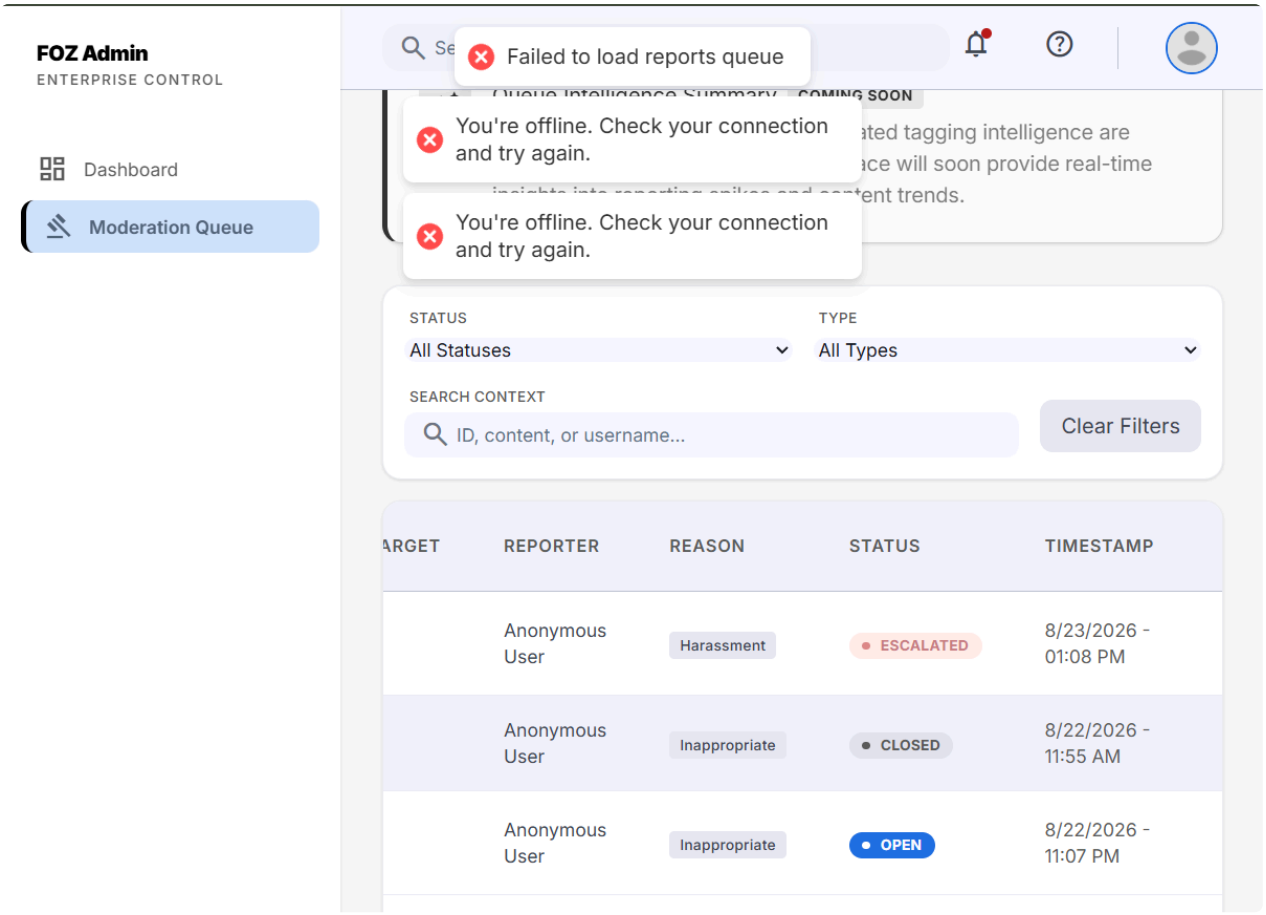
- Screen 3 (Alternative Flow 2):



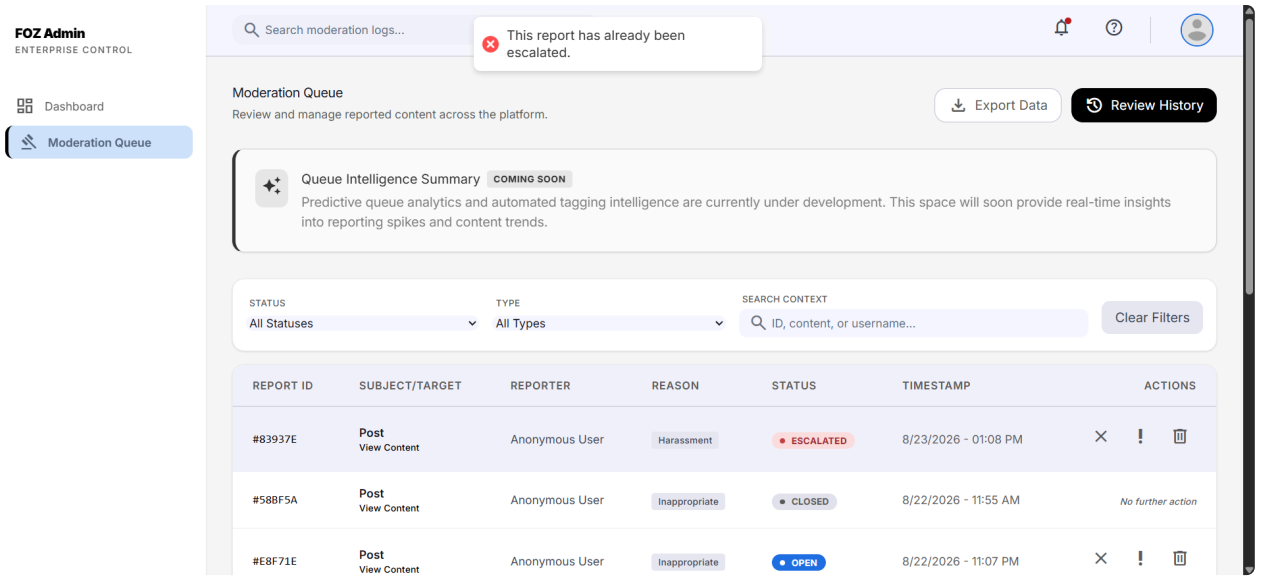
- Screen 4 (Alternative Flow 3):



- Screen 5 (Alternative Flow 4):



- **Screen 6 (Alternative Flow 5):**



Use-Case Specification: Delete Any Content

Reviewed and Edited by: Nguyen Hai Nam | Reviewed by: Vo An Phu

Project Name: FOZ
Version: 1.1

Document identifier: FOZ-UCS-19

1. Delete Any Content

- **Use-Case ID:** UC-19
- **Actor(s):** Administrator

1.1 Brief Description

Allows an Administrator to permanently delete an authorized post or comment and its dependent comments, replies, reactions, and attachments.

2. Preconditions

- The user is authenticated and has role `administrator`.
- The target is an existing active post or comment selected from an escalated report or an authorized Admin content view.
- The target is eligible for permanent deletion and has not already been deleted.

3. Flow of Events

3.1 Basic Flow

1. The use case begins when an Administrator views an eligible target in the Admin Dashboard.

2. The Administrator selects the "Force Delete" option.
3. The system opens a confirmation modal showing the target and requiring an administrative rationale.
4. The Administrator enters a rationale of at least five words and clicks "Confirm Deletion".
5. The system validates the rationale, authorization, target state, and target type.
6. In one transaction, the system permanently deletes the target and its dependent comments or replies, reactions, and attachments, updates any related report to `closed`, and records a `CONTENT_FORCE_DELETE` entry in `audit_logs` containing the actor, target type, target ID, timestamp, and rationale.
7. The system invalidates affected feed, search, and moderation-queue data and displays a success message.

3.2 Alternative Flows

3.2.1 Administrator Cancels Deletion

1. If at step 4 the Administrator clicks "Cancel", closes the modal, or presses Escape.
2. The modal closes, no data is deleted, and the content remains intact.
3. The use case ends.

3.2.2 Missing or Insufficient Rationale

1. If at step 4 the Administrator clicks "Confirm Deletion" without entering a rationale or with a rationale shorter than five words after trimming whitespace.
2. The system displays: "Rationale must be at least 5 words long for content deletion."
3. The modal remains open, the deletion is prevented, and the flow resumes at step 4.

3.2.3 Content Not Found or Already Deleted

1. If at step 5 the target post or comment no longer exists or has already been deleted.
2. The system makes no changes and displays: "This content is no longer available."
3. The use case ends.

3.2.4 Deletion Transaction Fails

1. If at step 6 the content, dependent-record, report, or audit update fails.
2. The system rolls back the entire transaction and displays: "The content could not be deleted. Please try again."
3. The use case ends.

4. Special Requirements

- **Authorization:** Administrator permission must be enforced at the API boundary.

- **Audit:** Permanent deletion must be recorded with the action, actor, target, timestamp, and rationale.
- **Data Integrity:** Dependent comments, replies, reactions, attachments, and related report references must be handled without orphan records.
- **Atomicity:** Content deletion and audit logging must succeed or fail together.

5. Postconditions

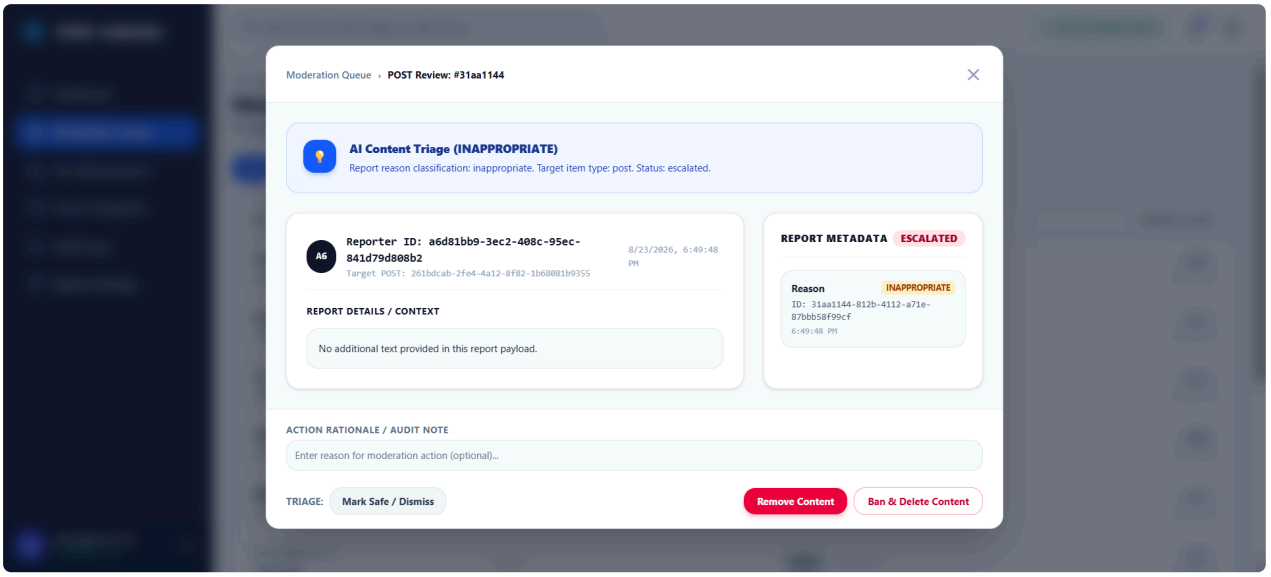
- The target content and its dependent comments or replies, reactions, and attachments no longer exist in the database.
- Related reports are closed or otherwise marked as resolved.
- The deletion is absent from public feeds, search results, and moderation queues.
- A corresponding audit entry exists.

6. Extension Points

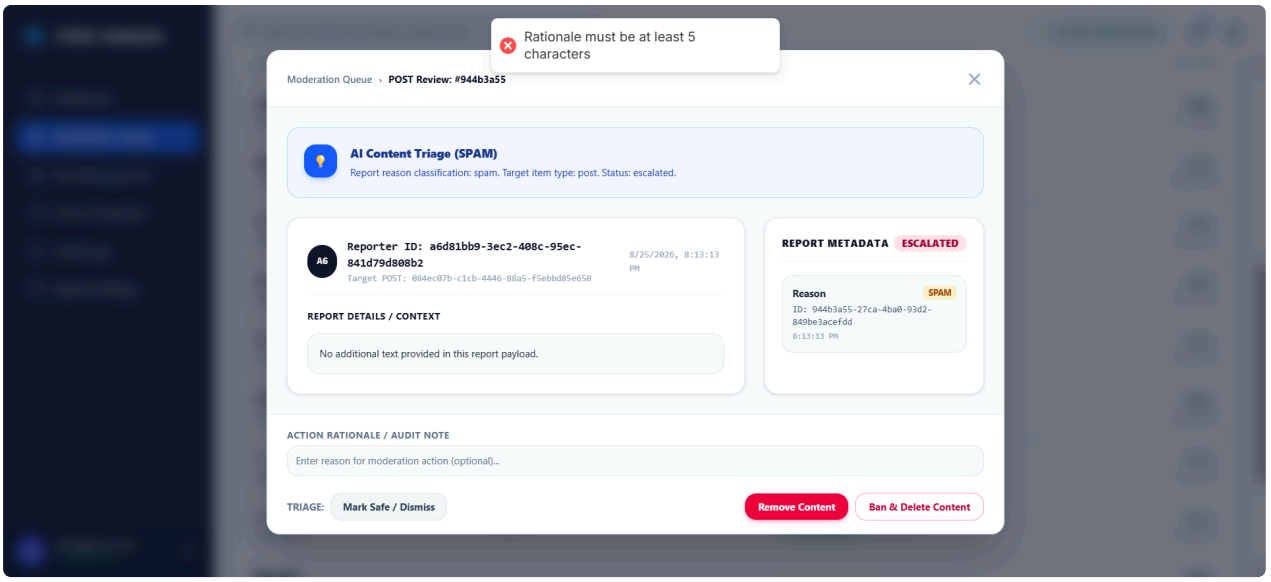
- None.

7. Prototype Requirement

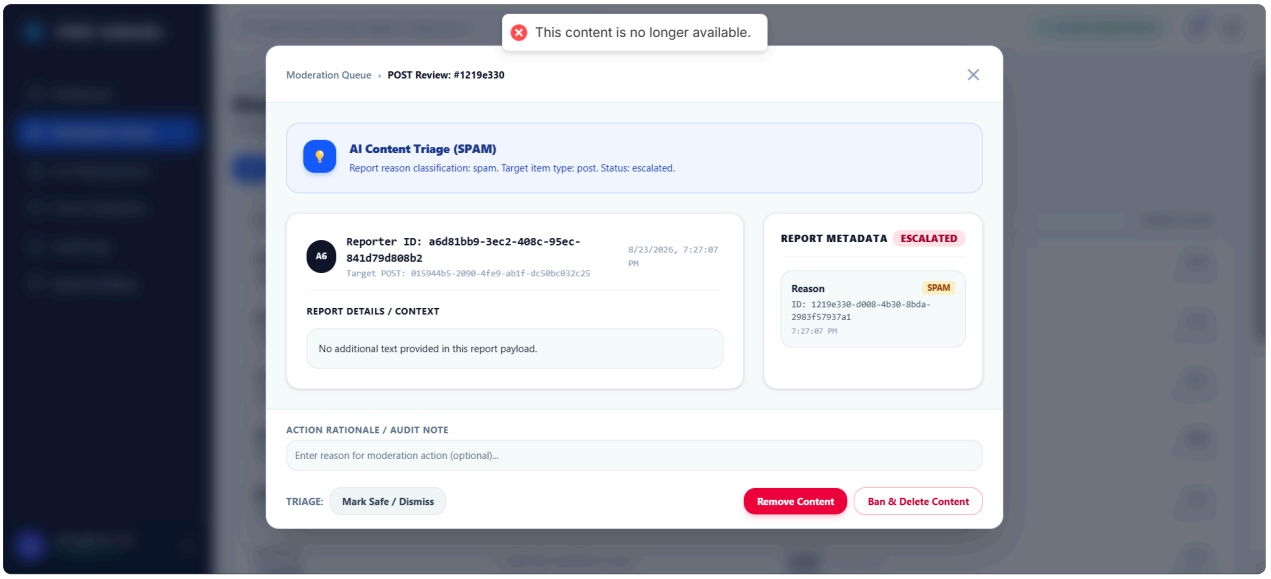
- Screen 1 (Basic Flow):



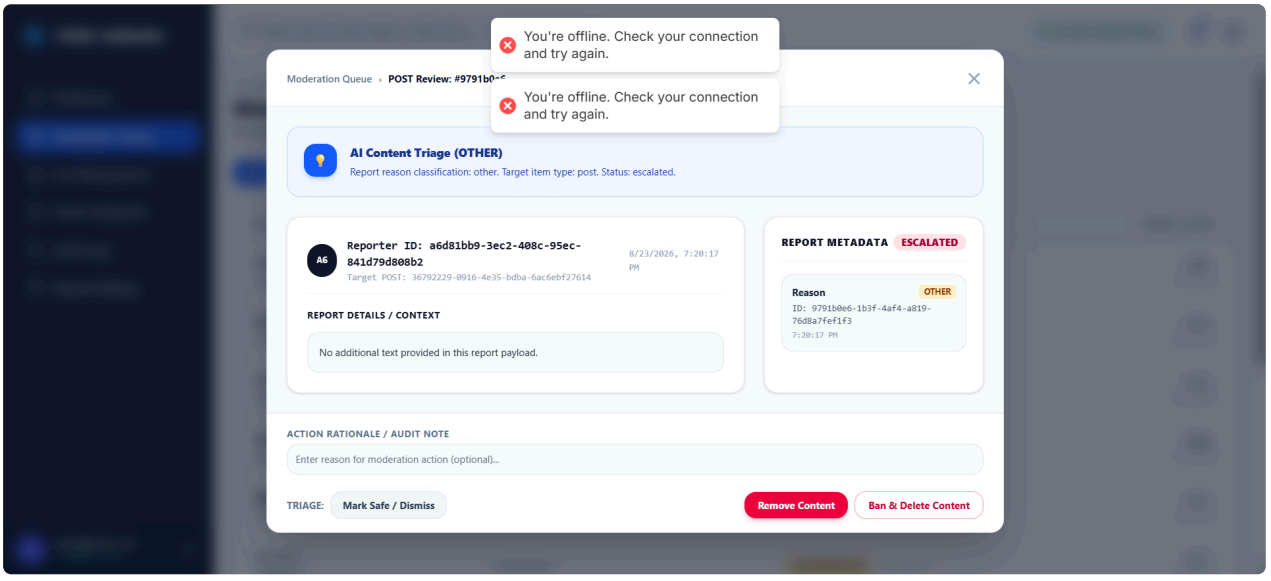
- Screen 2 (Alternative Flow 2):



- Screen 3 (Alternative Flow 3):



- Screen 3 (Alternative Flow 4):



Use-Case Specification: Permanently Remove User

Reviewed and Edited by: Nguyen Hai Nam | Reviewed by: Vo An Phu

Project Name: FOZ
Version: 1.1

Document identifier: FOZ-UCS-20

1. Permanently Remove User

- **Use-Case ID:** UC-20
- **Actor(s):** Administrator

1.1 Brief Description

Allows an Administrator to permanently remove an eligible user account, revoke all active sessions, and purge its profile, submitted reports, posts, comments, reactions, and attachments.

2. Preconditions

- The user is authenticated and has role `administrator`.
- The target account exists, has role `standard` or `moderator`, and is not the acting Administrator's account.
- Administrator accounts are not eligible for removal through this use case.

3. Flow of Events

3.1 Basic Flow

1. The use case begins when an Administrator opens the User Management page.
2. The Administrator clicks "Delete" next to the target user account.
3. The system opens the User Action Modal, explains that the action is permanent, and requires an administrative rationale of at least five words.
4. The Administrator enters the rationale and clicks "Delete Account".
5. The system validates the rationale, target account role, target state, and Administrator authorization.
6. In one transaction, the system revokes all active `refresh_sessions`, removes the user's profile, submitted reports, posts, comments, reactions, and attachments, deletes the `user_accounts` record, and records a `USER_DELETE` entry in `audit_logs` containing the actor, target, timestamp, and rationale.
7. The system invalidates the deleted user's session state and displays a success message. Because the account no longer exists, the user cannot authenticate again.

3.2 Alternative Flows

3.2.1 Administrator Cancels Deletion

1. If at step 4 the Administrator clicks "Cancel", closes the modal, or presses Escape.
2. The modal closes, no account data is deleted, and the use case ends.

3.2.2 Missing or Insufficient Rationale

1. If at step 4 the Administrator clicks "Delete Account" without entering a rationale or with a rationale shorter than five words after trimming whitespace.
2. The system displays: "Rationale must be at least 5 words long for account deletion."
3. The modal remains open, the deletion is prevented, and the flow resumes at step 4.

3.2.3 Data-Wipe Transaction Fails

1. If at step 6 a dependent-record deletion, session revocation, account deletion, or audit-log operation fails.
2. The system rolls back the entire transaction and displays: "The user account could not be removed. Please try again."
3. The use case ends.

4. Special Requirements

- **Authorization:** Administrator permission and target-account eligibility rules must be enforced at the API boundary.
- **Data Integrity:** Dependent records must be removed in a transaction with foreign keys handled correctly; no orphan records may remain.
- **Audit:** The account-removal action must be logged without storing unnecessary sensitive data.
- **Atomicity:** Session revocation, data removal, account deletion, and audit logging must succeed or fail together.

5. Postconditions

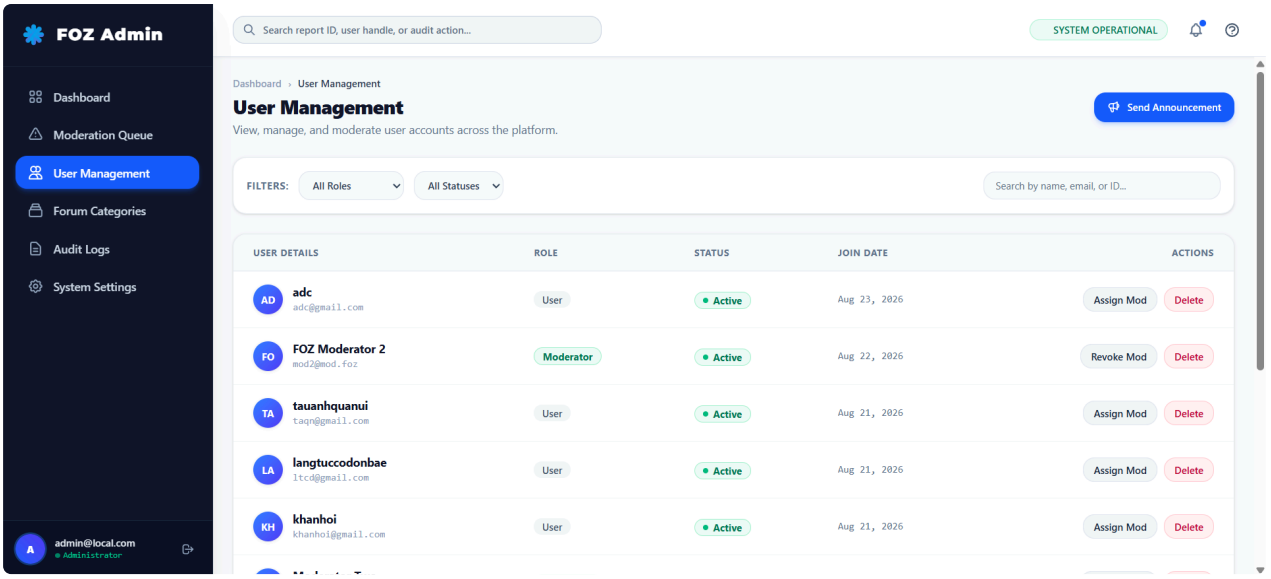
- The target `user_accounts` record no longer exists.
- The target user's profile, submitted reports, posts, comments, reactions, and attachments have been removed.
- All active sessions for the target user are revoked.
- A corresponding `USER_DELETE` audit entry exists.

6. Extension Points

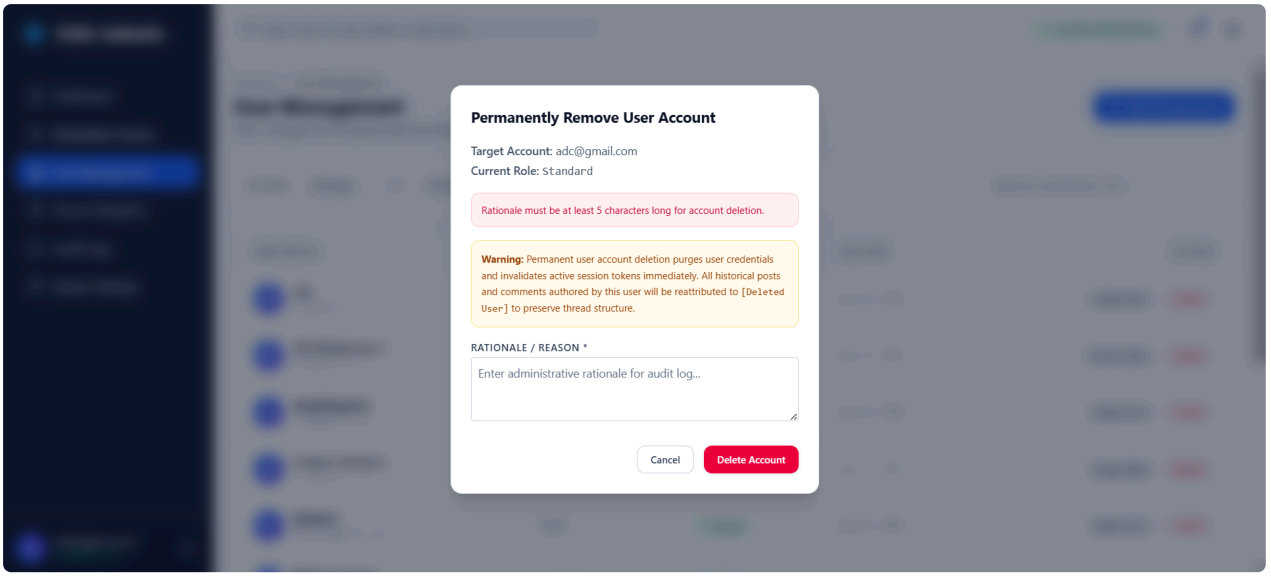
- None.

7. Prototype Requirement

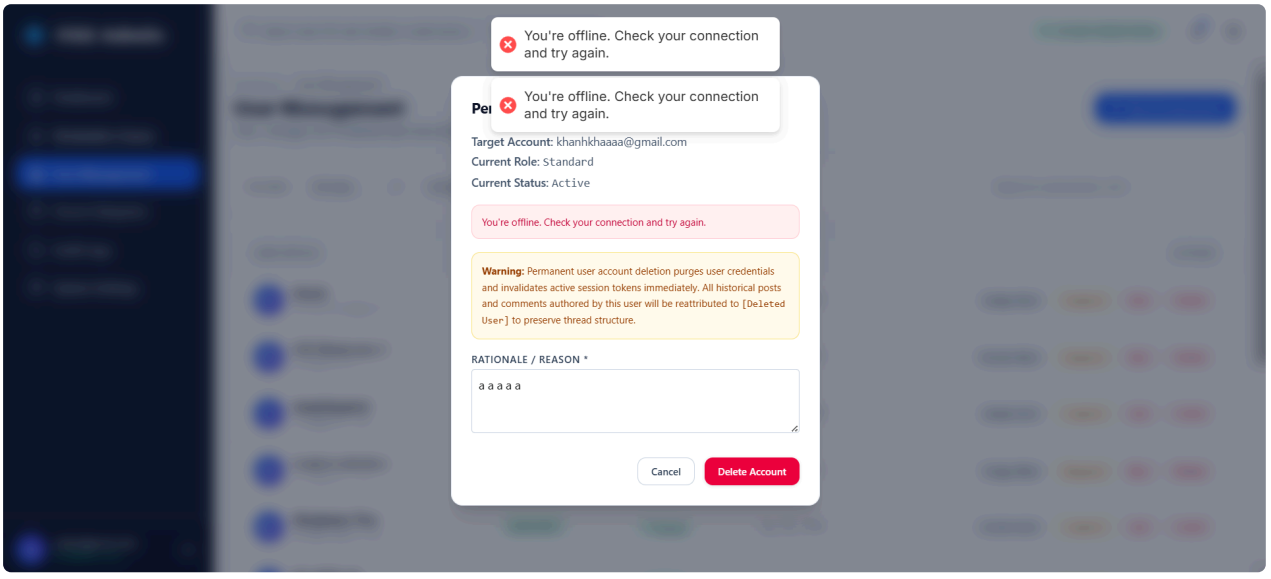
- Screen 1 (Basic Flow):



- Screen 2 (Alternative Flow 2):



- Screen 3 (Alternative Flow 3):



Use-Case Specification: Assign Moderator Role

Reviewed and Edited by: Nguyen Hai Nam | Reviewed by: Vo An Phu

Project Name: FOZ
Version: 1.1

Document identifier: FOZ-UCS-21

1. Assign Moderator Role

- **Use-Case ID:** UC-21
- **Actor(s):** Administrator

1.1 Brief Description

Allows an Administrator to promote an eligible Standard User to the Moderator role.

2. Preconditions

- The user is authenticated and has role `administrator`.
- The target account exists, is active, and currently has role `standard`.
- The target account is not the acting Administrator's account; administrator accounts are not eligible for this role assignment.

3. Flow of Events

3.1 Basic Flow

1. The use case begins when an Administrator opens the User Management page.
2. The Administrator selects "Assign Moderator" next to a target Standard User.
3. The system opens the User Action Modal displaying the target's current role and the permitted new role.
4. The Administrator selects `Moderator` and enters an administrative rationale of at least five words.
5. The Administrator clicks "Confirm Role Update".
6. The system validates the rationale, target account, current role, and Administrator authorization.
7. In one transaction, the system changes the target's `role` from `standard` to `moderator` and records an audit entry containing the Administrator, target user, previous role, new role, timestamp, and rationale.
8. The system displays a success message and refreshes the User Management page.

3.2 Alternative Flows

3.2.1 Administrator Cancels Role Assignment

1. If at step 5 the Administrator clicks "Cancel", closes the modal, or presses Escape.
2. The modal closes, no role changes are made, and the use case ends.

3.2.2 Missing or Insufficient Rationale

1. If at step 5 the Administrator clicks "Confirm Role Update" without entering a rationale or with a rationale shorter than five words after trimming whitespace.
2. The system displays: "Rationale must be at least 5 words long for role assignment."
3. The modal remains open, the role update is prevented, and the flow resumes at step 4.

3.2.3 Target Not Found or Not Eligible

1. If at step 6 the target account no longer exists, is inactive, is banned, or no longer has role `standard`.
2. The system rejects the request, displays: "This user is not eligible for Moderator assignment.", and makes no role change.
3. The use case ends.

3.2.4 Concurrent Role Update

1. If at step 7 another Administrator changes the target role before this transaction commits.
2. The system rolls back this request, displays the target's current role, and asks the Administrator to refresh the User Management page.
3. The use case ends.

3.2.5 Role Update or Audit Failure

1. If at step 7 the role update or audit write fails.
2. The system rolls back the transaction and displays: "The role could not be updated. Please try again."
3. The use case ends.

4. Special Requirements

- **Authorization:** Administrator permission and the allowed `standard` → `moderator` transition must be enforced at the API boundary.
- **Audit:** The previous role, new role, actor, target, timestamp, and rationale must be recorded.
- **Atomicity:** The role update and audit entry must succeed or fail together.

5. Postconditions

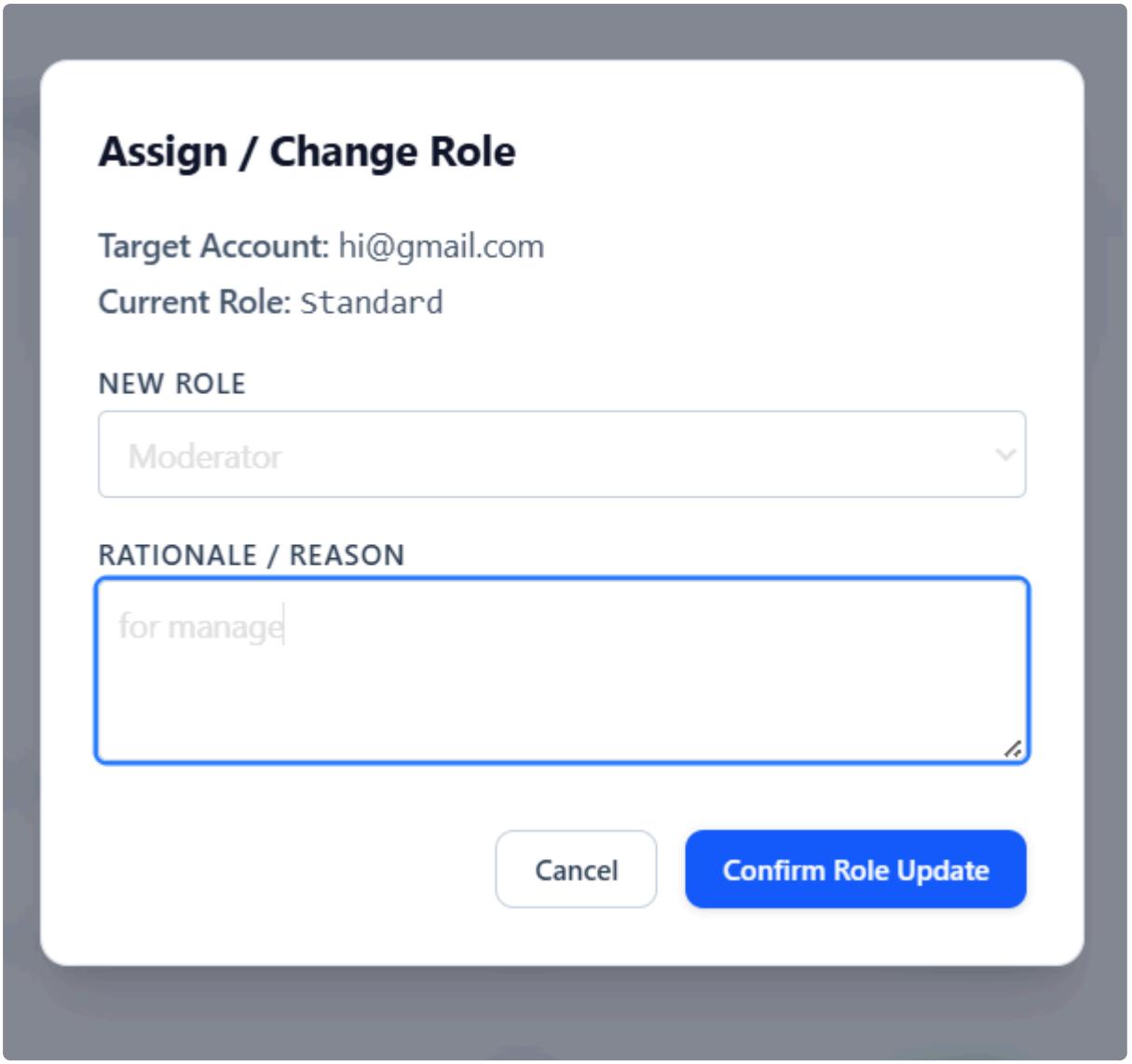
- The target user's role is `moderator` in `user_accounts`.
- An audit entry records the successful role transition.
- If an alternative flow occurs, the target role remains unchanged.

6. Extension Points

- None.

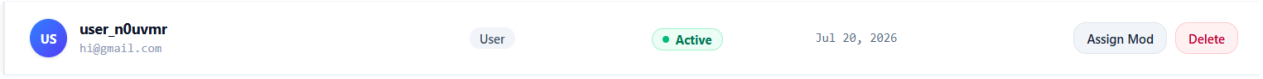
7. Prototype Requirement

- Screen 1 (Basic Flow):

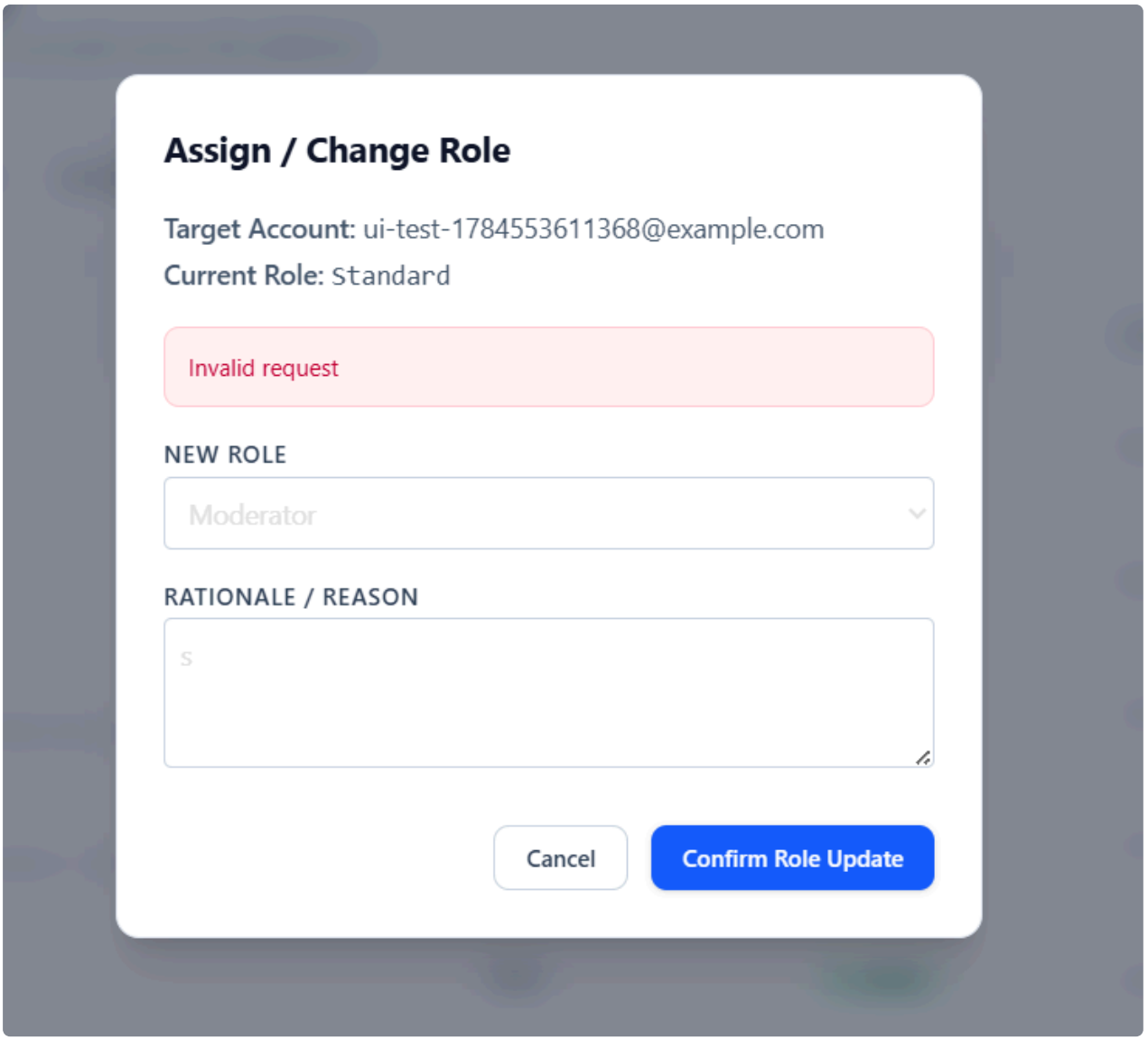


The Admin will assign Moderator role in the User Management

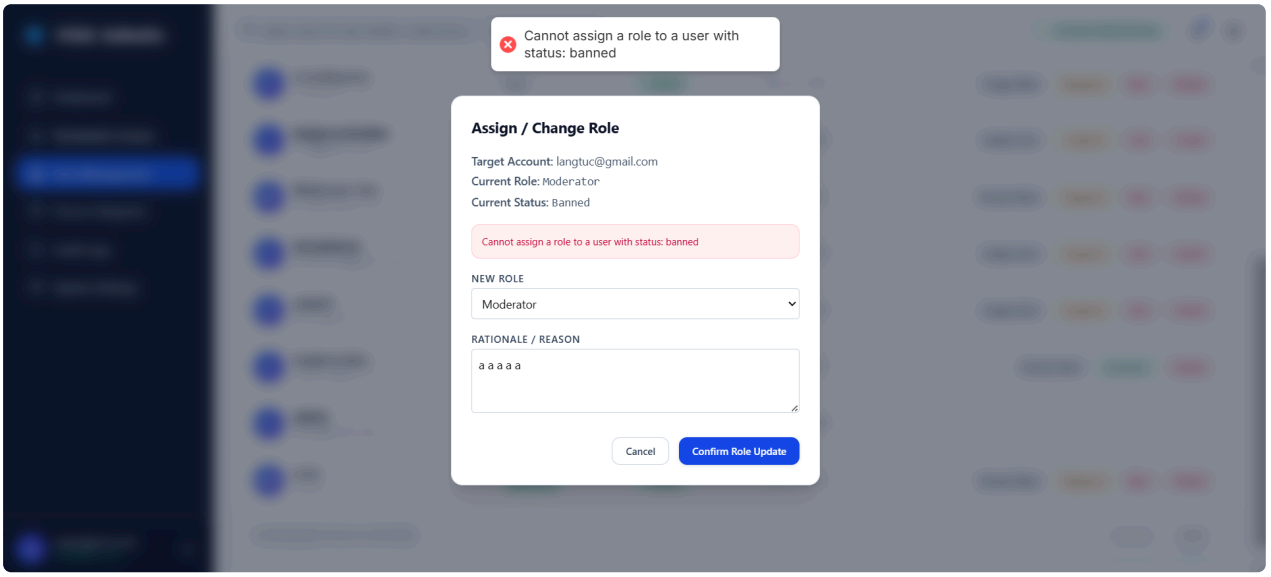
- Screen 2 (Alternative Flow 1):



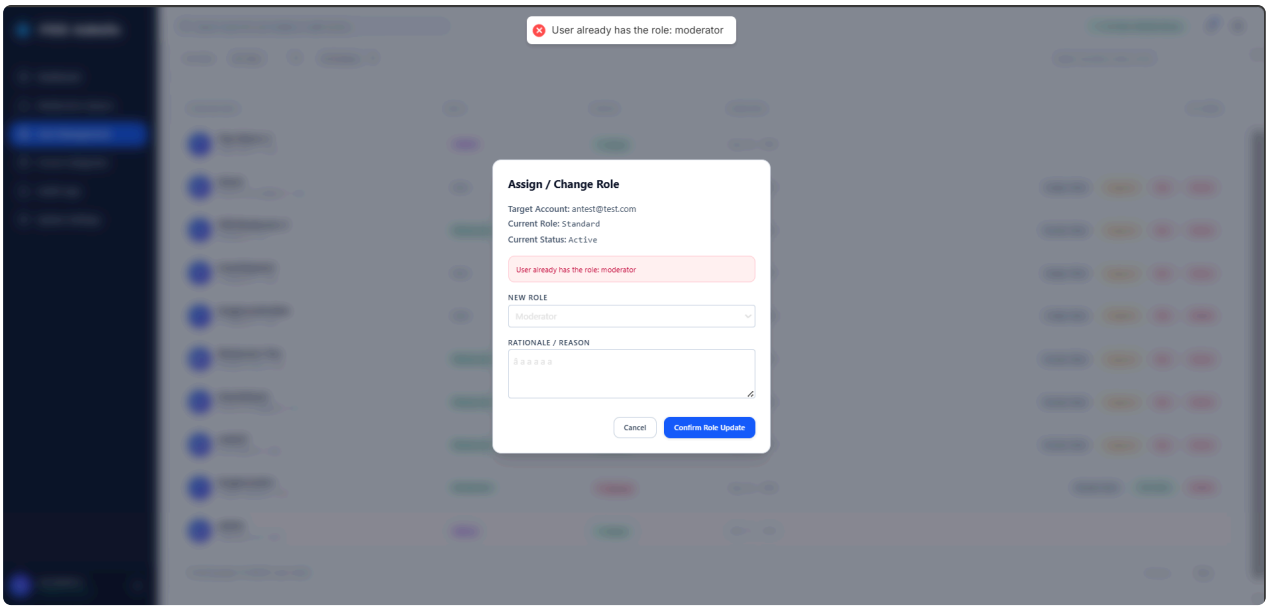
- Screen 3 (Alternative Flow 2):



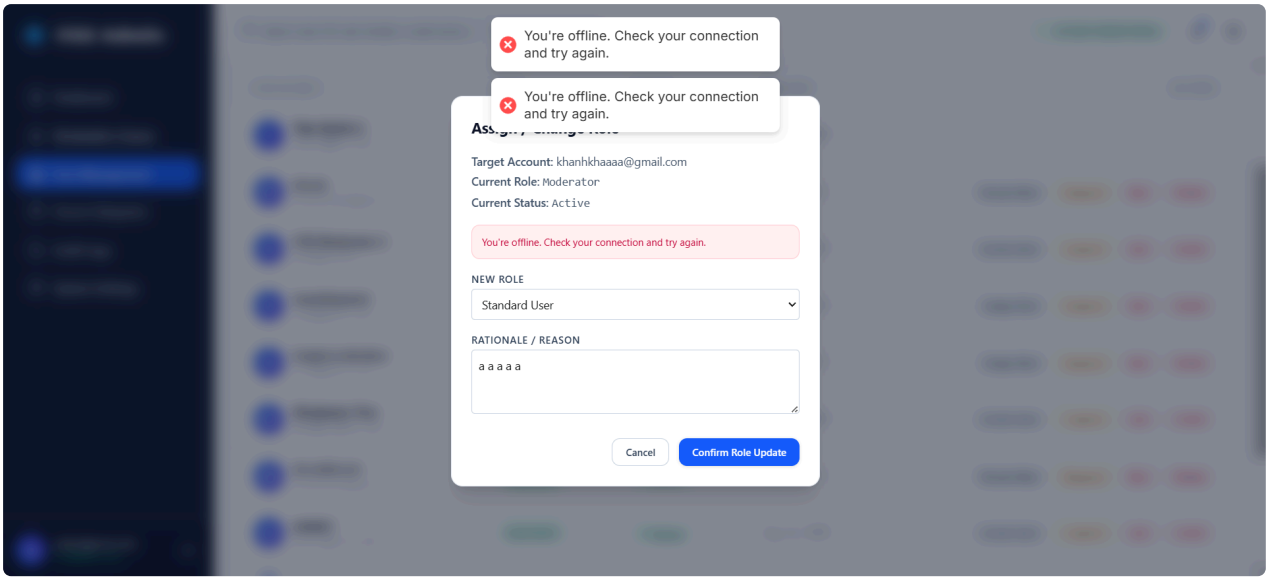
- Screen 3 (Alternative Flow 3):



- Screen 4 (Alternative Flow 4):



- Screen 5 (Alternative Flow 5):



B - Software Architecture: System Context Diagram

Work performed by: Nguyễn Hải Trọng | Reviewed by: Nguyễn Hải Nam, Võ An Phú, Nguyễn Tuấn An | Edited by: Võ An Phú, Nguyễn Tuấn An

1. Technology Stack

Frontend

Technology	Purpose
React 19	UI component framework (SPA)
TypeScript 6	Type-safe application code
Vite 8	Build tool and dev server with HMR

Technology	Purpose
Tailwind CSS 4	Utility-first styling (vanilla config, no UI library)
react-router-dom 7	Client-side routing
@tanstack/react-query 5	Server-state management and caching
zustand 5	Lightweight client-state management (auth store)
react-hook-form 7	Form state management and validation
react-hot-toast 2	Toast notification system
react-icons 5	Icon library
Vitest 4	Unit testing framework

Backend

Technology	Purpose
Node.js (LTS ≥ 24)	JavaScript runtime (ESM)
Express 5	Web framework for RESTful API
TypeScript 6	Type-safe server code
Drizzle ORM	SQL-visible ORM with type-safe queries
drizzle-kit	Database migration tooling
Zod 4	Runtime schema validation at API boundaries
bcrypt 6	Password hashing
jsonwebtoken 9	JWT access and refresh token management
express-rate-limit 8	Rate limiting for abuse-prone endpoints
multer 2	Multipart file upload handling
sharp 0.35	Server-side image processing and optimization
cookie-parser	HTTP cookie parsing for refresh tokens
Vitest 4	Unit and integration testing

Database

Technology	Purpose
PostgreSQL 16	Primary relational database (provisioned via Docker)

Technology	Purpose
postgres.js	Node.js PostgreSQL client driver

External Services

Service	Purpose
Cloudinary	Cloud-based image/media storage and CDN delivery
Google Generative AI API (Gemini)	AI-powered post & comment summarization (TL;DR) with fallback provider chain
Google OAuth 2.0	Social authentication (Google Sign-In)

Shared Module

Technology	Purpose
`src/shared/`	Zod schemas, DTOs, enums - single source of truth imported by both frontend and backend via `#shared/*` path aliases. No npm workspace, no codegen, no tRPC.

---

2. C4 Level 1 - System Context Diagram

This diagram shows FOZ as a single box, the actors who interact with it, and the external systems it depends on.

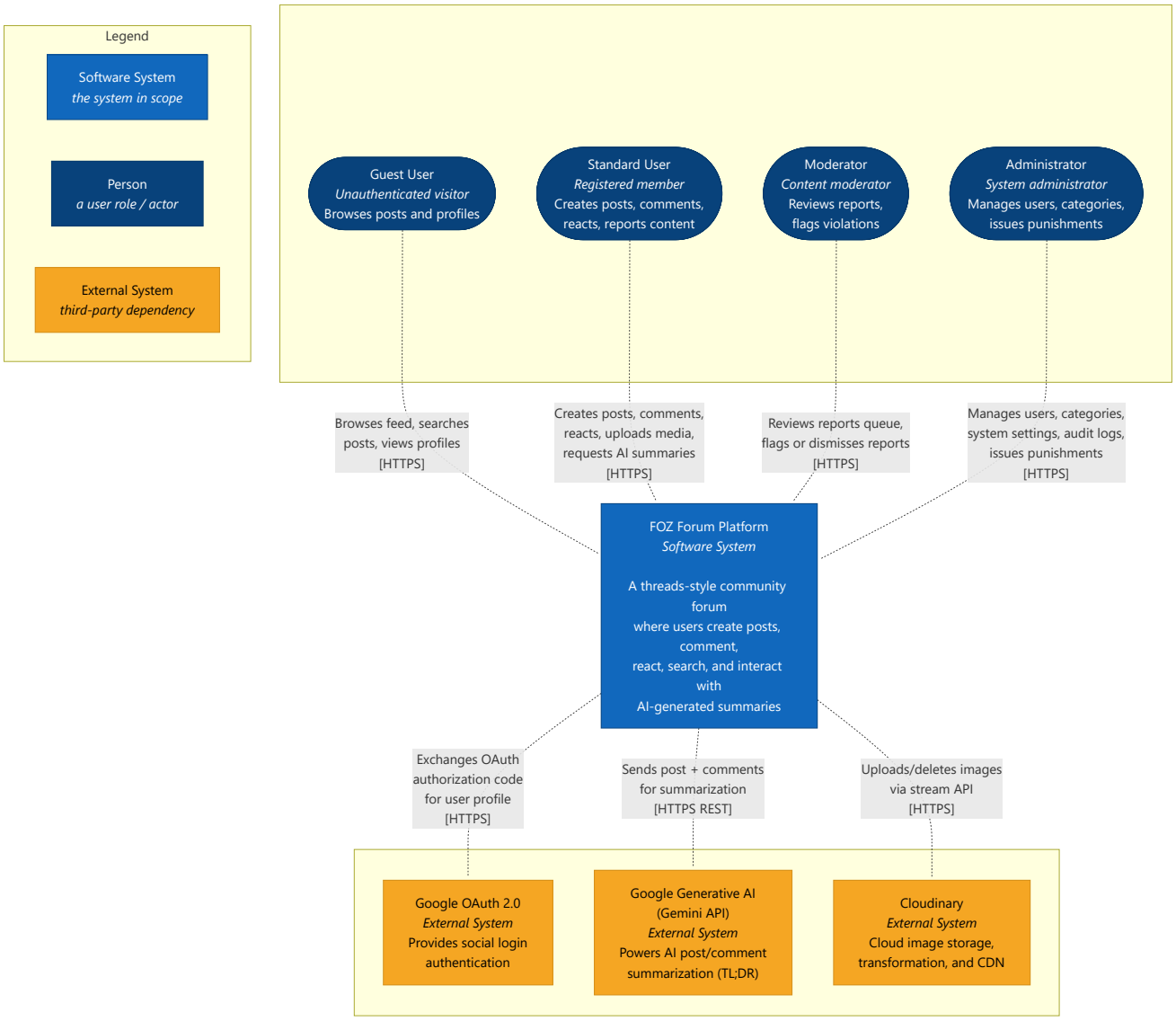


Diagram Explanation

Four actor types interact with the system:

- **Guest** — Unauthenticated. Can browse the feed, search, and view profiles. Cannot create content, react, report, or use AI features.
- **Standard User** — Authenticated member. Everything a guest can do, plus creating posts (text + images), commenting (nested replies), reacting, editing/deleting own content, managing their profile, reporting, requesting AI TL;DR summaries, and changing their password.
- **Moderator** — Standard user privileges plus the moderation queue (review, flag, or dismiss reports). Cannot issue punishments directly — this enforces separation of powers.
- **Administrator** — Moderator privileges plus the admin dashboard: user management (warn, suspend, ban), category management, system settings, audit logs, and punishment issuance.

The system depends on three external services:

- **Google OAuth 2.0** — Social login. The backend exchanges an auth code for a token, fetches the Google profile, and links or creates a FOZ account.
- **Google Generative AI (Gemini)** — AI post/comment summarization. Uses a fallback chain across multiple Gemini model versions for resilience.
- **Cloudinary** — Image storage and CDN. Attachments and avatars are streamed to Cloudinary; CDN URLs are stored in the database.

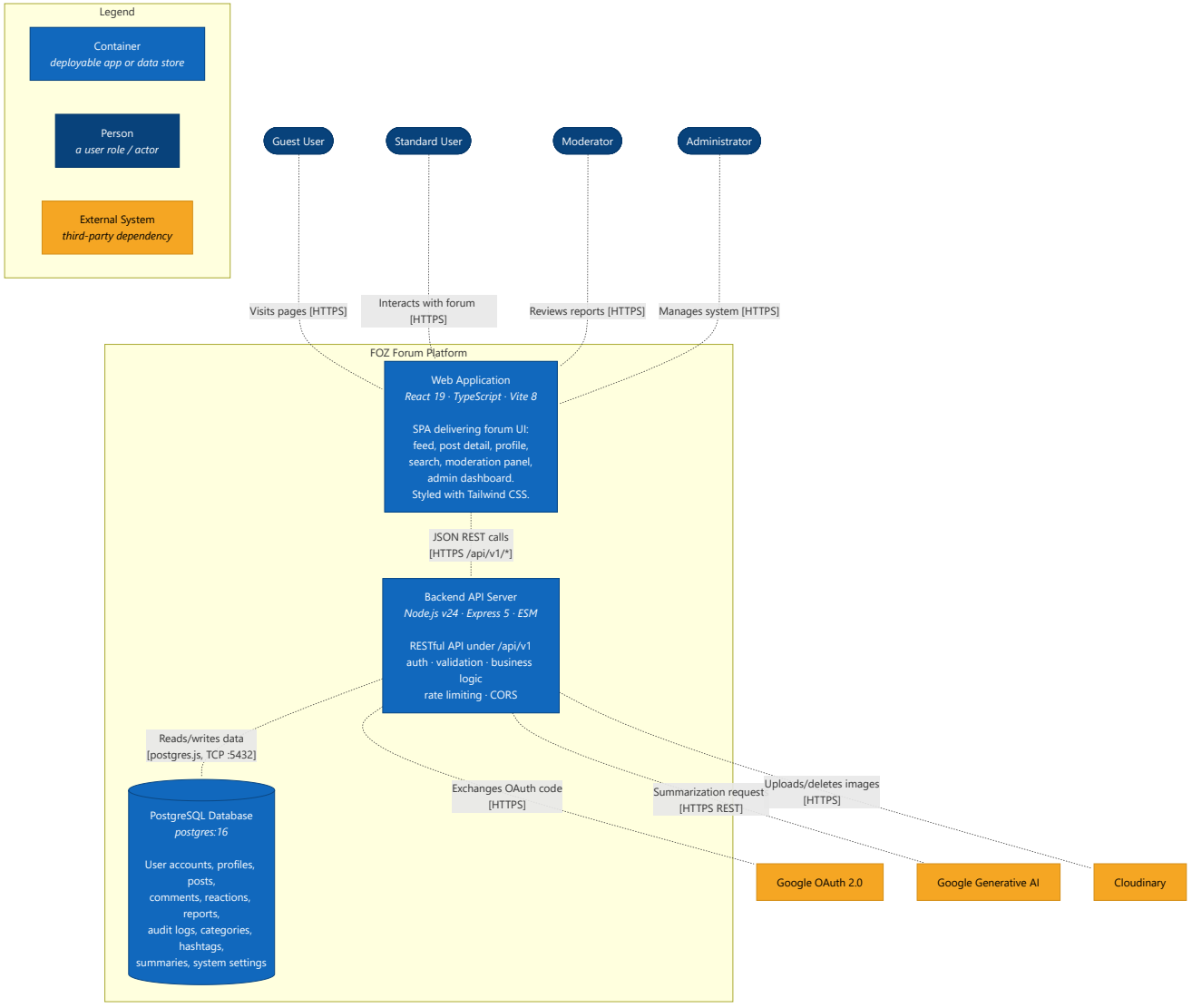
C - Software Architecture: Container Diagram and Component Diagram

Work performed by: Nguyễn Hải Nam | Reviewed by: Võ An Phú, Nguyễn Tuấn An | Edited by: Võ An Phú, Nguyễn Tuấn An

3. C4 Level 2 - Container Diagram

Zooms into FOZ to show its deployable units (containers), the technology each uses, and how they talk to each other. In C4 terms a "container" is a separately runnable process or data store.

3.1 Diagram



3.2 Container Descriptions

- **Web Application (SPA)** — Renders the entire forum UI: feed (latest + for-you), post detail with comments/reactions, profiles, search, moderation queue, admin dashboard, and auth screens. No direct DB access, all data comes from the API via REST.
  - **Technology**: React 19 + TypeScript + Vite 8, styled with Tailwind CSS v4. Key libs: `react-router-dom` (routing), `@tanstack/react-query` (server-state cache), `zustand` (auth state), `react-hook-form` (forms), `react-hot-toast` (toasts). Tested with Vitest + Testing Library.
  - **Communication**: Sends JSON requests to `/api/v1/*`. Dev: Vite proxies to `localhost:4000`. Prod: Vercel rewrites to the Render backend. Auth uses `Bearer` tokens; refresh uses an HttpOnly cookie.
- **Backend API Server** — RESTful API under `/api/v1` covering auth, posts, comments, reactions, profiles, search, upload, moderation, admin, hashtags, and a `/health` probe. Includes AI summarization, Zod validation, rate limiting, CORS, and a central error handler.
  - **Technology**: Node.js v24 (ESM) + Express 5. Auth: `jsonwebtoken` + `bcrypt`. DB: Drizzle ORM + `postgres.js` + `drizzle-kit` migrations. Validation: `zod`. Uploads: `multer` + `sharp`. Env: fail-fast Zod validation at startup.

    - **Communication**: Receives JSON from the SPA. Connects to Postgres via `postgres.js` (TCP :5432, pool max 10). Calls Google OAuth, Gemini, and Cloudinary over HTTPS.
- **PostgreSQL Database** — Stores all relational data: users, profiles, refresh sessions, posts, attachments, comments, reactions, reports, audit logs, categories, settings, hashtags, summaries, and social links.
    - **Technology**: `postgres:16` Docker service (`foz_db`, port 5432, volume `foz_pg_data`, database `foz_dev`). Migrations via `drizzle-kit`.
    - **Communication**: Accepts connections only from the Backend API Server (TCP :5432). No browser or SPA access.
- **Google OAuth 2.0** *(external)* — Social login. Backend exchanges auth codes for tokens and fetches user profiles. HTTPS only, called by the backend.
- **Google Generative AI / Gemini** *(external)* — On-demand AI TL;DR summaries (post snapshot, highlights, sentiment). Uses `gemini-3.6-flash` / `gemini-flash-latest` / `gemini-3.5-flash` with a fallback chain. HTTPS only, called by the backend.
- **Cloudinary** *(external)* — Image storage and CDN. Attachments and avatars are streamed up; CDN URLs are persisted in Postgres. HTTPS only, called by the backend.

3.3 Relationships Summary

From	To	Protocol	Description
User (all roles)	Web Application	HTTPS	Browsing, form submission, and UI interaction
Web Application	Backend API Server	HTTP/HTTPS JSON	REST calls to `/api/v1/*`. Dev: Vite proxies to `localhost:4000`. Authenticated calls pass `Bearer` tokens; refresh uses HTTP-only credentials
Backend API Server	PostgreSQL Database	SQL over TCP :5432	Drizzle ORM queries via `postgres-js` driver
Backend API Server	Google OAuth 2.0	HTTPS	Exchanges authorization codes for tokens, fetches user profiles
Backend API Server	Google Gemini API	HTTPS REST	Sends structured prompts, receives

From	To	Protocol	Description
			AI-generated summaries
Backend API Server	Cloudinary	HTTPS	Streams image uploads and issues delete requests

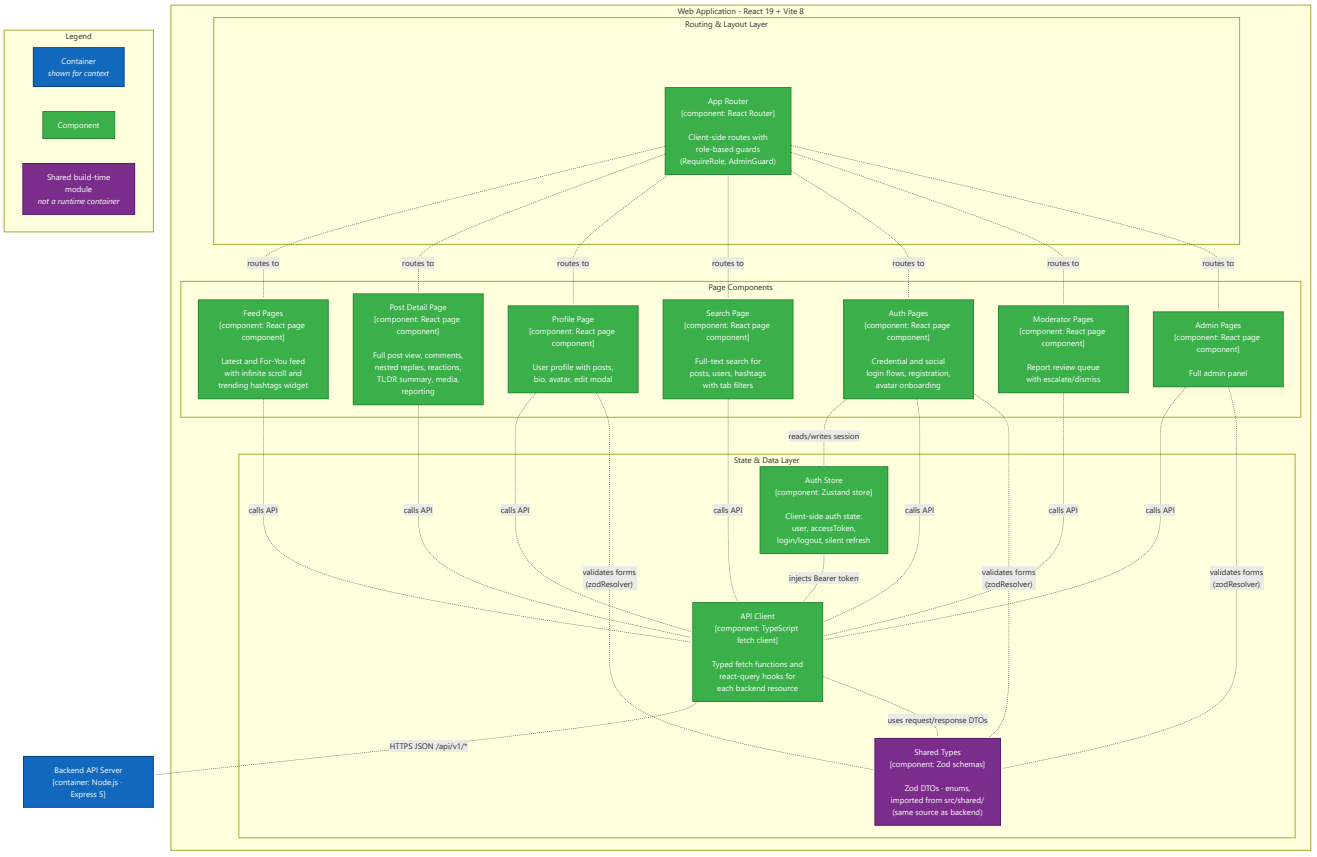
---

4. C4 Level 3 - Component Diagrams

These diagrams zoom into the Web Application and Backend API Server containers to show their internal components. The database and external systems appear for context.

4.1 Frontend - Web Application

4.1.1 Diagram



4.1.2 Component Descriptions

- **App Router (`App.tsx`)**
  - **Responsibility**: Declares all client-side routes (`/latest`, `/foryou`, `/posts/:id`, `/profiles/@:username`, `/search`, `/login`, `/register`, `/moderator`, `/admin/*`, etc.) and wraps protected routes with `RequireRole` and `AdminGuard` components that check the auth store for the required role.

    - **Relationships**: Receives navigation actions from the User; renders matching Page Components; injects global layout and dialog controls.
- **Feed Pages (`HomePage`)**
    - **Responsibility**: Renders the post feed in two modes: "Latest" (chronological) and "For You" (engagement-ranked). Supports infinite scroll via cursor-based pagination. Includes a TrendingHashtagsWidget sidebar.
    - **Relationships**: Calls APIClient for post, feed, and reaction data.
- **Post Detail Page (`PostPage`)**
    - **Responsibility**: Full post view with body text, media gallery, reaction controls, comment thread with nested replies, AI-generated TL;DR summary card, report button, and post edit/delete for the owner.
    - **Relationships**: Calls APIClient for post, comment, reaction, summary, and report data.
- **Profile Page (`ProfilePage`)**
    - **Responsibility**: Displays a user's profile (display name, bio, avatar, location, website) and their posts. Includes a ProfileEditModal for the profile owner to update their information and upload avatars.
    - **Relationships**: Calls APIClient for profile and user post data; validates the edit-profile form against Shared Types.
- **Search Page (`SearchPage`)**
    - **Responsibility**: Provides full-text search across posts, users, and hashtags with tab-based result filtering.
    - **Relationships**: Calls APIClient for search queries.
- **Auth Pages (`LoginPage`, `RegisterPage`, `OnboardingAvatarPage`, `ChangePasswordPage`, `SettingsPage`)**
    - **Responsibility**: Login page (email/password + Google OAuth), register page (with Zod-validated form), onboarding avatar upload, and change-password page. `SettingsPage` (`/settings`) is a newer general account-settings screen that also embeds the change-password form plus an appearance (theme) toggle — it and the standalone `ChangePasswordPage` currently coexist as two routes to the same change-password flow.
    - **Relationships**: Uses Auth Store to persist session; calls APIClient for auth endpoints; validates login/register forms directly against Shared Types via `zodResolver` (`LoginPage`, `RegisterPage`).
- **Moderator Pages (`ModeratorDashboardPage`, `ReportsQueuePage`)**
    - **Responsibility**: Moderator dashboard with report statistics and a reports queue page where moderators can review, flag (escalate), or dismiss user reports.
    - **Relationships**: Calls APIClient for moderation endpoints; guarded by RequireRole.
- **Admin Pages** (`AdminLoginPage`, `AdminDashboardPage`, `AdminUsersPage`, `AdminReportsPage`, `AdminCategoriesPage`, `AdminSettingsPage`, `AdminAuditLogsPage`) — Full admin panel: site stats, user management, report

management, category CRUD, system settings, audit logs. `AdminLoginPage` (`/admin/login`) is the one public, unguarded entry point; all other `/admin/*` routes are wrapped in `AdminGuard`, which redirects to it if the signed-in user is not an administrator. Calls APIClient for admin endpoints; `AdminLoginPage` validates its form directly against Shared Types (it reuses the same `LoginRequest` schema as the standard `LoginPage`).

- **Auth Store (`src/frontend/src/stores/auth.ts`)**
  - **Responsibility**: Zustand store managing client-side authentication state: `user` object, `accessToken`, `setSession`, `clearSession`, and `silentRefresh` (automatic token renewal via the refresh cookie).
  - **Relationships**: Read by APIClient for Bearer token injection; updated by Auth Pages upon login, registration, or session restoration.
- **API Client (`src/frontend/src/api/`)**
  - **Responsibility**: Modular API modules (`auth.ts`, `interactions.ts`, `post-summary.ts`, `profile.ts`, `search.ts`, `upload.ts`, `admin.ts`, `hashtags.ts`) providing typed `fetch` wrappers with `Bearer` authorization headers and `@tanstack/react-query` hooks for caching and data sync. A shared `network.ts` module detects offline state and surfaces a `NetworkError` toast when a request cannot reach the server.
  - **Relationships**: Invoked by all page components; reads access tokens from Auth Store; executes REST calls to Backend API Server; types request/response payloads against Shared Types.
- **Shared Types (`src/shared/`)**
  - **Responsibility**: The same Zod schemas, DTOs, and enums used by the backend (see 4.2), imported client-side via the `#shared/*` path alias. On the frontend it is compiled into the SPA bundle at build time rather than run as a service — it is not a runtime container of its own (see the note in Section D, "Shared Module"). Used both for typing API Client requests/responses and for `zodResolver`-driven form validation.
  - **Relationships**: Imported by API Client, Auth Pages (`LoginPage`, `RegisterPage`), Admin Pages (`AdminLoginPage`), and Profile Page (`ProfileEditModal`). Same source module as the backend's Shared Types component (4.2.1) — no network call, just a shared compile-time dependency.

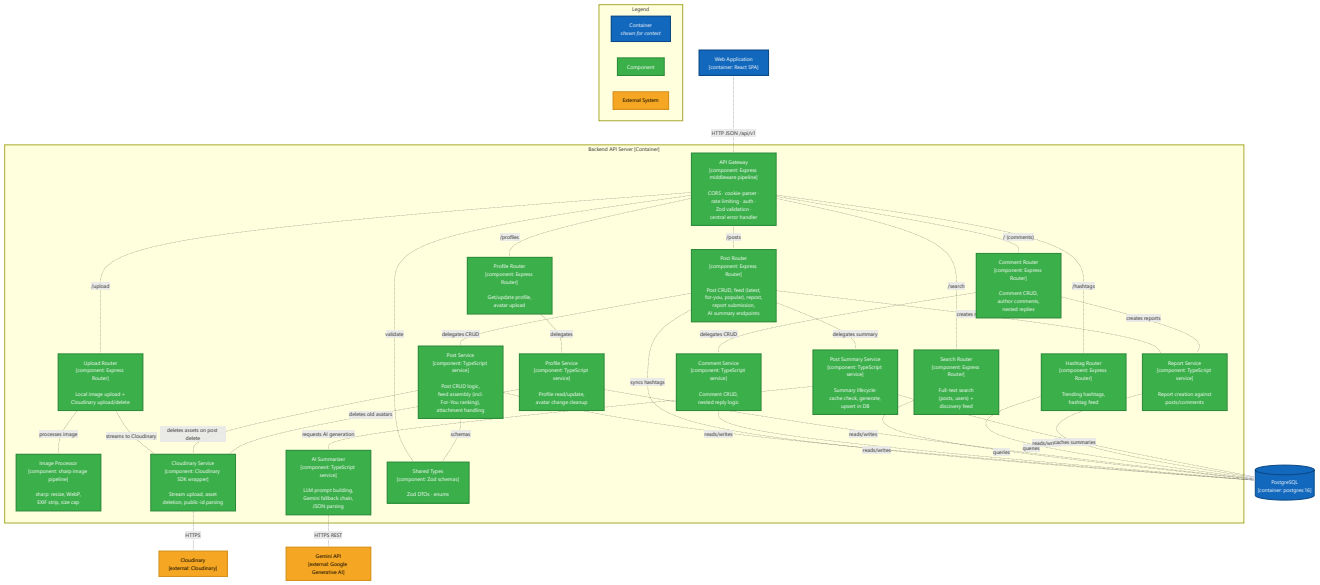
4.2 Backend - API Server

The backend is split into two diagrams for readability: **Diagram A** covers content and discovery (posts, comments, search, profiles, hashtags, media, reports, AI TL;DR); **Diagram B** covers identity and governance (auth, OAuth, reactions, moderation, admin).

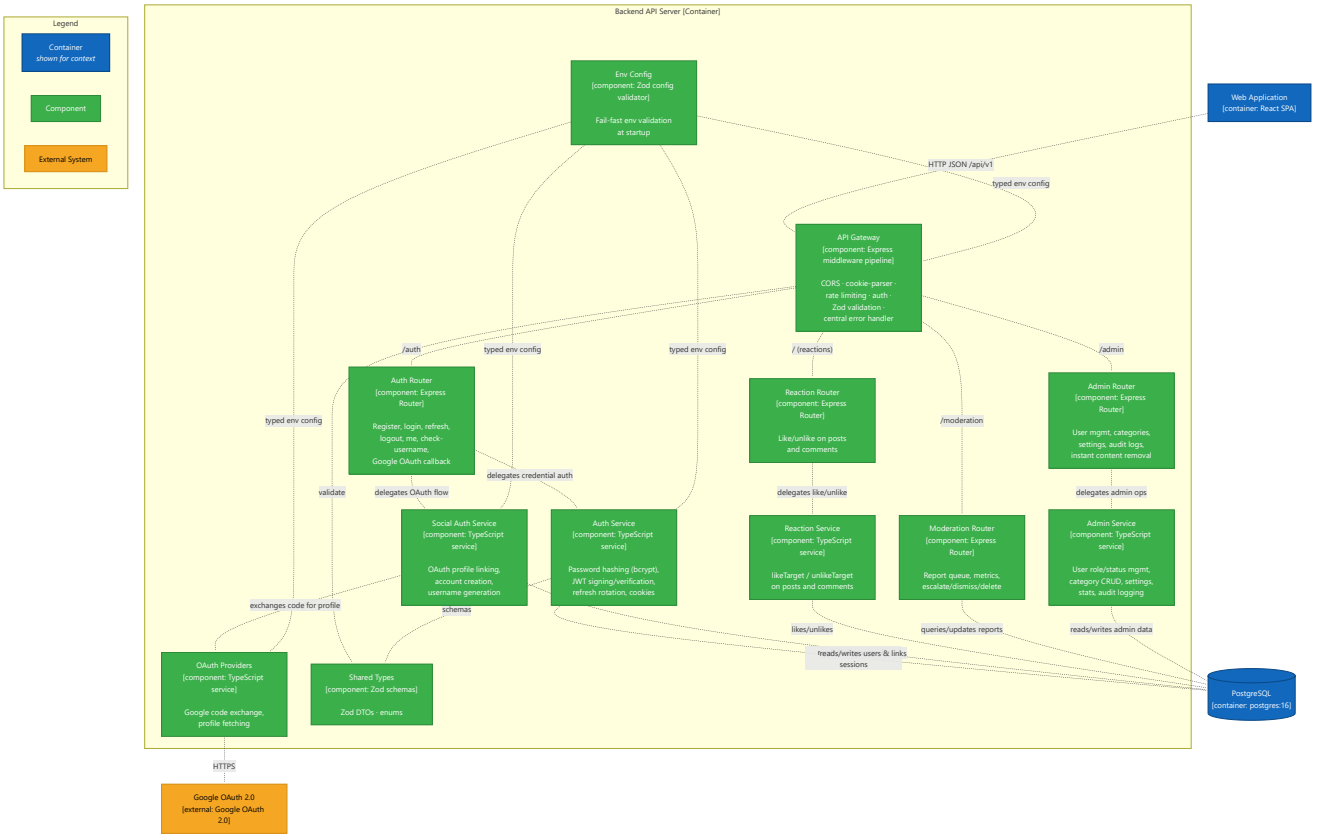
Why AI TL;DR is in Diagram A: the summary endpoints live on the Post Router (`/posts/:postId/summary`), and the summary service / AI summarizer are wired from there, not from a standalone router.

Why no Report Service in Diagram B: report *creation* lives on the Post Router and Comment Router, both in Diagram A; the Moderation Router only reads/updates reports directly against the database and has no service layer.

4.2.1 Diagram A - Content & Discovery (Feed · Search · Profile · Hashtags · Media · Reports · AI TL;DR)



4.2.2 Diagram B - Identity, Governance & AI (Auth · OAuth · Reactions · Moderation · Admin)



> **Note:** The API Gateway and Shared Types appear in both diagrams (shared infrastructure). The split is purely for presentation — the backend runs as **one container**.

4.2.3 Component Descriptions

Middleware Pipeline (API Gateway)

Middleware applied (in order) before routing in `src/backend/index.ts`:

Component	Responsibility	Source
CORS Middleware	Enforces origin-based CORS policy with credentials support. Allowed origins are configurable via `CORS_ORIGIN` env var.	`src/backend/index.ts`
Auth Middleware	Extracts and verifies the JWT access token from the `Authorization: Bearer` header. Hydrates `req.user` with `{ sub, role, status }`. Routes can be optionally or mandatorily authenticated (`requireAuth`, `optionalAuth`).	`src/backend/api/auth/`

Component	Responsibility	Source
Rate Limiter	Applies configurable per-window rate limits to abuse-prone endpoints (auth, AI summarization). Uses `express-rate-limit` with in-memory store.	`src/backend/lib/rate-limit.ts`
Zod Validation	Validates request bodies and parameters against Zod schemas from `src/shared/` at the API boundary. Rejects invalid requests with structured 422 error responses.	Inline in route handlers
Central Error Handler	The single place that maps thrown errors to HTTP JSON responses: `ZodError` -> 422, `AuthError` -> status from error, payload-too-large -> 413, unhandled -> 500. No error swallowing anywhere else.	`src/backend/index.ts`

Route Layer

Router	Mount Point	Endpoints
Auth Router	`/api/v1/auth`	`POST /register`, `POST /login`, `POST /refresh`, `POST /logout`, `GET /me`, `GET /check-username`, `GET /google`, `GET /google/callback`, `PATCH /password`
Post Router	`/api/v1/posts`	`GET /` (feed via `?type=latest|for-you`, `?authorId=`), `POST /`, `PATCH /:postId`, `DELETE /:postId`, `POST /:postId/repost`, `DELETE /:postId/repost`, `POST /:postId/report`, `GET /:postId/summary`, `POST /:postId/summary/refresh`

Router	Mount Point	Endpoints
Comment Router	mounted at `/` (no prefix)	`GET /posts/:postId/comments`, `POST /posts/:postId/comments`, `GET /comments` (by `?authorId=`), `PATCH /comments/:commentId`, `DELETE /comments/:commentId`, `POST /comments/:commentId/report`
Reaction Router	mounted at `/` (no prefix)	`POST /posts/:postId/reaction`, `DELETE /posts/:postId/reaction`, `POST /comments/:commentId/reaction`, `DELETE /comments/:commentId/reaction` (like/unlike on posts and comments)
Profile Router	`/api/v1/profiles`	`GET /by-username/:username`, `GET /:userId`, `PATCH /me`, `POST /me/avatar`
Search Router	`/api/v1/search`	`GET /discovery` (suggested creators, trending discussions, topics), `GET /` (unified search, `?type=posts|users|all`)
Upload Router	`/api/v1/upload`	`POST /image` (multer → sharp → local disk), `POST /cloudinary` (stream upload), `DELETE /cloudinary/{*publicId}`
Moderation Router	`/api/v1/moderation`	`GET /metrics`, `GET /reports`, `PATCH /reports/:reportId/status`, `POST /reports/:reportId/action` (delete_content / dismiss / escalate)
Admin Router	`/api/v1/admin`	`GET /metrics`, `GET /users`, `PUT`

Router	Mount Point	Endpoints
		`/users/:userId/role`, `DELETE /users/:userId`, `GET /audit-logs`, `GET /reports`, `POST /reports/:id/dismiss`, `DELETE /content/:type/:id`, `GET/POST /categories`, `PUT /categories/:id`, `DELETE /categories/:id`, `GET /settings`, `PUT /settings`
Hashtag Router	`/api/v1/hashtags`	`GET /trending`, `GET /:tag/posts`

Service Layer

Component	Responsibility	Key Relationships
Auth Service (`src/backend/api/auth/auth.service.ts`)	Core authentication logic: password hashing and verification (bcrypt), JWT access token signing and verification (HS256, 15-min TTL), refresh token issuance with rotation (7-day, HttpOnly cookie), cookie construction, and session management. Defines the `AuthError` class used by the central error handler.	Reads/writes `user_accounts`, `profiles`, `refresh_sessions` tables.
Social Auth Service (`src/backend/services/social-auth.service.ts`)	Handles the OAuth account-linking flow: finds existing social links, links social profiles to existing email-matched accounts, or creates new user accounts with generated usernames. Issues session tokens matching the standard auth flow.	Calls OAuth Providers; reads/writes `user_accounts`, `user_social_links` tables.
OAuth Providers (`src/backend/services/oauth-providers.service.ts`)	Implements provider-specific OAuth logic. Currently supports Google: exchanges authorization codes for tokens via the	Calls Google OAuth 2.0 (external).

Component	Responsibility	Key Relationships
	Google token endpoint, then fetches the user profile from the Google userinfo API. A Facebook route file exists (`facebook-auth.routes.ts`) but is an unimplemented stub — not a live feature.	
Post Service (`src/backend/services/interactions/posts.ts`)	Business logic for post CRUD: creation (with content and attachments), reading (single post with author/reactions/comment count), updating, soft-deletion (respecting ownership), repost create/remove, and feed assembly with cursor-based pagination. Includes the For-You feed algorithm (engagement-weighted scoring via `feed-ranking.ts`). Hashtag sync on create/update is done by the **Post Router** directly, not this service.	Reads/writes `posts` , `post_attachments` ; calls Cloudinary Service to delete attachments on post delete.
Comment Service (`src/backend/services/interactions/comments.ts`)	Business logic for comment CRUD: creation (with post-existence check and optional parent-comment for nesting), updating, and soft-deletion with ownership enforcement.	Reads/writes `comments` table.
Reaction Service (`src/backend/services/interactions/reactions.ts`)	Like/unlike logic: `likeTarget` adds a reaction, `unlikeTarget` removes it. Currently supports the "like" reaction kind on both posts and comments.	Reads/writes `reactions` table.
Report Service (`src/backend/services/interactions/reports.ts`)	Creates user reports against posts or comments with a reason type and optional details. Report creation is invoked by the **Post Router** (`POST /posts/:postId/report`)	Reads/writes `reports` table. Called by Post Router and Comment Router.

Component	Responsibility	Key Relationships
	and the **Comment Router** (`POST /comments/:commentId/report`); the Moderation Router reads/updates reports directly via `db` and does not use this service.	
AI Summarizer (`src/backend/services/ai-summarizer.ts`)	Core AI logic: builds a structured prompt from post content and comment bodies, calls the Gemini API with a fallback chain of model versions (`gemini-3.6-flash -> gemini-flash-latest -> gemini-3.5-flash`), parses the JSON response into `{ postSnapshot, keyHighlights, sentiment }` , and falls back to a text-truncation heuristic if all API calls fail.	Calls Gemini API (external).
Post Summary Service (`src/backend/services/post-summary.service.ts`)	Orchestrates the summary lifecycle: checks if a cached summary exists and is still fresh, triggers AI Summarizer if not, and upserts the result into the database.	Uses AI Summarizer; reads/writes `post_summaries` table.
Profile Service (`src/backend/api/profile/profile.service.ts`)	Profile read by user ID / username and update. On avatar change, deletes the previous Cloudinary avatar asset (best-effort). Defines the `ProfileError` class.	Reads/writes `profiles` ; calls Cloudinary Service for old-avatar cleanup.
Image Processor (`src/backend/services/image-processor.ts`)	Wraps `sharp` : validates format, auto-orients, resizes to per-upload-type limits, converts to WebP, strips EXIF, and enforces a 2MB output cap with quality step-down.	Used by Upload Router (`POST /upload/image`); writes to local disk, no DB or Cloudinary.
Cloudinary Service (`src/backend/services/cloudinary.ts`)	Wraps the Cloudinary SDK: streams file buffers to Cloudinary's upload API,	Calls Cloudinary (external). Used by Upload Router (`/upload/cloudinary`), Post Service (attachment

Component	Responsibility	Key Relationships
	extracts public IDs from URLs, and deletes assets.	cleanup), and Profile Service (avatar cleanup).
Admin Service	Admin business logic: dashboard statistics (user/post/comment/report counts), user management (role changes, status changes with audit logging), forum category CRUD, system settings management, and audit log retrieval.	Reads/writes `admin_audit_logs`, `forum_categories`, `system_settings`, `reports`, `user_accounts` tables.

Data Access (`src/backend/db/`)

Database access is handled by Drizzle ORM repository modules. These are an implementation detail of the service layer — each service imports the repositories it needs directly. Key tables:

Table Group	Tables	Used By
User & Auth	`user_accounts`, `profiles`, `refresh_sessions`, `user_social_links`	Auth Service, Social Auth Service
Content	`posts`, `comments`, `reactions`, `post_attachments`	Post Service, Comment Service, Reaction Service
Reports & Moderation	`reports`	Report Service, Admin Service
Hashtags	`hashtags`, `post_hashtags`	Post Service, Hashtag Router
Summaries	`post_summaries`	Post Summary Service
Admin	`admin_audit_logs`, `forum_categories`, `system_settings`	Admin Service

Infrastructure

Component	Responsibility
Env Config (`src/backend/env.ts`)	Validates all environment variables at startup using a Zod schema. Fails fast with clear error messages if required variables are missing or malformed. Exports the

Component	Responsibility
	typed `env` object used throughout the backend, including the API Gateway bootstrap (`index.ts`), Auth Service, Social Auth Service, and OAuth Providers (shown as direct dependencies in Diagram B).

D - Deployment Diagram

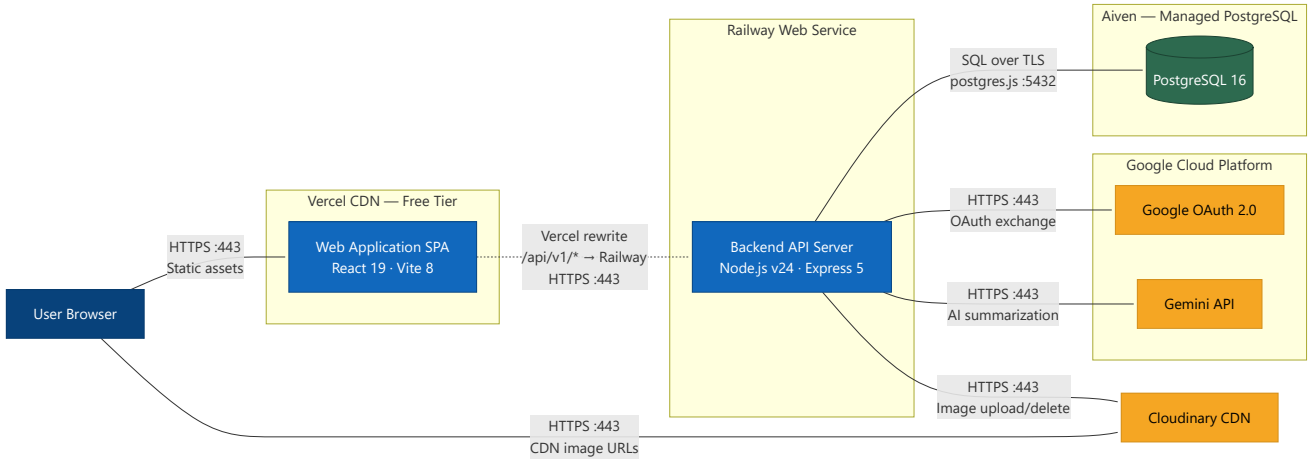
Work performed by: Nguyễn Tuấn An | Reviewed by: Võ An Phú | Edited by: Võ An Phú

> **Version**: 1.3.0
> **Last Updated**: 2026-08-22
>
> This document maps the containers defined in the Architecture Document to the physical and cloud infrastructure that runs them, for both **development** and **production** environments. Diagrams use Mermaid syntax following C4 deployment conventions.

NOTE: Due to data leak concerns, there are some variables that are not included in this document.

1. Production Deployment Diagram

1.1 Diagram



How the request flows

1. The **User Browser** loads static assets (HTML, JS, CSS) from **Vercel CDN** over HTTPS.
2. Once the React SPA is running in the browser, all API calls go to the same Vercel domain under `/api/v1/*` .
3. **Vercel rewrite rules** (defined in `vercel.json`) transparently proxy those requests to `https://foz-backend.up.railway.app/api/v1/*` — the browser never contacts

Railway directly.

4. The **Backend API Server** on Railway connects to **Aiven PostgreSQL** via the `postgres.js` driver (TLS-encrypted, port 5432), and makes outbound HTTPS calls to **Google OAuth 2.0**, **Gemini API**, and **Cloudinary** as needed.
5. The browser loads images directly from **Cloudinary CDN** URLs stored in the database.

1.2 Node Descriptions

Node	Infrastructure / Cloud Service	Containers Deployed	Notes
User Device	Desktop or mobile device running a modern web browser (Chrome, Firefox, Safari, Edge)	**Web Application (SPA)** — downloaded and executed client-side as static JS/CSS/HTML	The React 19 SPA runs entirely in the browser after the initial page load. All server communication is via REST API calls.
Vercel — Edge Network / CDN	**Vercel** (free tier). Global edge CDN with serverless rewrite rules.	**Web Application (SPA)** — static build output (`src/frontend/dist`) served from Vercel's CDN.	Vercel serves the static SPA bundle and uses rewrite rules (`vercel.json`) to proxy all `/api/v1/*` requests to the Railway-hosted backend (`foz-backend.up.railway.app`). This eliminates CORS issues in production and provides a unified domain for the user. Build command: `npm run build` .
Railway — Web Service	**Railway**. Cloud platform & container hosting.	**Backend API Server** — Node.js v24 Express 5 application running as a Railway Web Service.	The backend runs as a single long-lived containerized service. Railway handles environment variables, continuous deployment from GitHub, and domain routing for Google OAuth callback URL

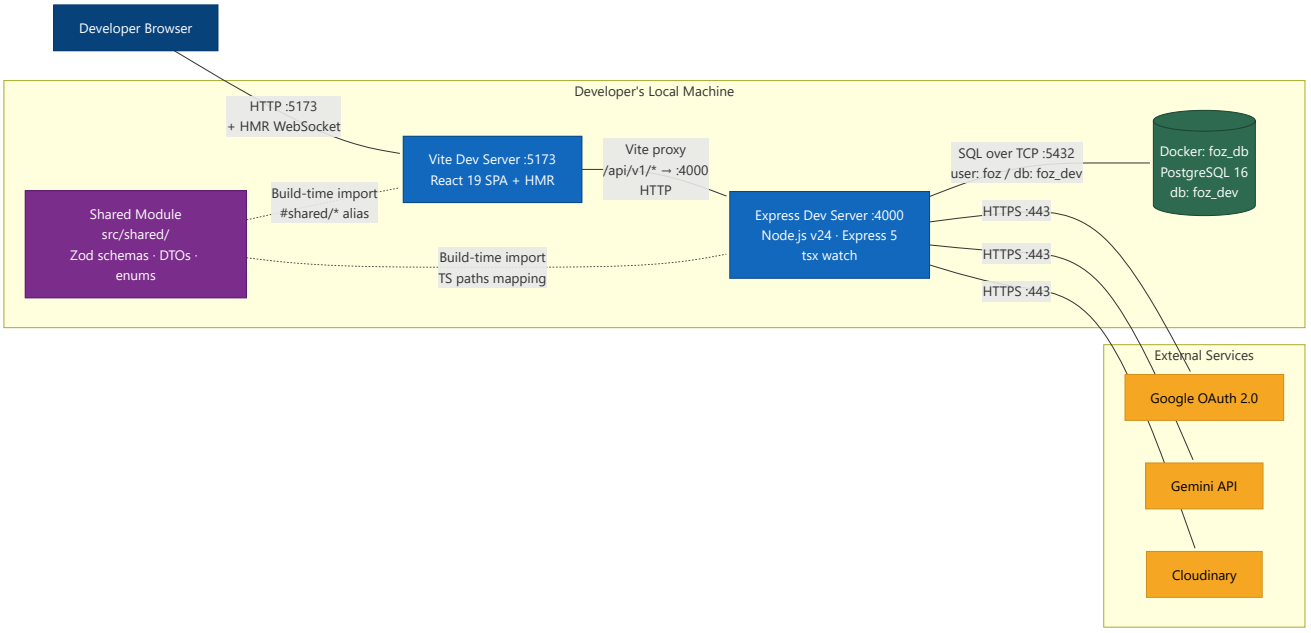
Node	Infrastructure / Cloud Service	Containers Deployed	Notes
			construction (`https://foz-backend.up.railway.app`).
Aiven — Managed PostgreSQL	**Aiven**. Fully managed cloud database service.	**PostgreSQL Database** — `postgres:16` -compatible managed database.	Aiven provides a highly available, cloud-managed PostgreSQL 16 instance. The backend connects via `DATABASE_URL` using the `postgres.js` driver over TLS-encrypted TCP on port 5432. Schema migrations are managed via `drizzle-kit` .
Google Cloud Platform	**Google Cloud** (API-based services).	**Google OAuth 2.0** — social login identity provider. **Google Generative AI (Gemini API)** — AI summarization service.	Two independent Google services: OAuth 2.0 for authentication and Gemini API for post/comment TL;DR summarization with a fallback model chain (`gemini-3.6-flash` → `gemini-flash-latest` → `gemini-3.5-flash`).
Cloudinary	**Cloudinary** (free tier). Cloud-based media management and CDN.	**Cloudinary Media Service** — image storage, transformation, and CDN delivery.	Post attachments and profile avatars are uploaded from the backend via Cloudinary's streaming API. CDN URLs are stored in the database and served directly to browsers for read access.

1.3 Communication Protocols Summary

From	To	Protocol	Port	Description
User Browser	Vercel CDN	HTTPS	443	Initial SPA download (HTML, JS, CSS, static assets)
Vercel CDN (rewrite)	Railway Backend	HTTPS	443	Vercel transparently rewrites `/api/v1/*` to `foz-backend.up.railway.app`. The browser never contacts Railway directly. Authenticated requests include `Authorization: Bearer <token>` header; refresh uses HTTP-only cookies.
Railway Backend	Aiven PostgreSQL	PostgreSQL wire protocol over TLS	5432	SQL queries via `postgres.js` driver through Drizzle ORM
Railway Backend	Google OAuth 2.0	HTTPS	443	Authorization code exchange and user profile fetch
Railway Backend	Google Gemini API	HTTPS REST	443	Structured prompt submission and JSON response parsing
Railway Backend	Cloudinary	HTTPS	443	Image stream upload and asset deletion
User Browser	Cloudinary CDN	HTTPS	443	Direct image retrieval via CDN-delivered URLs

2. Development Deployment Diagram

2.1 Diagram



Key development details

- **Vite proxy**: The Vite config (`src/frontend/vite.config.ts`) proxies both `/api/v1` and `/health` to `http://localhost:4000` , eliminating CORS issues during local development.
- **Backend runner**: The backend is launched via `tsx watch src/backend/index.ts` (defined as the `dev:be` script in `package.json`), which provides hot-reload on file changes.
- **Database**: PostgreSQL 16 runs as Docker service `foz_db` via `docker-compose.yml` with credentials `foz/foz` , database `foz_dev` , data persisted in volume `foz_pg_data` , and a health check (`pg_isready -U foz -d foz_dev`).
- **Shared Module**: Not a running process — `src/shared/` is imported at build time by both Vite (via `#shared` alias) and the backend (via TypeScript `paths` mapping).

2.2 Node Descriptions

Node	Infrastructure	Containers / Processes Running	Notes
Developer Browser	Local browser (Chrome, Firefox, Edge)	**Web Application (SPA)** — React app loaded from `http://localhost:5173`	HMR (Hot Module Replacement) via Vite's WebSocket connection provides instant feedback

Node	Infrastructure	Containers / Processes Running	Notes
			during development.
Vite Dev Server	Local machine, Node.js process on **port 5173**	**Web Application (SPA)** — Vite 8 dev server serving the React 19 SPA with HMR	Proxies `/api/v1` and `/health` to `http://localhost:4000`.
Express Dev Server	Local machine, Node.js process on **port 4000**	**Backend API Server** — Express 5 RESTful API running in development mode	Launched via `tsx watch` for hot-reload. Environment variables loaded from `.env` file.
Docker Container (foz_db)	Local machine, **Docker** running `docker-compose.yml`	**PostgreSQL 16** — `postgres:16` image as service `foz_db`	Credentials: user `foz`, password `foz`, database `foz_dev`. Port `5432` exposed to localhost. Data persisted in Docker volume `foz_pg_data`.
Shared Module	Local filesystem (`src/shared/`)	**Shared Module** — TypeScript source (Zod schemas, DTOs, enums)	Not a running process. Consumed at build time via `#shared/*` path aliases. Changes trigger HMR/rebuild in consumers.
Google Cloud Platform	Google Cloud (same APIs as production)	**Google OAuth 2.0**, **Gemini API**	Requires `GOOGLE_CLIENT_ID`, `GOOGLE_CLIENT_SECRET`, and `AI_API_KEY` in `.env`. OAuth callback defaults to `http://localhost:4000/api/v1/auth/google/callback`.
Cloudinary	Cloudinary Cloud (same API as production)	**Cloudinary Media Service**	Requires `CLOUDINARY_CLOUD_NAME`, `CLOUDINARY_API_KEY`, and `CLOUDINARY_API_SECRET` in `.env`.

2.3 Communication Protocols Summary

From	To	Protocol	Port	Description
Browser	Vite Dev Server	HTTP + WebSocket	5173	Page loads and HMR updates
Vite Dev Server	Express Dev Server	HTTP (Vite proxy)	4000	Proxied REST API calls (`/api/v1/*` and `/health` → `localhost:4000`)
Express Dev Server	Docker PostgreSQL	PostgreSQL wire protocol (TCP)	5432	SQL queries via `postgres.js` / Drizzle ORM (unencrypted locally)
Express Dev Server	Google OAuth 2.0	HTTPS	443	Authorization code exchange and user profile fetch
Express Dev Server	Gemini API	HTTPS REST	443	AI summarization requests
Express Dev Server	Cloudinary	HTTPS	443	Image upload and deletion

---

3. Environment Comparison

Aspect	Development	Production
Frontend Hosting	Vite dev server on `localhost:5173`	Vercel Edge CDN (global, static build)
Backend Hosting	Express on `localhost:4000` via `tsx watch`	Railway Web Service (`foz-backend.up.railway.app`)
Database	Docker container `foz_db` (`postgres:16` , local)	Aiven Managed PostgreSQL (cloud, TLS-encrypted)
API Proxying	Vite proxy (`/api/v1` and `/health` → `localhost:4000`)	Vercel rewrite rules (`/api/v1/*` → Railway)

Aspect	Development	Production
TLS/HTTPS	HTTP (no TLS locally)	HTTPS everywhere (Vercel, Railway, Aiven all enforce TLS)
Image Storage	Cloudinary (same cloud service)	Cloudinary (same cloud service)
OAuth Callback	`http://localhost:4000/api/v1/auth/google/callback`	`https://foz-backend.up.railway.app/api/v1/auth/google/callback`
Shared Module	Live source imports with HMR rebuild	Compiled into static bundles at build time
Database Credentials	Hardcoded in `docker-compose.yml` (`foz/foz`)	Managed via Aiven console + `DATABASE_URL` env var
Cold Start / Uptime	None (always running)	Continuous service running on Railway

E - Implement 2 Functional Groups using Spec Kit

Work performed by: Võ An Phú | Reviewed by: Nguyễn Tuấn An | Edited by: Nguyễn Tuấn An

- **Video Demo:** can be found at: `youtube-link.txt` or `https://youtu.be/ETm_nWagoRE?si=to6VentWdVtbCzoN`
- **Source Code:** source code for current progress can be found at folder: `src/`
- **Spec Kit Artifacts:** generated Spec Kit artifacts for current progress can be found at folder: `specs/`

F - AI Usage Report and Weekly Report

Work performed by: Nguyễn Tuấn An | Reviewed by: Phạm Đinh Gia Khánh | Edited by: Phạm Đinh Gia Khánh

AI Usage Report

Declaration of No AI Use

We declare that no AI tools were used in peer review tasks.

AI Usage Notes

AI Usage Report of each member can be found in folder: `ai-logs/`

Weekly Report

Sprint Planning

========= 25/7, Sprint 4 planning ==========

Team members present:

- Nguyễn Tuấn An
- Nguyễn Hải Nam
- Nguyễn Hải Trọng
- Phạm Đinh Gia Khánh
- Võ An Phú

Team members absent: None

Reports:

- Nguyễn Tuấn An:
  - List all possible tasks for PA4 and for project development process, getting ideas from the team members to make sure all tasks are covered.
  - Estimate time for each task and assign them to team members.

Actions:

- All team members agreed to the tasks and their assignments.

Summary of the meeting:

- The team has finalized the list of tasks for the sprint, including both PA4 and project development process tasks. All team members have agreed on their assignments and estimated time for each task.

Scrum Meetings

========= 2/8, Scrum 1 ==========

Nguyễn Tuấn An

1. Fix ai usage log skill, fix post creation function, implement post summarize function, fix UI in home + comment sections, deploy app to website, add image compression, add multiple images scrolling effect
2. Write report D, plan for PA5
3. None

Nguyễn Hải Nam

1. Write and Draw Software Architecture: Container Diagram and Component Diagram, Implement Admin view, Review Software Architecture: System Context Diagram

2. Integration and System test
3. None

Nguyễn Hải Trọng

1. Write and Draw Software Architecture: System Context Diagram
2. Review report D and write report A
3. None

Phạm Đinh Gia Khánh

1. Design Admin UI, implement Moderator view
2. Review report A
3. None

Võ An Phú

1. Implemented for you feed, fixed create post pro5 picture out of sync, fix picture not showing in /foryou
2. Write report and make video for E, review C report
3. None

========= 8/8, Scrum 2 ==========

Nguyễn Tuấn An

1. Integrate cloud; design logo; fix for you button on guest; add guide when hovering navbar; add client side process for fast Like response; fix redirect when register/login; add click post/image to view detail; add read more button; add sort comment; fix search use mock data; redesign login UI; implement login with Google; redesign report format; Review and edit B C reports; write D report
2. Plan for PA5, assign tasks to team members.
3. None

Nguyễn Hải Nam

1. Revised and rewrite UC14 to UC21, Implement hashtag feature, polish profile page ui
2. Waiting for new tasks
3. None

Nguyễn Hải Trọng

1. Write part B and edit UC01 to UC06 in report A
2. Waiting for new tasks for PA5, fixing some issues in Github
3. None

Phạm Đinh Gia Khánh

1. Implement post report, upgrade signup design, review and edit A report,
2. Waiting for new tasks, optimize some functions
3. None

Võ An Phú

1. Make video for E: Post creation, For you feed generation; Review and edit C report; implemented setting; fix dark mode readmore
2. Wait task
3. None

Sprint Review

========= 8/8 Sprint Review ==========

Team members present:

- Nguyễn Tuấn An
- Nguyễn Hải Nam
- Nguyễn Hải Trọng
- Phạm Đinh Gia Khánh
- Võ An Phú

Team members absent: None

What went well: discover several potential vulnerabilities, revise project based on lecturers' feedbacks.

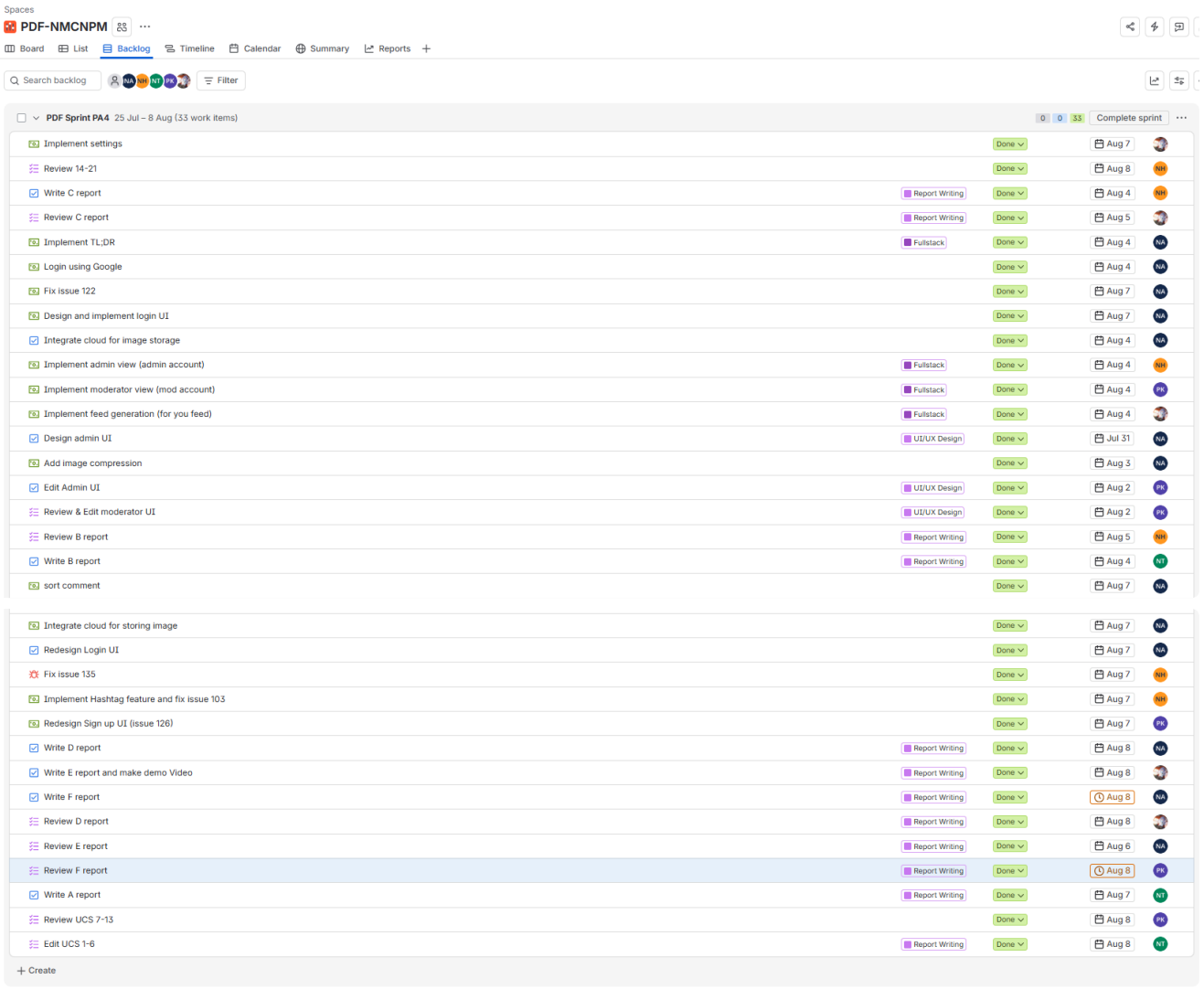
What went wrong: some tasks quality not good and need refinement.

What problems and what caused the problems: foundation of knowledge weak.

What can be done differently in the next sprint to improve: team members should learn more.

What lessons we could learn: the importance of learning and grinding.

Jira Board


Spaces
PDF-NMCNPM
Board List Backlog Timeline Calendar Summary Reports
Search backlog Filter
PDF Sprint PA4 25 Jul – 8 Aug (33 work items) Complete sprint
Implement settings Done Aug 7
Review 14-21 Done Aug 8
Write C report Report Writing Done Aug 4
Review C report Report Writing Done Aug 5
Implement TL;DR Fullstack Done Aug 4
Login using Google Done Aug 4
Fix issue 122 Done Aug 7
Design and implement login UI Done Aug 7
Integrate cloud for image storage Done Aug 4
Implement admin view (admin account) Fullstack Done Aug 4
Implement moderator view (mod account) Fullstack Done Aug 4
Implement feed generation (for you feed) Fullstack Done Aug 4
Design admin UI UI/UX Design Done Jul 31
Add image compression Done Aug 3
Edit Admin UI UI/UX Design Done Aug 2
Review & Edit moderator UI UI/UX Design Done Aug 2
Review B report Report Writing Done Aug 5
Write B report Report Writing Done Aug 4
sort comment Done Aug 7
Integrate cloud for storing image Done Aug 7
Redesign Login UI Done Aug 7
Fix issue 135 Done Aug 7
Implement Hashtag feature and fix issue 103 Done Aug 7
Redesign Sign up UI (issue 126) Done Aug 7
Write D report Report Writing Done Aug 8
Write E report and make demo Video Report Writing Done Aug 8
Write F report Report Writing Done Aug 8
Review D report Report Writing Done Aug 8
Review E report Report Writing Done Aug 6
Review F report Report Writing Done Aug 8
Write A report Report Writing Done Aug 7
Review UCS 7-13 Done Aug 8
Edit UCS 1-6 Report Writing Done Aug 8
Create